

ABES Engineering College, Ghaziabad

Department of ASH



Estd. 2000

Web Designing

Year/ Sem. : 1st year/ I sem.

Subject Name : WD Workshop-I

Subject Code : 25VA151

Faculty Name : Ms. Deepti Deshmukh , Mr. Jitendra Kumar Chauhan,
Mr. Sanjeev Soni , Ms. Pranshi Verma , Ms. Monika ,
Mr. Rohit Pratap Singh

Approved by AICTE, New Delhi & Affiliated to Dr. APJ Abdul Kalam Technical University, UP,
Lucknow

NBA Accredited Courses (CSE, IT, ECE, EN)

19th KM Stone, NH-24, Ghaziabad-201009, UP



ABES Engineering College, Ghaziabad

VISION

To take ABES Engineering College to such a level that, it is at par with the leading institutions of the world in providing leadership to the international education system and be amongst the top-rated institutions of the world by providing a transformative education to create leaders and innovators embedded in traditional Indian values.

MISSION

- To create an ambiance for healthy teaching learning process.
- To nurture the students and infuse in them:
 - A passion to excel professionally
 - A spirit to be of utmost use to the industry, corporate sector and the society at large
 - An intense desire to take challenging responsibilities and leadership roles
 - A craving to be wholesome good human beings
- To develop an environment for creating new knowledge through research and by thriving to explore innovative ideas.

QUALITY POLICY

To continuously thrive to provide a congenial and wholesome academic environment and a healthy culture for faculty, staff and students which would motivate teacher's full participation with passion and develop an intense desire in the students to acquire comprehensive education and hence become a useful and confident human resource for the industry and academia.

VALUES

Respect – We treat people the way we want to be treated

Integrity – We believe in doing things right when no one is watching

Accountability – We take full responsibility of our actions and are accountable for them

Transparency – We communicate openly amongst ourselves and all our stakeholders to avoid any misconception.

Excellence – We endeavor to think out-of-the-box and deliver maximum value to all our Stakeholders.



ABES Engineering College, Ghaziabad

Department of ASH

VISION

The department of ASH has a distinct vision to nurture and inculcate knowledge and understanding of engineering sciences amongst the budding technocrats and groom the students with intellectual and professional strength. ASH is an anvil to chisel future professionals with the skills of engineering, imbued with social, ethical and moral values, so that the pupils can expedite their efforts to build technologies from knowledge and be a part of the workforce, instrumental in the development of the nation's technological and economic progress.

MISSION

M1: To play a pivotal role in laying the foundations of engineering education by building the professional strengths of the students in basic sciences and engineering disciplines and strengthening the foundation of aspiring engineers.

M2: To provide a unique learning environment for the students, preparing the students at par with the global technical workforce, building confidence, character and human values.

M3: To upgrade the quality of technical education and produce dynamic engineers and entrepreneurs for the country, with a sole aim to spot ABESEC amongst the best educational institutions of global standards



PROGRAMME OUTCOMES (POs)

PO 1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO 2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO 3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO 4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO 6. The engineer and society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.

PO 7. Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.

PO 9. Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO 11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO 12. Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



ABES Engineering College, Ghaziabad

B.Tech. First Year, Semester-I Common to all branches)

Course Code	25VA151	Course Name	Web Designing	L	3	T	0	P	0	C	0
-------------	---------	-------------	---------------	---	---	---	---	---	---	---	---

Course Objective

- * Introduce students to the fundamentals of web technologies and the full-stack development environment. *
- Familiarize students with modern tools like VS Code and Git for development and version control practices.
- * Provide hands-on experience with HTML, CSS, and GitHub for collaborative development.
- * Develop practical skills in creating structured, styled, and interactive web pages.

Syllabus

Unit-1	Introduction to Web Technology	
Introduction to Web Technology, Full Stack Development, Web Designing, Web Development, Client Server Architecture, Introduction to Text Editors, Installation & Set up of VS Code Editor, Introduction of Git & GitHub, Installation & Configuration of Git Account, Introduction to HTML & HTML Features, HTML Page Structure, Elements in HTML (Semantic and Non-Semantic), Types of Elements (Inline/Block), Text formatting: headings, paragraphs, HTML Lists, Unordered Lists, Ordered Lists, Description Lists, Nesting and Mixed Lists, Lists for Navigation Menus, HTML links, Images and attributes, Comment in HTML, HTML Formatting Elements / Special Character & Symbol, Text related Elements (Quotation and Citation Elements), Using Multimedia (Image/Video/Audio/YouTube) in HTML, Creating & Using Hyperlinks, Accessibility and Semantic Considerations, Practical	10	
Unit-2	Web Page Development using HTML	
HTML Table: Creating and using Tables, Basic Table Structure, Table Headers and Captions, HTML Table: Rowspan and Colspan, Table Attributes, Responsive Tables, HTML Table: Nested Tables, Accessibility in Tables, Practical, HTML Form: The <form> Element, Input Elements, Label and Fieldset, HTML Form: Select and Option Tags, TextArea, Button Element, HTML Form: Form Attributes, Input Validation, Accessibility and Usability, What is Git and why use it?, Git setup and config (git config), Creating a local repository: git init, Tracking changes: add, commit, status, log, .gitignore basics, Practical	9	
Unit-3	Version Control with Git & GitHub and CSS Fundamentals	
Creating a GitHub account, Connecting local repos to GitHub: remote add, push, Cloning and pulling repositories, GitHub repository structure and README.md, Basic collaboration terms: fork, pull request, branch (intro), Branching: git branch, git checkout, git switch, Merging branches, handling merge conflicts, GitHub collaboration workflow, Best practices for commits, branches, and messages, Practical with all steps of git, CSS selectors: element, class, ID, Color, fonts, text alignment, font size, Box model: padding, margin, border, Box model: padding, margin, border, Applying styles: inline, internal, external	11	
Unit-4	CSS for Advanced Styling and Responsive Layout	
display: block, inline, inline-block, float and clear, Introduction to Flexbox: justify-content, align-items, flex-direction, Practical in CSS, Units: px, %, em, rem, vh, vw, Media queries and breakpoints, Media queries and breakpoints, Mobile-first design, Responsive navigation concepts, Practicals, Project planning and design, Version control in large projects, Deploy using GitHub Pages, Final portfolio polishing, Project using HTML & CSS	10	

Total No. of Lectures 40

Text Books
1. Jon Duckett, HTML and CSS: Design and Build Websites, Wiley.
2. Jennifer Robbins, Learning Web Design, O'Reilly.
Reference Books
1. Ethan Brown, Web Development with Node and Express, O'Reilly.
2. Paul McFedries, HTML, CSS & JavaScript Web Publishing in One Hour a Day, SAMS.

Internal Practical Examination Evaluation Process (Weightage 50%)

Components of Internal Marks (50) For Lab Courses			
Exam	Lab ST1 (20)	Lab ST2 (20)	Attendance
Lab	Quiz: 5 Lab Records & Viva:10 Exp. Conduction: 5	Quiz: 5 Lab Records & Viva: 0 Exp. Conduction: 5	10
Duration	During the Regular Time Table of the Lab	During the Regular Time Table of the Lab	Criteria decided By EC
Syllabus	After First 5 Experiments	After First 10 Experiments	

External Practical Examination Evaluation Process (Weightage 50%)

- Duration: 2 Hours
- Marks: 50 (Viva Voce: 25 + Practical Exam: 25)
- Syllabus: All Experiments
- Project Evaluation: (Project Execution/ Viva Voce/Report

Table of Index

	Unit-01 Introduction to Web Technology	
1.0	Introduction to Web Technology Introduction to Web Technology, Full Stack Development, Web Designing	1-5
1.1	Client–Server Architecture & Text Editors	6-10
1.2	Installation & Setup of VS C ode Editor Installation & Set up of VS Code Editor, Introduction of Git & Git Hub, Installation & Configuration of Git Account	10-38
1.3	Introduction to HTML & HTML Features, HTML Page Structure, Elements in HTML (Semantic and Non Semantic)	39-45
1.4	Types of Elements (Inline/Block), Text formatting- headings, paragraphs	45-49
1.5	HTML Lists: Unordered Lists, Ordered Lists, Description Lists, Nesting and Mixed Lists, Lists for Navigation Menus	50-59
1.6	HTML Links, Images and attributes, Comment in HTML, HTML Formatting Elements/ Special Character & Symbol	60-69
1.7	Text related Elements (Quotation and Citation Elements), Using Multimedia (Image/video/Audio/YouTube) in HTML, Creating & Using Hyperlinks	70-74
1.8	Accessibility & Semantic Considerations	74-85
	Unit-02 Web Page Development using HTML	
2.0	HTML Table:Creating and using Tables,Basic Table Structure,Table Headers and Captions	86-90
2.1	HTML Table: Rowspan and Colspan, Table Attributes, Responsive Tables	90-98
2.2	HTML Table: Nested Tables, Accessibility in Tables, Practical	99-105
2.3	HTML Form: The <form> Element, Input Elements, Label and Field set	106-111
2.4	HTML Form: Select and Option Tags, Text Area, Button Element	112-113
2.5	HTML Form: Form Attributes, Input Validation, Accessibility and Usability	113-120
2.6	What is Git and why use it?, Git setup and config (git config), Creating a local repository: git init	120-128
2.7	Tracking changes: add, commit, status, log, .git ignore basics, Practical	129-130
	Unit-03 Version Control with Git & GitHub and CSS Fundamentals	
3.0	Creating a GitHub account, Connecting local repos to GitHub: remote add, push	131-139
3.1	Cloning and pulling repositories	139-141
3.2	GitHub repository structure and README.md	142-145
3.3	Basic collaboration terms: fork, pull request, branch (intro)	145-155
3.4	Branching: git branch, git checkout, git switch, merging branches, handling merge conflicts	156-173
3.5	GitHub collaboration workflow, Best practices for commits, branches, and messages	173-176
3.6	CSS selectors: element, class, ID	177-178

3.7	CSS Properties: Color, fonts, text alignment, font-size, Box model: padding, margin, border	178-181
3.8	Applying styles: inline, internal, external	182-185
	Unit 04 CSS for Advanced Styling and Responsive Layout	
4.0	Display: block, inline, inline-block, float and clear	186-191
4.1	Introduction to Flexbox: justify-content, align-items, flex-direction	192-195
4.2	Units: px, %, em, rem, vh, vw,Media queries and breakpoints	195-198
4.3	Media Queries & Breakpoints, Mobile-first design	198-202
4.4	Responsive navigation concepts, Practical	203-207
4.5	Project planning and design, Version control in large projects	207-213
4.6	Deploy using GitHub Pages, Final portfolio polishing	213-214

1.0 Introduction to Web Technology Introduction to Web Technology, Full Stack Development, Web Designing

1.0.0 What is the web?

The web, also known as the World Wide Web (WWW).

The Web (World Wide Web) is a system of websites and webpages that can be accessed through the Internet using browsers.

1.0.1 Evolution of Web

Web 1.0: Static, read-only websites.

Web 2.0: Interactive, user-driven content (social media, blogs).

Web 3.0: AI-driven, decentralized, blockchain-based, semantic web.

Some web components

Web–Page

A document which can be displayed in a web browser such as Firefox, Google Chrome, Opera, Microsoft Internet Explorer or Edge, or Apple's Safari.

Web-Site

Collection of web pages which are grouped together and usually connected together in various ways.

Web Server

A web server is a computer that runs websites.

A web server is a software that Serves web content through the HTTP protocol

1.0.2 Web Technology

“Web Technology is the technology used to make and run websites on the Internet.”

“Web Technology refers to the collection of tools, languages, and protocols that allow computers and devices to communicate through the World Wide Web (WWW).”

These technologies allow users to browse websites, interact with online content, and share information over the internet.

Components of Web Technology

1. HTML (Hypertext Markup Language)

- HTML is the foundational markup language of the web.
- It defines the structure of web pages by specifying elements such as headings, paragraphs, lists, images, and links.
- HTML acts as the starting point for any website or web application.

2. CSS (Cascading Style Sheets):

- While HTML defines the structure, CSS is responsible for the appearance.
- It controls the design, including colors, fonts, layout, and responsiveness, ensuring that websites look good across various devices and screen sizes.

3. JavaScript:

- JavaScript is a programming language that adds interactivity to websites.
- It enables dynamic content, such as clickable buttons, animations, and real-time form validation.
- JavaScript is often used alongside HTML and CSS to create seamless user experiences.

4. Web Browsers:

- Browsers like Google Chrome, Mozilla Firefox, and Safari interpret HTML, CSS, and JavaScript and render websites on users' devices.
- They play a crucial role in ensuring a smooth user experience.

5. Web Servers:

- Web servers host websites and serve content over the internet.
- They store files and data that make up a website and deliver them to users when requested.
- Web servers use protocols like **HTTP** (Hypertext Transfer Protocol) and **HTTPS** (HTTP Secure) to communicate with browsers.

6. Web Protocols:

- Protocols are the rules that govern communication between web servers and browsers.
- **HTTP** is the most common protocol, but **HTTPS**, which encrypts data for security, is becoming more prevalent.

- Newer protocols like **HTTP/2** and **HTTP/3** offer faster and more efficient communication.

7. APIs (Application Programming Interfaces):

- APIs are essential for modern web applications.
- They allow different software systems to communicate with each other.
- **Example**, a weather website may use an API to fetch real-time data from a third-party weather service.

8. Databases:

- Databases store and manage data for websites and applications.
- They allow for efficient storage, retrieval, and updating of data.
- Examples of databases used in web technology include **MySQL**, **MongoDB**, and **PostgreSQL**.

1.03 Types of Web Technologies

1. Frontend Technologies

Frontend technologies are used to build the user-facing part of a website or web application. They include:

- **HTML, CSS, and JavaScript:** The trio of core languages for designing and developing user interfaces.
- **Frontend Frameworks:** Tools like **React**, **Vue.js**, and **Angular** help developers quickly build complex, interactive user interfaces and manage state.

2. Backend Technologies

Backend technologies are used to build and manage the server-side of web applications. These include:

- **Server-side Languages:** Python, Ruby, PHP, and **Node.js** are popular backend programming languages used to handle business logic, manage databases, and process user requests.
- **Database Management Systems:** **MySQL**, **MongoDB**, and **PostgreSQL** are commonly used to store and retrieve data on the server-side.
- **Web Servers:** **Apache**, **Nginx**, and cloud services like **AWS** and **Google Cloud** host and manage websites.

3. Full-Stack Technologies

Full-stack development involves both frontend and backend development. Frameworks like **Django** (Python), **Ruby on Rails** (Ruby), and **Node.js** (JavaScript) allow developers to handle both client-side and server-side development.

4. Web APIs

Web APIs enable communication between different software components. Some popular API types include:

- **RESTful APIs:** Representational State Transfer (REST) APIs are widely used for client-server communication.
- **GraphQL:** A modern alternative to REST APIs, allowing clients to request only the data they need.
- **WebSockets:** WebSockets allow for real-time, two-way communication between the client and server.

1.04 Web Designing & Development

1.04A Web Design

It involves the color scheme, layout, information flow, and all aspects of UI/UX (user interface and user experience).

Web design covers everything related to a website's visual aesthetics and usability

Adobe Creative Cloud. Software like Photoshop and Illustrator help designers create design elements for web pages, such as graphics and logos.

Graphic design. Graphic design involves crafting visuals and layouts to make the website visually appealing and enhance the overall web experience.

UI/UX design. UI/UX design focuses on creating intuitive user interfaces and seamless user experiences by prioritizing ease of use and navigation.

Figma. This browser-based design tool helps designers do collaborative design work and prototyping, as well as create high-fidelity mock-ups.

1.04B Web Development

Front-end developers

Front-end developers focus on what users interact with on the website. They use HTML, CSS, and JavaScript to bring web designs to life. Key tools and technologies include:

1.04.1.1 CSS preprocessors like LESS or Sass for efficient style management.

1.04.1.2 JavaScript frameworks such as AngularJS, React, Ember.js, and Vue.js have revolutionized how developers build dynamic and responsive web applications.

1.04.1.3 Libraries like jQuery.

1.04.1.4 Git and GitHub for version control.

They also use on-site search engine optimization (SEO) to make sure the website ranks high in search engine results.

Back-end developers

Back-end developers handle the data and server-side functions of web applications. Essential tools and skills include:

1.04.1.5 Programming languages like PHP, Python, Java, and C#.

1.04.1.6 Frameworks such as Ruby on Rails, Symfony, and .NET that streamline building robust server-side applications.

1.04.1.7 Database management systems, including MySQL, MongoDB, and PostgreSQL.

1.04.1.8 Security and authentication technologies such as OAuth and Passport for protecting data integrity and user privacy.

Front-end developer	Back-end developer
It is Part of website User Can see Code and Interact it.	It is brain of website that Code never visible to End User
Run on the client side browser	Run on the server side browser
HTML, CSS, JavaScript (and frameworks like Bootstrap, React).	PHP, Python, Java, Node.js and Databases like MySQL, MongoDB.
Designs the layout, look and feel of the website.	Handles the logic, database, and server operations.
Limited Security	Advanced Security

1.1 Client–Server Architecture & Text Editors

1.1.0 Definition & Concept

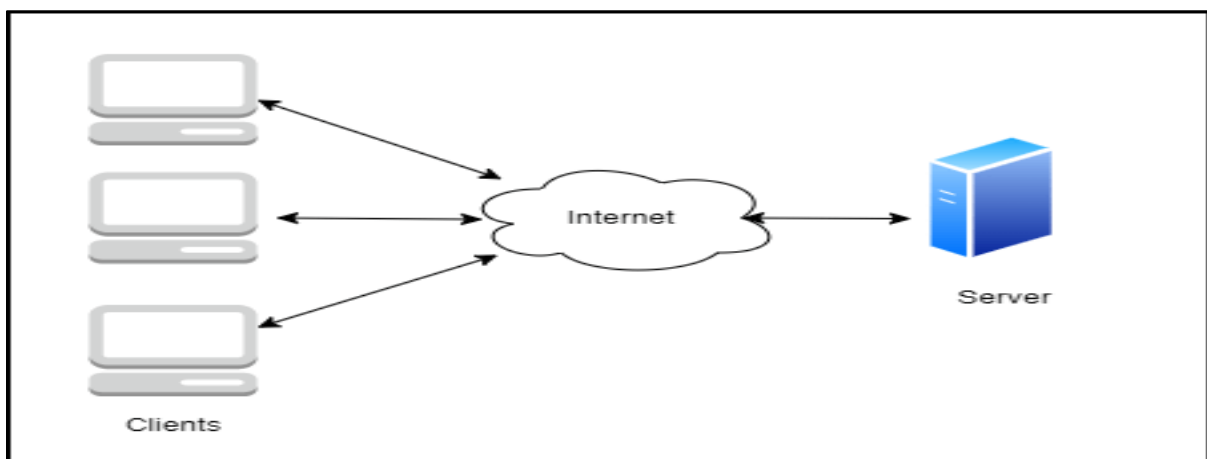
Client-Server Architecture is a model where tasks are divided between two entities:

(i) Client = Request

(ii) Server = Response

A computing model where the client requests services and the server processes and responds.

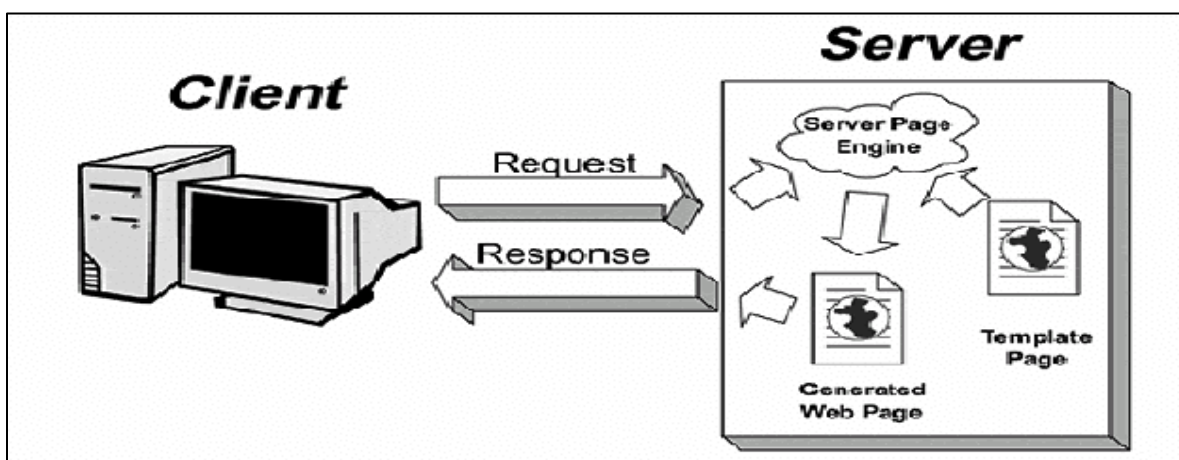
Client-Server Model is a distributed architecture where clients request services and servers provide them



a) **Client:** An application that requests services from the server, such as data retrieval, storage, calculations, or other functions.

b) **Server:** An application that processes client requests, sends responses, or performs specified actions. The server and client may reside on the same machine or different devices across the network. Effective Communication occurs via predefined protocols like HTTP, FTP, or SMTP.

How client requests and server responds



1.1.1 Types of Client-Server Architecture

1. 2-tier Architecture

Client directly communicates with Server.

Two tier architecture primarily has two parts, a client tier and a server tier. The client tier sends a request to the server tier and the server tier responds with the desired information.

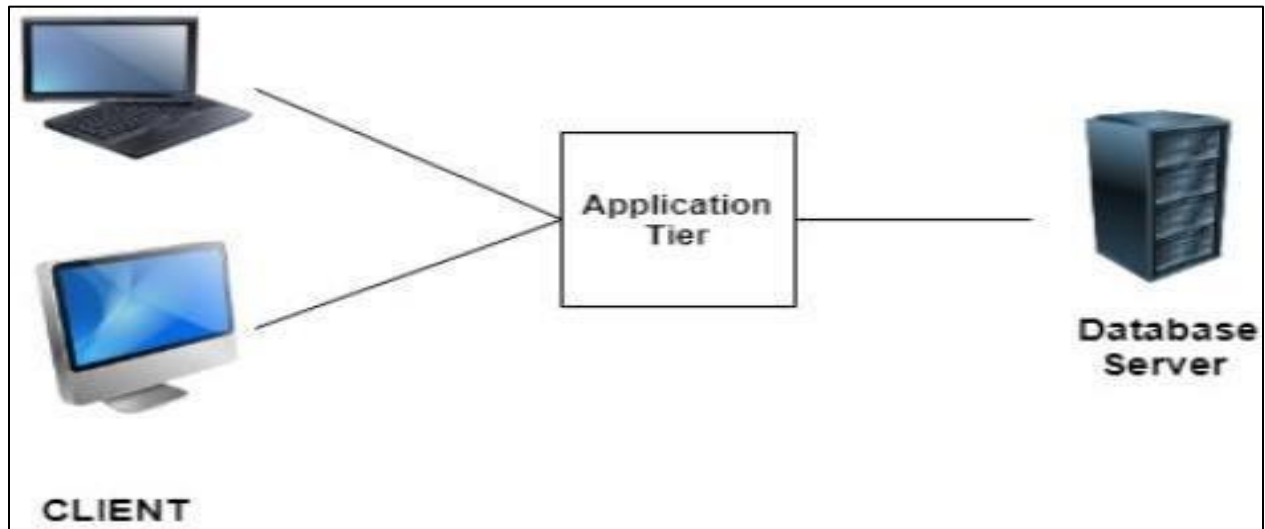


2. 3-tierArchitecture

Three tier architecture has three layers namely client, application and data layer.

Client → Application Server → Database.

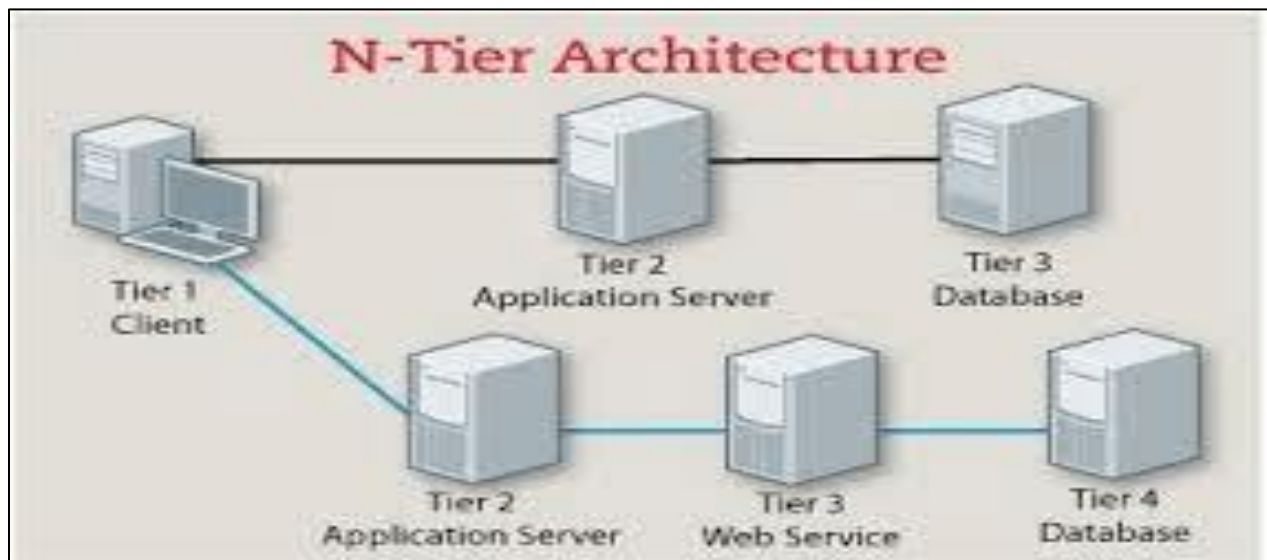
client layer is the one that requests the information. In this case it could be the GUI, web interface etc. The application layer acts as an interface between the client and data layer. It helps in communication and also provides security. The data layer is the one that actually contains the required data.



3. N-tier Architecture

N-tier architecture is also called Distributed Architecture or Multi-tier Architecture. Multiple layers of servers for scalability.

It is similar to three-tier architecture but the number of the application server is increased and represented in individual tiers



1.1.2 Introduction to Text Editors

A Text Editor is software used to write and edit code.

Text editor is a program that lets the user edit text. Whenever you're able to type some text in the input, you are editing the text, and that input is somehow a text editor. By this definition, we can call the HTML input element a basic text editor.

1.12A Basic text editors

The simplest form of text editors is the basic one. In a basic text editor, you can create and edit plain text without any styling or formatting.

- TextEdit (Mac)
- Notepad (Windows)
- Nano (Linux)
- HTML textarea (Web)

1.12B Integrated Development Environments

IDEs are some sort of special text editors that are designed for the programming and software development purposes. Syntax highlighting, intelligence and code completion, version controlling and debugging are the benefits of integrated development environments.

- Visual Studio
- Eclipse
- Jet Brains Products like PyCharm and WebStorm

1.12C Difference between Text Editor & IDE

Text Editor: Lightweight, mainly for writing code (Notepad, Sublime, Atom).

- **IDE:** Advanced, includes debugging, compilation, version control (VS Code, IntelliJ).

Feature	Text Editor	IDE (Integrated Development Environment)
Definition	A simple tool to write and edit plain text or code.	A complete software suite that provides tools for writing, testing, and debugging code.
Main Purpose	Only for writing and editing text/code.	For software development (coding, compiling, debugging, project management).
Complexity	Lightweight and simple.	Heavy, feature-rich, and advanced.
Examples	Notepad, Notepad++, Sublime Text, VS Code (when used just for editing).	Eclipse, PyCharm, IntelliJ IDEA, Visual Studio, Android Studio.
Features	Syntax highlighting, basic editing.	Code editor + Debugger + Compiler/Interpreter + Build automation + Version control integration.
Usage	Best for small coding tasks or quick edits.	Best for large projects and full application development.
Resource Requirement	Low system resources (RAM, CPU).	Requires more system resources.

1.2 Installation & Setup of VS Code Editor Installation & Set up of VS Code Editor, Introduction of Git & Git Hub, Installation & Configuration of Git Account

1.2.0 Installation & Setup

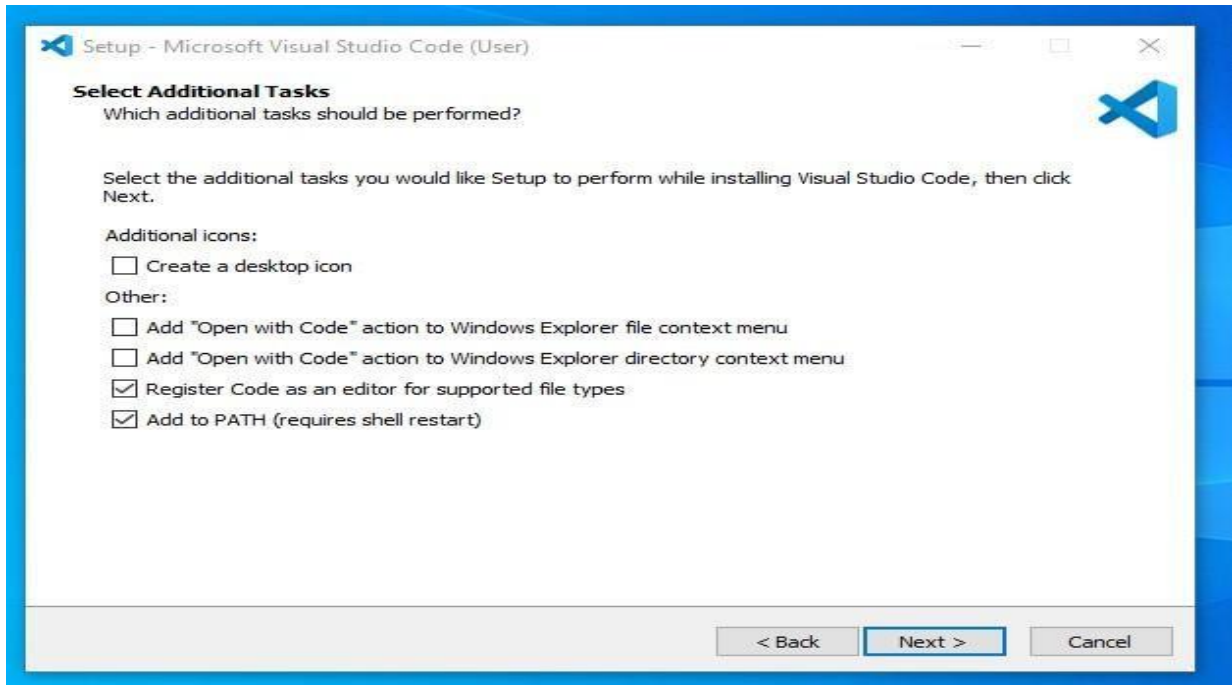
Step 1: Downloading VS Code (from official site)

Visit <https://code.visualstudio.com/> and download the installer.

Press the "**Download for Windows**" button on the website to start the download of the Visual Studio Code Application.

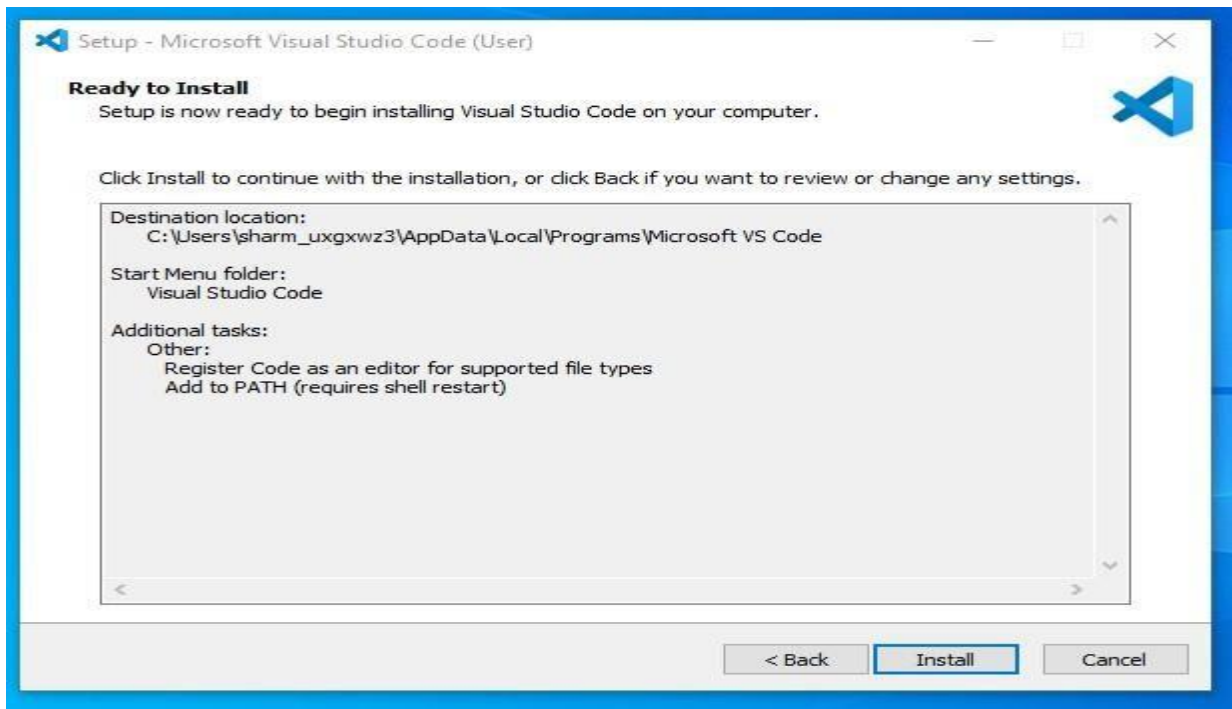
Step 3: Choose the location data for running the Visual Studio Code

Choose the location data for running the Visual Studio Code. It will then ask you to browse the location. Then click on the **Next** button.



Step 4: installation setup.

Then it will ask to begin the installation setup. Click on the **Install** button.



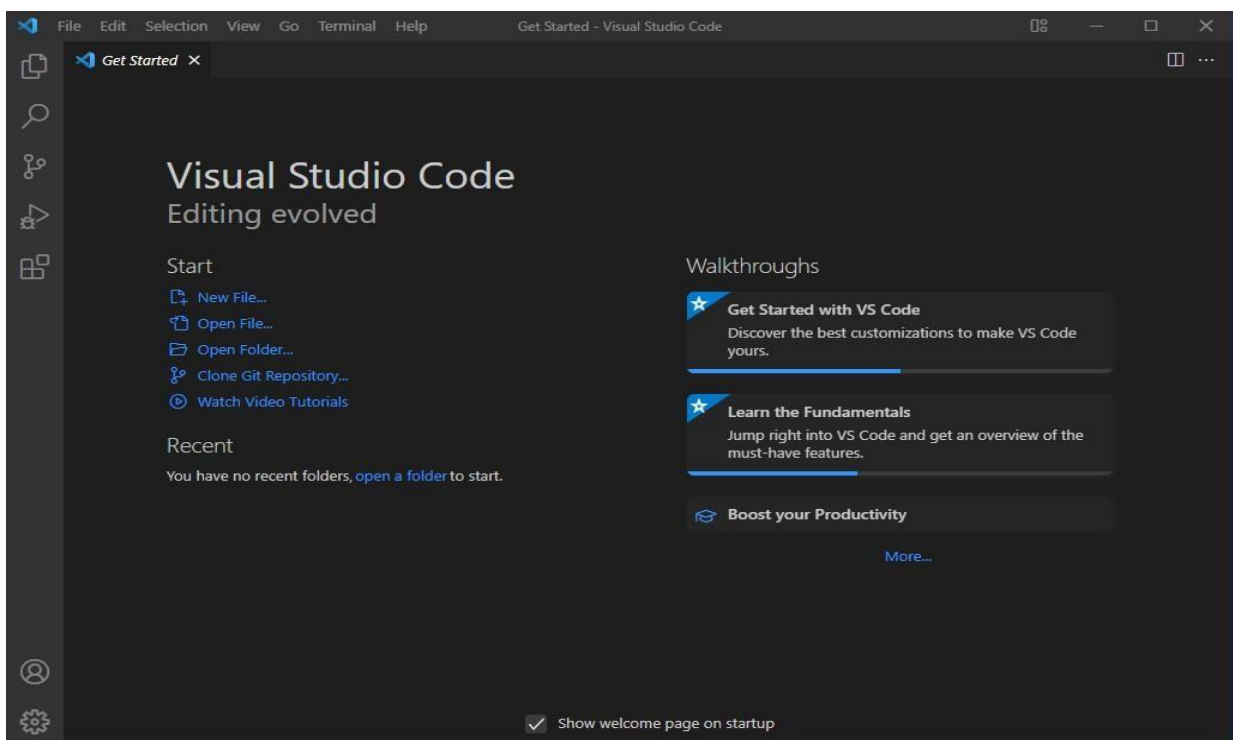
Step 5: Launch Visual Studio Code

After the Installation setup for Visual Studio Code is finished, it will show a window like this below. Tick the "**Launch Visual Studio Code**" checkbox and then click **Next**.



Step 6: Start Your Coding in Visual Studio

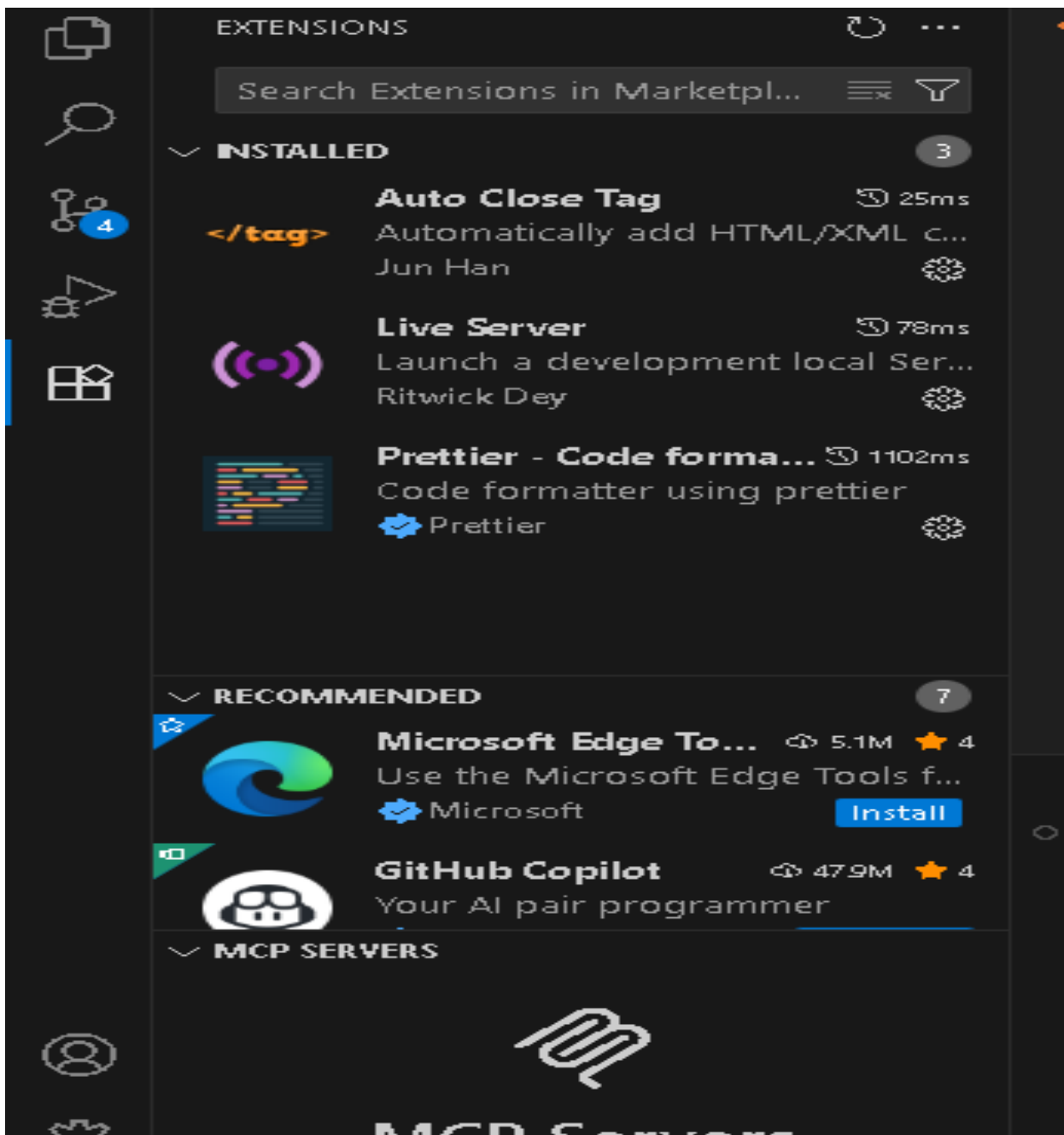
After the previous step, the Visual Studio Code window opens successfully. Now you can create a new file in the Visual Studio Code window and choose a language of yours to begin your programming journey!



1.2.1 How to Add Extensions in VS Code

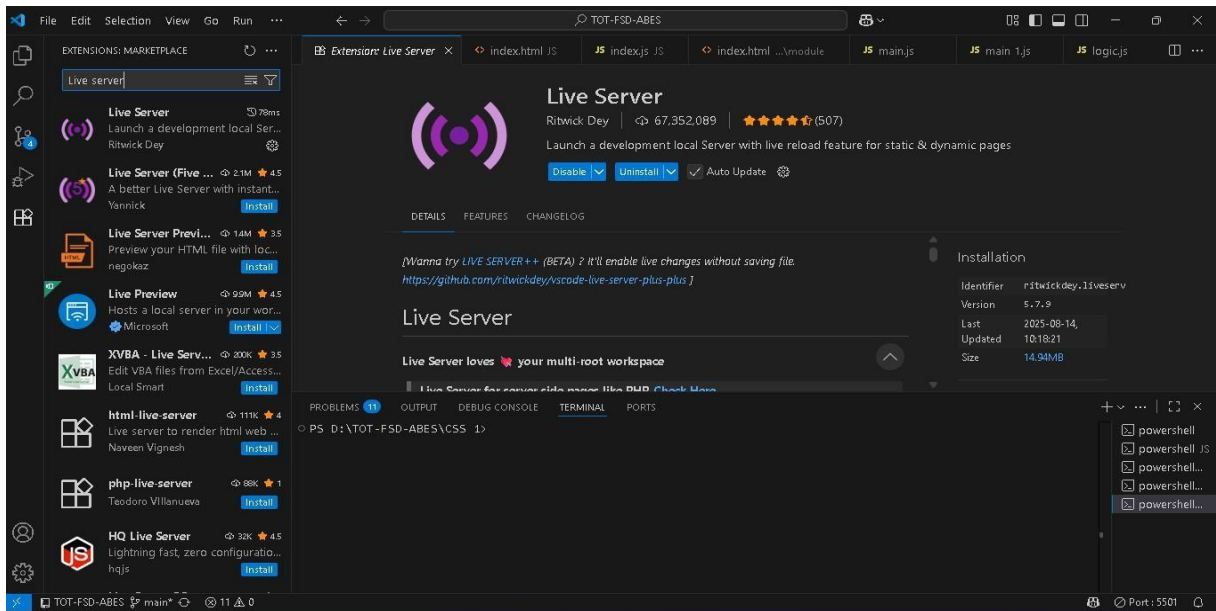
Step 1: Open Extensions Panel

- Open Visual Studio Code.
- On the left sidebar, click the Extensions icon (looks like 4 small squares).
- Shortcut: Ctrl + Shift + X



Step 2: Search for an Extension

- At the top of the Extensions panel, type the name of the extension you want.
- Example: Python, C/C++, Prettier - Code Formatter, Live Server



Step 3: Install the Extension

- From the search results, click the Install button next to the extension.
- It will download and install automatically.

Step 4: Reload if Needed

- Some extensions require a reload of VS Code.
- If prompted, click Reload.

Step 5: Manage Extensions

- Go to the Installed tab in the Extensions panel.
- From here you can:
 - Enable / Disable
 - Update
 - Uninstall

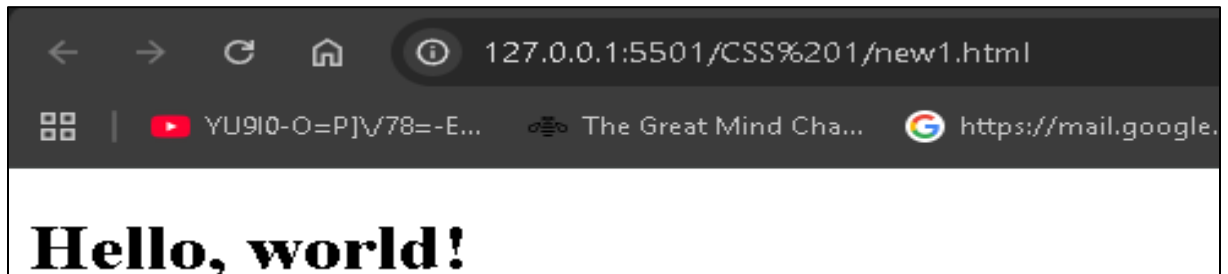
First sample HTML Program in VS Code:

- Open **Visual Studio Code**.
- Create a **new folder** (for example: MyFirstHTML).
- Inside the folder, create a new file and save it as:
index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Web Page</title>
</head>
<body>
  <h1>Hello, world!</h1>
</body>
</html>
```

Output

Output: Displays 'Hello, Web Technology!' in browser.



Second HTML Program in VS Code

- Open **Visual Studio Code**.
- Create a **new folder** (for example: MySecondHTML).
- Inside the folder, create a new file and save it as:
New.html

```
<!DOCTYPE html>
<html>
<head>
  <title>College Info</title>
</head>
<body>
  <h1> ABES Engineering College </h1>
  <h2>Ghaziabad </h2>
  <h3>ASH- Department</h3>
</body>
</html>
```

Output:



ASSIGNMENT -1

1. Define Web Technologies. Explain different types of Web Technologies with suitable examples.
2. Difference between Front-End Developer and Back-End Developer.
3. Explain Client-Server Architecture with diagram
4. Create a simple HTML webpage in VS Code with the following details:
Instructions:
 1. Create a new folder named Web-Assignment on your computer.
 2. Inside the folder, create a file named index.html.
 3. The webpage must include the following:
 - A title in the browser tab: *"My first HTML Page"*.
 - A main heading (H1) containing your Name, Father's Name, and Mother's Name.
 - A subheading (H2) mentioning your Branch and Section.
 - A paragraph (P) that contains your address in 3–4 lines.
 4. Save the file and run it in a web browser to check the output.

Solution:

1. Define Web Technologies. Explain different types of Web Technologies with suitable examples.

Web Technology is the technology used to make and run websites on the Internet.

Web Technology refers to the collection of tools, languages, and protocols that allow computers and devices to communicate through the World Wide Web (WWW).”

These technologies allow users to browse websites, interact with online content, and share information over the internet.

Types of Web Technologies

Frontend Technologies

Frontend technologies are used to build the user-facing part of a website or web application. They include:

- **HTML, CSS, and JavaScript:** The trio of core languages for designing and developing user interfaces.
- **Frontend Frameworks:** Tools like **React**, **Vue.js**, and **Angular** help developers quickly build complex, interactive user interfaces and manage state.

Backend Technologies

Backend technologies are used to build and manage the server-side of web applications. These include:

- **Server-side Languages:** Python, Ruby, PHP, and **Node.js** are popular backend programming languages used to handle business logic, manage databases, and process user requests.
- **Database Management Systems:** **MySQL**, **MongoDB**, and **PostgreSQL** are commonly used to store and retrieve data on the server-side.
- **Web Servers:** **Apache**, **Nginx**, and cloud services like **AWS** and **Google Cloud** host and manage websites.

Full-Stack Technologies

Full-stack development involves both frontend and backend development. Frameworks like **Django** (Python), **Ruby on Rails** (Ruby), and **Node.js** (JavaScript) allow developers to handle both client-side and server-side development.

Web APIs

Web APIs enable communication between different software components. Some popular API types include:

- **RESTful APIs:** Representational State Transfer (REST) APIs are widely used for client-server communication.
- **GraphQL:** A modern alternative to REST APIs, allowing clients to request only the data they need.
- **WebSockets:** WebSockets allow for real-time, two-way communication between the client and server.

2.Difference between Front-End Developer and Back-End Developer.

Front-end developer	Back-end developer
It is Part of website User Can see Code and Interact it.	It is brain of website that Code never visible to End User
Run on the client side browser	Run on the server side browser
HTML, CSS, JavaScript (and frameworks like Bootstrap, React).	PHP, Python, Java, Node.js and Databases like MySQL, MongoDB.
Designs the layout, look and feel of the website.	Handles the logic, database, and server operations.
Limited Security	Advanced Security

3.Explain Client-Server Architecture with diagram

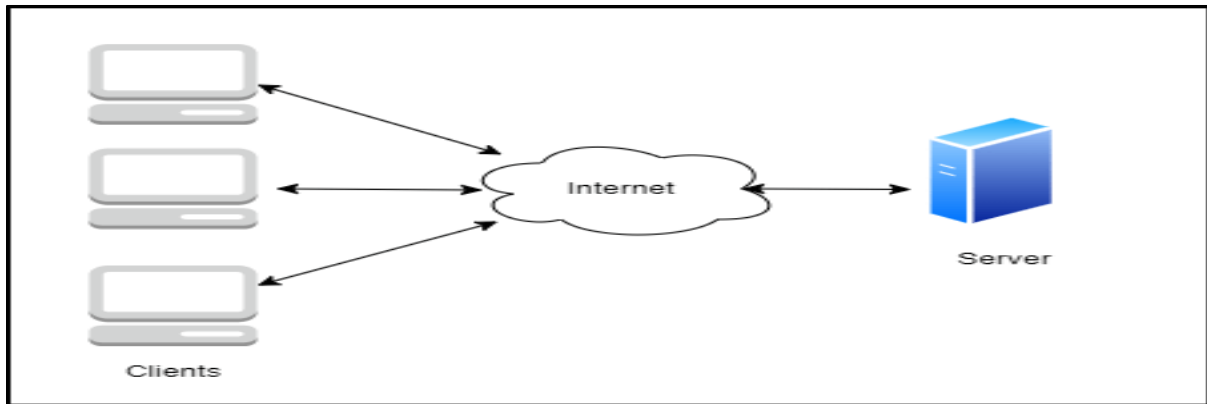
Client-Server Architecture is a model where tasks are divided between two entities:

(i)Client = Request

(ii)Server = Response

A computing model where the client requests services and the server processes and responds.

Client-Server Model is a distributed architecture where clients request services and servers provide them



Client: An application that requests services from the server, such as data retrieval, storage, calculations, or other functions.

Server: An application that processes client requests, sends responses, or performs specified actions. The server and client may reside on the same machine or different devices across the network. Effective Communication occurs via predefined protocols like HTTP, FTP, or SMTP.

4. Create a simple HTML webpage in VS Code with the following details

Instructions:

- a. Create a new folder named **Web_Assignment** on your computer.
- b. Inside the folder, create a file named **index.html**.
- c. The webpage must include the following:
 - i. A **title** in the browser tab: *“My first HTML Page”*.
 - ii. A **main heading (H1)** containing your **Name, Father’s Name, and Mother’s Name**.
 - iii. A **subheading (H2)** mentioning your **Branch and Section**
 - iv. A **paragraph (P)** that contains your **address** in 3–4 lines.
- d. Save the file and **run it in a web browser** to check the output

```

<!DOCTYPE html>
<html lang="en">
<head>
<title>My first HTML Page</title>
</head>
<h1>
Name: ABES Engineering College<br >
Founded: 1 July 2000<br >
Address: Ghaziabad, Uttar Pradesh 201009
</h1>
<h2>
Branches: B.Tech , M.Tech ,
MCA , Business School
</h2>
<p>
Vision :
To take ABES Engineering College to such a level that, it is at par with the leading institutions of the
world in providing leadership to the international education system and be amongst the top rated
institutions of the world by providing a transformative education to create leaders and innovators
embedded in traditional Indian values.
</p>
<p>
Mission :
<ul>
<li>To create an ambiance for healthy teaching-learning process.</li>
<li>To nurture the students and infuse in them
<ul>
<li>A passion to excel professionally.</li>
<li>A spirit to be of utmost use to the industry, corporate sector and the society at large.</li>
<li>An intense desire to take challenging responsibilities and leadership roles.</li>
<li>A craving to be wholesome good human beings.</li>
</ul>
</li>
<li>To develop an environment for creating new knowledge through research and by thriving to explore
innovative ideas.</li>
</ul>
</p>

```

Output:

Name: ABES Engineering College

Founded: 1 July 2000

Address: Ghaziabad, Uttar Pradesh 201009

Branches: B.Tech , M.Tech , MCA , Business School

Vision To take ABES Engineering College to such a level that, it is at par with the leading institutions of the world in providing leadership to the international education system and be amongst the top rated institutions of the world by providing a transformative education to create leaders and innovators embedded in traditional Indian values.

Mission

- To create an ambience for healthy teaching-learning process.
- To nurture the students and infuse in them
 - A passion to excel professionally.
 - A spirit to be of utmost use to the industry, corporate sector and the society at large.
 - An intense desire to take challenging responsibilities and leadership roles.
 - A craving to be wholesome good human beings.
- To develop an environment for creating new knowledge through research and by thriving to explore innovative ideas.

1.2.2 Introduction of Git & GitHub

What is a Version Control System (VCS)?

Imagine you are writing your **Project Report**.

- Day 1: You save it as report.docx.
- Day 2: You add more content and save as report-final.docx.
- Day 3: You make changes and save as report-final2.docx.

Soon, your folder looks like this:

report.docx
report-final.docx
report-final2.docx
report-final2-new.docx

Confusing, right? Which one is the latest? Which one has the correct data?
What if you want to **go back** to Day 1's version?

Version Control System (like Git) solves this problem!

- Git keeps track of every change (like a time machine).
- You can go back to *any previous version* easily.
- You don't need to create multiple final-final.docx files anymore!

Definition:

A **Version Control System (VCS)** is a tool that **records changes to files over time** so that you can recall specific versions later.

◆ What is Git?

Think of **Git as a CCTV Camera for your code**.

- It watches every change (who did it, when, what was changed).
- It stores snapshots (commits) of your project.
- You can rewind and see the history.
- Multiple developers can work together and Git will track "who changed what".

◆ In technical words:

Git is a **distributed version control system**. It runs on your **local machine** and manages code versions, branches, and merging.

- **Git** = Version Control System (VCS).
- Tracks project changes → allows rollback.
- Enables collaboration among developers.

◆ What is GitHub?

If **Git** is your **CCTV camera at home**, then **GitHub** is your **cloud storage (like Google Drive)** where you upload those **CCTV recordings**.

- GitHub is an **online platform** where you can store your Git repositories.
- A **cloud-based platform** hosting Git repositories.
- It helps you **share code with teammates**, manage collaboration, and contribute to open-source.
- Provides features like:
 - **Pull requests** (suggesting changes)
 - **Issues** (tracking bugs/tasks)
 - **Forks** (copy of someone else's repo)
 - **Branches** (parallel development)
- Think of Git as **your notebook** and GitHub as **Google Drive for your notebook**.
- **Git vs GitHub – Relation & Difference**

Git	GitHub
Tool to track changes (VCS).	Platform to host Git repositories.
Works on your local system .	Works online (cloud) .
Can be used alone (without GitHub).	Cannot work without Git.
Example: Track project locally.	Example: Share project with your team worldwide.

- Git = **Notebook** (where you write your code & track changes).
- GitHub = **Library** (where you store your notebook so that others can read, borrow, or contribute).

Why Use Both Git & GitHub Together?

- **Git alone** → You can track changes locally, but **can't collaborate easily**.
- **GitHub alone** → It's just a website, can't track versions without Git.
- **Together** → You get **version control + cloud hosting + collaboration tools**.

Example:

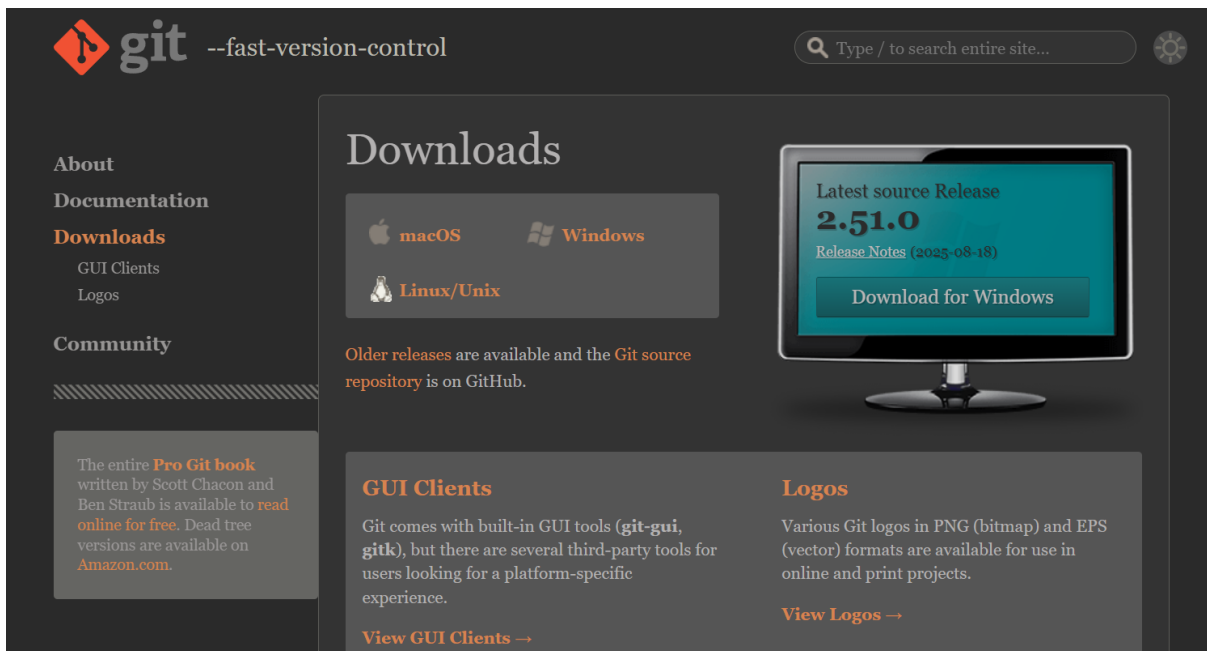
Imagine you and your friends are building a **mini web project**:

- On your laptop, Git tracks your changes (like saving checkpoints).
- You push it to GitHub → your friends can see and pull updates.
- They add new features → push again → you pull back.
- GitHub acts as a **shared code hub**, Git acts as the **history manager**.

1.2.3 Installation & Configuration of Git Account

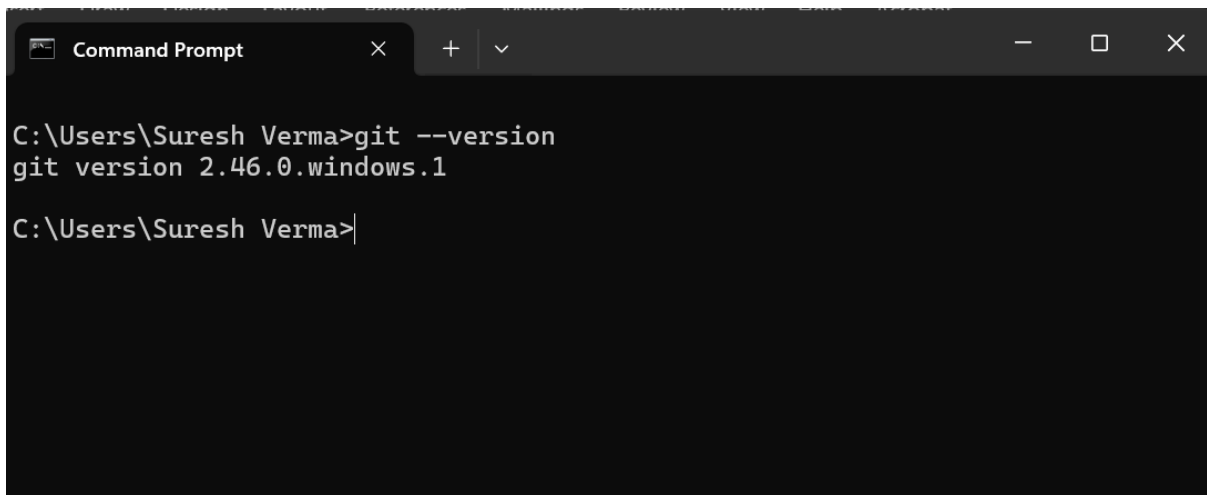
Steps:

1. Download: <https://git-scm.com/downloads>



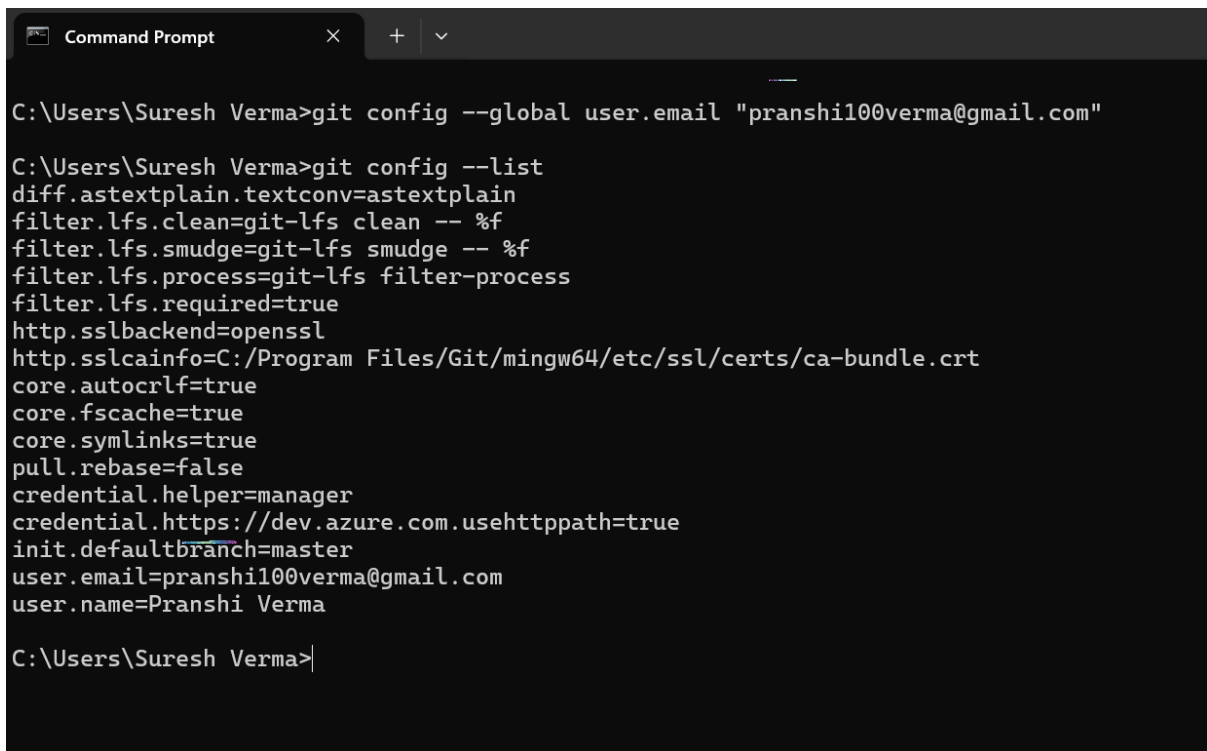
2. Install with default settings.
3. Open terminal (CMD/VS Code terminal) → check version:

git --version



Configure Git (first-time only):

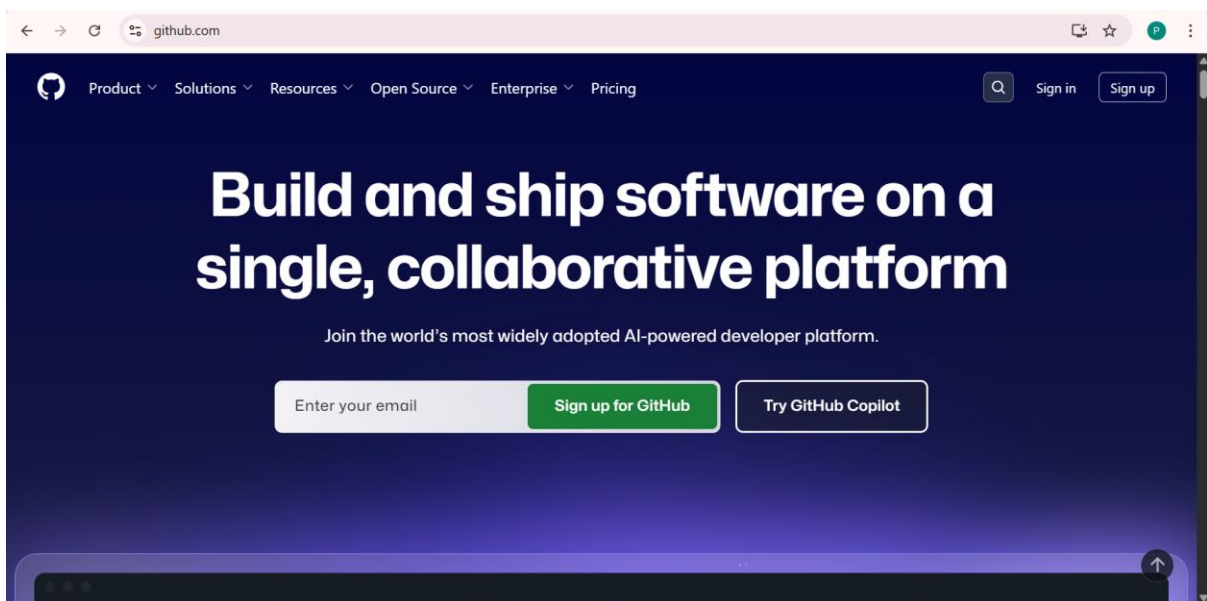
```
git config --global user.name "Your Name"  
git config --global user.email "your_email@example.com"  
git config --list # check configuration
```



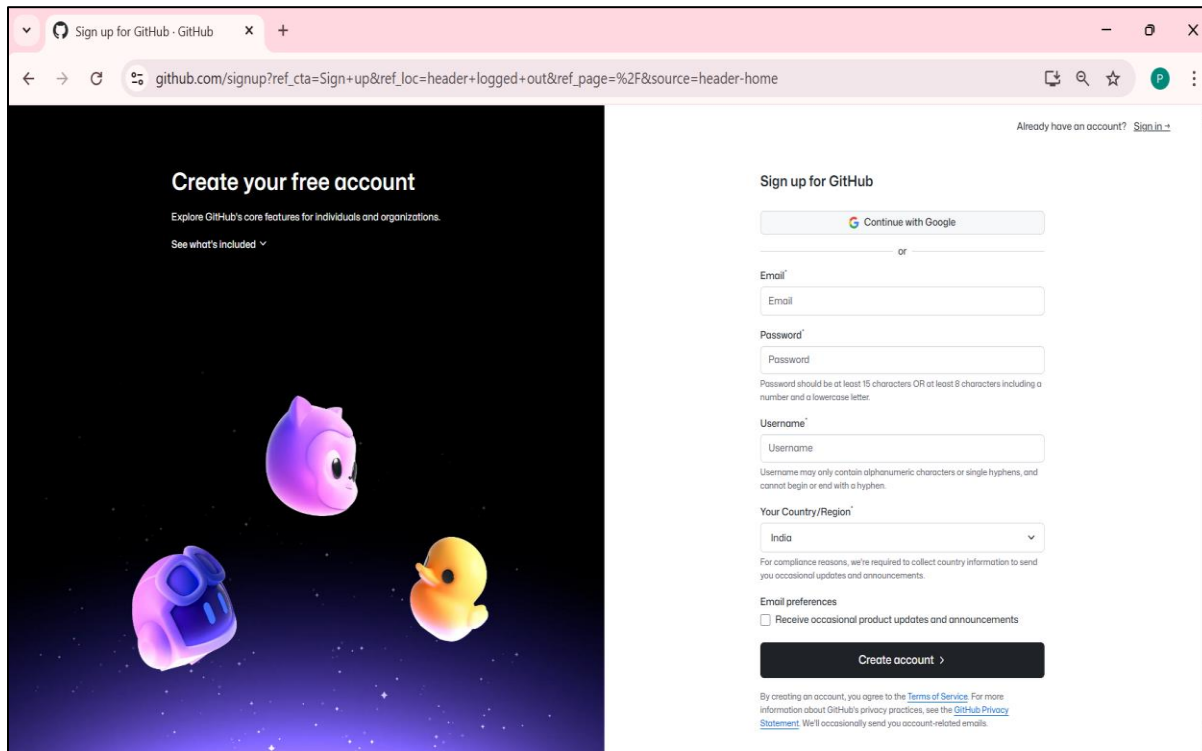
```
Command Prompt  
C:\Users\Suresh Verma>git config --global user.email "pranshi100verma@gmail.com"  
  
C:\Users\Suresh Verma>git config --list  
diff.astextplain.textconv=astextplain  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
http.sslbackend=openssl  
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt  
core.autocrlf=true  
core.fscache=true  
core.symlinks=true  
pull.rebase=false  
credential.helper=manager  
credential.https://dev.azure.com.usehttppath=true  
init.defaultbranch=master  
user.email=pranshi100verma@gmail.com  
user.name=Pranshi Verma  
  
C:\Users\Suresh Verma>
```

4. Creating a GitHub Account

1. Go to <https://github.com/>



2. Sign up with email → verify → choose username.



Sign up for GitHub · GitHub

github.com/signup?ref_cta=Sign+up&ref_loc=header+logged+out&ref_page=%2F&source=header-home

Create your free account

Explore GitHub's core features for individuals and organizations.
[See what's included](#)

Already have an account? [Sign in](#)

Sign up for GitHub

[Continue with Google](#)

or

Email*

Password*

Username*

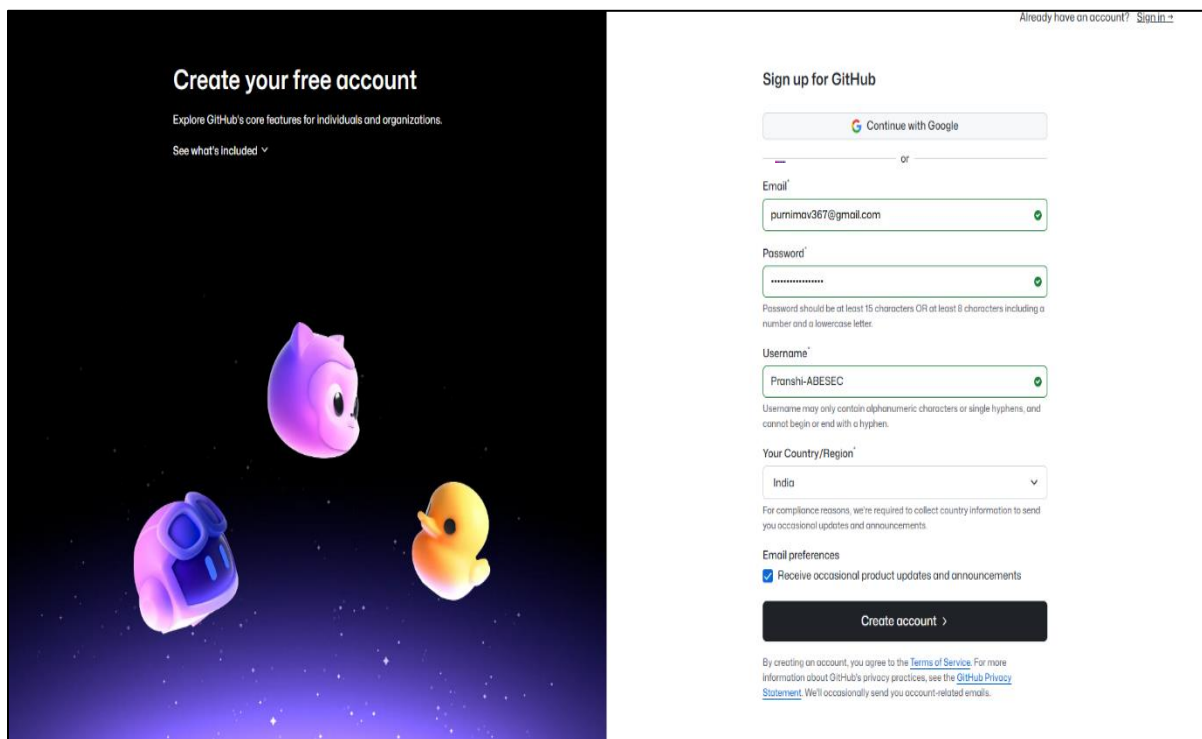
Your Country/Region*

Email preferences

☐ Receive occasional product updates and announcements

[Create account](#)

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.



Sign up for GitHub · GitHub

github.com/signup?ref_cta=Sign+up&ref_loc=header+logged+out&ref_page=%2F&source=header-home

Create your free account

Explore GitHub's core features for individuals and organizations.
[See what's included](#)

Already have an account? [Sign in](#)

Sign up for GitHub

[Continue with Google](#)

or

Email*

purnimov367@gmail.com

Password*

Username*

Purnishi-ABESEC

Your Country/Region*

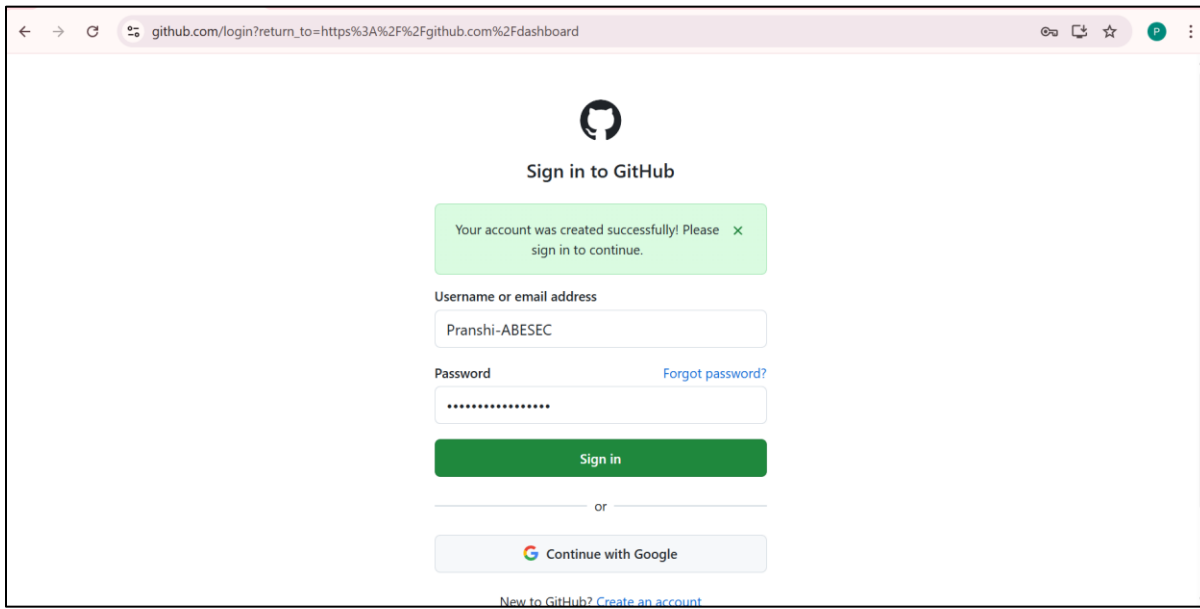
India

Email preferences

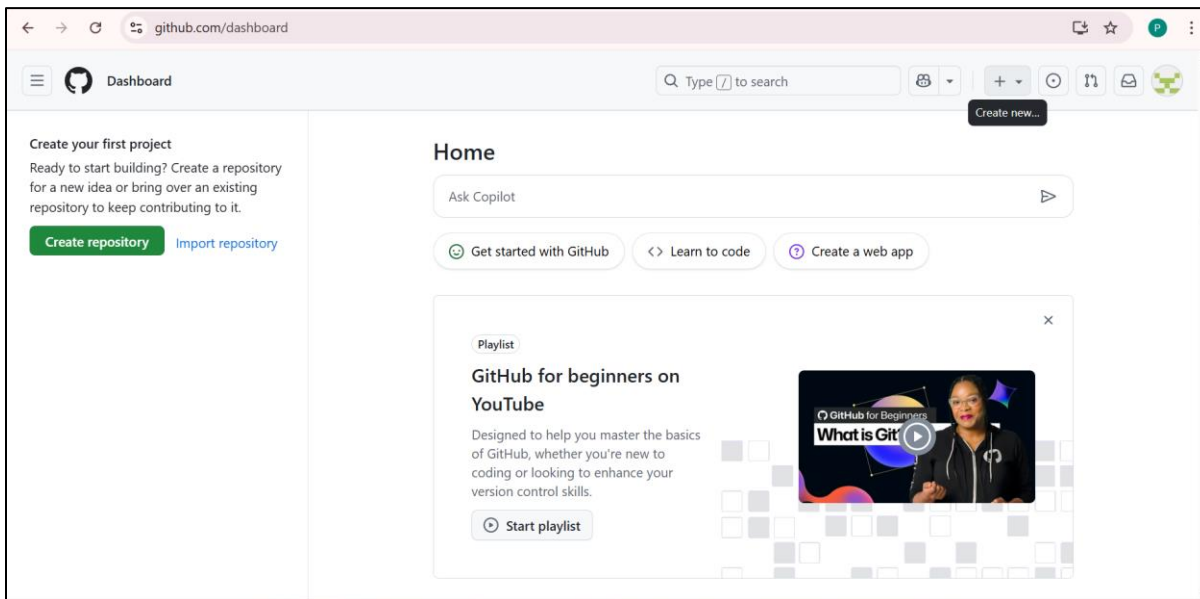
☒ Receive occasional product updates and announcements

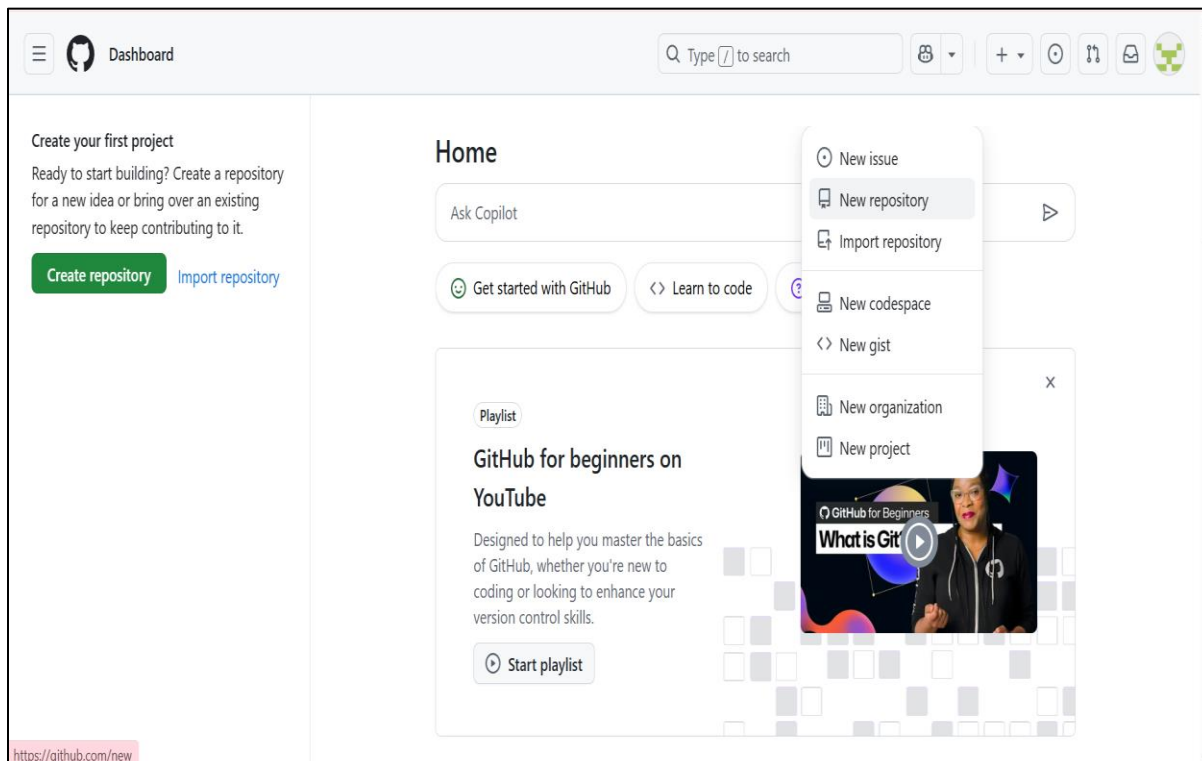
[Create account](#)

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.

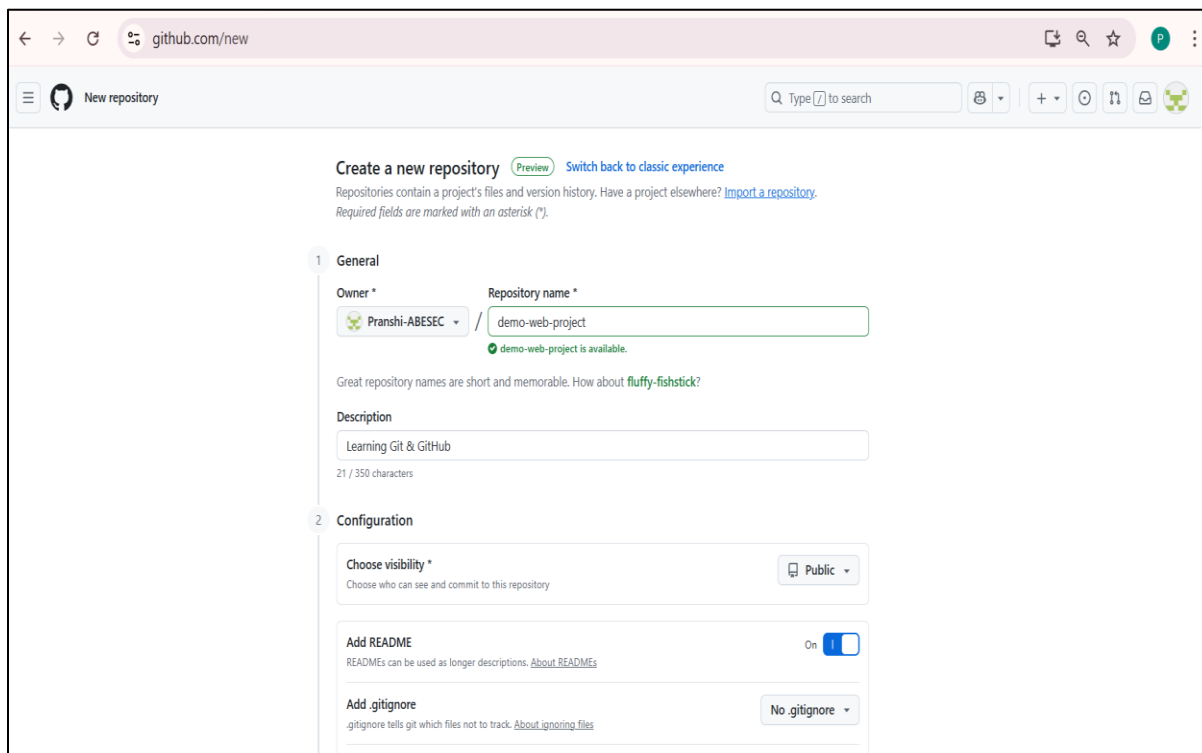


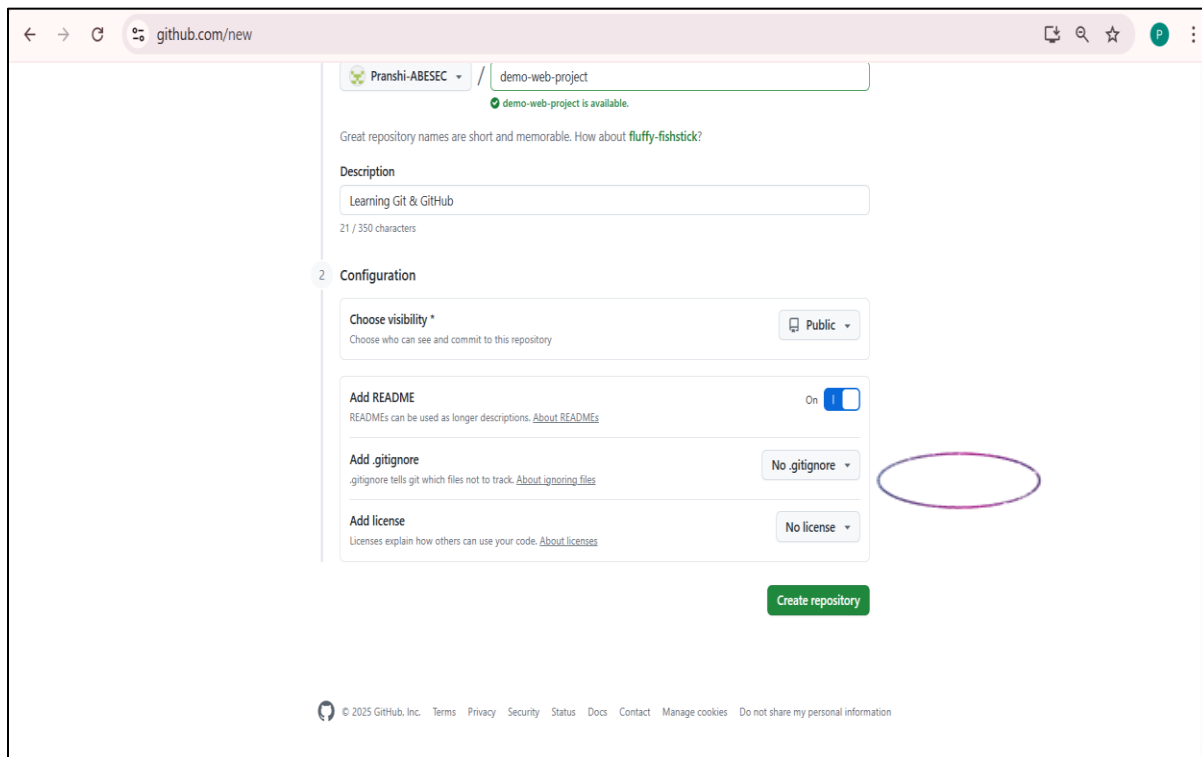
3. Create a new repository:





- Repository Name: demo-web-project
- Description: "Learning Git & GitHub"
- Public → Add README.md → Create Repository.





Creating a Local Repository

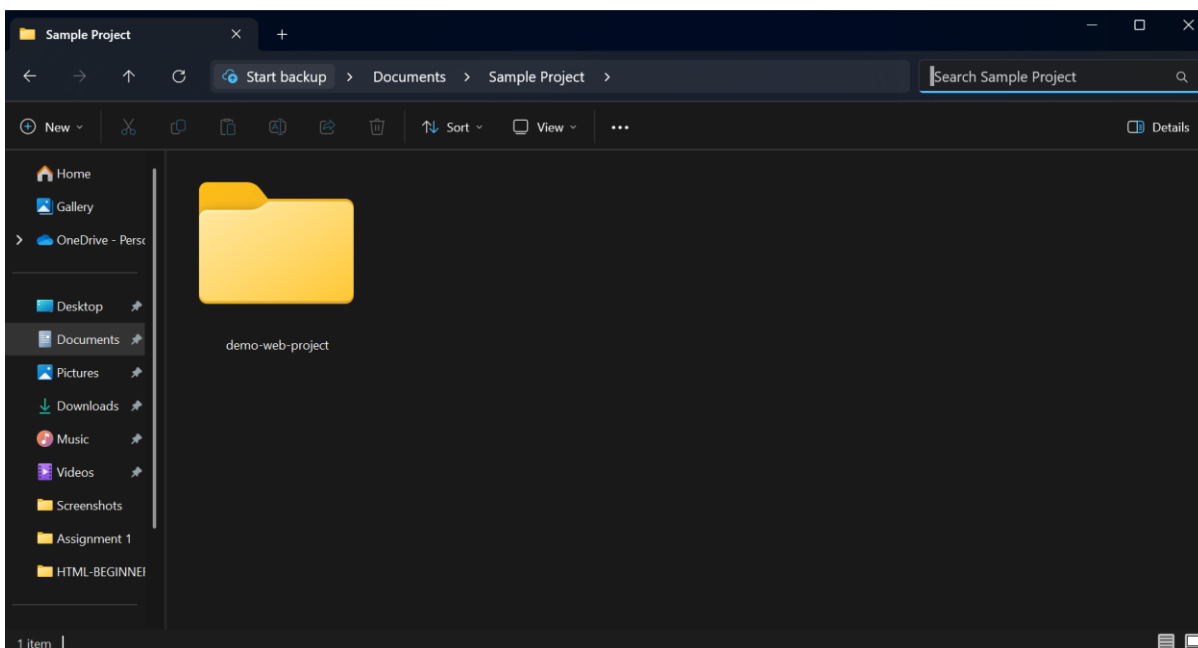
A **local repository** is a project folder on your computer that Git will track. This is the **first step** before pushing anything to GitHub.

You can create it in two ways:

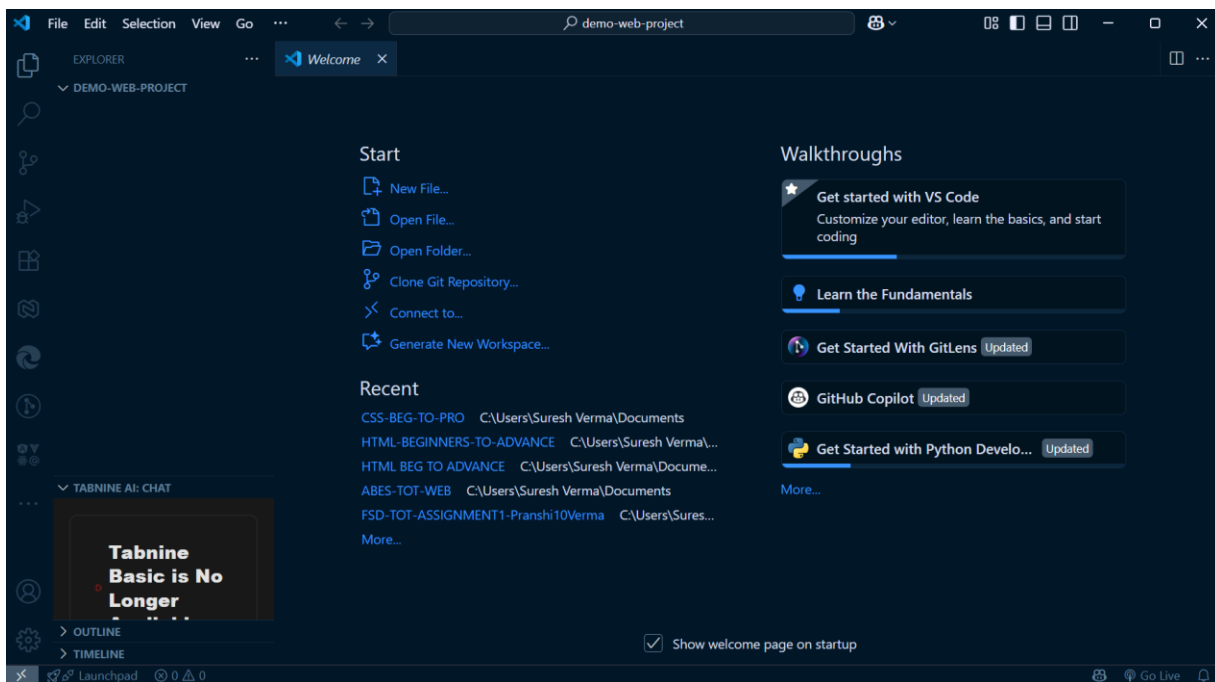
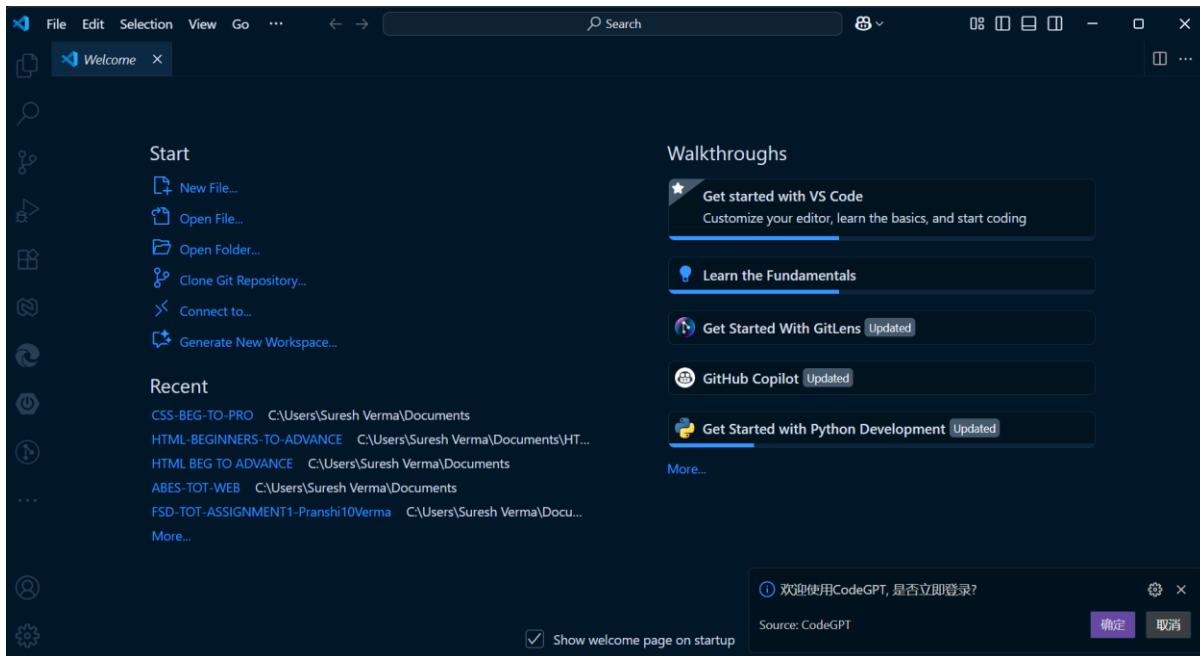
Method A – Using VS Code (Beginner-Friendly UI)

1. Create a project folder

- Example: demo-web-project



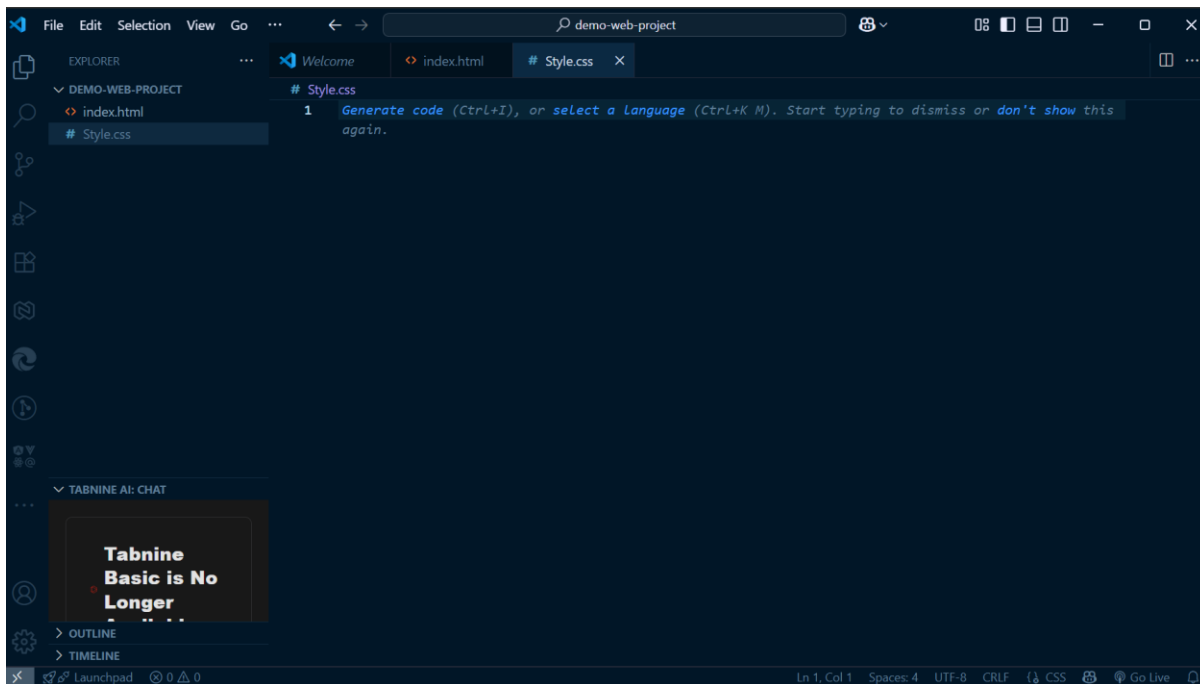
- Open it in VS Code (**File** → **Open Folder**).



2. Create project files

Example:

- index.html
- style.css



index.html

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8" />

<title>My First GitHub Project</title>

</head>

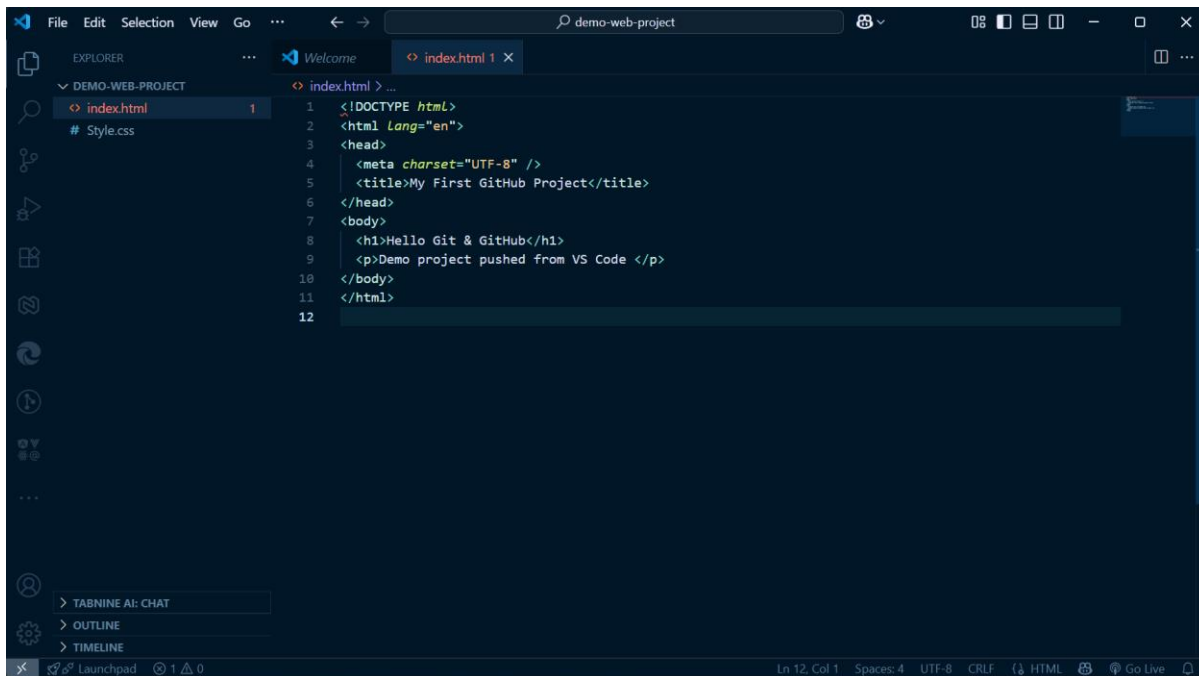
<body>

<h1>Hello Git & GitHub</h1>

<p>Demo project pushed from VS Code </p>

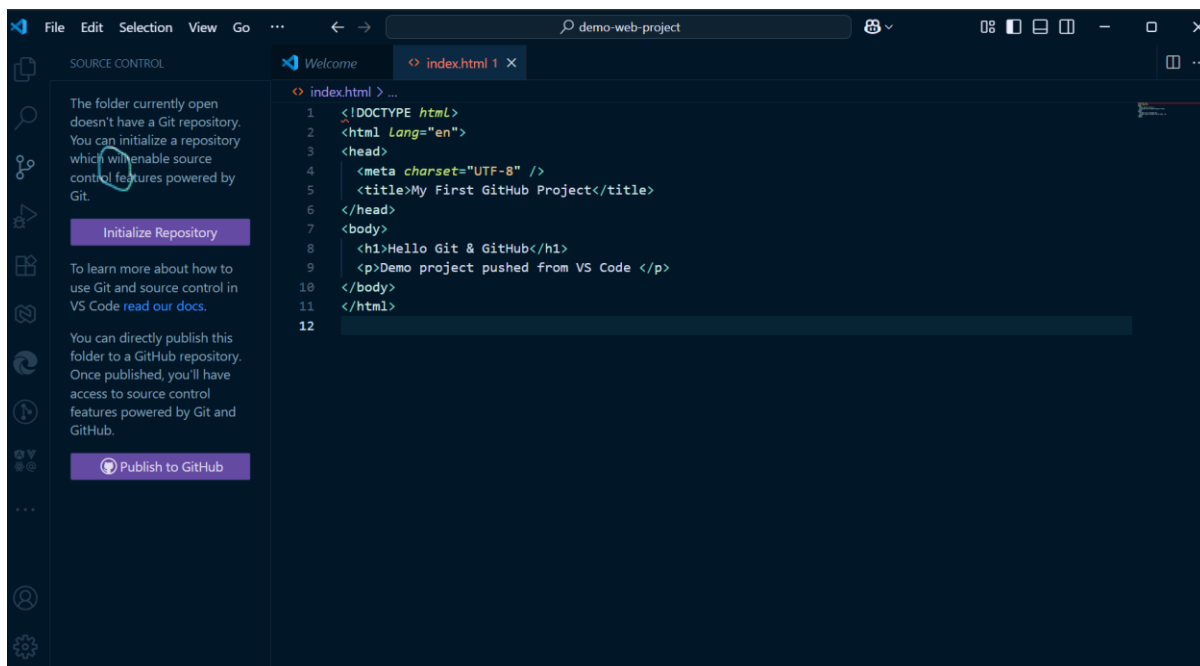
</body>

</html>



3. Initialize Git (create local repository)

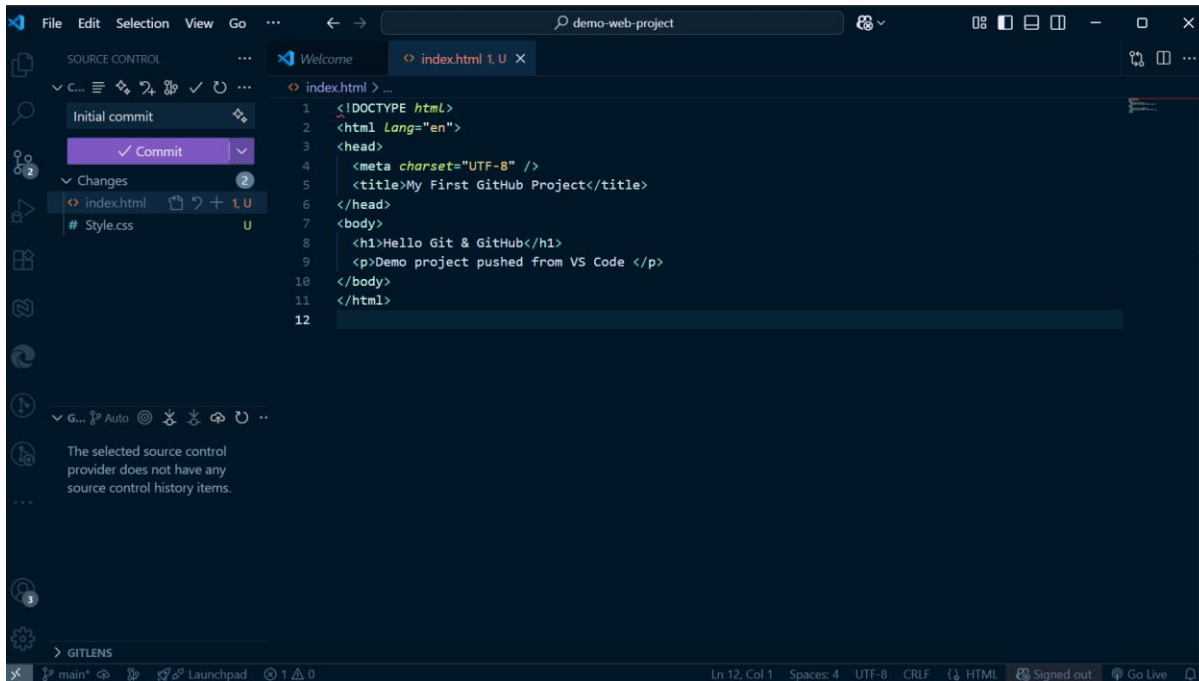
- Go to the **Source Control** panel in VS Code (branch icon on sidebar).



- Click **Initialize Repository**.
- This creates a **local Git repository** inside the folder.

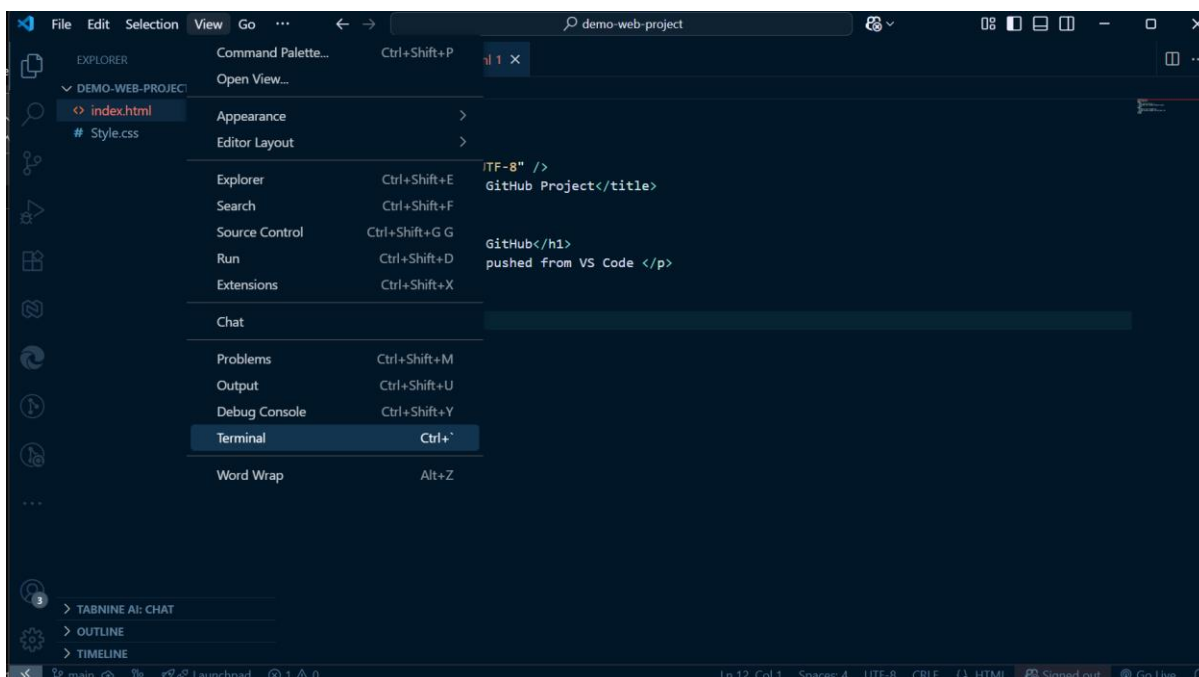
4. Make your first commit

- Stage changes (+ button).
- Enter a commit message: "Initial commit".
- Click **Commit**.

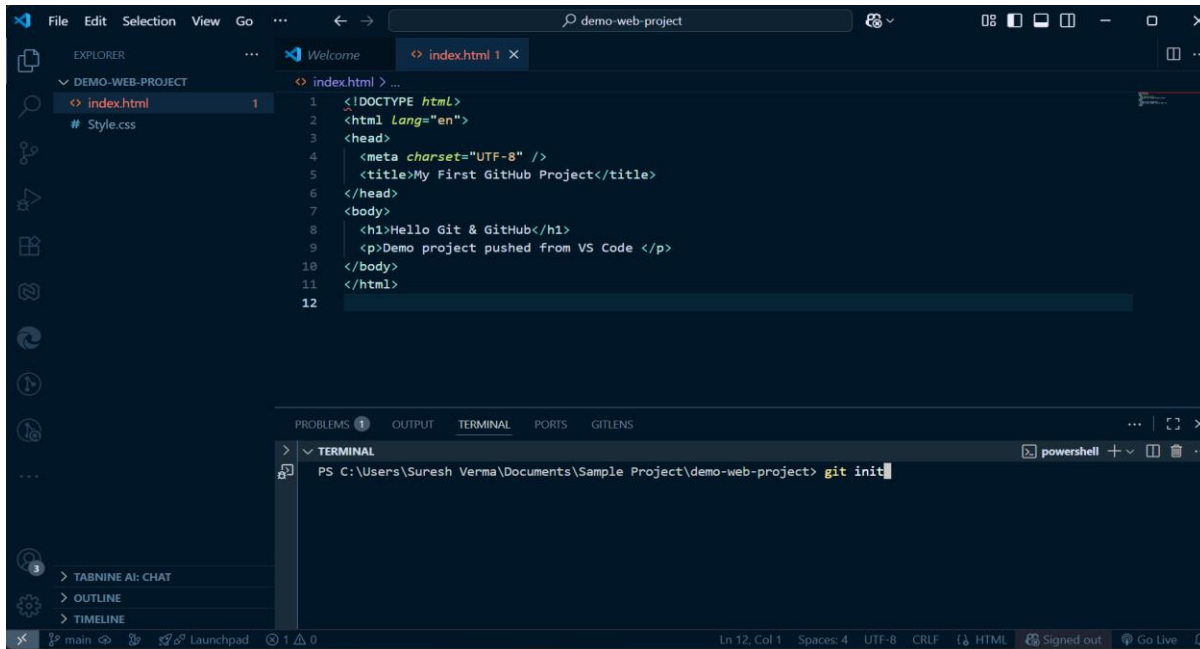


Method B – Using Terminal

1. Open **VS Code Terminal** (Ctrl + ~) or CMD.
2. Navigate to your project folder: `cd path/to/demo-web-project`



Initialize the repository: git init



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows a project named 'DEMO-WEB-PROJECT' with two files: 'index.html' and 'Style.css'. The 'index.html' file is open in the editor, showing a basic HTML structure with a title 'My First GitHub Project' and a body containing 'Hello Git & GitHub' and 'Demo project pushed from VS Code'. The Terminal panel at the bottom shows the command 'git init' being executed in a PowerShell window. The status bar at the bottom indicates the file is 'Signed out' and 'Go Live' is available.

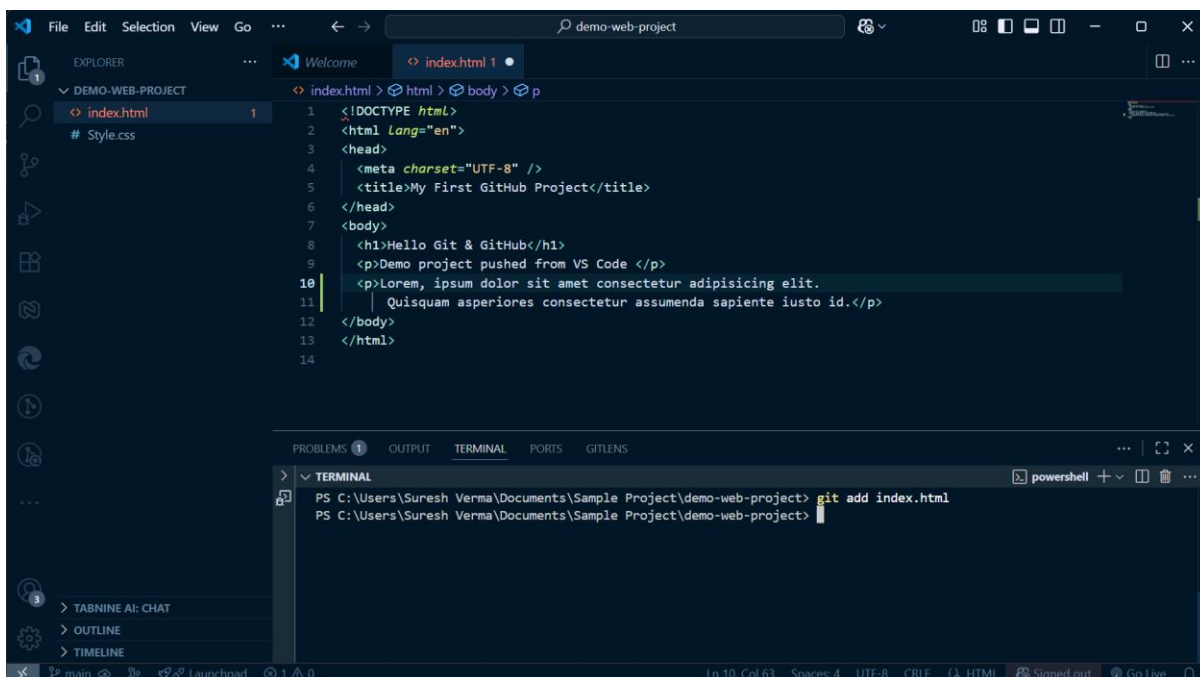
- This command creates a **hidden .git folder**, which means Git is now tracking your project.
- Add and commit files:

Add a Single File

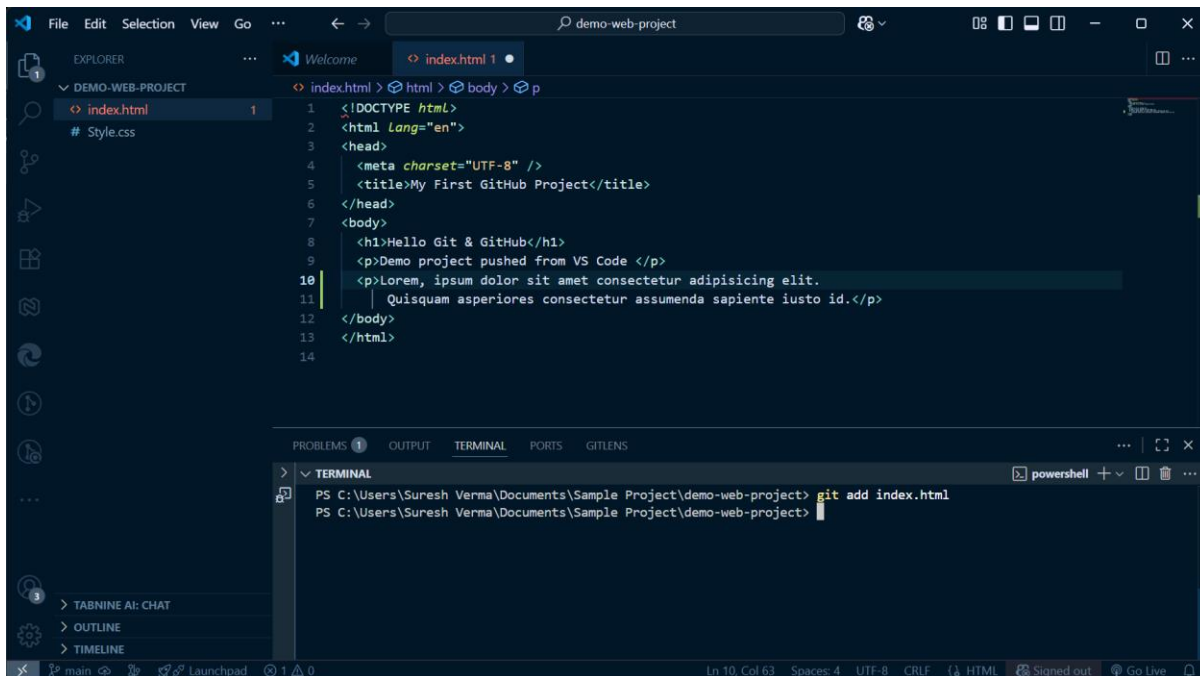
If you only want to add **one file** to the staging area (for example, index.html): *git add index.html*

Add Multiple Specific Files

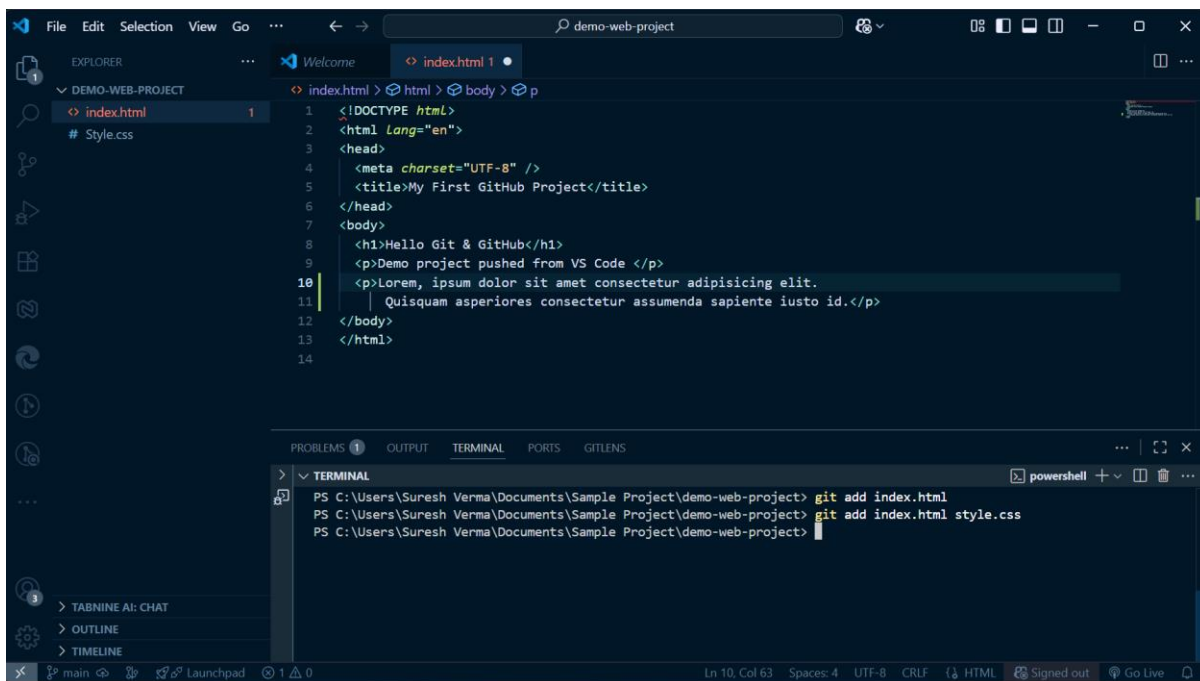
You can also add more than one file at a time by naming them: *Git add index.html*



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows the 'DEMO-WEB-PROJECT' with 'index.html' and 'Style.css'. The 'index.html' file is open in the editor, showing the same basic HTML structure as before, but with additional text added to the body: 'Lorem, ipsum dolor sit amet consectetur adipisicing elit. Quisquam asperiores consectetur assumenda sapiente iusto id.' The Terminal panel at the bottom shows the command 'git add index.html' being executed in a PowerShell window. The status bar at the bottom indicates the file is 'Signed out' and 'Go Live' is available.



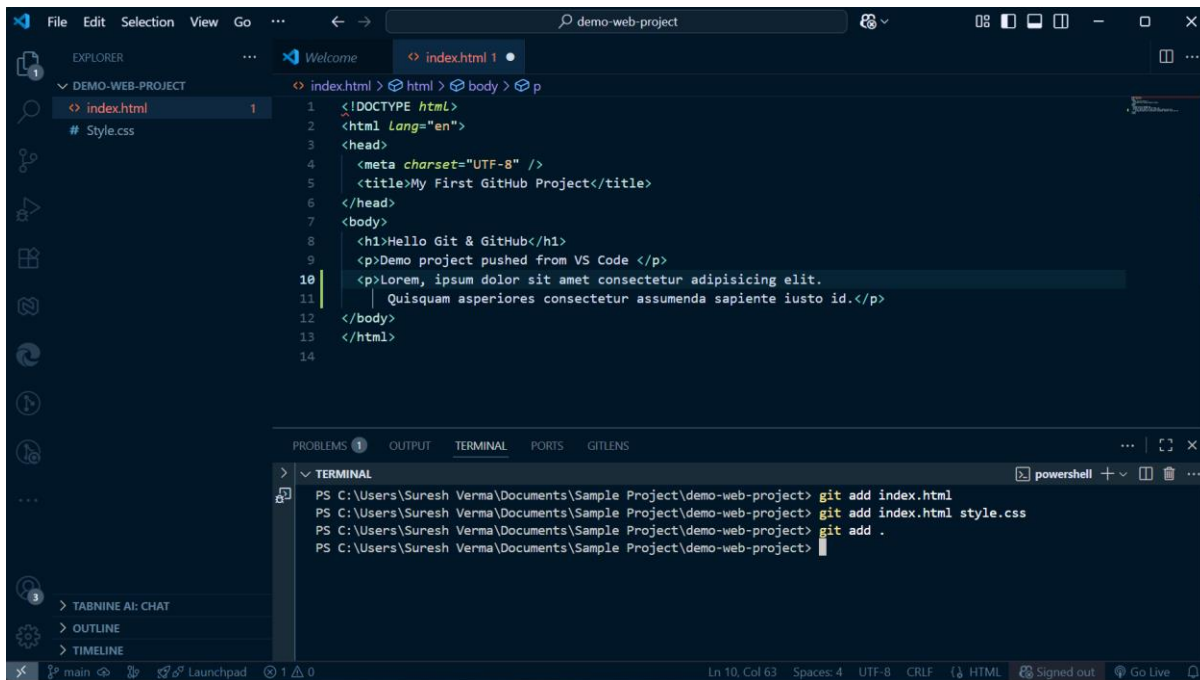
git add index.html style.css



Add All Files (Recommended when you want everything)

If you want to add **all files in the project folder**:

git add .



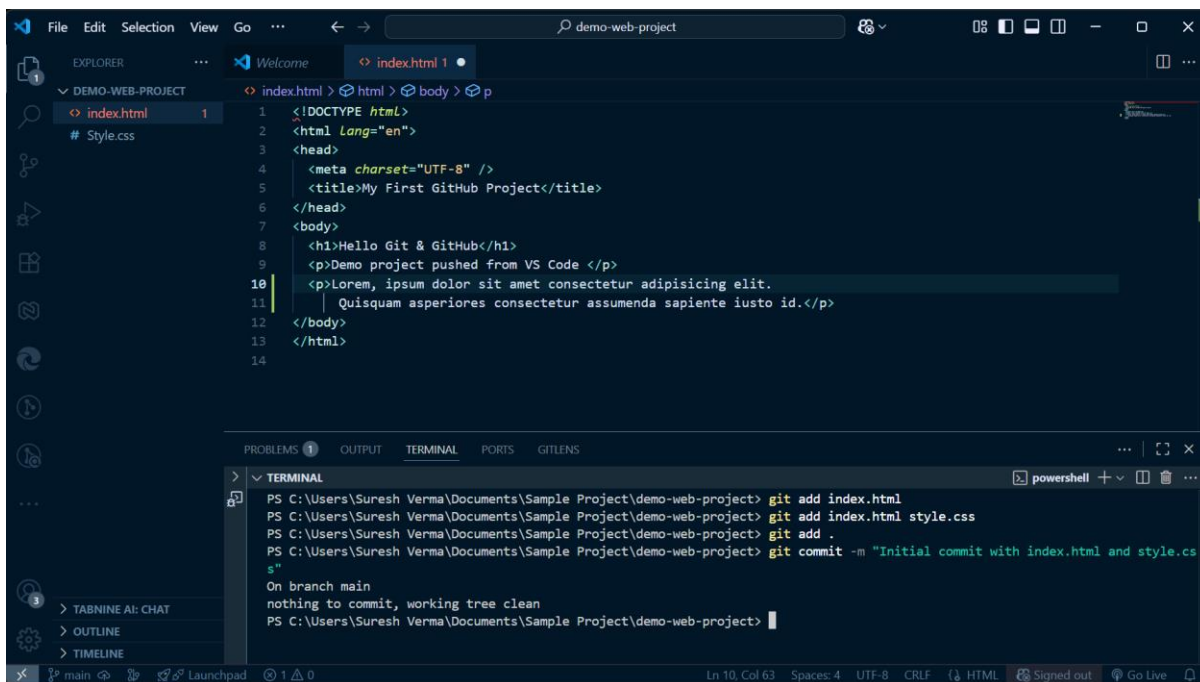
The screenshot shows the Visual Studio Code interface with a project named 'demo-web-project'. The Explorer pane on the left shows 'index.html' and 'style.css'. The editor displays the content of 'index.html', which is an HTML document with a title 'My First GitHub Project' and some placeholder text. The terminal at the bottom shows the following commands and output:

```
PS C:\Users\Suresh Verma\Documents\Sample Project\demo-web-project> git add index.html
PS C:\Users\Suresh Verma\Documents\Sample Project\demo-web-project> git add index.html style.css
PS C:\Users\Suresh Verma\Documents\Sample Project\demo-web-project> git add .
PS C:\Users\Suresh Verma\Documents\Sample Project\demo-web-project>
```

Commit the Changes

After adding files, you need to **commit** them with a message:

`git commit -m "Initial commit with index.html and style.css"`



The screenshot shows the Visual Studio Code interface with the same project. The terminal now shows the execution of the commit command and its output:

```
PS C:\Users\Suresh Verma\Documents\Sample Project\demo-web-project> git add index.html
PS C:\Users\Suresh Verma\Documents\Sample Project\demo-web-project> git add index.html style.css
PS C:\Users\Suresh Verma\Documents\Sample Project\demo-web-project> git add .
PS C:\Users\Suresh Verma\Documents\Sample Project\demo-web-project> git commit -m "Initial commit with index.html and style.css"
On branch main
nothing to commit, working tree clean
PS C:\Users\Suresh Verma\Documents\Sample Project\demo-web-project>
```

A **commit** is like saving a snapshot of your project at that point in time.

Now we will link our local repository with the GitHub repository (remote).

Now your project is on GitHub!

1.3 Introduction to HTML & HTML Features, HTML Page Structure, Elements in HTML (Semantic and Non Semantic)

1.3.0 What is HTML?

- HTML stands for Hyper Text Markup Language
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

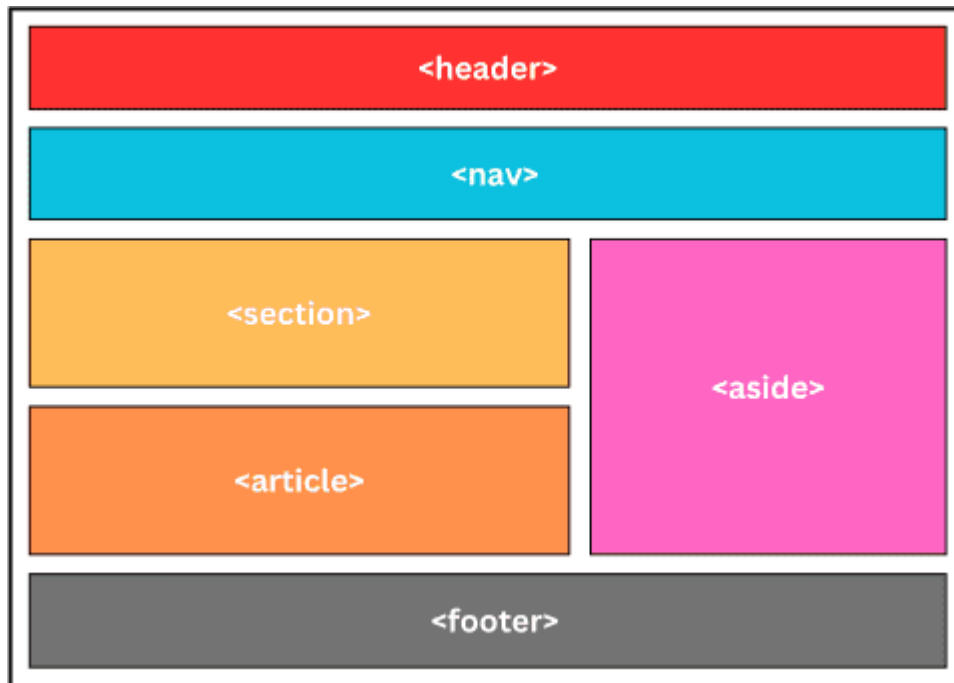
1.3.1 HTML 5 Features

HTML5 was introduced having some new features following are

- Error tolerance
- Save as html or .htm
- Pass data in “.....” or ‘.....’
- Name Casing <HTML> or <html>
- Semantic Elements
- Audio and Video Support
- Canvas Elements
- Geolocation API
- Local Storage
- Responsive Images
- Web Workers
- Drag and Drop API
- Form Enhancements
- Web Sockets
- Micro Data

1.3.2 HTML Page Structure

An HTML page is structured using a hierarchical arrangement of elements, providing a logical organization for the content. The fundamental structure includes:



`<!DOCTYPE html>` Declaration:

This declaration, placed at the very beginning of an HTML document, informs the browser that the document is an HTML5 document. It is not an HTML tag but a directive.

`<html>` Element:

This is the root element of an HTML page, encapsulating all other HTML elements. It signifies the beginning and end of the HTML document.

`<head>` Element:

Located within the `<html>` element, the `<head>` contains metadata about the HTML document. This information is not directly displayed on the web page but is crucial for browser rendering, search engine optimization, and other functionalities. Common elements within the `<head>` include:

`<title>`: Defines the title of the document, which appears in the browser tab or window title bar.

`<meta>`: Provides various metadata, such as character set (charset), viewport settings, and descriptions for search engines.

`<link>`: Links external resources like stylesheets (.css files) or favicons.

`<style>`: Embeds internal CSS styles directly within the HTML document.

`<script>`: Embeds or links external JavaScript code for interactive functionality.

<body> Element:

Also located within the <html> element, the <body> contains all the visible content of the web page. This is where the actual content that users interact with is placed. Examples of elements found within the <body> include:

Headings (<h1> to <h6>): Define titles and subtitles for sections of content.

Paragraphs (<p>): Enclose blocks of text.

Links (<a>): Create hyperlinks to other web pages or resources.

Images (): Embed images into the page.

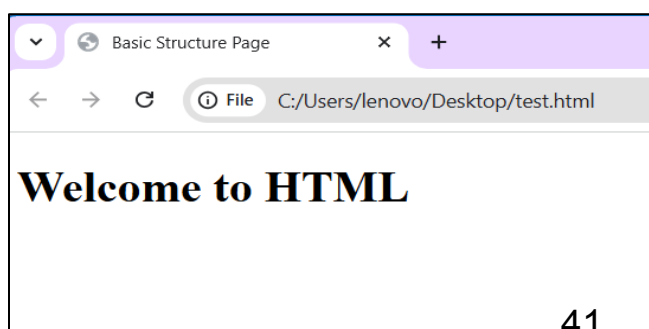
Lists (, ,): Create unordered or ordered lists of items.

Divisions (<div>): Generic container for grouping other elements for styling or scripting purposes. Provide more meaningful structure to the content, improving accessibility and SEO.

This nested structure, starting from the <!DOCTYPE html> and encompassing the <html>, <head>, and <body> elements, forms the fundamental framework of every HTML page.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Basic Structure Page</title>
</head>
<body>
  <h1>Welcome to HTML</h1>
</body>
</html>
```

Output:



1.3.3 Elements in HTML (Semantic and Non Semantic)

A tag is a keyword that defines how a web browser should format and display a web page. HTML tags are the basic building blocks of any website. In html, codes(normal text) enclosed in brackets written in usually paired.

See below example:

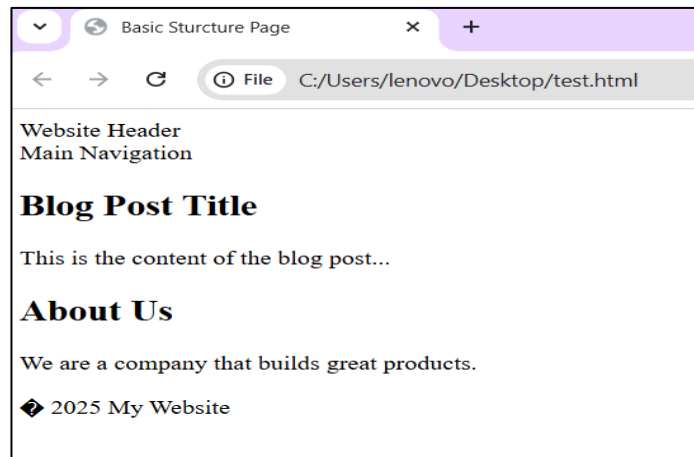
<TITLE>My Web Page</TITLE>

Note: - HTML is not case sensitive. <TITLE> = <title> =

<TitLE> In HTML,

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Basic Sturcture Page</title>
  </head>
  <body>
    <header>Website Header</header>
    <nav>Main Navigation</nav>
    <main>
      <article>
        <h2>Blog Post Title</h2>
        <p>This is the content of the blog post...</p>
      </article>
      <section>
        <h2>About Us</h2>
        <p>We are a company that builds great products.</p>
      </section>
    </main>
    <footer>© 2025 My Website</footer>
  </body>
```

Output:



Semantic tags define the meaning of the content they contain, while non-semantic tags only old content and do not indicate its role on the page.

Semantic tags are both human- and machine-readable, and they can help users and machines understand the context and importance of the content.

Non-semantic tags, on the other hand, are used for styling purposes or as containers for other elements.

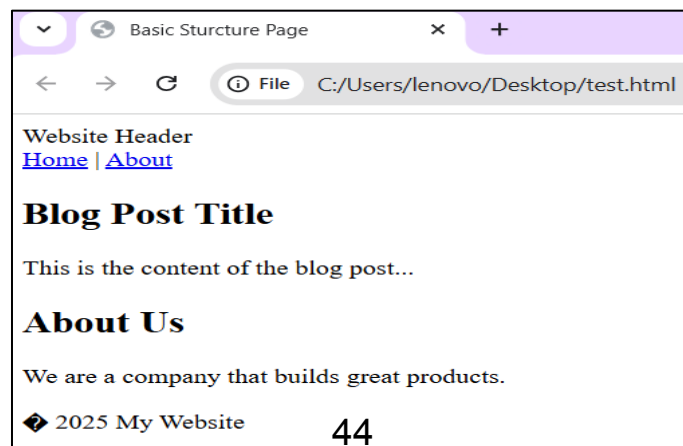
For Example: the `<div>` tag does not suggest the content it will carry, however using the `<p>` tag suggests it can be used to hold paragraph information.


```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Basic Sturcture Page</title>
  </head>
  <body>
    <div id="header">Website Header</div>
    <div id="menu"><a href="#">Home</a> | <a href="#">About</a></div>
    <div id="content">
      <div class="post">
        <h2>Blog Post Title</h2>
        <p>This is the content of the blog post...</p>
      </div>
      <div class="info">
        <h2>About Us</h2>
        <p>We are a company that builds great products.</p>
      </div>
    </div>
    <div id="footer">© 2025 My Website</div>
  </body>

```

Output:



There are many reasons why you should write Semantic tags instead of normal HTML tags, to leverage SEO, accessibility, and browser compatibility.

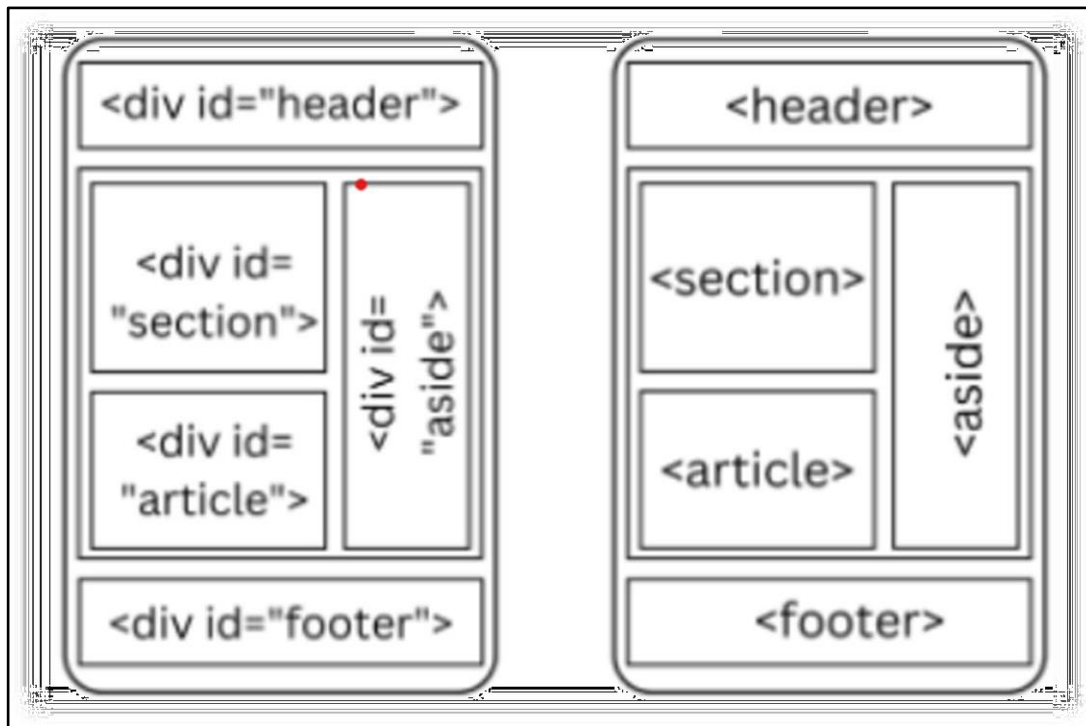


Fig: Non-Semantic and Semantic Tags

1.4 Types of Elements (Inline/Block), Text formatting- headings, paragraphs

1.4.0 Inline and Block Elements

HTML elements can be categorized in several ways, primarily based on their display behavior and their purpose.

1. Block-level Elements:

These elements typically start on a new line and take up the full available width of their parent container.

They are used to structure the main sections of a web page.

Examples include:

<address>,<article>,<aside>,<blockquote>,<canvas>,<dd>,<div>,<dl>,<dt>,<fieldset>,<figcaption>,<figure>,<footer>,<form>,<h1>...<h6>,<header>,<hr>,,<main>,<nav>,<noscript>,,<p>,<pre>,<section>,<table>,<tfoot>,,<video>.

2. Inline Elements:

These elements do not start on a new line and only take up as much width as their content requires. They are used to style or structure content within a block-level element.

Examples include:

<a>,<abbr>,<acronym>,,<bdo>,<big>,
,<button>,<cite>,<code>,<dfn>,,<i>,,<input>.

<kbd>,<label>,<map>,<object>,<output>,<q>,<samp>,<script>,<select>,<small>,,,<sub>,<sup>,<textarea>,<time>,<tt>,<var>.

```
<!DOCTYPE html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>Basic Sturcture Page</title>

  </head>

  <body>

    <!--block level elements-->

    <div style="border: 1px solid red">This is a div (block)</div>

    <p style="border: 1px solid green">This is a paragraph (block)</p>

    <h1 style="border: 1px solid blue">This is a heading (block)</h1>

    <section style="border: 1px solid
      orange"> A section is also block-level
    </section>

    <!--inline elements-->

    <span style="border: 1px solid red">This is span (inline)</span>

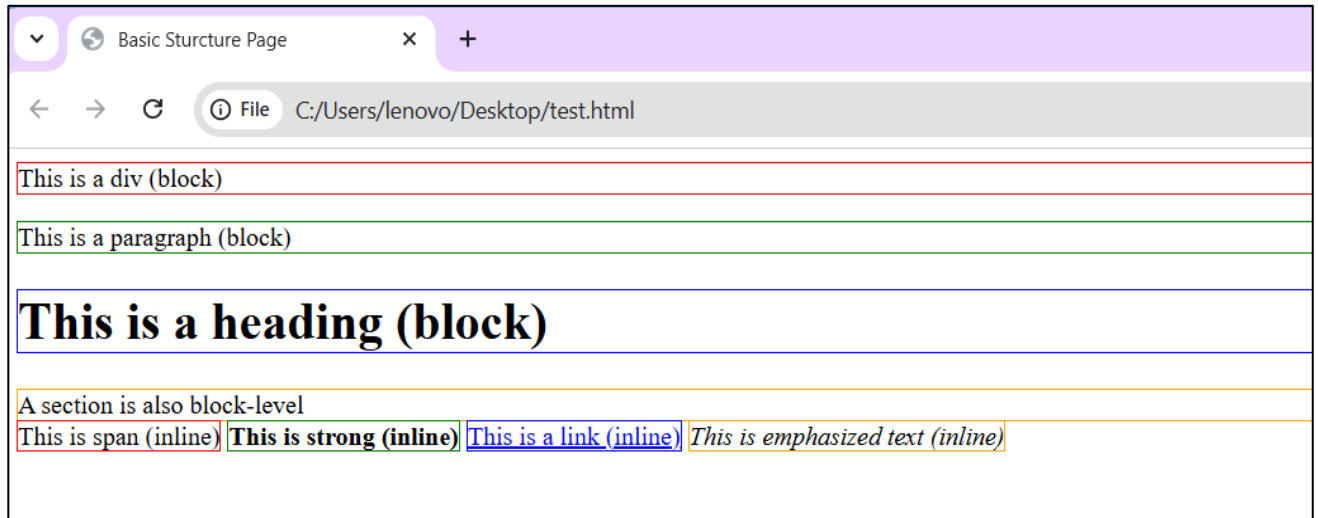
    <strong style="border: 1px solid green">This is strong (inline)</strong>

    <a href="#" style="border: 1px solid blue">This is a link (inline)</a>

    <em style="border: 1px solid orange">This is emphasized text (inline)</em>

  </body>
```

Output:



Tag	Description
<html> ... </html>	Declares the Web page to be written in HTML
<head> ... </head>	Delimits the page's head
<title> ... </title>	Defines the title (not displayed on the page)
<body> ... </body>	Delimits the page's body
<h n> ... </h n>	Delimits a level <i>n</i> heading
 ... 	Set ... in boldface
<i> ... </i>	Set ... in italics
<center> ... </center>	Center ... on the page horizontally
...	Brackets an unordered (bulleted) list
...	Brackets a numbered list
...	Brackets an item in an ordered or numbered list
 	Forces a line break here
<p>	Starts a paragraph
<hr>	Inserts a horizontal rule
	Displays an image here
 ... 	Defines a hyperlink

1.4.1 Text formatting- headings, paragraphs

HTML Headings

- Inside the BODY element, heading elements H1 through H6 are generally used for major divisions of the document. Headings are permitted to appear in any order, but you will obtain the best results when your documents are displayed in a browser if you follow these guidelines:
- H1: should be used as the highest level of heading, H2 as the next highest, and so forth.
- You should not skip heading levels: e.g., an H3 should not appear after an H1, unless there is an H2 between them.

```
<HTML>

<HEAD>

<TITLE> Example Page</TITLE>

</HEAD>

<BODY>

<H1> Heading 1 </H1>

<H2> Heading 2 </H2>

<H3> Heading 3 </H3>

<H4> Heading 4 </H4>

<H5> Heading 5 </H5>

<H6> Heading 6 </H6>

</BODY>

</HTML>
```

HTML Paragraph

Paragraphs allow you to add text to a document in such a way that it will automatically adjust the end of line to suite the window size of the browser in which it is being displayed. Each line of text will stretch the entire length of the window.

```
<!DOCTYPE html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>My First Page!!!!</title>

  </head>

  <body bgcolor="rgb(255,242,255)" text="yellow">

    <!--html elements-->

    <!--paragraph element-->

    <p>

      Lorem ipsum dolor sit amet consectetur adipisicing elit. Odio, nisi! Dicta

      nemo porro, aperiam ex, vero necessitatibus nostrum debitis obcaecati

      officiis nam numquam minima nisi tempora itaque expedita amet voluptatum!

    </p>

  </body>

</html>
```

1.5 HTML Lists: Unordered Lists, Ordered Lists, Description Lists, Nesting and Mixed Lists, Lists for Navigation Menus

An HTML list helps present data on web pages in a structured manner, either in an ordered or unordered format, making the content easier to read and visually attractive. Lists play a key role in creating organized and accessible content in web development.

Types of HTML Lists

1.5.0 Unordered Lists (): These lists are used for items that do not need to be in any specific order. The list items are typically marked with bullets.

1.5.1 Ordered Lists (): These lists are used when the order of the items is important. Each item in an ordered list is typically marked with numbers or letters.

Attributes:

type → numbering style (1, a, A, i, I)

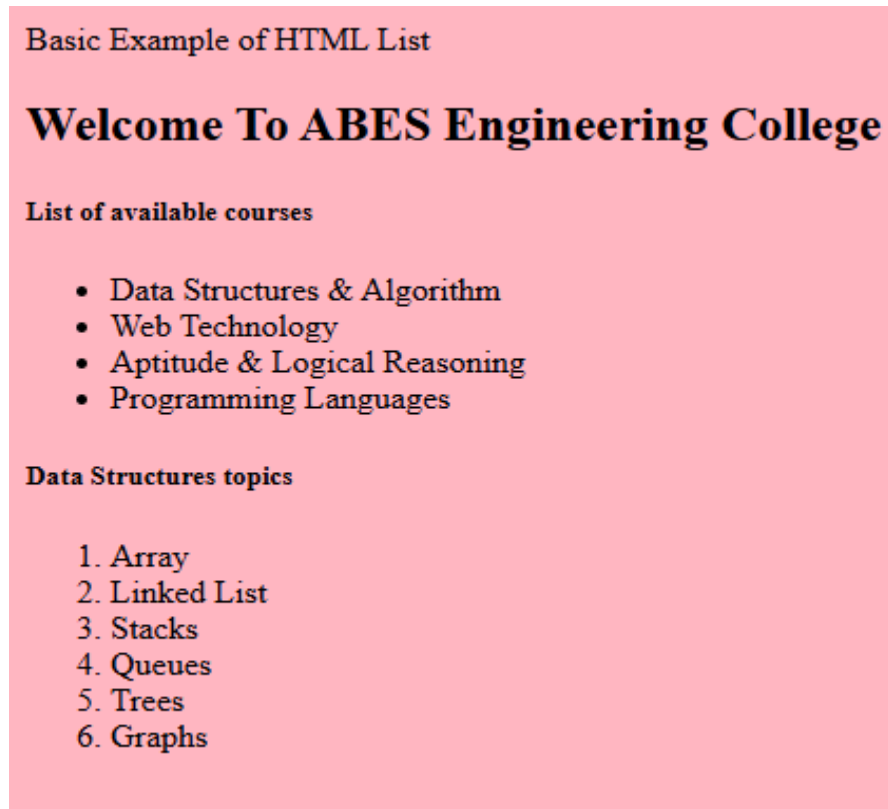
start → starting number/letter

reversed → reverses order

Example:

```
<html> </head> <body>
Basic Example of HTML List
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>HTML</title>
</head> <body>
  <h2>Welcome To ABES Engineering College </h2>
  <h5>List of available courses</h5>
  <ul>
    <li>Data Structures & Algorithm</li>
    <li>Web Technology</li>
    <li>Aptitude & Logical Reasoning</li>
    <li>Programming Languages</li>
  </ul>
  <h5>Data Structures topics</h5>
  <ol>
    <li>Array</li>
    <li>Linked List</li>
    <li>Stacks</li>
    <li>Queues</li>
    <li>Trees</li>
    <li>Graphs</li>
  </ol> </body> </html>
```


Output:



1.5.2 Description Lists (<dl>): These lists are used to contain terms and their corresponding descriptions.

Contains:

1. <dt> → **Definition Term**
2. <dd> → **Definition Description**

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
content="width=device-width, initial-scale=1.0">
  <title>HTML</title>
</head>
<body style="background-color: #FFB6C1;">
  <h2>Frequently Asked Questions</h2>
<dl>
  <dt>What is HTML? </dt>
  <dd>HTML stands for HyperText Markup Language and is used to create web pages. </dd>
  <dt>What is CSS? </dt>
  <dd>CSS stands for Cascading Style Sheets and controls the look and layout of web pages.
</dd>
  <dt>What is JavaScript? </dt>
  <dd>JavaScript is a programming language that adds interactivity to websites. </dd>
</dl>
  </body> </html> </body>
</html>
```

Output:

Frequently Asked Questions

What is HTML?

HTML stands for HyperText Markup Language and is used to create web pages.

What is CSS?

CSS stands for Cascading Style Sheets and controls the look and layout of web pages.

What is JavaScript?

JavaScript is a programming language that adds interactivity to websites.

1.5.4 Nesting lists

Nesting lists means placing one list **inside an item of another list**. The nested list is usually wrapped inside an `<li>` element of the parent list.

The purpose of Nesting List:

- ✓ To represent **hierarchical data** (e.g., categories and subcategories).
- ✓ To create **multi-level navigation menus**.
- ✓ To organize **outlines, directories, or structured content**.

Key Rules for Nesting Lists:

The nested list **must be inside an `<li>` element** of the parent list.

Nesting can be:

- **Unordered inside unordered** (`<ul>` in `<ul>`).
- **Ordered inside unordered** (`<ol>` in `<ul>`), and vice versa.
- ✓ Indentation (visually) helps users understand the hierarchy, but this is **done via CSS**, not by adding spaces in HTML.

Types of Nesting

1. **Unordered → Unordered** (Common in multi-level bullet points)
2. **Ordered → Ordered** (Common in outlines or steps with sub-steps)
3. **Mixed Nesting** (Combination of `<ul>` and `<ol>`)
4. **Nesting inside Description List** (less common, but possible inside `<dd>`)

Example of Nesting Lists:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>HTML</title>
</head>
<body style="background-color: #FFB6C1;">
<h3>Shopping List</h3>
<ul>
  <li>Fruits
    <ul>
      <li>Apple</li>
      <li>Mango</li>
      <li>Orange</li>
    </ul>
  </li>
  <li>Vegetables
    <ul>
      <li>Carrot</li>
      <li>Potato</li>
      <li>Tomato</li>
    </ul>
  </li>
</ul>
</body>
</html>
```

Output:



1.5.5 Mixed lists

Mixed lists in HTML refer to the combination of **ordered lists ()** and **unordered lists ()** **within each other**. This means that an ordered list can contain an unordered list as a nested list or vice versa. The purpose of using mixed lists is to represent complex hierarchical information where some items need to be displayed in a specific order, while others do not.

Mixed Lists uses:

- **To organize information with different importance levels:** For example, main steps (ordered) and sub-options (unordered).
- **To create structured outlines:** Where the top level is numbered, and the sub-points are bulleted.
- **To maintain clarity in instructions, menus, and structured content.**

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>HTML</title>
</head>
<body style="background-color: #FFB6C1;">
  <h3>Web Development Topics</h3>
  <ol>
    <li>Frontend
      <ul>
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
      </ul>
    </li>
    <li>Backend
      <ul>
        <li>Node.js</li>
        <li>PHP</li>
        <li>Python</li>
      </ul>
    </li>
    <li>Database
      <ul>
        <li>MySQL</li>
        <li>MongoDB</li>
      </ul>
    </li>
  </ol>
</body>
</html>
```

Output:



1.5.7 Lists for Navigation Menus

Navigation menus are a crucial part of any website, allowing users to move between different pages or sections easily. In HTML, navigation menus are commonly created using unordered lists (`<ul>`) because the order of menu items is usually not important. These lists are typically placed inside a `<nav>` element, which provides semantic meaning, indicating that the section is for navigation purposes.

The best practice for creating navigation menus is to wrap each navigation link inside an `<li>` element, and then place all `<li>` elements within a `<ul>`. Each menu item is usually linked using the `<a>` (anchor) tag inside the list item. This structure ensures that the menu is both semantic and accessible, improving usability for screen readers and assistive technologies.

The uses of Navigation list:

Semantic Structure: Using `<nav>` with `<ul>` and `<li>` clearly defines the navigation section.

Accessibility: Screen readers can interpret lists properly, making navigation easier for visually impaired users.

Styling Flexibility: Lists provide an easy structure for applying CSS to create horizontal or vertical menus.

Consistency: Most browsers and frameworks support list-based menus, making it a standard practice.

Example:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>HTML</title>

</head>

<body style="background-color: #FFB6C1;">

  <nav>

    <span><a href="index.html">Home</a></span>&nbsp;&nbsp;&nbsp;&nbsp; 

    <span><a href="about.html">About</a></span>&nbsp;&nbsp;&nbsp;&nbsp; 

    <span><a href="services.html">Services</a></span>&nbsp;&nbsp;&nbsp;&nbsp; 

    <span><a href="contact.html">Contact</a></span>

  </nav>

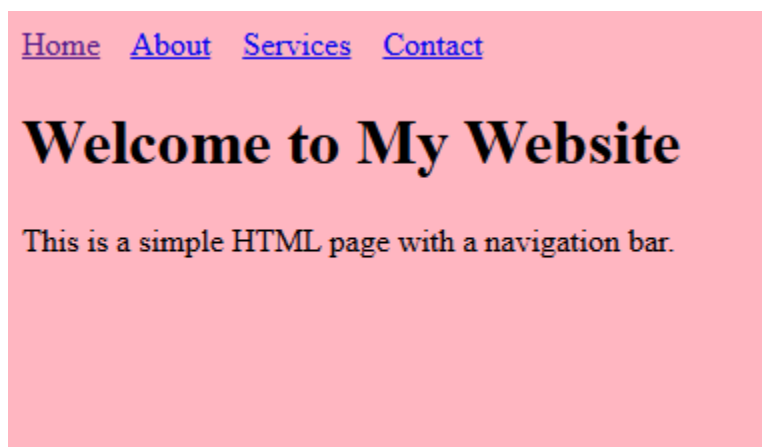
  <h1>Welcome to My Website</h1>

  <p>This is a simple HTML page with a navigation bar.</p>

</body>

</html>
```

Output:



1.6 HTML Links, Images and attributes, Comment in HTML, HTML Formatting Elements/ Special Character & Symbol

1.6.0 HTML Links

HTML links are hyperlinks.

You can click on a link and jump to another document.

When you move the mouse over a link, the mouse arrow will turn into a little hand.

HTML Links - Syntax

The HTML **<a>** tag defines a hyperlink. It has the following syntax:

```
<a href="url">link text</a>
```

The most important attribute of the **<a>** element is the href attribute, which indicates the link's destination.

```
<!DOCTYPE html>
<html>
<body>

<h1>HTML Links</h1>

<p><a href="https://www.abes.ac.in/">Welcome ABES EC</a></p>

</body>
</html>
```

1.6.1 HTML Image

The HTML **** tag is used to embed an image in a web page.

Images are not technically inserted into a web page; images are linked to web pages. The **** tag creates a holding space for the referenced image.

The **** tag is empty; it contains attributes only, and does not have a closing tag.

The **** tag has two required attributes:

src - Specifies the path to the image

alt - Specifies an alternate text for the image

```
<!DOCTYPE html>

<html>

<body>


<h2>Alternative text</h2>


<p>The alt attribute should reflect the image content, so users who cannot see the image get an
understanding of what the image contains:</p>





</body>

</html>
```

1.6.2 HTML Comments

```
<!DOCTYPE html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>HTML Comments</title>

  </head>

  <body> <!--body start -->

  <!--section is started from here! -->

  <section>

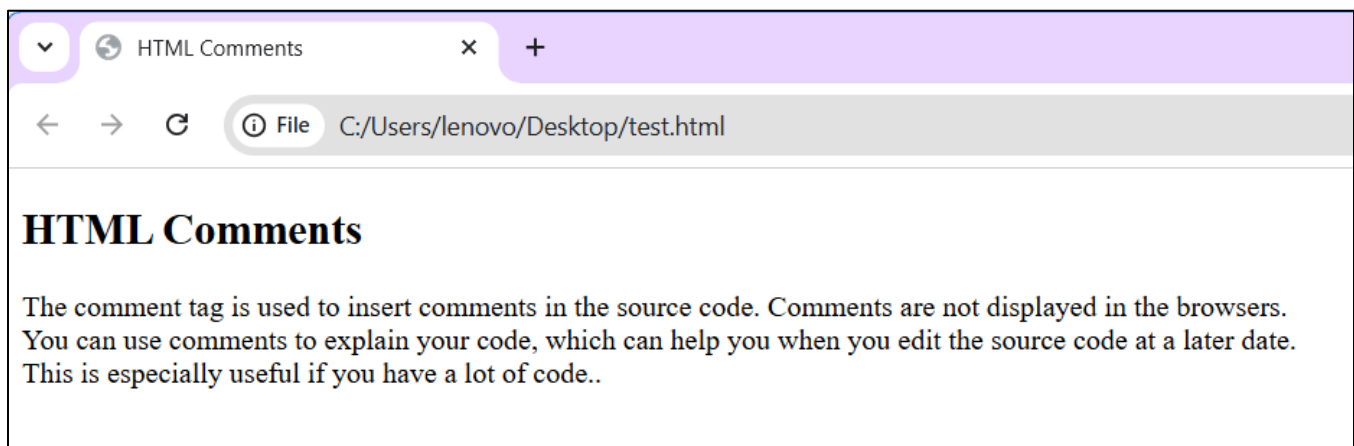
    <h2> HTML Comments</h2>
    <p>The comment tag is used to insert comments in the source code. Comments are not displayed in the
    browsers.<br> You can use comments to explain your code, which can help you when you edit the source code at a
    later date. <br>This is especially useful if you have a lot of code..</p>

  </section>

  </body> <!--body end--></html>
-----
```

Comments can be inserted into the HTML code to make it more readable and understandable. Comments are ignored by the browser and are not displayed.

Output:



1.6.3 HTML Formatting Elements

In this chapter, you will learn how to enhance your page with Bold, Italics, and other character formatting options.

`<FONT SIZE="+2"> Two sizes bigger</FONT>`

The size attribute can be set as an absolute value from 1 to 7 or as a relative value using the “+” or “-” sign. Normal text size is 3 (from -2 to +4).

`<FONT COLOR="#RRGGBB">this text has color</FONT>`

Color = “#RRGGBB” The COLOR attribute of the FONT element.

`<B> Bold </B>`

`<I> Italic </I>`

`<U> Underline </U>`

`<PRE> Preformatted </PRE>`

Text enclosed by PRE tags is displayed in a mono-spaced font. Spaces and line breaks are supported without additional elements or special characters.

`<EM> Emphasis </EM>`

Browsers usually display this as italics.

`<STRONG> STRONG </STRONG>`

Browsers display this as bold.

`<TT> TELETYPE </TT>`

Text is displayed in a mono-spaced font. A typewriter text, e.g. fixed-width font.

`<STRIKE> strike-through text</STRIKE>`

DEL is used for STRIKE at the latest browsers

`<BIG> places text in a big font</BIG>`

`<SMALL> places text in a small font</SMALL>`

`<SUB> places text in subscript position </SUB>`

^{places text in superscript style position}

<ins>showing new inserted text</ins>

<mark>used for highlighting</mark>

1.6.4 HTML ENTITIES or Characters & Symbols

Some characters are reserved in HTML.

If you use the less than (<) or greater than (>) signs in your HTML text, the browser might mix them with tags.

- Entity names or entity numbers can be used to display reserved HTML characters.
- Entity names look like this: &entity_name;
- Entity numbers look like this: &#entity_number;

These Characters are recognized in HTML as they begin with an ampersand and end with with a semi- colon e.g. &value; The value will either be an entity name or a standard ASCII character number. They are called escape sequences.

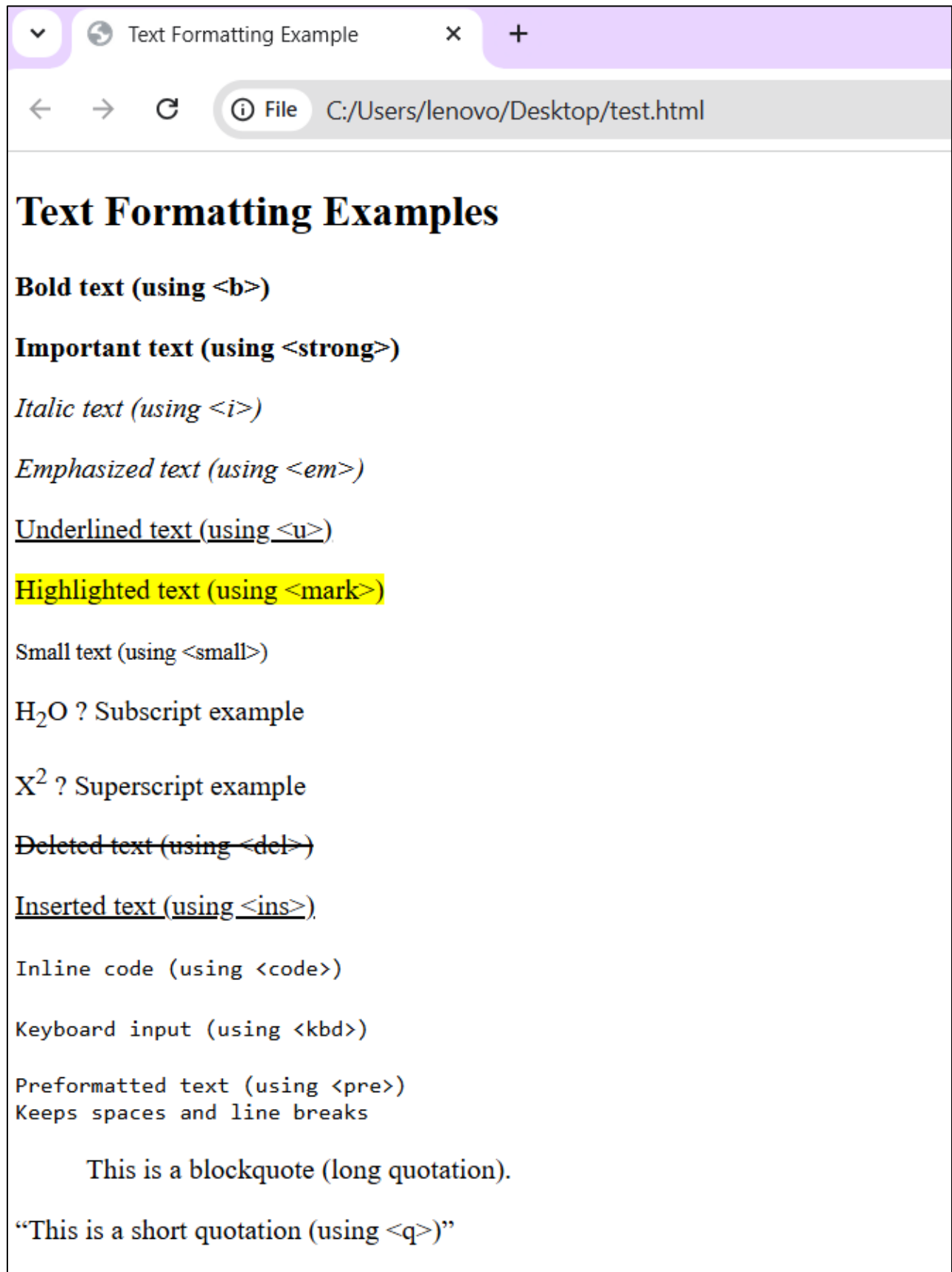
The next table represents some of the more commonly used special characters. For a comprehensive listing.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Text Formatting Example</title>
</head>
<body>
  <h2>Text Formatting Examples</h2> <p><b>Bold text (using
    &lt;b>)</b></p> <p><strong>Important text (using
    &lt;strong>)</strong></p> <p><i>Italic text (using
    &lt;i>)</i></p> <p><em>Emphasized text (using
    &lt;em>)</em></p> <p><u>Underlined text (using
    &lt;u>)</u></p> <p><mark>Highlighted text (using
    &lt;mark>)</mark></p> <p><small>Small text (using
    &lt;small>)</small></p> <p>H<sub>2</sub>O → Subscript
    example</p> <p>X<sup>2</sup> → Superscript example</p>
    <p><del>Deleted text (using &lt;del>)</del></p>
    <p><ins>Inserted text (using &lt;ins>)</ins></p>
    <p><code>Inline code (using &lt;code>)</code></p>
    <p><kbd>Keyboard input (using &lt;kbd>)</kbd></p>
    <p><pre>
Preformatted text (using &lt;pre>)</pre>
  Keeps spaces and line breaks
</pre></p>
    <p><blockquote cite="https://www.example.com">
  This is a blockquote (long quotation).
</blockquote></p>
    <p><q>This is a short quotation (using &lt;q>)</q></p>
  </body></html>

```

Output:



Special Characters & Symbols OR ENTITIES

Special Character	Entity Name	Special Character	Entity Name
Ampersand	& &	Greater-than sign	> >
Asterisk	∗ *	Less-than sign	< <
Cent sign	¢ ¢	Non-breaking space	 ;
Copyright	© ©	Quotation mark	" "
Fraction one qtr	¼ ¼	Registration mark	® ®
Fraction one half	½ ½	Trademark sign	™ ™

Additional escape sequences support-accented characters, such as:

- **ö** a lowercase o with an umlaut: ö
- **ñ** a lowercase n with a tilde: ñ
- **È** an uppercase E with a grave accent: È

NOTE: Unlike the rest of HTML, the escape sequences are case sensitive. You cannot, for instance, use < instead of <.


```
<html lang="en">

<head>

  <meta charset="UTF-8" />

  <title>HTML Entities Example</title>

</head>

<body>

  <h2>Special Characters & Entities</h2>

  <p>Less than: &lt;</p>

  <p>Greater than: &gt;</p>

  <p>Ampersand: &amp;</p>

  <p>Double Quote: &quot;Hello&quot;</p>

  <p>Single Quote: &apos;World&apos;</p>

  <p>Non-breaking space: Hello&nbsp;World (keeps them together)</p>

  <p>Copyright: &copy; 2025 MySite</p>

  <p>Registered Trademark: &reg;</p>

  <p>Trademark: &trade;</p>

  <p>Euro: &euro; 50</p>

  <p>Pound: &pound; 100</p>

  <p>Yen: &yen; 500</p>

  <p>Indian Rupee: &#8377; 999 (₹)</p>

  <p>Arrow Right: &rarr;</p>

  <p>Arrow Left: &larr;</p>

  <p>Arrow Up: &uarr;</p>

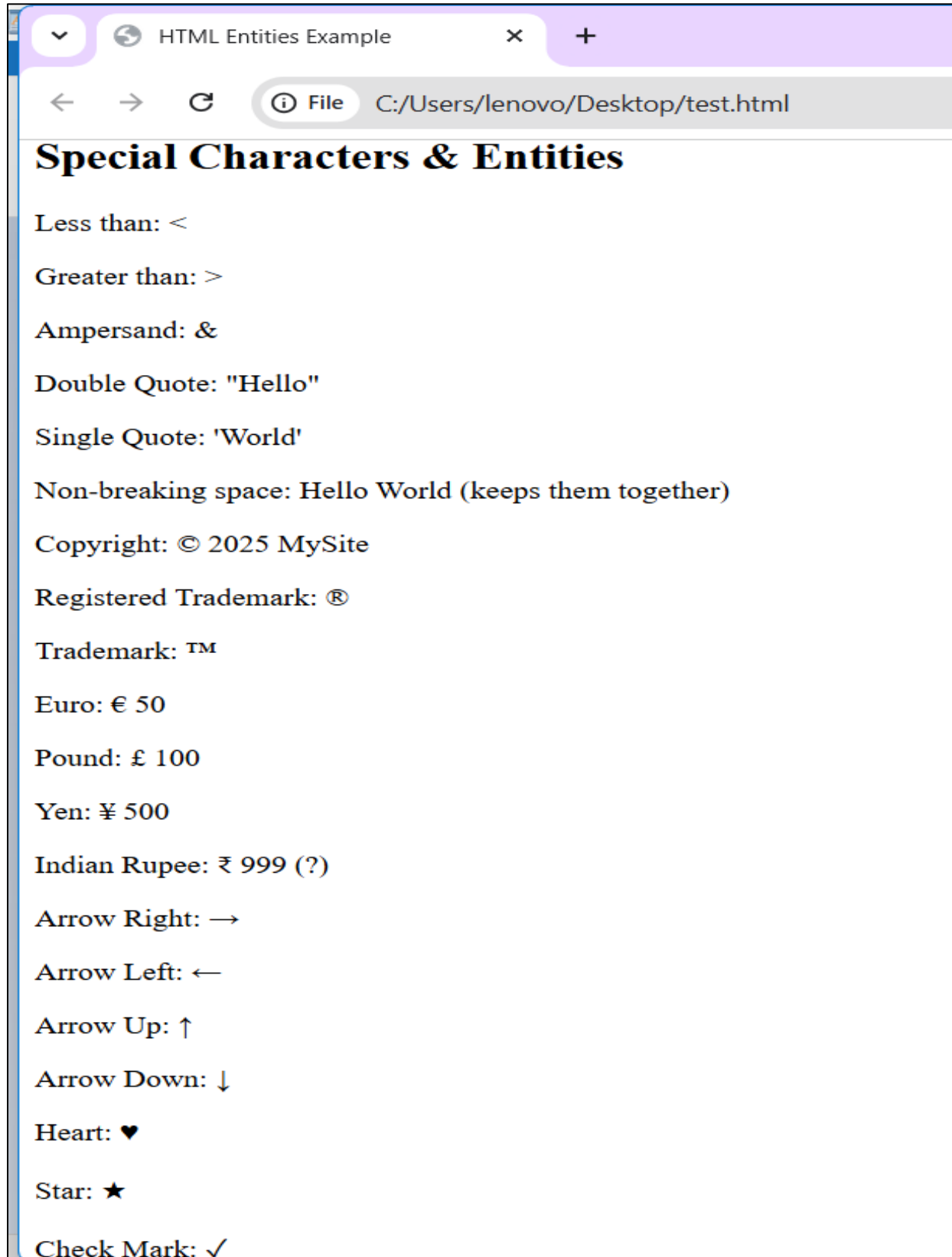
  <p>Arrow Down: &darr;</p>

  <p>Heart: &hearts;</p>

</body>

</html>
```

Output:



1.7 Text related Elements (Quotation and Citation Elements), Using Multimedia (Image/video/Audio/YouTube) in HTML, Creating & Using Hyperlinks

1.7.0 Text related Elements (Quotation and Citation Elements)

HTML provides specific elements for semantically marking up quoted text and citations, enhancing accessibility and structure.

- **<abbr>** Defines an abbreviation or acronym
- **<bdo>** Defines the text direction
- **<blockquote>** Defines a section that is quoted from another source
- **<cite>** Defines the title of a work
- **<q>** Defines a short inline quotation
- **<address>** Defines contact information for the author/owner of a document

1.7.1 Using Multimedia (Image/video/Audio/you tube) in HTML

A variety of tags such as the tag, <video> tag, and <audio> tag are available in HTML to include media on your web page. Multimedia combines different media, such as images, audio, and videos. Users will have a better experience when multimedia is embedded into HTML.

Media Tag	Description
<u><audio></u>	An inline element is used to embed sound files into a web page.
<u><video></u>	Used to embed video files into a web page.
<u><source></u>	Used to attach multimedia files like audio, video, and pictures.
<u><embed></u>	Used for embedding external applications, generally multimedia content like audio or video, into an HTML document.<
<u><track></u>	Specifies text tracks for media components, specifically for audio and video elements.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Media Tags Example</title>
</head>
<body>
  <h1>Media Tags in HTML</h1>
  <h2>Image</h2> <!-- Image Example -->
  
  <p>This is an image using the <code>&lt;img&gt;</code> tag.</p>

  <!-- Audio Example --><h2>Audio</h2>

  <audio controls>
    <source src="sample-audio.mp3" type="audio/mpeg" />
    <source src="sample-audio.ogg" type="audio/ogg" />
    Your browser does not support the audio element.
  </audio>

  <p>This is an audio player using the <code>&lt;audio&gt;</code>
tag.</p> <h2>Video</h2> <!-- Video Example --> <video width="400"
controls>

  <source src="sample-video.mp4" type="video/mp4" />
  <source src="sample-video.ogg" type="video/ogg" />
  Your browser does not support the video tag. </video>

  <p>This is a video player using the <code>&lt;video&gt;</code> tag.</p>

  <!-- Iframe Example -->
  <h2>Embedded Video (Iframe)</h2>
  <iframe width="400" height="250" src="https://www.youtube.com/embed/dQw4w9WgXcQ"
title="YouTube Video" frameborder="0" allowfullscreen ></iframe>

  <p>This uses <code>&lt;iframe&gt;</code> to embed a YouTube video.</p>
</body> </html>

```

Output:

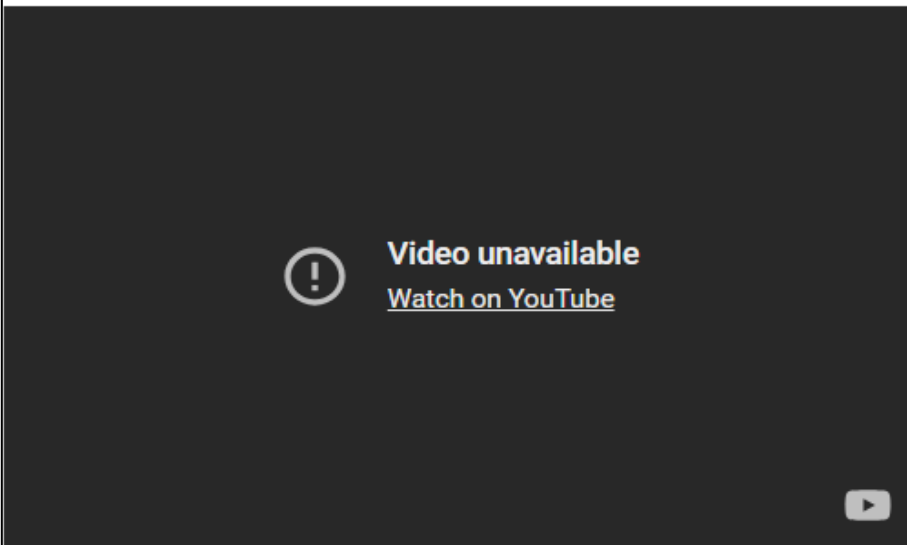


Video



This is a video player using the `<video>` tag.

Embedded Video (Iframe)



This uses `<iframe>` to embed a YouTube video.

Advantage of Media tag:

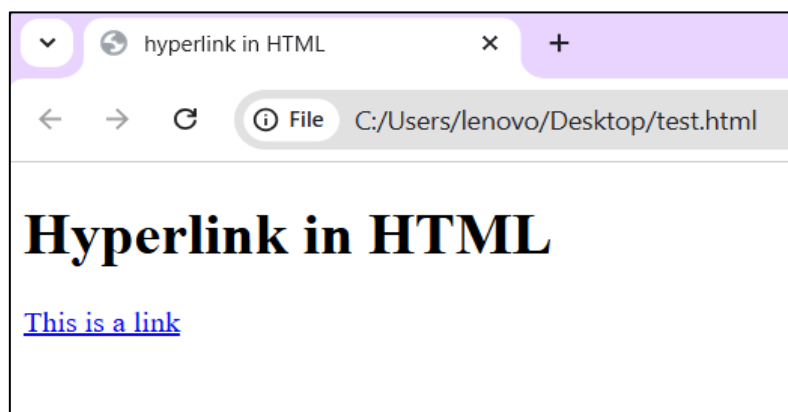
- Plugins are no longer required
- fashion than imported third-party
- Native (built-in) controls are provided by the browser.
- Accessibilities (keyboard, mouse) are built-in automatically

1.7.3 Creating & Using Hyperlinks

The <a> HTML element (or anchor element), with its href attribute, creates a hyperlink to web pages, files, email addresses, locations in the same page, or anything else a URL can address.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>hyperlink in HTML</title>
  </head>
  <body>
    <h1>Hyperlink in HTML</h1>
    <!-- Hyperlink Example -->
    <a href="http://www.facebook.com" target= " _blank" > This is a link</a>
  </body>
</html>
```

Output:



1.8 Accessibility & Semantic Considerations

Semantic elements = elements with a meaning.

Accessibility considerations focus on designing websites for users of all abilities, while semantic HTML provides the structural meaning and context that assistive technologies like

screen readers rely on to interpret content for visually impaired users and enable keyboard navigations.

When using lists in HTML, it is important to follow semantic and accessibility best practices to ensure that content is understandable and navigable for all users, including those using assistive technologies. **Semantic HTML means using the correct tags for their intended purpose:** `<ul>` for unordered lists, `<ol>` for ordered lists, `<dl>` for description lists, and `<nav>` for navigation menus. This not only improves readability for developers but also allows screen readers and other assistive tools to interpret the structure of the content accurately.

For navigation menus, wrapping menu items inside `<li>` elements within a `<ul>` and enclosing the whole menu in a `<nav>` element provides clear semantic meaning. Each link (`<a>` tag) should have descriptive text, avoiding generic phrases like “Click here,” so users understand the purpose of each link. Proper use of semantic tags helps screen readers announce the type and number of items in a list, enabling users to navigate efficiently.

Other accessibility considerations include maintaining sufficient color contrast, providing visible focus indicators for keyboard navigation, and avoiding excessive list nesting that may confuse users. Following these semantic and accessibility principles ensures that HTML lists are both meaningful and user-friendly across different devices and abilities.

Accessibility & Semantic Features in this Example: semantic glossary using a description list (`<dl>`). Each term (`<dt>`) like “HTML” or “CSS” is paired with its description (`<dd>`), making the relationship clear. Semantic tags help screen readers announce terms and definitions correctly, improving accessibility. Optional id attributes allow linking directly to specific terms. CSS can style the list for readability without affecting its structure, ensuring the glossary is both user-friendly and accessible.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body style="background-color: #ffb6c1;">
  <h2>Web Development Glossary</h2>
  <dl>
    <dt id="html-term">HTML</dt>
    <dd>
      HyperText Markup Language, the standard language for creating web pages.
    </dd>
    <dt id="css-term">CSS</dt>
    <dd>
      Cascading Style Sheets, used for styling and layout of web pages.
    </dd>
    <dt id="js-term">JavaScript</dt>
    <dd>
      A programming language used to make web pages interactive.
    </dd>
    <dt id="api-term">API</dt>
    <dd>
      Application Programming Interface, a set of rules that allows software
      programs to communicate with each other.
    </dd>
  </dl>
</body> </html>
```

Output:

Web Development Glossary

HTML

HyperText Markup Language, the standard language for creating web pages.

CSS

Cascading Style Sheets, used for styling and layout of web pages.

JavaScript

A programming language used to make web pages interactive.

API

Application Programming Interface, a set of rules that allows software programs to communicate with each other.

Practical Use case:

1. A company website lists its main navigation items like Home, About, Services, and Contact. Additionally, a product page lists key features using bullet points.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>HTML</title>
</head> <body style="background-color: rgb(180, 233, 120);">
  <!-- Navigation Menu -->
  <h3>Company Navigation Menu</h3>
  <nav aria-label="Main Navigation">
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="about.html">About</a></li>
      <li><a href="services.html">Services</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul> </nav> <!-- Product Features -->
  <h3>Product Key Features</h3>
  <div class="features">
    <ul>
      <li>High-speed performance</li>
      <li>Durable design</li>
      <li>Affordable price</li>
      <li>24/7 customer support</li>
    </ul> </div> </body> </html>
```

Output:



2. A cooking website lists a recipe where steps must be followed in order, such as mixing ingredients, baking, and serving.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>HTML</title>
</head>
<body>
  <h1>Welcome to My Cooking Page</h1>
  <p>This page contains a simple recipe for baking a cake.</p>

  <h3>Recipe: How to Bake a Cake</h3>
  <ol>
    <li>Preheat the oven to 180°C (350°F).</li>
    <li>Mix flour, sugar, eggs, and butter in a bowl.</li>
    <li>Pour the batter into a greased baking tray.</li>
    <li>Bake for 25–30 minutes or until golden brown.</li>
    <li>Remove from oven and let it cool.</li>
    <li>Serve and enjoy!</li>
  </ol>
</body>
</html>
```

Output:

Welcome to My Cooking Page

This page contains a simple recipe for baking a cake.

Recipe: How to Bake a Cake

1. Preheat the oven to 180°C (350°F).
2. Mix flour, sugar, eggs, and butter in a bowl.
3. Pour the batter into a greased baking tray.
4. Bake for 25–30 minutes or until golden brown.
5. Remove from oven and let it cool.
6. Serve and enjoy!

3. A web development tutorial defines terms like HTML, CSS, and JavaScript.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>HTML</title>
</head>
<body style="background-color: rgb(180, 233, 120);">
  <h3>Web Development Glossary</h3>
  <dl>
    <dt>HTML</dt>
    <dd>HyperText Markup Language, used to structure content on the web.</dd>
    <dt>CSS</dt>
    <dd>Cascading Style Sheets, used to style and layout web pages.</dd>
    <dt>JavaScript</dt>
    <dd>A programming language that makes web pages interactive.</dd>
    <dt>API</dt>
    <dd>Application Programming Interface, allows communication between
software programs.</dd>
  </dl>
</body>
</html>
```

Output:

Web Development Glossary

HTML

HyperText Markup Language, used to structure content on the web.

CSS

Cascading Style Sheets, used to style and layout web pages.

JavaScript

A programming language that makes web pages interactive.

API

Application Programming Interface, allows communication between software programs.

4. An e-commerce website has a main category “Products” with subcategories “Electronics” and “Clothing.” Electronics further includes “Mobiles” and “Laptops.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>HTML</title>
</head>
<body style="background-color: rgb(180, 233, 120);">
  <h3>Product Categories</h3>
  <ul>
    <li>Products
      <ul>
        <li>Electronics
          <ul>
            <li>Mobiles</li>
            <li>Laptops</li>
          </ul>
        </li>
        <li>Clothing</li>
      </ul>
    </li>
    <li>About Us</li>
    <li>Contact</li>
  </ul>
</body>
</html>
```

Output:

Product Categories

- Products
 - Electronics
 - Mobiles
 - Laptops
 - Clothing
- About Us
- Contact

2.0 HTML Table:Creating and using Tables,Basic Table Structure,Table Headers and Captions

2.0.0 HTML Table

An HTML table is a way to organize data in rows and columns. We create it with the `<table>` element in HTML. Tables mainly show tabular data like schedules, product lists, or reports.

Basic HTML Table Structure

An HTML table is created using the `<table>` tag. Inside the table, we use:

- [`<tr>`](#): Table Row
- [`<th>`](#): Table Header
- [`<td>`](#): Table Data

Basic Example:

```
index.html > html
1  <!-- index.html -->
2  <!DOCTYPE html>
3  <html lang="en">
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport"
7          content="width=device-width, initial-scale=1.0">
8      <title>HTML</title>
9  </head>
10 <body>
11     <table>
12         <tr>
13             <th>Firstname</th>
14             <th>Lastname</th>
15             <th>Age</th>
16         </tr>
17         <tr>
18             <td>Priya</td>
19             <td>Sharma</td>
20             <td>24</td>
21         </tr>
22         <tr>
23             <td>Arun</td>
24             <td>Singh</td>
25             <td>32</td>
26         </tr>
27         <tr>
28             <td>Sam</td>
29             <td>Watson</td>
30             <td>41</td>
31         </tr>
32     </table>
33 </body>
34 </html>
```

Output:

Firstname	Lastname	Age
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Tag	Description
<u><table></u>	Defines the structure for organizing data in rows and columns on a web page.
<u><tr></u>	Represents a row in an HTML table that contains individual cells.
<u><th></u>	Shows a table header cell, which usually holds titles or headings.
<u><td></u>	Represents a standard data cell that holds content or data.
<u><caption></u>	Provides a title or description for the entire table.
<u><thead></u>	Defines the header section of a table, often containing column labels.
<u><tbody></u>	Represents the main content area of a table, separating it from the header or footer.
<u><tfoot></u>	Specifies the footer section of a table that typically holds summaries or totals.
<u><col></u>	Defines attributes for table columns that can apply to multiple columns at once.
<u><colgroup></u>	Groups together a set of columns in a table so you can apply formatting or properties to all of them at once.

2.0.1 Basic Table Structure in HTML

HTML tables are not just a collection of rows and columns; they can be **logically grouped into sections** for better organization, accessibility, and styling. The three main grouping tags are:

- 1) **<thead>**: Groups header rows.
- 2) **<tbody>**: Groups body rows.
- 3) **<tfoot>**: Groups footer rows.

Example:

```
index.html > html > body > table
1  <!-- index.html -->
2  <!DOCTYPE html>
3  <html lang="en">
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport"
7          content="width=device-width, initial-scale=1.0">
8      <title>HTML</title>
9  </head>
10 <body>
11     <table>
12         <thead>
13             <tr><th>Header 1</th><th>Header 2</th></tr>
14         </thead>
15         <tbody>
16             <tr><td>Data 1</td><td>Data 2</td></tr>
17         </tbody>
18         <tfoot>
19             <tr><td colspan="2">Footer</td></tr>
20         </tfoot>
21     </table>
22 </body>
23 </html>
```

Output

Header 1 Header 2	
Data 1	Data 2
Footer	

2.0.2 Table Headers and Captions

Headers and captions help organize and present data in tables.

The table heading is important because it labels the columns. The **<th>** (table header) element defines these headings.

Captions give a title or explanation for the table. The **<caption>** element is placed right after the opening table tag.

Employee Record		
Name	Age	City
Kelly Hu	21	Honolulu, Hawaii
Julia Roberts	23	Smyrna, Georgia

Syntax to Create Table's Header and Caption:

The following is the syntax to create a header and caption for an HTML table:

```
<table>
<caption>Description of table</caption>
<tr>
  <th>heading 1</th>
  <th>heading 2</th>
  <th>heading 3</th>
</tr>
</table>
```

2.1 HTML Table: Rowspan and Colspan, Table Attributes, Responsive Tables

2.1.0 HTML Table Colspan & Rowspan

HTML tables can have cells that span over multiple rows and/or columns.

NAME		

APRIL		

2022		
FIESTA		

HTML Table - Colspan

To make a cell span over multiple columns, use the colspan attribute:

Example:

```
index.html > html > body > table
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5  table, th, td {
6      border: 1px solid black;
7      border-collapse: collapse;
8  }
9  </style>
10 </head>
11 <body>
12
13 <h2>Cell that spans two columns</h2>
14 <p>To make a cell span more than one column, use the colspan attribute.</p>
15
16 <table style="width:50%">
17     <tr>
18         <th colspan="2">Name</th>
19         <th>Age</th>
20     </tr>
21     <tr>
22         <td>Jill</td>
23         <td>Smith</td>
24         <td>43</td>
25     </tr>
26     <tr>
27         <td>Eve</td>
28         <td>Jackson</td>
29         <td>57</td>
30     </tr>
31 </table>
32 </body>
33 </html>
```


Output:

Cell that spans two columns

To make a cell span more than one column, use the colspan attribute.

Name		Age
Jill	Smith	43
Eve	Jackson	57

HTML Table - Rowspan

To make a cell span over multiple rows, use the rowspan attribute:

Example:

```
index.html > html > body > table
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5  table, th, td {
6    border: 1px solid black;
7    border-collapse: collapse;
8  }
9  </style>
10 </head>
11 <body>
12
13 <h2>Cell that spans two rows</h2>
14 <p>To make a cell span more than one row, use the rowspan attribute.</p>
15
16 <table style="width:50%">
17   <tr>
18     <th>Name</th>
19     <td>Jill</td>
20   </tr>
21   <tr>
22     <th rowspan="2">Phone</th>
23     <td>555-1234</td>
24   </tr>
25   <tr>
26     <td>555-8745</td>
27   </tr>
28 </table>
29 </body>
30 </html>
31
```

Output:

Cell that spans two rows

To make a cell span more than one row, use the rowspan attribute.

Name	Jill
Phone	555-1234
	555-8745

2.1.1 HTML Table Attributes

HTML tables can be customized using attributes to control appearance, spacing, and alignment. Many of these attributes are outdated in HTML5, but they still help in understanding older code. For modern projects, CSS is the preferred option.

Most common attributes are:

- **border:** The **HTML <table> border Attribute** is used to specify the border of a table. It sets the border around the table cells. This attribute defines the visual presentation of the table by setting the thickness of the borders. A higher value results in a thicker border. Alternatively, setting the border attribute to "0" removes the borders entirely.

Syntax:

<table border="value">

The value represents the thickness of the border in pixels.

border="1": This means the table will have a border of 1 pixel.

border="0": This will remove the borders from the table, leaving it borderless.

```

1 <html>
2 <head>
3 <style>
4 table, th, td {
5     border: 1px solid black;
6     border-collapse: collapse;
7 }
8 </style>
9 </head>
10 <body>
11     <h1>This is Border Attribute</h1>
12     <h2>HTML table border Attribute</h2>
13     <table border="1">
14         <caption>
15             Author Details
16         </caption>
17
18         <tr>
19             <th>NAME</th>
20             <th>AGE</th>
21             <th>BRANCH</th>
22         </tr>
23         <tr>
24             <td>Shalini</td>
25             <td>22</td>
26             <td>CSE</td>
27         </tr>
28         <tr>
29             <td>RAM</td>
30             <td>21</td>
31             <td>ECE</td>
32         </tr>
33     </table>
34 </body>
35 </html>

```

Output:

This is Border Attribute		
HTML table border Attribute		
Author Details		
NAME	AGE	BRANCH
Shalini	22	CSE
RAM	21	ECE

- **Cellpadding:** The HTML `<table>` **cell padding Attribute** is used to specify the space between the cell content and cell wall. The cellpadding attribute is set in terms of pixels. By default, the padding is set to 0.

Syntax: `<table cellpadding="pixels">`

Note: The `<table>` cellpadding Attribute is not supported by [HTML5](#).

```
index.html x
index.html > html > body > h1
1 <html>
2 <head>
3   <title>
4     HTML table cellpadding Attribute
5   </title>
6 </head>
7
8 <body>
9   <h1>HTML TABLE Attribute</h1>
10
11   <h2>
12     HTML table cellpadding Attribute
13   </h2>
14
15   <table border="1" cellpadding="15">
16     <caption>Author Details</caption>
17
18     <tr>
19       <th>NAME</th>
20       <th>AGE</th>
21       <th>BRANCH</th>
22     </tr>
23     <tr>
24       <td>BITTU</td>
25       <td>22</td>
26       <td>CSE</td>
27     </tr>
28     <tr>
29       <td>RAM</td>
30       <td>21</td>
31       <td>ECE</td>
32     </tr>
33   </table>
34 </body></html>
```

Output:

HTML TABLE Attribute		
HTML table cellpadding Attribute		
Author Details		
NAME	AGE	BRANCH
BITTU	22	CSE
RAM	21	ECE

- **Cellspacing:** The cellspacing attribute in HTML is used to define the space between cells in a table. It controls the amount of space, in pixels, between the table cells. This spacing can make the table's content more readable by adding extra room between cells.

Syntax: <table cellspacing="pixels">

```
index.html > html > head > title
1  <html>
2  /
3
4  <body>
5
6      <h1 style="color: green;">
7          HTML table Attribute
8      </h1>
9
10     <h2>HTML table cellspacing Attribute</h2>
11
12     <table border="1" cellspacing="15">
13         <caption>
14             Author Details
15         </caption>
16
17         <tr>
18             <th>NAME</th>
19             <th>AGE</th>
20             <th>BRANCH</th>
21         </tr>
22         <tr>
23             <td>BITTU</td>
24             <td>22</td>
25             <td>CSE</td>
26         </tr>
27         <tr>
28             <td>RAM</td>
29             <td>21</td>
30             <td>ECE</td>
31         </tr>
32     </table>
33 </body></html>
34
35
```

Output:

HTML table Attribute		
HTML table cellpadding Attribute		
Author Details		
NAME	AGE	BRANCH
BITTU	22	CSE
RAM	21	ECE

- **Align:** The **HTML <table> align Attribute** specify the table's alignment but CSS properties like margin and text-align are preferred for table alignment. To align content within table rows or cells, use the align attribute within [<tr>](#) and [<td>](#) tag or apply CSS styles

Syntax:

<table align="left | right | center">

Table Alignment Attributes	
Attribute	Description
left	Aligns the table to the left side of the page. This is the default setting.
right	Aligns the table to the right side of the page.
center	Aligns the table in the horizontal center of the page.

Example:

```
index.html X
index.html > html
1  <html><head>
2      <title>
3          HTML table align Attribute
4      </title>
5  </head>
6
7  <body>
8
9      <h2>HTML table align Attribute</h2>
10     <table align="left" border="1">
11         <caption>
12             Author Details
13         </caption>
14         <tr>
15             <th>NAME</th>
16             <th>AGE</th>
17             <th>BRANCH</th>
18         </tr>
19         <tr>
20             <td>BITTU</td>
21             <td>22</td>
22             <td>CSE</td>
23         </tr>
24         <tr>
25             <td>RAM</td>
26             <td>21</td>
27             <td>ECE</td>
28         </tr>
29     </table>
30 </body>
31 </html>
32
```

Output:

HTML table align Attribute		
Author Details		
NAME	AGE	BRANCH
BITTU	22	CSE
RAM	21	ECE

2.2 HTML Table: Nested Tables, Accessibility in Tables, Practical

2.2.0 Nested Tables

In HTML, tables are created using the `<table>` tag. To nest a table, put one table inside another. The outer table is your main table, while the inner table is the one you are nesting inside.

To create nested Table:

- First, you create the outer table using the `<table>` tag.
- Then, within one of the `<td>` tags (which are used to define individual cells in the table), you insert another `<table>` tag to create the inner table.

Example:


```

1  <html><head>
5  </head>
6  <body>
7      <h2>About Table</h2>
8      <p><b>Nested tables</b></p>
9
10     <table border="2">
11         <tr>
12             <td>Main Table row 1 column 1</td>
13             <td>Main Table column 2
14                 <table border="2">
15                     <tr>
16                         <td>Inner Table row 1 column 1</td>
17                         <td>Inner Table row 1 column 2</td>
18                     </tr>
19                     <tr>
20                         <td>Inner Table row 2 column 1</td>
21                         <td>Inner Table row 2 column 2</td>
22                     </tr>
23                     <tr>
24                         <td>Inner Table row 3 column 1</td>
25                         <td>Inner Table row 3 column 2</td>
26                     </tr>
27                 </table>
28             </td>
29         </tr>
30         <tr>
31             <td>Main Table row 2 column 1</td>
32             <td>Main Table row 2 column 2</td>
33         </tr>
34     </table>
35 </body>
36 </html>

```

Output:

Nested tables	
Main Table row 1 column 1	Main Table column 2
	Inner Table row 1 column 1
	Inner Table row 1 column 2
	Inner Table row 2 column 1
Main Table row 2 column 1	Inner Table row 2 column 2
	Inner Table row 3 column 1
	Inner Table row 3 column 2
Main Table row 2 column 1	Main Table row 2 column 2

2.2.1 HTML Table Accessibility

Tables help organize data visually, but **screen readers require additional information** to interpret them correctly. Without proper markup, users with visual impairments cannot understand the relationship between rows and columns as easily as sighted users. This is why **semantic HTML elements** are crucial.

Main Accessibility Features:

1. Use Semantic Tags (<thead>, <tbody>, <tfoot>)

These tags provide structure and meaning to your table:

- a) **<thead>**: Groups the header rows of the table.
- b) **<tbody>**: Contains the main body rows (data rows).
- c) **<tfoot>**: Groups the footer rows, often used for totals or summaries.

Using these tags helps assistive technologies and search engines understand the role of each section. It also improves styling and layout control with CSS.

2. Add scope Attribute for Headers

The scope attribute clarifies whether a header cell applies to a row, a column, or a group:

- a) **scope="col"** → Column header
- b) **scope="row"** → Row header
- c) **scope="colgroup"** → Multiple columns
- d) **scope="rowgroup"** → Multiple rows

This ensures that screen readers can associate the correct headers with data cells, improving accessibility for visually impaired users.

3. Use <caption> for Description

The <caption> tag provides a brief description of the table's purpose and content.

- Placed immediately after the <table> tag.
- Helps users (including those using screen readers) understand the context without reading all the data.

Example:

```
<table>
  <caption>Student Attendance Report</caption>
</table>
```

4. Add aria-describedby for Screen Readers

The `aria-describedby` attribute can link the table to additional descriptive text (like a summary or explanation) outside the table.

Example:

```
<div id="table-info">This table shows student attendance for the month
of July.</div>
<table aria-describedby="table-info">
  ...
</table>
```

5. Avoid Using Tables for Layout Purposes

Tables should **only** be used for displaying tabular data—not for page layout.

- Using tables for layout creates confusion for screen readers and breaks semantic structure.
- For layouts, use **CSS grid or flexbox** instead.

2.2.2 Practical Questions

Creating and Using Tables

1. Create a simple HTML table to display the following student data:
Name, Roll No, Subject, Marks for at least **5 students**.
2. Add a **caption** to the table describing the data.
3. Apply basic table structure with `<thead>`, `<tbody>`, and `<tfoot>` tags.

```
index.html x
index.html > html > body > table
1 <html><head>
2   <title>
3     HTML table
4   </title>
5 </head>
6 <body>
7   <table border="">
8     <caption>Student Marks Data</caption>
9     <thead>
10      <tr>
11        <th scope="col">Name</th>
12        <th scope="col">Roll No</th>
13        <th scope="col">Subject</th>
14        <th scope="col">Marks</th>
15      </tr>
16    </thead>
17    <tbody>
18      <tr><td>Amit</td><td>101</td><td>Math</td><td>90</td></tr>
19      <tr><td>Riya</td><td>102</td><td>Science</td><td>88</td></tr>
20      <tr><td>Karan</td><td>103</td><td>English</td><td>75</td></tr>
21      <tr><td>Pooja</td><td>104</td><td>Math</td><td>82</td></tr>
22      <tr><td>Rahul</td><td>105</td><td>Science</td><td>95</td></tr>
23    </tbody>
24    <tfoot>
25      <tr>
26        <td colspan="4">Total Students: 5</td>
27      </tr>
28    </tfoot>
29  </table>
30
31 </body>
32 </html>
33
```

Output:

Student Marks Data			
Name	Roll No	Subject	Marks
Amit	101	Math	90
Riya	102	Science	88
Karan	103	English	75
Pooja	104	Math	82
Rahul	105	Science	95
Total Students: 5			

Rowspan and Colspan

Create a timetable for your class using HTML tables:

1. Use rowspan for subjects that cover multiple time slots.
2. Use colspan for breaks or combined sessions.

```
index.html x
index.html > html > body > table > tbody > tr > td

 2 <html>
 3 <head>
 4
 5 </head>
 6 <body>
 7
 8     <table border="1" align="center">
 9         <caption>Class Timetable</caption>
10         <thead>
11             <tr>
12                 <th>Day</th>
13                 <th>8am-9am</th>
14                 <th>9am-10am</th>
15                 <th>10am-11am</th>
16                 <th>12pm-1pm</th>
17             </tr>
18         </thead>
19         <tbody>
20             <tr>
21                 <td>Mon</td>
22                 <td rowspan="2">Math</td>
23                 <td>Science</td>
24                 <td colspan="2">Lab</td>
25             </tr>
26             <tr>
27                 <td>Tue</td>
28                 <td>English</td>
29                 <td>Break</td>
30                 <td>History</td>
31             </tr>
32         </tbody>
33     </table>
34 </body>
35 </html>
```

Output:

Class Timetable				
Day	8am-9am	9am-10am	10am-11am	12pm-1pm
Mon	Math	Science	Lab	
Tue		English	Break	History

Nested Tables

Create a table where one of the cells contains **another table** (nested table) showing additional details.

```
index.html x
index.html > html > head > title
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Order List</title>
5
6 </head>
7 <body>
8   <table border="1" align="center">
9     <caption>Order Details</caption>
10    <tr>
11      <th>Order ID</th>
12      <th>Customer</th>
13      <th>Items</th>
14    </tr>
15    <tr>
16      <td>101</td>
17      <td>Amit</td>
18      <td>
19        <table>
20          <tr><th>Item</th><th>Qty</th></tr>
21          <tr><td>Laptop</td><td>1</td></tr>
22          <tr><td>Mouse</td><td>2</td></tr>
23        </table>
24      </td>
25    </tr>
26  </table>
27 </body>
28 </html>
29
30
```

Output:

Order Details								
Order ID	Customer	Items						
101	Amit	<table><tr><th>Item</th><th>Qty</th></tr><tr><td>Laptop</td><td>1</td></tr><tr><td>Mouse</td><td>2</td></tr></table>	Item	Qty	Laptop	1	Mouse	2
Item	Qty							
Laptop	1							
Mouse	2							

2.3 HTML Form: The <form> Element, Input Elements, Label and Field set

2.3.0 HTML Form

Forms add the ability to web pages to not only provide the person viewing the document with dynamic information but also to obtain information from the person viewing it, and process the information.

Objectives: Upon completing this section, you should be able to

- Create a FORM.
- Add elements to a FORM.
- Define CGI (Common Gateway Interface).
- Describe the purpose of a CGI Application.
- Specify an action for the FORM.
- Forms work in all browsers.
- Forms are Platform Independent.

To insert a form we use the <FORM></FORM> tags. The rest of the form elements must be inserted in between the form tags.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML Form</title>
</head>
<body>
  <!--form start-->
  <form name="" method="" action="" target="">
  </form>
  <!--form end-->
</body>
</html>
```

2.3.1 Form Element Attributes

- **ACTION:** It is the URL of the CGI (Common Gateway Interface) program that is going to accept the data from the form, process it, and send a response back to the browser.
- **METHOD:** GET (default) or POST specifies which HTTP method will be used to send the form's contents to the web server. The CGI application should be written to accept the data from either method.
- **NAME:** It is a form name used by VBScript or JavaScript.
- **TARGET:** It is the target frame where the response page will show up.

2.3.2 Form Input Elements

HTML form input elements are interactive controls used within an HTML <form> element to collect user input.

- Form elements have properties: Text boxes, Password boxes, Checkboxes, Option (Radio) buttons, Submit, Reset, File, Hidden and Image.
-

The properties are specified in the TYPE Attribute of the HTML element <INPUT></INPUT>.

```
<!DOCTYPE html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>HTML Form</title>

  </head>

  <body>

    <!--form start-->

    <form name="" method="" action="" target="">

      <input

        type="" name="" value="" checked

        size="" maxlength="" minlength="" required />

    </form> <!--form end-->

  </body></html>
```


<INPUT> Element's Properties	
TYPE	Type of INPUT entry field
NAME	Variable name passed to CGI application
VALUE	The data associated with the variable name to be passed to the CGI application
CHECKED	Button/box checked
SIZE	Number of visible characters in text field
MAXLENGTH	Maximum number of characters accepted
MINLENGTH	Minimum number of characters accepted

2.33 Form Input Elements, Label and Fieldset

Input Type	Description
<u><label></u>	It defines labels for <form> elements.
<u><input></u>	It is used to get input data from various types such as text, password, email, etc by changing its type.
<u><input type="text"></u>	Defines a one-line text input field
<u><input type="password"></u>	Defines a password field
<u><input type="submit"></u>	Defines a submit button
<u><input type="reset"></u>	Defines a reset button
<u><input type="radio"></u>	Defines a radio button
<u><input type="email"></u>	Validates that the input is a valid email address.

Input Type	Description
<u><input type="number"></u>	Allows the user to enter a number. You can specify min, max, and step attributes for range.
<u><input type="checkbox"></u>	Used for checkboxes where the user can select multiple options.
<u><input type="date"></u>	Allows the user to select a date from a calendar.
<u><input type="time"></u>	Allows the user to select a time.
<u><input type="file"></u>	Allows the user to select a file to upload.
<u><button></u>	It defines a clickable button to control other elements or execute a functionality.
<u><select></u>	It is used to create a drop-down list.
<u><textarea></u>	It is used to get input long text content.
<u><fieldset></u>	It is used to draw a box around other form elements and group the related data.
<u><legend></u>	It defines a caption for fieldset elements
<u><datalist></u>	It is used to specify pre-defined list options for input controls.
<u><output></u>	It displays the output of performed calculations.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML Form</title>
</head>
<body>
  <form>
    <fieldset>
      <legend>User Personal Information</legend>
      <label for="name">Enter your full name:</label>
      <input type="text" id="name" name="name" required /><br/>
      <label for="email">Enter your email:</label>
      <input type="email" id="email" name="email" required /><br/>
      <label for="password">Enter your password:</label>
      <input type="password" id="password" name="pass" required /><br/>
      <label for="confirmPassword">Confirm your password:</label>
      <input type="password" id="confirmPassword" name="confirmPass" required /> <br/>
    </fieldset>
    <select>
      <!-- option tag starts -->
      <option>Choose an option</option>
      <option value="html">HTML</option>
      <option value="java">JAVA</option>
      <option value="C++">C++</option>
      <option value="php">PHP</option>
      <!-- option tag ends -->
    </select>
```

```
<label>Enter your gender:</label>

  <div class="gender-group">

    <input type="radio" name="gender" value="male" id="male" required />

    <label for="male">Male</label>

    <input type="radio" name="gender" value="female" id="female" />

    <label for="female">Female</label>

    <input type="radio" name="gender" value="others" id="others" />

    <label for="others">Others</label>

  </div>
<br/>

  <label for="dob">Enter your Date of Birth:</label>

  <input type="date" id="dob" name="dob" required /><br/>

  <label for="address">Enter your Address:</label>

  <textarea id="address" name="address" required></textarea>

<br/>

  <input type="submit" value="Submit" />

</fieldset>

</form>
</body>
</html>
```

2.4 HTML Form: Select and Option Tags, TextArea, Button Element

Input Type	Description
<code><select></code>	It is used to create a drop-down list.
<code><option></code>	It is used to define options in a drop-down list.
<code><optgroup></code>	It is used to define group-related options in a drop-down list.
<code><textarea></code>	It is used to get input long text content.
<code><input type="submit"></code>	Defines a submit button
<code><input type="reset"></code>	Defines a reset button

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML Form</title>
</head>
<body>
  <form>
    <h2>HTML optgroup Tag</h2>
    <select>
```

```

<select>
  <!-- optgroup tag starts -->
  <optgroup label="Programming Languages">
    <option value="C">C</option>
    <option value="C++">C++</option>
    <option value="Java">Java</option>
  </optgroup>

  <optgroup label="Scripting Language">
    <option value="JavaScript">JavaScript</option>
    <option value="PHP">PHP</option>
    <option value="Shell">Shell</option>
  </optgroup>
  <!-- optgroup tag ends -->
</select>
</form>
</body>
</html>

```

See above example under 2.4 for textarea and button.

2.5 HTML Form: Form Attributes, Input Validation, Accessibility and Usability

2.5.0 Form Attribute See under 2.4 titled Form Attribute.

Attributes that are unique to particular input types—or attributes which are common to all input types but have special behaviors when used on a given input type—are instead documented on those types' pages.

Attributes for the <input> element include the global HTML attributes and additionally:

Attribute	Type(s)	Description
accept	file	Hint for expected file type in file upload controls
alt	image	alt attribute for the image type. Required for accessibility
autocapitalize	all except url, email, and password	Controls automatic capitalization in inputted text.
autocomplete	all except checkbox, radio, and buttons	Hint for form autofill feature
capture	file	Media capture input method in file upload controls
checked	checkbox, radio	Whether the command or control is checked
dirname	hidden, text, search, url, tel, email	Name of form field to use for sending the element's directionality in form submission
disabled	all	Whether the form control is disabled
form	all	Associates the control with a form element
formaction	image, submit	URL to use for form submission
formenctype	image, submit	Form data set encoding type to use for form submission
formmethod	image, submit	HTTP method to use for form submission
formnovalidate	image, submit	Bypass form control validation for form submission
formtarget	image, submit	Browsing context for form submission
height	image	Same as height attribute for ; vertical dimension
list	all except hidden, password, checkbox, radio, and buttons	Value of the id attribute of the <datalist> of autocomplete options

<u>max</u>	date, month, week, time, datetime - local, number, range	Maximum value
<u>maxlength</u>	text, search, url, tel, email, password	Maximum length (number of characters) of value
<u>min</u>	date, month, week, time, datetime - local, number, range	Minimum value
<u>minlength</u>	text, search, url, tel, email, password	Minimum length (number of characters) of value
<u>multiple</u>	email, file	Boolean. Whether to allow multiple values
<u>name</u>	all	Name of the form control. Submitted with the form as part of a name/value pair
<u>pattern</u>	text, search, url, tel, email, password	Pattern the value must match to be valid
<u>placeholder</u>	text, search, url, tel, email, password, number	Text that appears in the form control when it has no value set
<u>popovertarget</u>	button	Designates an <input type="button"> as a control for a popover element
<u>popovertargetaction</u>	button	Specifies the action that a popover control should perform

readonly	all except hidden, range, color, checkbox, radio, and buttons	Boolean. The value is not editable
required	all except hidden, range, color, and buttons	Boolean. A value is required or must be checked for the form to be submittable
size	text, search, url, tel, email, password	Size of the control
src	image	Same as src attribute for ; address of image resource
step	date, month, week, time, datetime-local, number, range	Incremental values that are valid
type	all	Type of form control
value	all except image	The value of the control. When specified in the HTML, corresponds to the initial value
width	image	Same as width attribute for

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML Form</title>
  </head>
  <body>
    <form action="page.html" method="post">
      <label>
        Fruit:
        <input type="text" name="fruit" dirname="fruit-dir" value="cherry" />
      </label>
      <input type="submit" />
    </form>
  </body>
</html>
```

2.5.1 Input Validation

Some input types automatically validate format:

- `type="email"` → must contain @ and .
- `type="url"` → must start with `http://` or `https://`
- `type="number"` → must be numeric
- `type="date"` → must be a valid date

List of Validation Attributes:

- `required` → field must be filled

- min / max → numeric/date range
- minlength / maxlength → character length limits
- pattern → custom regex rule
- step → numeric intervals
- type → built-in format validation
- novalidate → disable validation

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML Form</title>
  </head>
  <body>
    <form action="page.html" method="post">
      <label>Email:</label>
      <input type="email" name="email" required><br/>
      <label>Age (18-60):</label>
      <input type="number" name="age" min="18" max="60" required><br/>
      <label>Username (5-10 chars):</label>
      <input type="text" name="username" minlength="5" maxlength="10" required><br/>
      <label>Only Letters:</label>
      <input type="text" name="letters" pattern="[A-Za-z]+" required><br/>
      <label>Even Number:</label>
      <input type="number" name="even" step="2" required><br/>
      <label>Website:</label><input type="url" name="website" required><br/>
      <input type="submit" />
    </form>
    <!-- <form novalidate>
      <input type="email" name="email" required>
      <input type="submit"></form> -->
  </body>
</html>

```

2.5.2 Accessibility and Usability

When including inputs, it is an accessibility requirement to add labels alongside. This is needed so those who use assistive technologies can tell what the input is for. Also, clicking or touching a label gives focus to the label's associated form control. This improves the accessibility and usability for sighted users, increases the area a user can click or touch to activate the form control. This is especially useful (and even needed) for radio buttons and checkboxes, which are tiny.

See above example under 2.4 and 2.5 how to associate the `<label>` with an `<input>` element.

2.6 What is Git and why use it?, Git setup and config (git config), Creating a local repository: git init

2.6.0 What is Git and why do we use it?

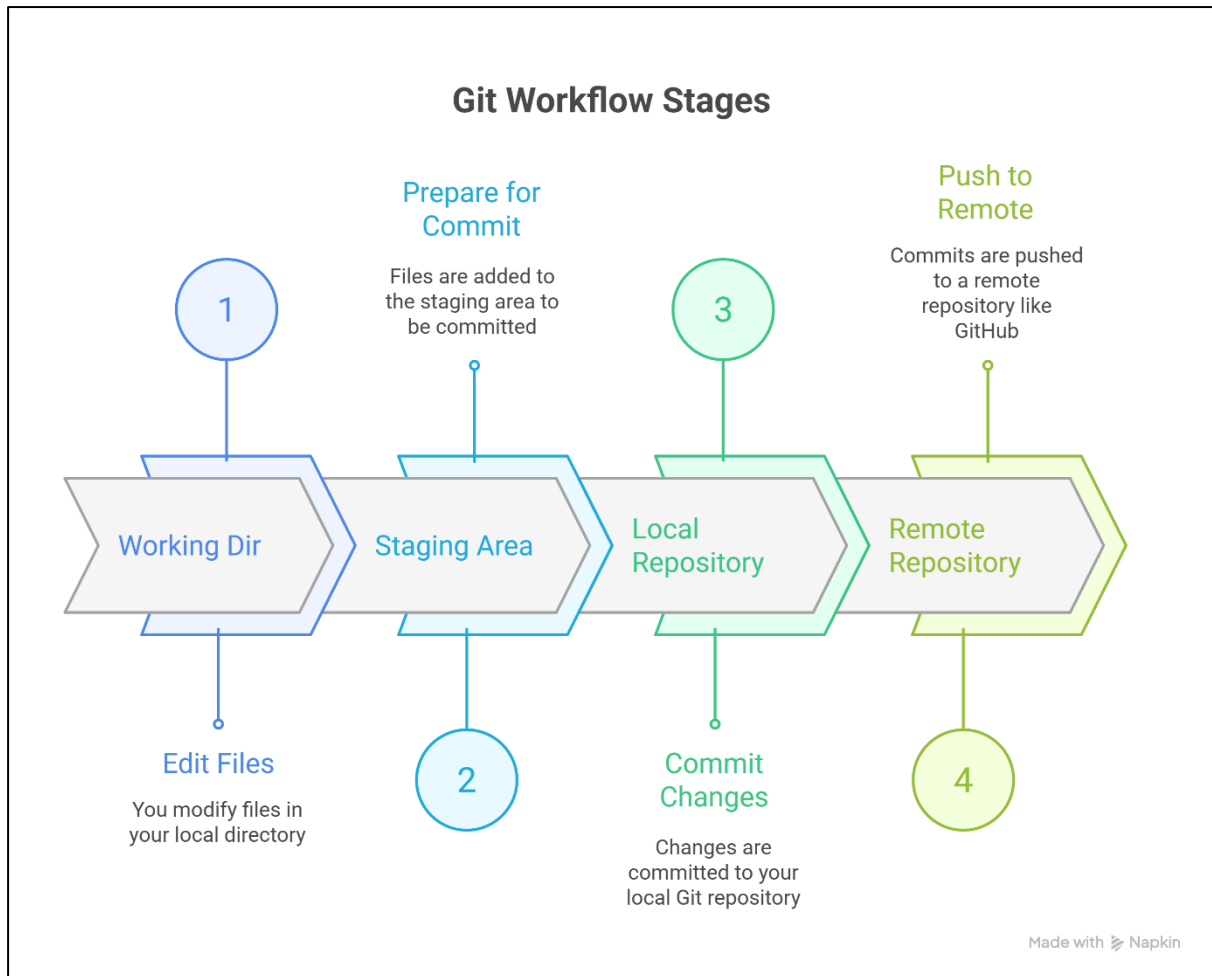
- Git is a **Version Control System (VCS)**, means Every developer has a full copy of repo.
- It helps keep track of every change you make to files in a project.
- If something breaks, you can go back to an earlier version.
- Multiple people can work on the same project without disturbing each other's work.
- Git is widely used in the software industry to manage projects and teamwork.

Key points:

- Git runs on your local computer.
- GitHub is a website (cloud) where Git projects can be stored and shared.
- Together, Git + GitHub = teamwork + backup + tracking changes.

Git has four important areas:

1. **Working Directory** – Where you actually edit files (your project folder).
2. **Staging Area (Index)** – A temporary area where you put the changes you want to commit.
3. **Local Repository** – The permanent history of commits stored on your computer.
4. **Remote Repository (GitHub/GitLab/Bitbucket)** – The copy of your repository stored on the server for sharing and collaboration.



Quick check: In VS Code, open the Terminal and run:

`git --version`

If you see a version (e.g., git version 2.45.x), you're set.

2.6.1 First-time Git setup in VS Code (one-time)

1. Open VS Code → **Terminal** → **New Terminal**.
2. Set your identity (appears in commit history):

Creating a GitHub Account

1. Go to <https://github.com/>
2. Sign up with email → verify → choose username.
3. Create a new repository:
 - Repository Name: web-project
 - Description: " Learn Git from Scratch !! "
 - Public → Add README.md → Create Repository.

github.com/new

Updated Fortinet N... how to give self intr... PB Academy GATE CS Preparatio... Master Course : Agil... AWS Certified Soluti... SOLUTION LINKS F... All Bookmarks

New repository

Type / to search

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

- General**

Owner *
Firstyearpranshivarma-Educator

Repository name *
web-project
web-project is available.

Great repository names are short and memorable. How about [ideal-octo-enigma](#)?

Description
Learn Git from Scratch !!
25 / 350 characters
- Configuration**

Choose visibility *
Choose who can see and commit to this repository
Public

github.com/new

Updated Fortinet N... how to give self intr... PB Academy GATE CS Preparatio... Master Course : Agil... AWS Certified Soluti... SOLUTION LINKS F... All Bookmarks

Great repository names are short and memorable. How about [ideal-octo-enigma](#)?

Description
Learn Git from Scratch !!
25 / 350 characters

- Configuration**

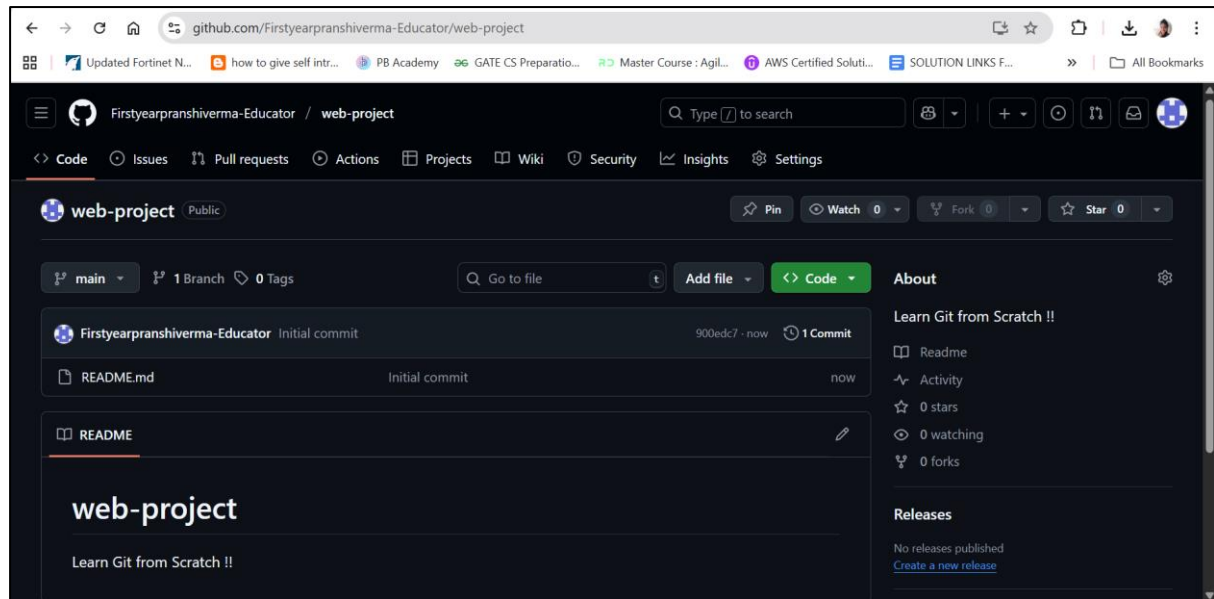
Choose visibility *
Choose who can see and commit to this repository
Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#)
On

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)
No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#)
No license

Create repository



We will use this GitHub Repo to push our project code remotely after few steps.

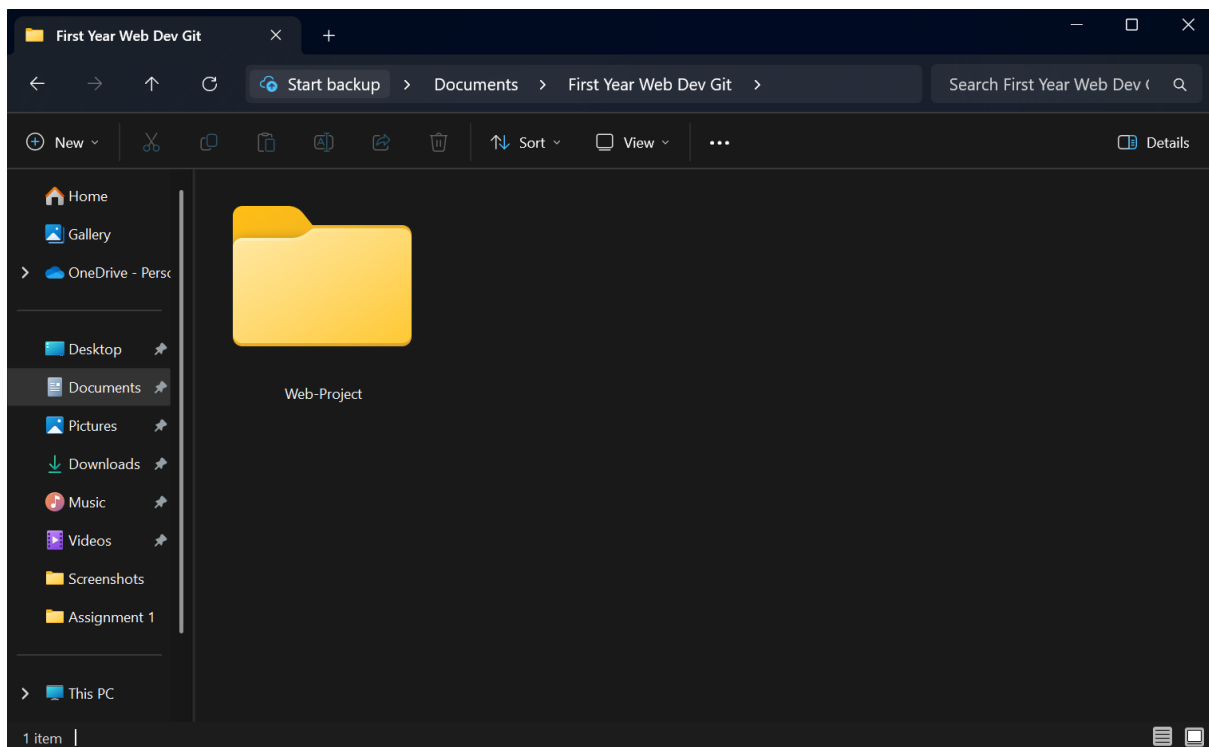
2.6.2 Creating a Local Repository

A **local repository** is a project folder on your computer that Git will track. This is the **first step** before pushing anything to GitHub.

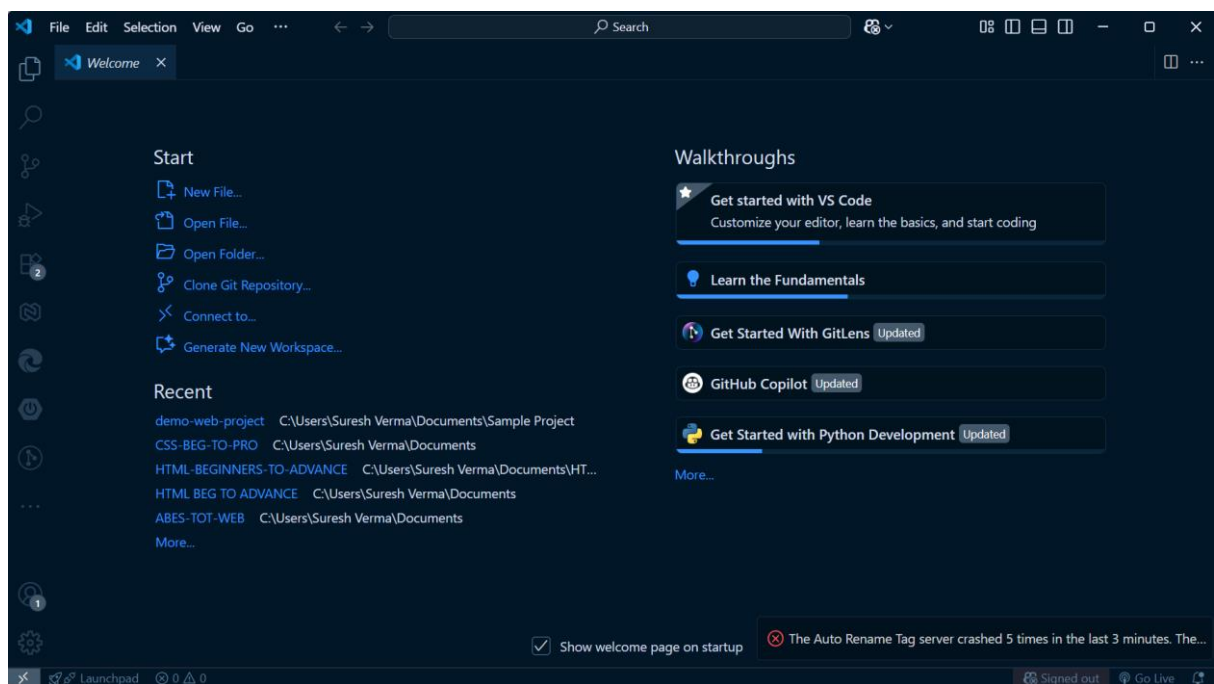
Using VS Code (Beginner-Friendly UI)

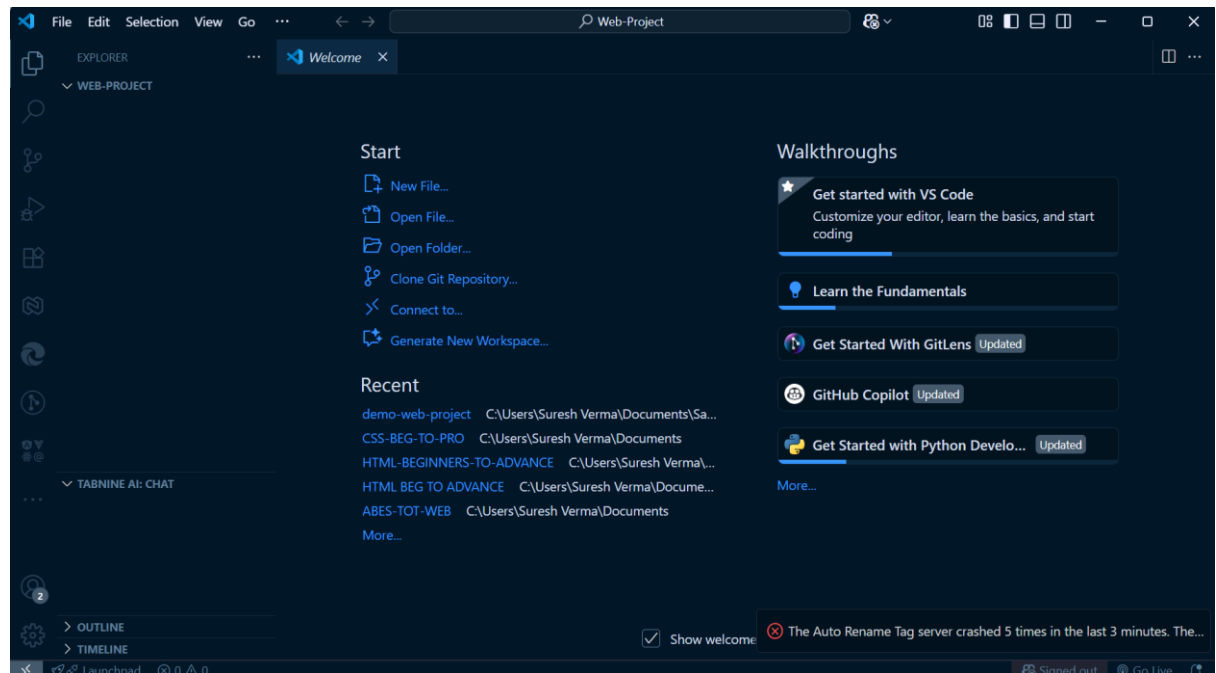
Create a project folder

Example: web-project



- Open it in VS Code (**File → Open Folder**).





Create project files

Example: index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8" />
```

```
<title>My First GitHub Project</title>
```

```
</head>
```

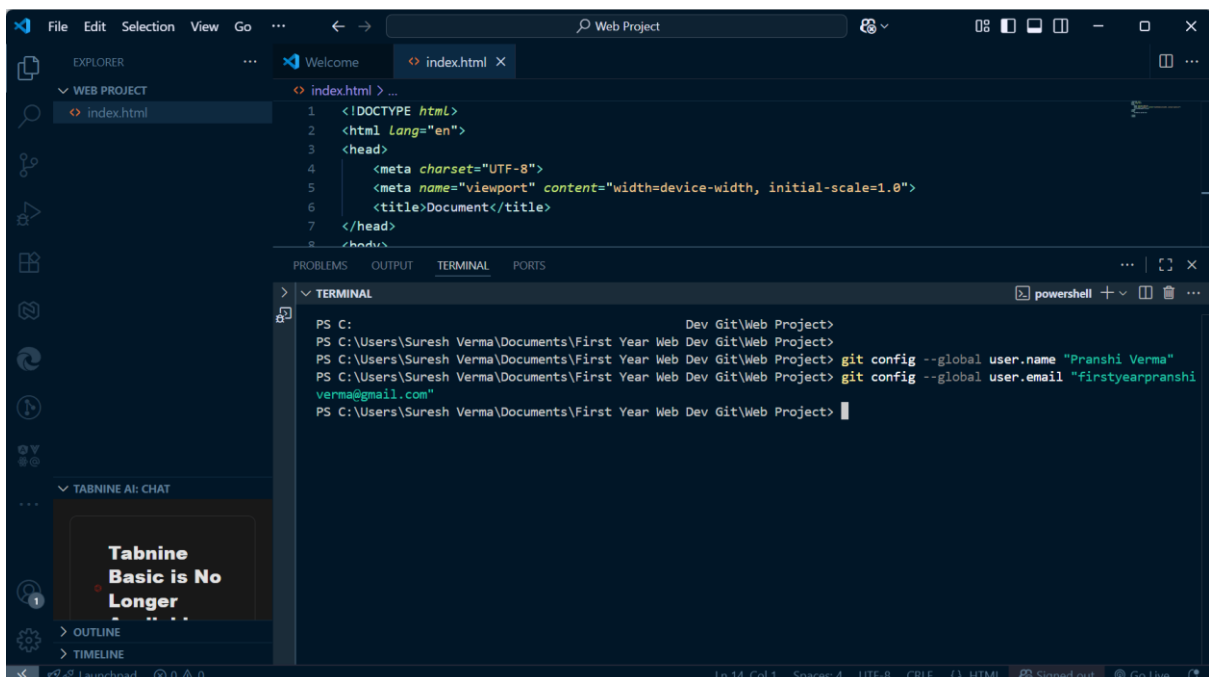
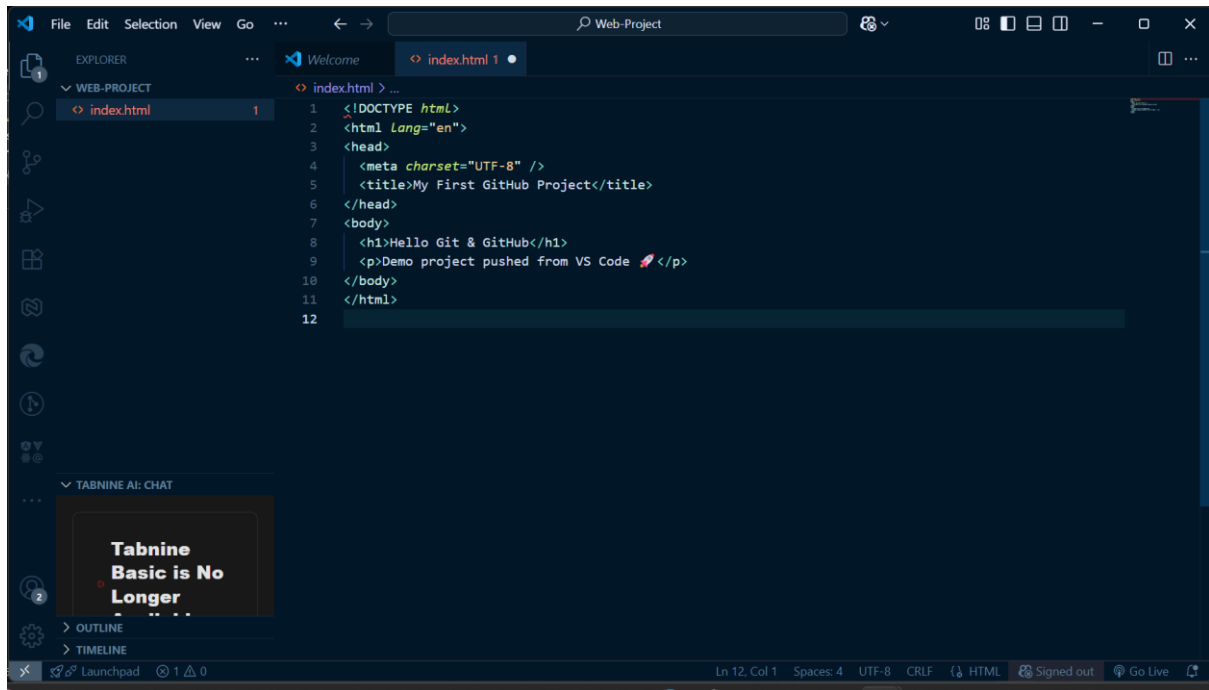
```
<body>
```

```
<h1>Hello Git & GitHub</h1>
```

<p>Demo project pushed from VS Code </p>

</body>

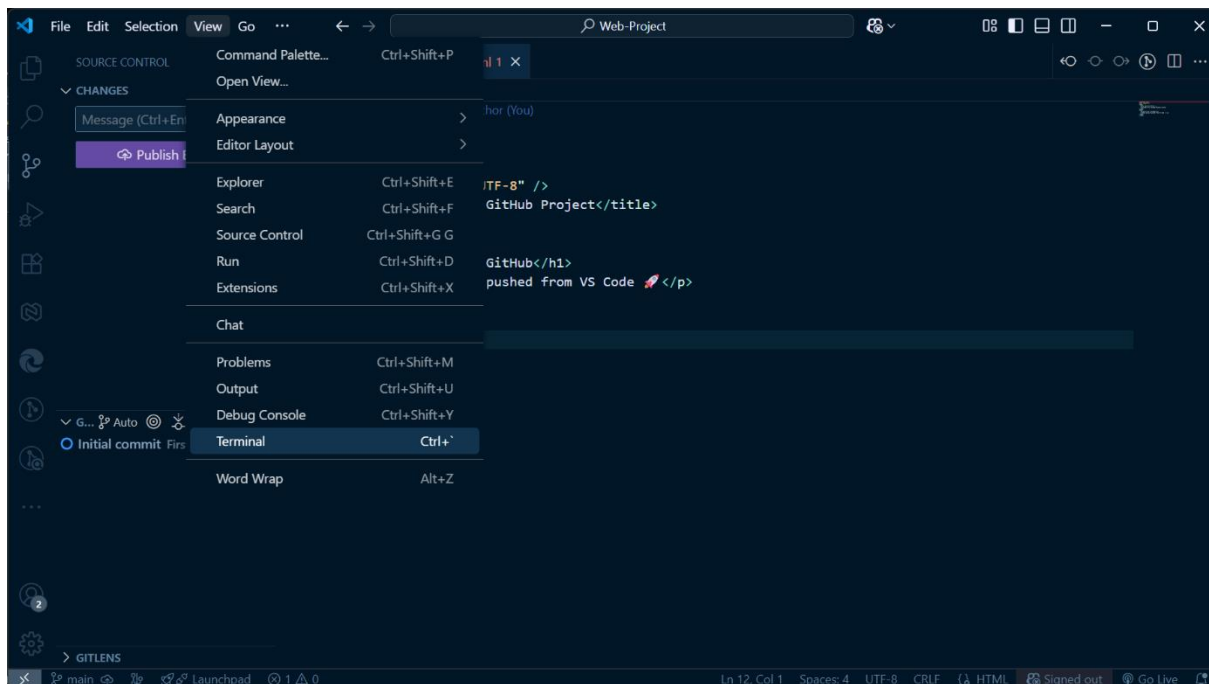
</html>



Initialize Git (create local repository)

- Where to Perform This Step?

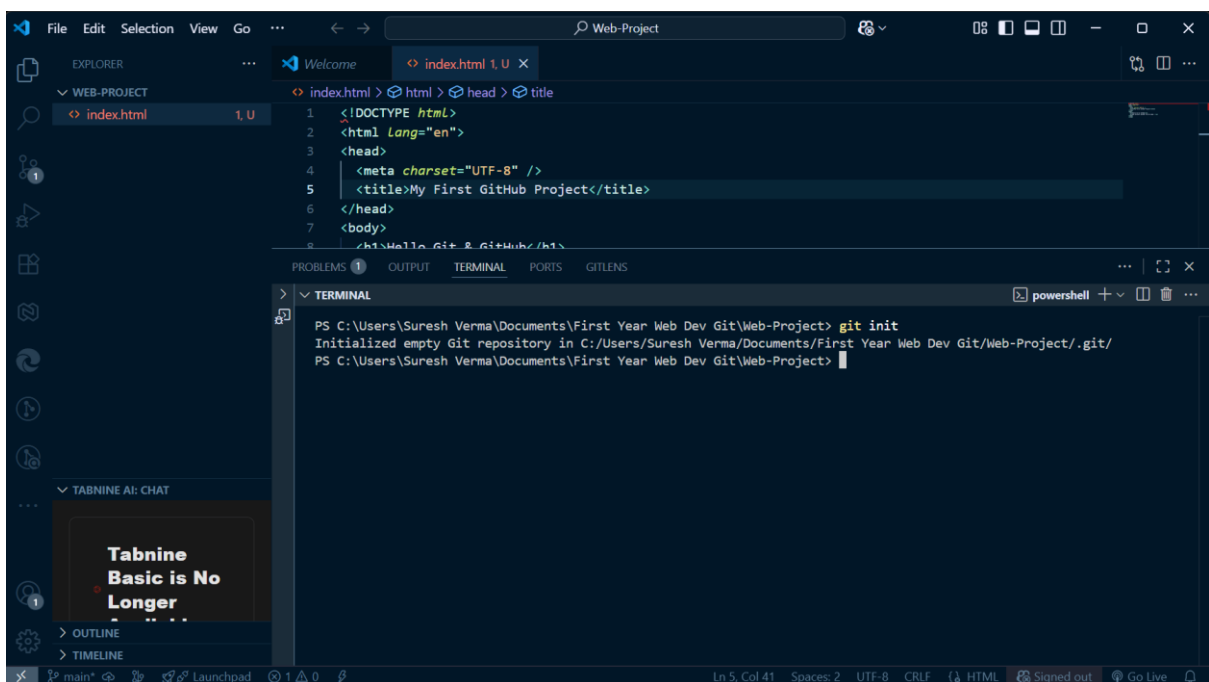
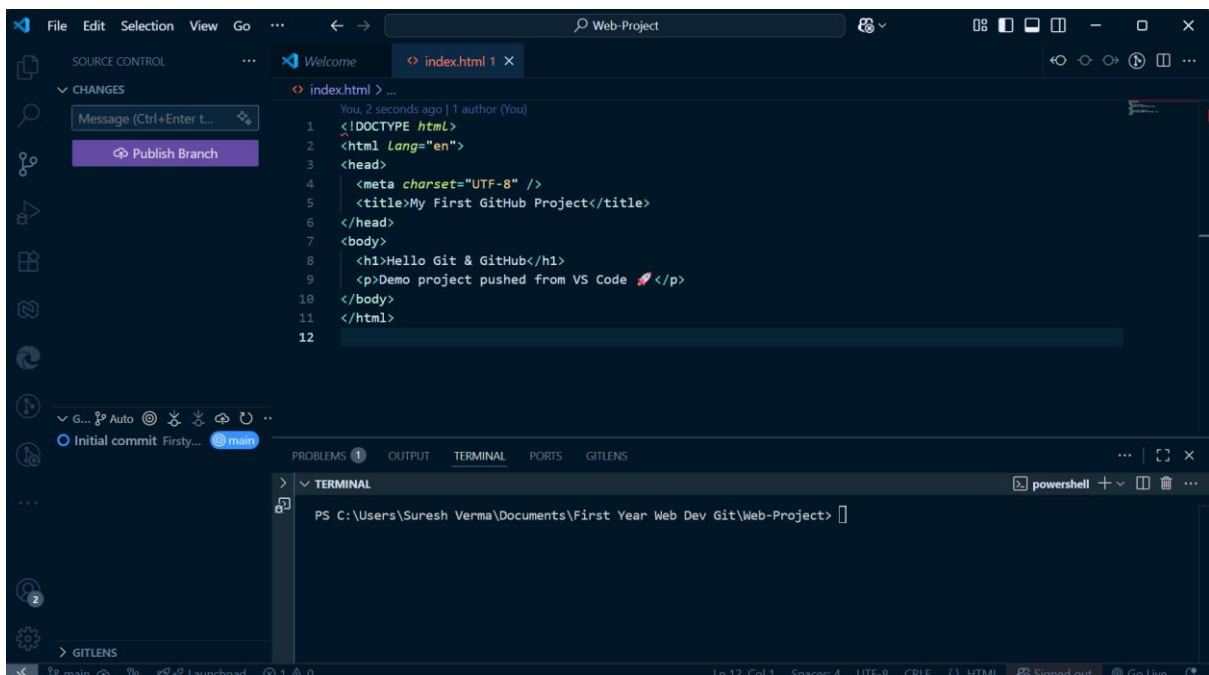
- This step is performed **in your terminal/command prompt (VS Code terminal or Git Bash)** — not on GitHub’s website.
- **In terminal:**



Use the commands:

Initialize the repository:

Using the command `git init`



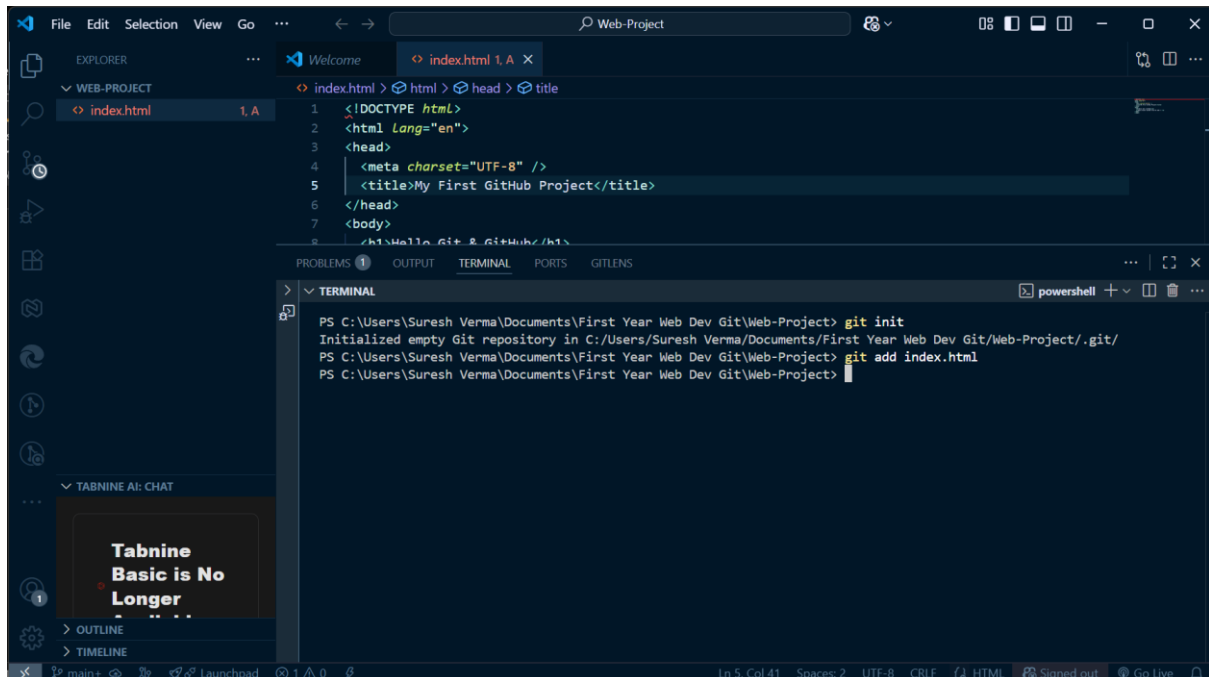
- This command creates a **hidden .git folder**, which means Git is now tracking your project.
- Add and commit files:

2.7 Tracking changes: add, commit, status, log,.gitignore basics,Practical

2.7.0 Add a Single File

If you only want to add **one file** to the staging area (for example, index.html):

`git add index.html`



Add Multiple Specific Files

You can also add more than one file at a time by naming them:

`git add index.html style.css`

Add All Files (Recommended when you want to add everything)

If you want to add **all files in the project folder**:

`git add .`

2.7.1 Commit the Changes

After adding files, you need to **commit** them with a message:

`git commit -m "Initial commit with index.html"`

```
File Edit Selection View Go ... Web-Project
EXPLORER
  WEB-PROJECT
    index.html 1
index.html 1
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <title>My First GitHub Project</title>
6 </head>
7 <body>
8   <h1>Hello Git & GitHub</h1>
9 </body>
10 </html>
11
12
TERMINAL
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git init
Initialized empty Git repository in C:/Users/Suresh Verma/Documents/First Year Web Dev Git/Web-Project/.git/
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git add index.html
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git commit -m "Initial commit with index.html"
[main (root-commit) 17532ae] Initial commit with index.html
1 file changed, 11 insertions(+)
create mode 100644 index.html
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project>
```

A **commit** is like saving a snapshot of your project at that point in time.

Now, project file is saved in local repository.

```
File Edit Selection View Go Run ... Web Project
EXPLORER
  WEB-PROJECT
    index.html 1
index.html 1
1 <html Lang="en">
2 <head>
3   <meta charset="UTF-8" />
4   <title>My First GitHub Project</title>
5 </head>
6 <body>
7   <h1>Hello Git & GitHub</h1>
8   <p>Demo project pushed from VS Code</p>
9 </body>
10 </html>
11
12
TERMINAL
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git remote add origin https://github.com/Firstyearpranshive
rma-Educator/web-project
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git remote -v
origin  https://github.com/Firstyearpranshive rma-Educator/web-project (fetch)
origin  https://github.com/Firstyearpranshive rma-Educator/web-project (push)
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project>
```

3.0 Creating a GitHub account, Connecting local repos to GitHub: remote add, push

3.0.0 Creating a GitHub Account

1. Go to <https://github.com/>
2. Sign up with email → verify → choose username.
3. Create a new repository:
 - Repository Name: web-project
 - Description: " Learn Git from Scratch !! "
 - Public → Add README.md → Create Repository.

The screenshot shows the 'Create a new repository' page on GitHub. The 'General' tab is selected. The 'Owner' is 'Firstyearpranshivarma-Educator'. The 'Repository name' is 'web-project', with a green checkmark indicating it is available. The 'Description' is 'Learn Git from Scratch !!'. The 'Configuration' tab is partially visible below.

github.com/new

New repository

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner *

Firstyearpranshivarma-Educator

Repository name *

web-project

web-project is available.

Great repository names are short and memorable. How about [ideal-octo-enigma?](#)

Description

Learn Git from Scratch !!

25 / 350 characters

2 Configuration

Choose visibility *

Public

The screenshot shows the 'Create a new repository' page on GitHub, with the 'Configuration' tab selected. The 'Description' is 'Learn Git from Scratch !!'. The 'Choose visibility' is set to 'Public'. The 'Add README' toggle is turned 'On'. The 'Add .gitignore' dropdown is set to 'No .gitignore'. The 'Add license' dropdown is set to 'No license'. A green 'Create repository' button is at the bottom.

github.com/new

Great repository names are short and memorable. How about [ideal-octo-enigma?](#)

Description

Learn Git from Scratch !!

25 / 350 characters

2 Configuration

Choose visibility *

Public

Choose who can see and commit to this repository

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

On

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

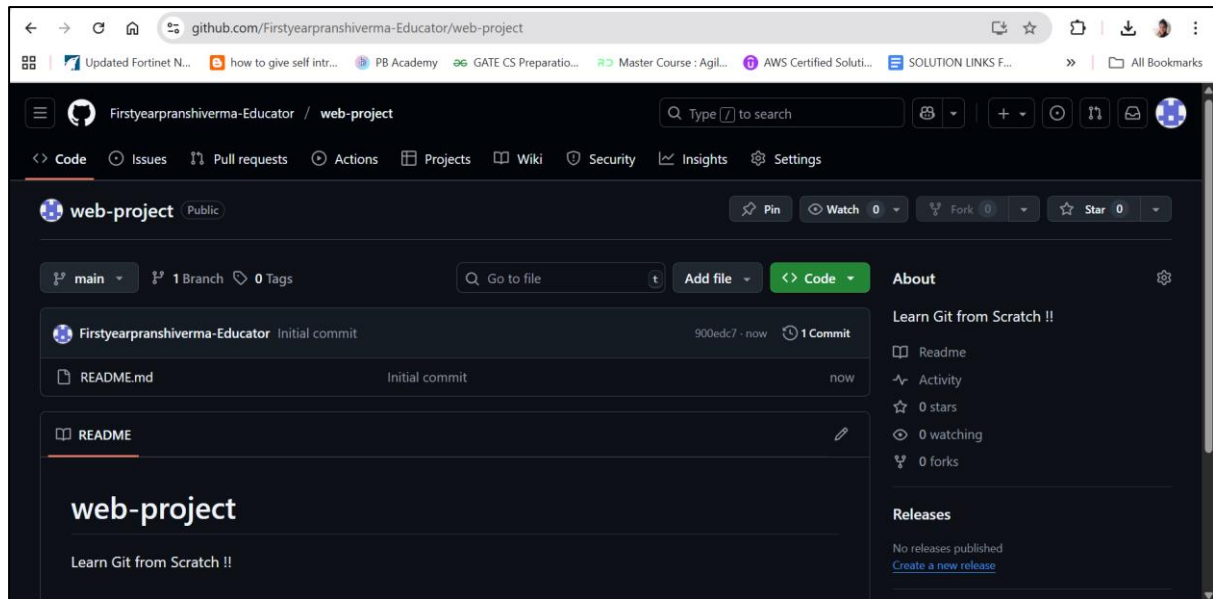
No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository



We will use this GitHub Repo to push our project code remotely after few steps.

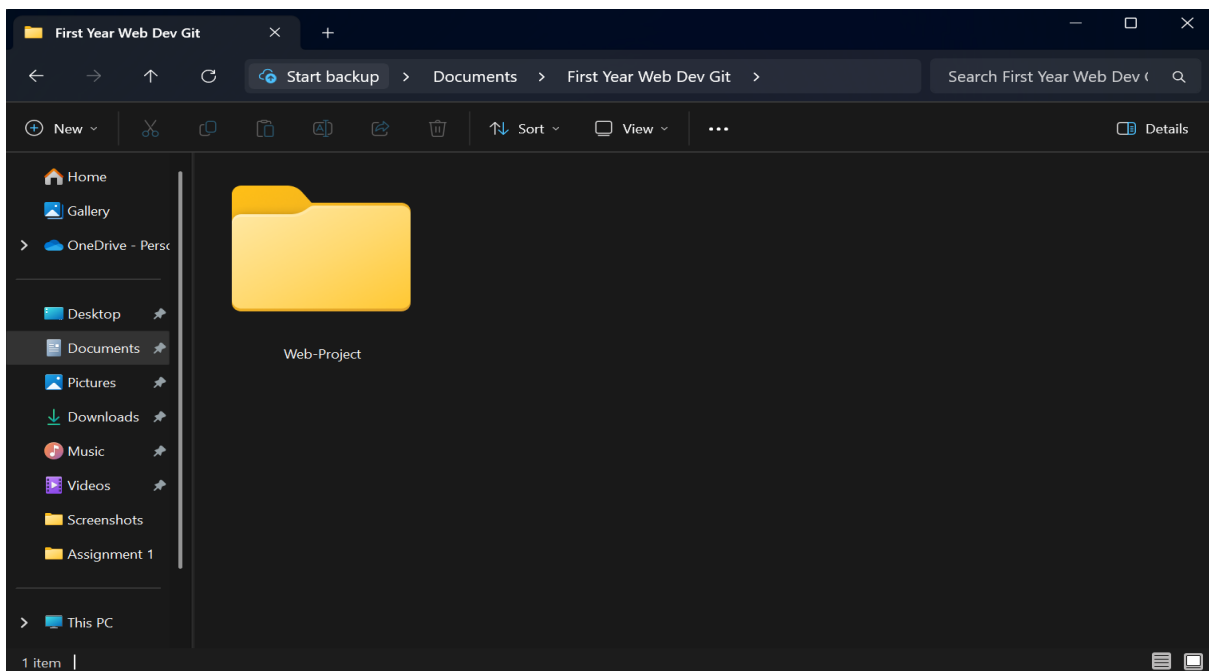
3.0.1 Creating a Local Repository

A **local repository** is a project folder on your computer that Git will track. This is the **first step** before pushing anything to GitHub.

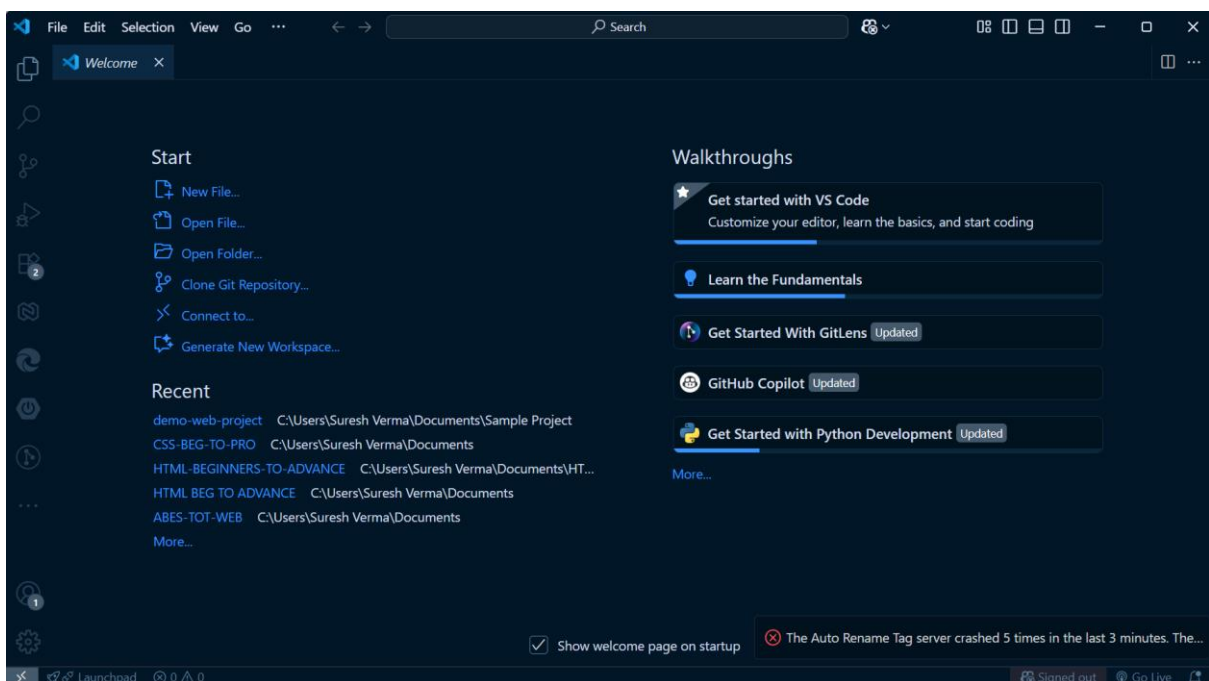
Using VS Code (Beginner-Friendly UI)

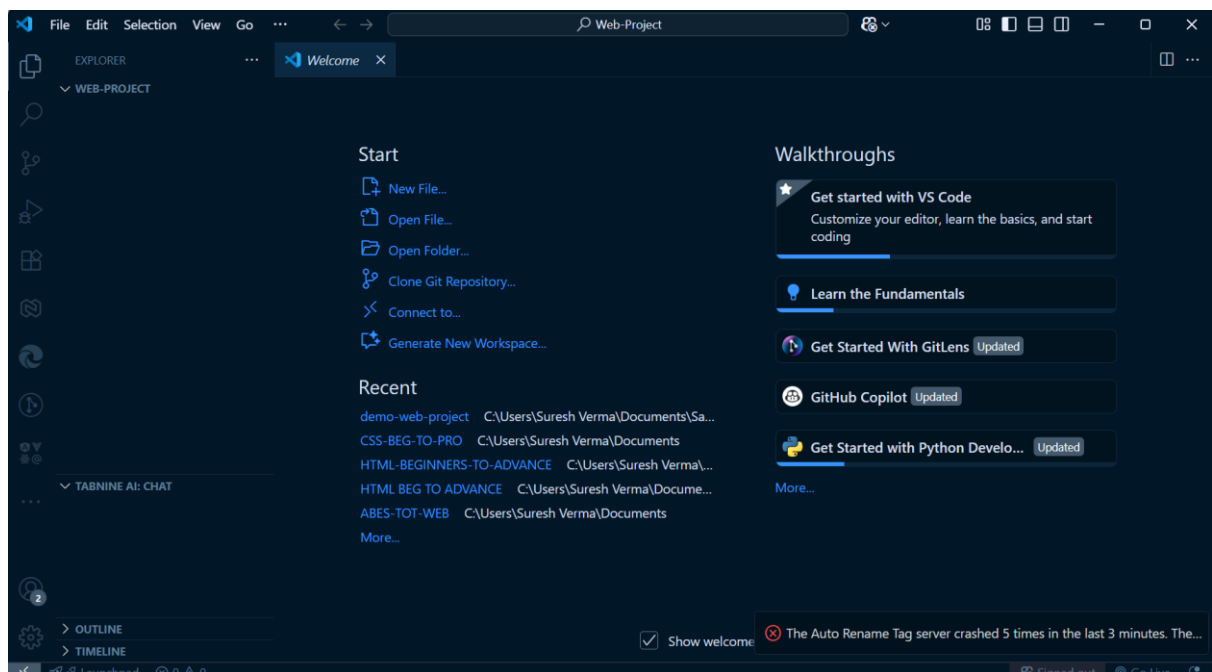
Create a project folder

Example: web-project



- Open it in VS Code (**File** → **Open Folder**).





Create project files

Example:

- index.html

index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8" />
```

```
<title>My First GitHub Project</title>
```

```
</head>
```

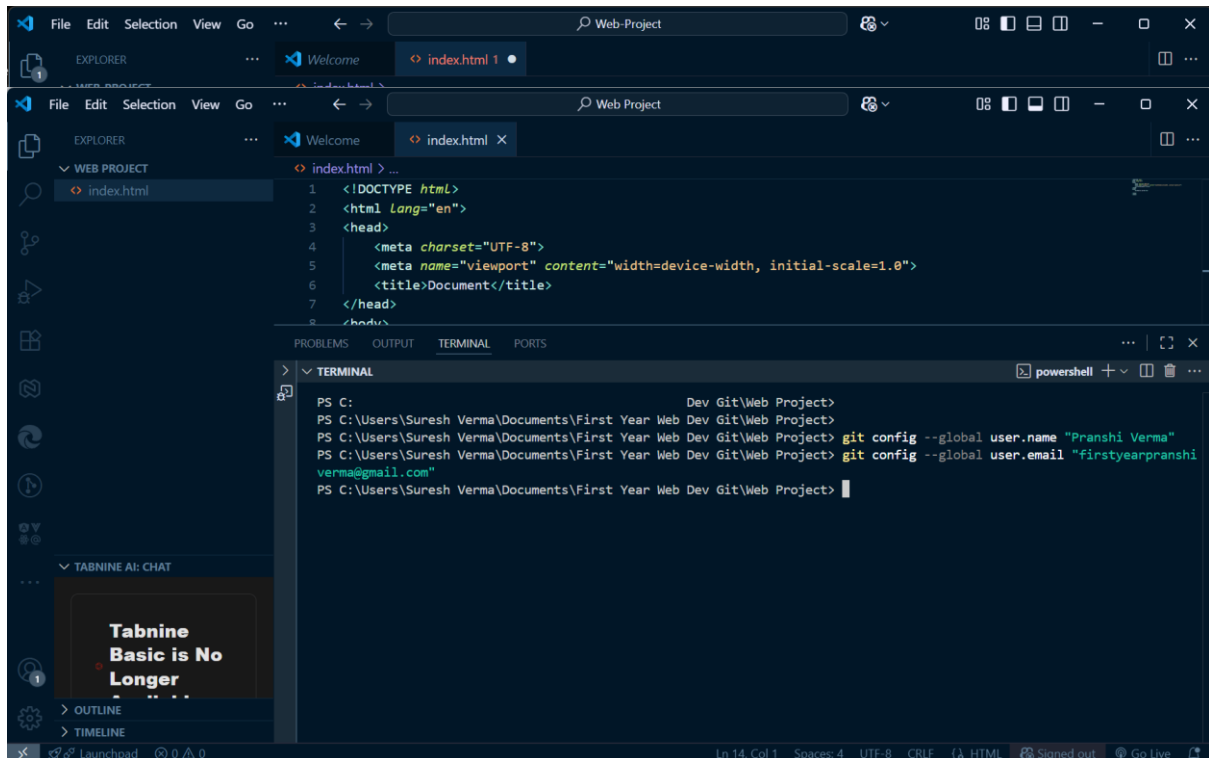
```
<body>
```

```
<h1>Hello Git & GitHub</h1>
```

<p>Demo project pushed from VS Code </p>

</body>

</html>

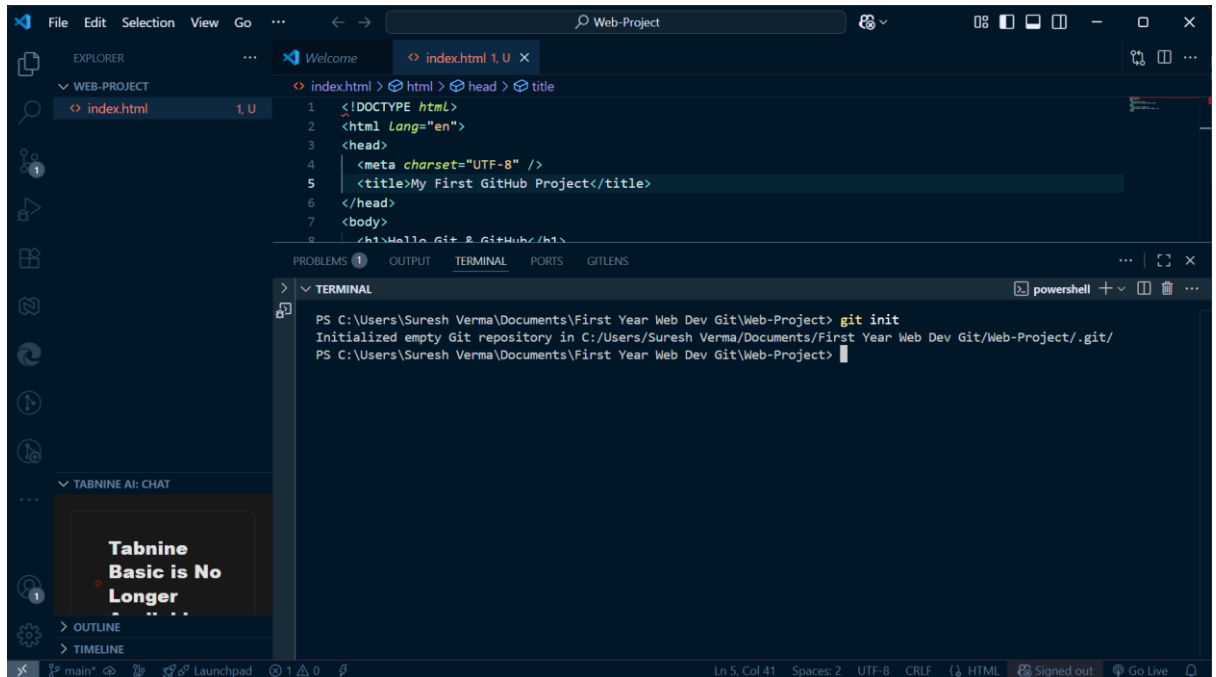


Initialize Git (create local repository)

- **Where to Perform This Step?**
- This step is performed **in your terminal/command prompt (VS Code terminal or Git Bash)** — not on GitHub's website.

In terminal:

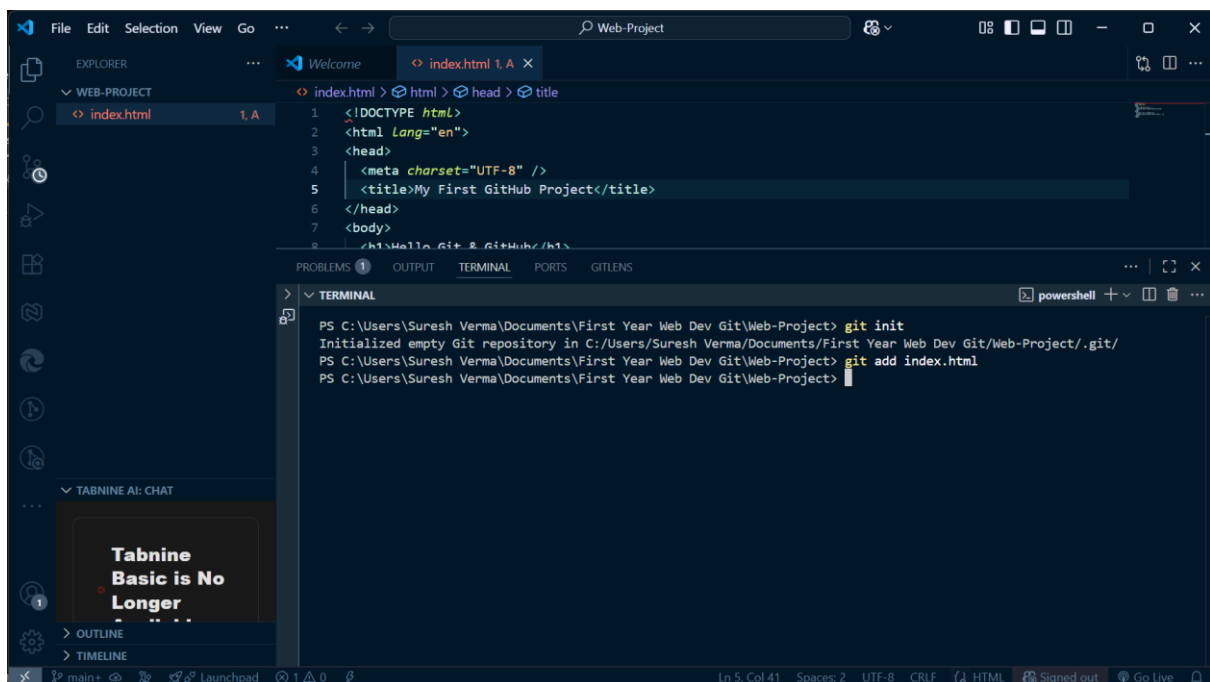
- Use the commands: Using the command `git init`
- This command creates a **hidden .git folder**, which means Git is now tracking your project.



Add a Single File

If you only want to add **one file** to the staging area (for example, index.html):

`git add index.html`



Add Multiple Specific Files

You can also add more than one file at a time by naming them:

```
git add index.html style.css
```

Add All Files (Recommended when you want to add everything)

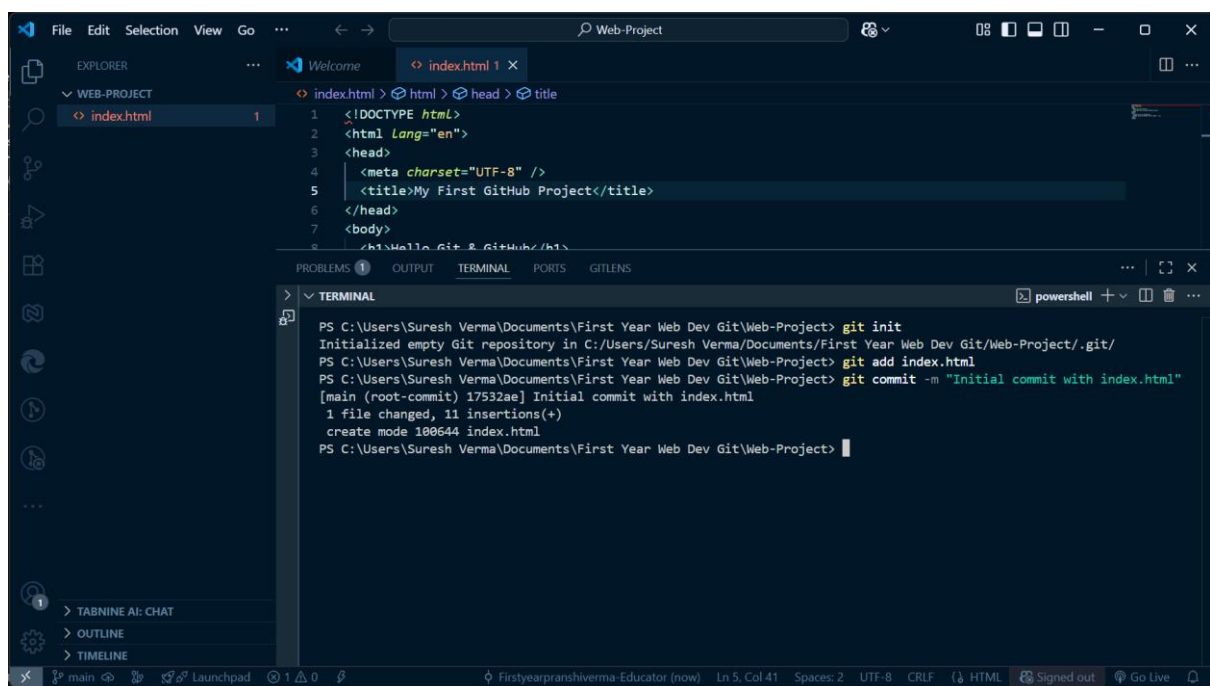
If you want to add **all files in the project folder**:

```
git add .
```

Commit the Changes

After adding files, you need to **commit** them with a message:

```
git commit -m "Initial commit with index.html"
```

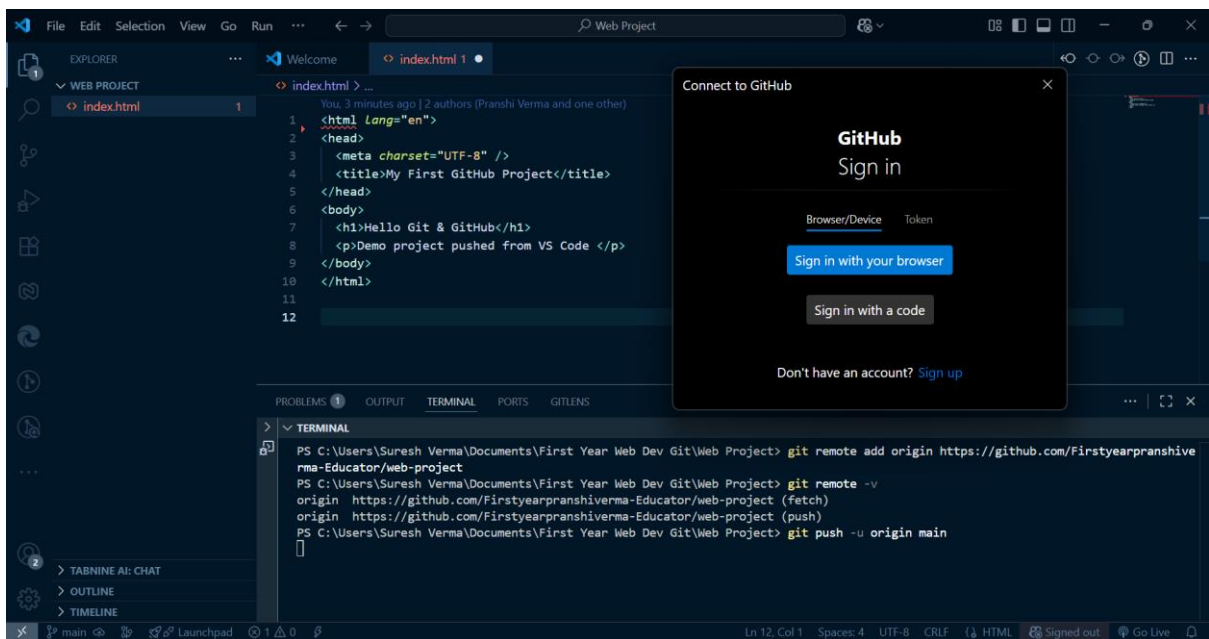
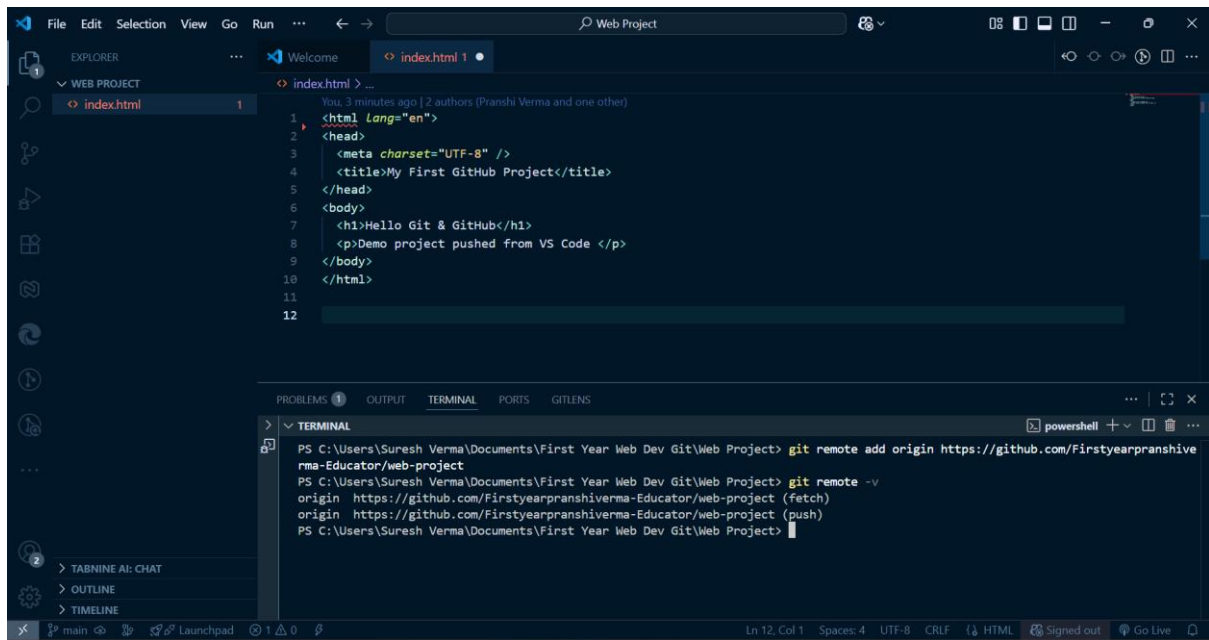


The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a folder named 'WEB-PROJECT' containing a file 'index.html'. The main editor area displays the content of 'index.html', which is an HTML file with a title 'My First GitHub Project'. The Terminal pane at the bottom shows the execution of the following commands:

```
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git init
Initialized empty Git repository in C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project\.git\
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git add index.html
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git commit -m "Initial commit with index.html"
[main (root-commit) 17532ae] Initial commit with index.html
1 file changed, 11 insertions(+)
create mode 100644 index.html
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project>
```

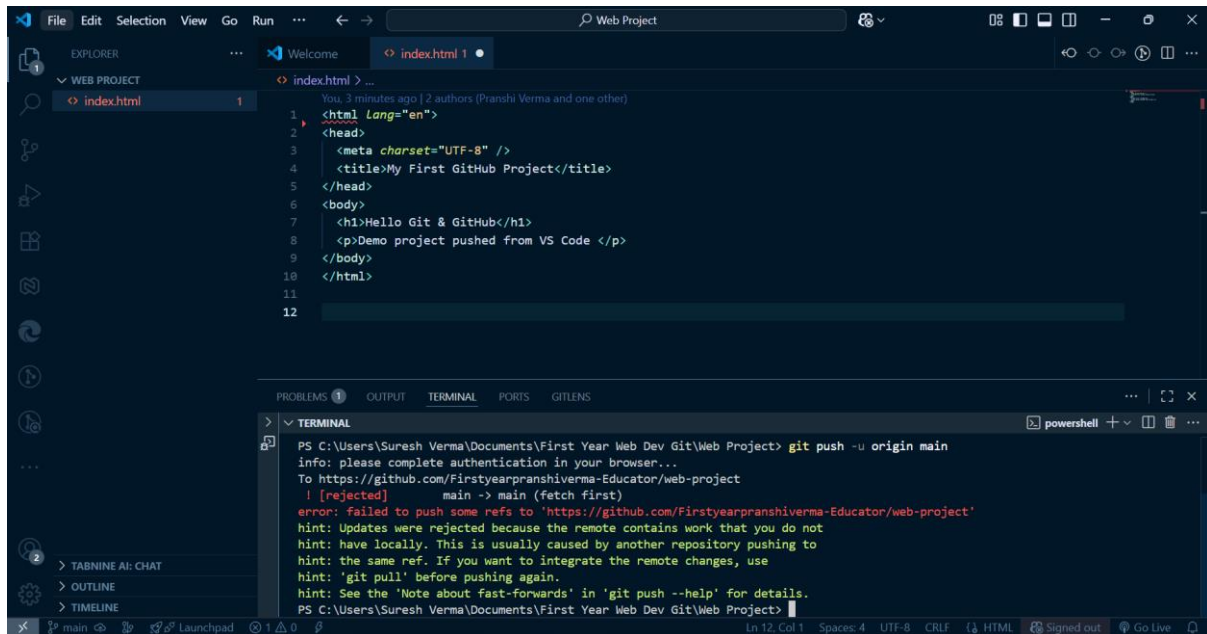
A **commit** is like saving a snapshot of your project at that point in time.

Now, project file is saved in local repository.



Pull & Sync changes from GitHub into VS Code

- **Pull:** download new commits and merge them into your local branch.
- **Fetch:** check for new commits without merging.



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'WEB PROJECT' with a file 'index.html'. The main editor displays the content of 'index.html', which is an HTML document with a title 'My First GitHub Project' and a body containing 'Hello Git & GitHub' and 'Demo project pushed from VS Code'. The bottom panel shows the TERMINAL window with a PowerShell session. The user has run 'git push -u origin main', which resulted in a 'rejected' error because the remote repository contains work that is not locally present. The error message suggests using 'git pull' before pushing again.

```
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git push -u origin main
info: please complete authentication in your browser...
To https://github.com/Firstyearpranshivarma-Educator/web-project
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/Firstyearpranshivarma-Educator/web-project'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project>
```

3.1 Cloning and pulling repositories

3.1.0 Cloning repositories

When you clone a repository, you copy the repository from GitHub.com to your local machine. You can clone a repository from GitHub.com to your local computer to make it easier to fix merge conflicts, add or remove files, and push larger commits.

3.1.1 Pulling a Repository

What is pulling?

Pulling means **fetching the latest changes** (commits, updates, new files) from the remote GitHub repository and merging them into your **local repository**.

Where does it happen?

You perform git pull inside your **local project folder** (the one you cloned earlier).

Command:

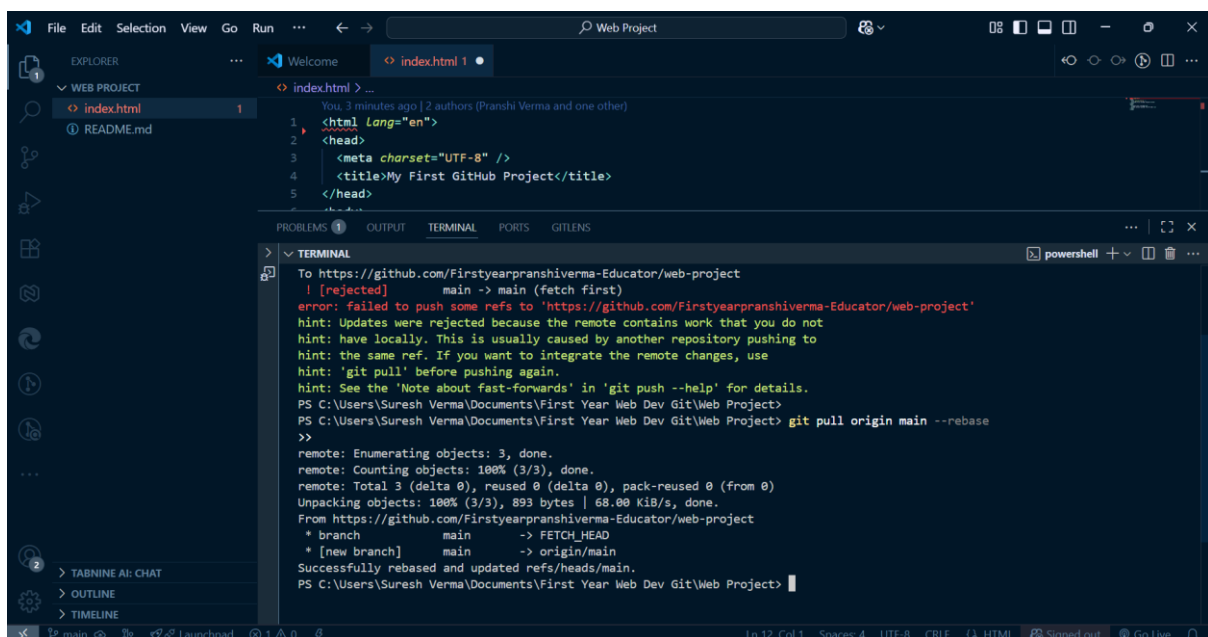
- `git pull origin main`
- `origin` refers to your GitHub repository.
- `main` is the branch name (sometimes `master` or another branch).

When do we use pull?

- If your team members (or you from another system) made changes on GitHub, you run `git pull` to **sync your local project** with the latest version.
- This avoids conflicts and keeps your work up-to-date.

In simple terms:

- **Clone** = "Download the whole project to my computer."
- **Pull** = "Update my local project with the latest changes from GitHub."

A screenshot of the Visual Studio Code interface. The Explorer pane on the left shows a 'WEB PROJECT' with files 'index.html' and 'README.md'. The editor pane shows the content of 'index.html', which is an HTML document with a title 'My First GitHub Project'. The Terminal pane at the bottom shows the output of a 'git pull' command. The output indicates that the remote repository has updates that are not in the local repository, and the local repository is successfully updated by fetching the remote changes and rebasing.

```
To https://github.com/Firstyearpranshivarma-Educator/web-project
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/Firstyearpranshivarma-Educator/web-project'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git pull origin main --rebase
>>
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 893 bytes | 68.00 KiB/s, done.
From https://github.com/Firstyearpranshivarma-Educator/web-project
* branch      main      -> FETCH_HEAD
* [new branch]      main      -> origin/main
Successfully rebased and updated refs/heads/main.
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project>
```

So Pull remote first, then push.

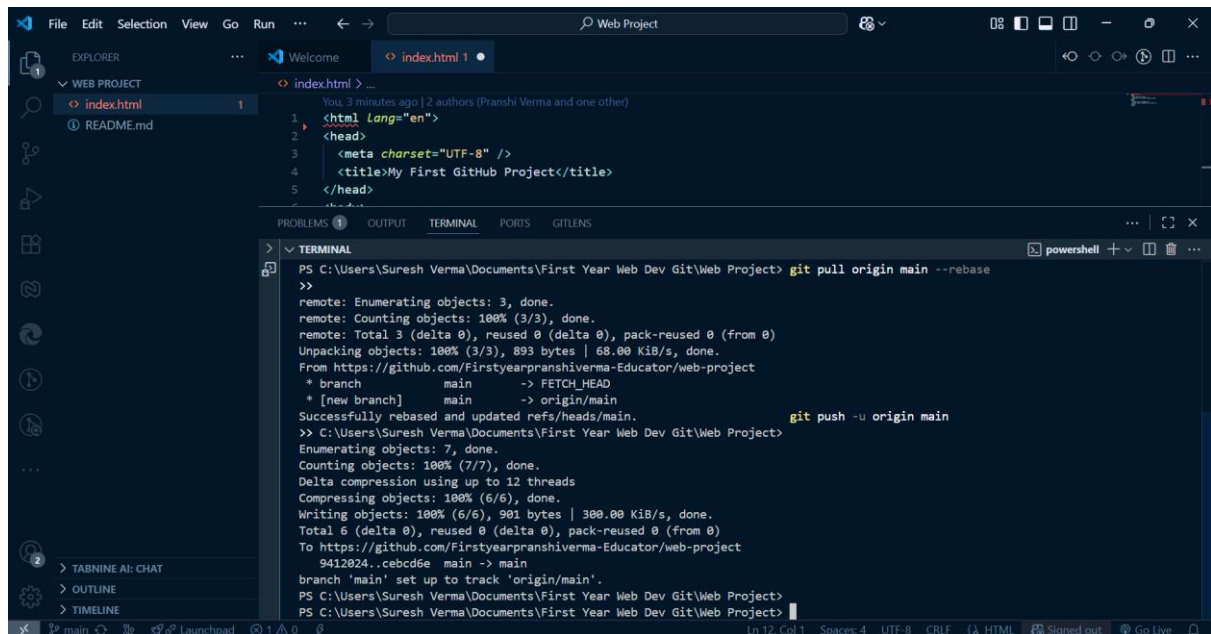
This way you combine your local and GitHub changes. Run:

- If there are no conflicts, your local commit will be placed “on top” of the remote commit.
- Then push:

Run the command:

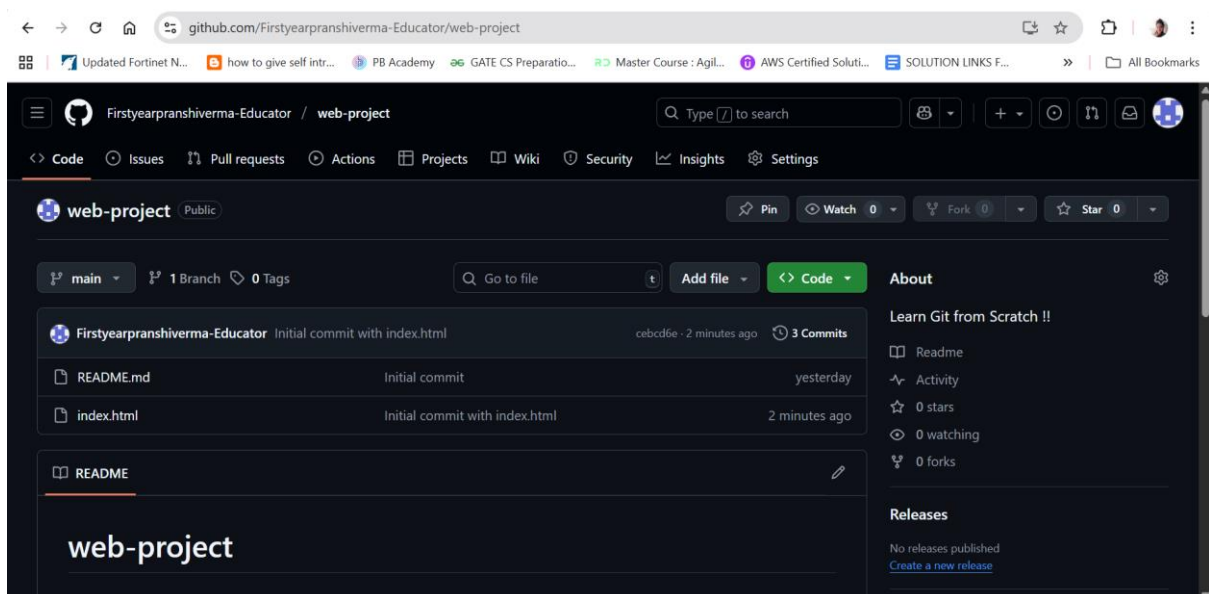
`git push -u origin main`

This means Upload my local main branch to the remote repository named origin, and set up a tracking relationship so future pushes/pulls are easier.



```
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git pull origin main --rebase
>>
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 893 bytes | 68.00 KiB/s, done.
From https://github.com/Firstyearpranshiverma-Educator/web-project
* branch      main      -> FETCH_HEAD
* [new branch] main      -> origin/main
Successfully rebased and updated refs/heads/main.
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 901 bytes | 300.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Firstyearpranshiverma-Educator/web-project
9412824..cebcd6e  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project>
```

To verify this, please refresh the GitHub repository and see the changes in files available in repository.



3.2 GitHub repository structure and README.md

3.2.0 GitHub Repository Structure

When you create or work with a GitHub repository, you will notice some important files and folders. Each has a specific purpose and makes your project more professional and maintainable:

3.2.1 README.md

What is README.md?

- A **README.md** file is written in **Markdown format** (file extension: .md).
- It provides an **introduction and overview of your project** so that visitors can quickly understand:
 - **What the project is about**
 - **How to install and run it**
 - **Features and usage**
 - **Contribution guidelines**
 - **Credits and contributors**

Example of a README.md file:

1. Project Title & Description

My Web Project

A simple web project to demonstrate Git & GitHub basics.

- The # symbol in Markdown means **Heading 1**.
 - Provide a **clear project name** and a **one-line description**.
-

2. Installation Instructions

Installation

Clone the repo:

<https://github.com/Firstyearpranshiverma-Educator/web-project.git>

Open the folder and run:
npm install (if Node.js project)

- Always guide users **step by step** on how to set up your project.
-

3. Usage Instructions

Usage

Open `index.html` in your browser.

- Show how to **run the project** after installation.
-

4. Contribution Guidelines

Contribution

Anyone can fork this repository, create a new branch, make improvements, and open a Pull Request for review.

- **Fork** → Copy someone else's repository into your own GitHub account.
 - You can **make changes freely** in your fork without affecting the original project.
 - Later, you can **create a Pull Request (PR)** to merge your changes into the original repository.
(We will discuss Fork and Pull Request in detail in upcoming lectures.)
-

5. Screenshots or Examples

Screenshots

We can add images like images/demo.png or any other demo images here.

- **Visuals** make your README attractive and easy to understand.
-

6. Credits & Contributors

##Credits & Contributors

- ****Author / Maintainer: **** [Your Name](https://github.com/your-username)
- ****Contributors: ****

- [Contributor 1] (<https://github.com/contributor1>) – Feature XYZ
- [Contributor 2] (<https://github.com/contributor2>) – Bug fixes & testing

Special thanks to:

- Open-source libraries and frameworks used in this project
- Community members who gave feedback and suggestions

Explanation:

- **Author / Maintainer** → The person who created or maintains the repo.
- **Contributor's** → People who added code, fixed bugs, improved docs, etc.
- **Special Thanks** → Credit libraries, tutorials, or external help. (*GitHub also auto-generates a contributors list based on commits, but mentioning key contributors manually is a good practice.*)

Best Practice:

- Always write a **clear and meaningful README.md** because it's the **first impression** of your project on GitHub.
 - Keep it **organized, simple, and helpful** for anyone who visits your repository.
-

2. .gitignore

- This file tells Git which files or folders **should not be tracked** or uploaded to GitHub.
- Example use cases:
 - Ignoring dependencies like `node_modules/` in Node.js projects
 - Ignoring log files (`.log`)
 - Ignoring environment configuration files (`.env`) for security
- Example .gitignore content:

```
node_modules/
```

```
.env
```

```
*.log
```

3. LICENSE

- This file specifies the **license type** under which your project is distributed.

- It defines what other developers can and cannot do with your code (e.g., MIT, Apache 2.0, GPL).
- Example:
 - MIT License → Free to use, copy, modify with attribution
 - GPL License → Derivative projects must also be open-source

Including a **LICENSE file** makes your project legally safe and encourages contributions.

4. src/ (Source Code Folder)

- The src/ directory usually contains the **main source code** of your project.
- Example structure for a web project:

src/

|— index.html

|— style.css

|— script.js

|— images/

- Keeping source files inside src/ makes your repository **organized and professional**.

Key Takeaway

- **README.md** → First impression (must be meaningful and well-written).
 - **.gitignore** → Keeps unnecessary files out of Git.
 - **LICENSE** → Defines usage rights of your project.
 - **src/** → Main code lives here (clean and organized).
-

3.3 Basic collaboration terms: fork, pull request, branch (intro)

3.3.0 Basic Collaboration Terms: Fork

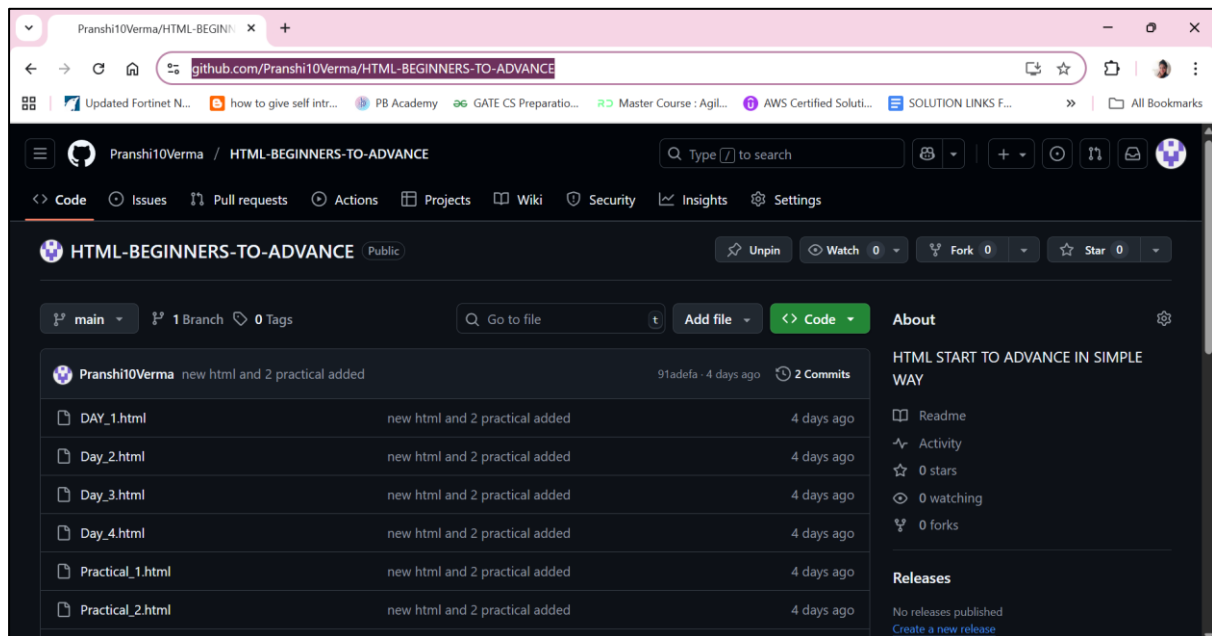
- A **Fork** is a personal copy of someone else's repository on your GitHub account.
- It allows you to experiment with changes **without affecting the original repository**.

- Commonly used in **open-source projects** where you don't have write access to the original repo.

Example:

Suppose there is a public repo:

👉 <https://github.com/Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE>



Step 1: Open the repository on GitHub

- Go to the repo link in your browser.
- At the top-right of the page, you'll see a **Fork** button.
- This button is used to create your own copy of the repo inside your GitHub account.

👉 Think of Fork as “Save a copy to my account.”

Step 2: Click on Fork

- Click the **Fork** button.
- GitHub will ask where you want to fork the repository (if you belong to multiple organizations, you can select your personal account).
- Optionally, you can rename the repository during the fork.
- Then click **Create Fork**.

github.com/Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE/fork

Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk (*).

Owner *
Firstyearpranshivarma-Educator

Repository name *
Fork HTML-BEGINNERS-TO-ADVANCE
Your new repository will be created as Fork-HTML-BEGINNERS-TO-ADVANCE.
✓ The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description (optional)
HTML START TO ADVANCE IN SIMPLE WAY

☒ Copy the `main` branch only
Contribute back to Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE by adding your own branch. [Learn more.](#)

📘 You are creating a fork in your personal account.

Create fork

You can click **Fork** (on GitHub UI) → it creates a copy in your account:

github.com/Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE/fork

Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk (*).

Owner *
Firstyearpranshivarma-Educator

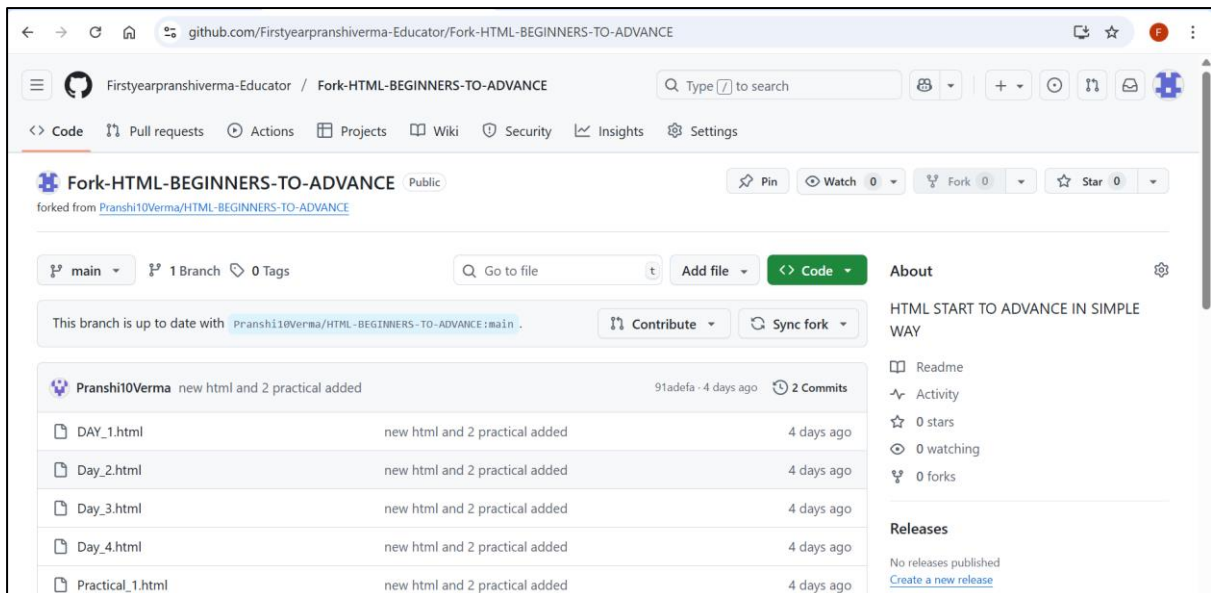
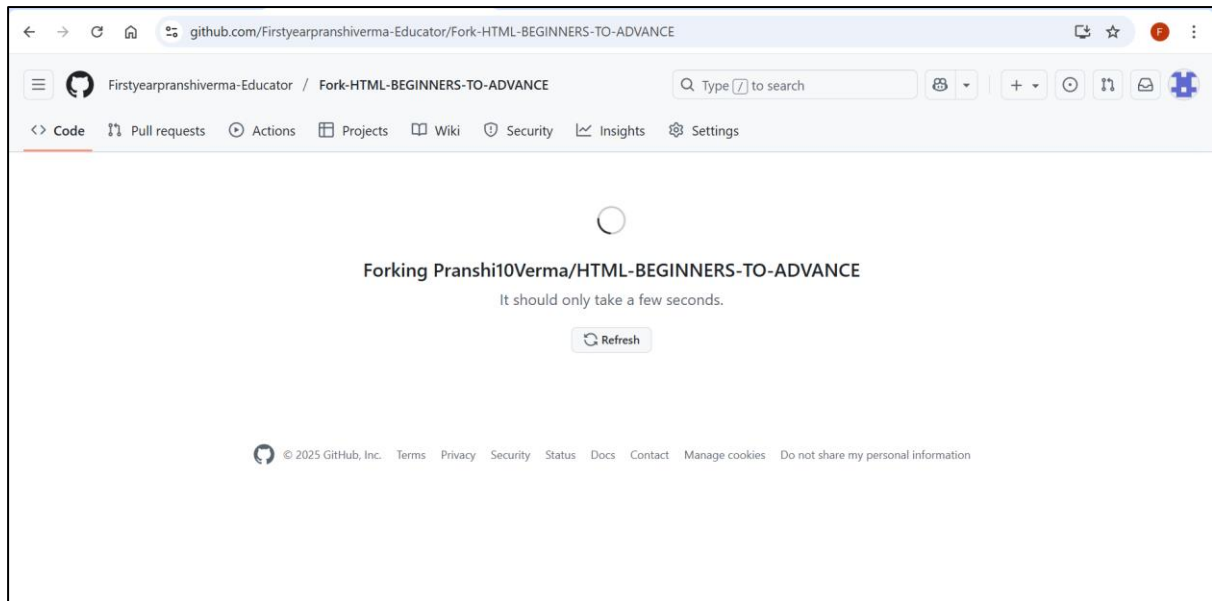
Repository name *
Fork HTML-BEGINNERS-TO-ADVANCE
Your new repository will be created as Fork-HTML-BEGINNERS-TO-ADVANCE.
✓ The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description (optional)
HTML START TO ADVANCE IN SIMPLE WAY

☒ Copy the `main` branch only
Contribute back to Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE by adding your own branch. [Learn more.](#)

📘 You are creating a fork in your personal account.



 <https://github.com/Firstyearpranshiverma-Educator/Fork-HTML-BEGINNERS-TO-ADVANCE>

Now you can make changes safely in your copy.

3.3.1 Basic Collaboration Terms: Pull Request

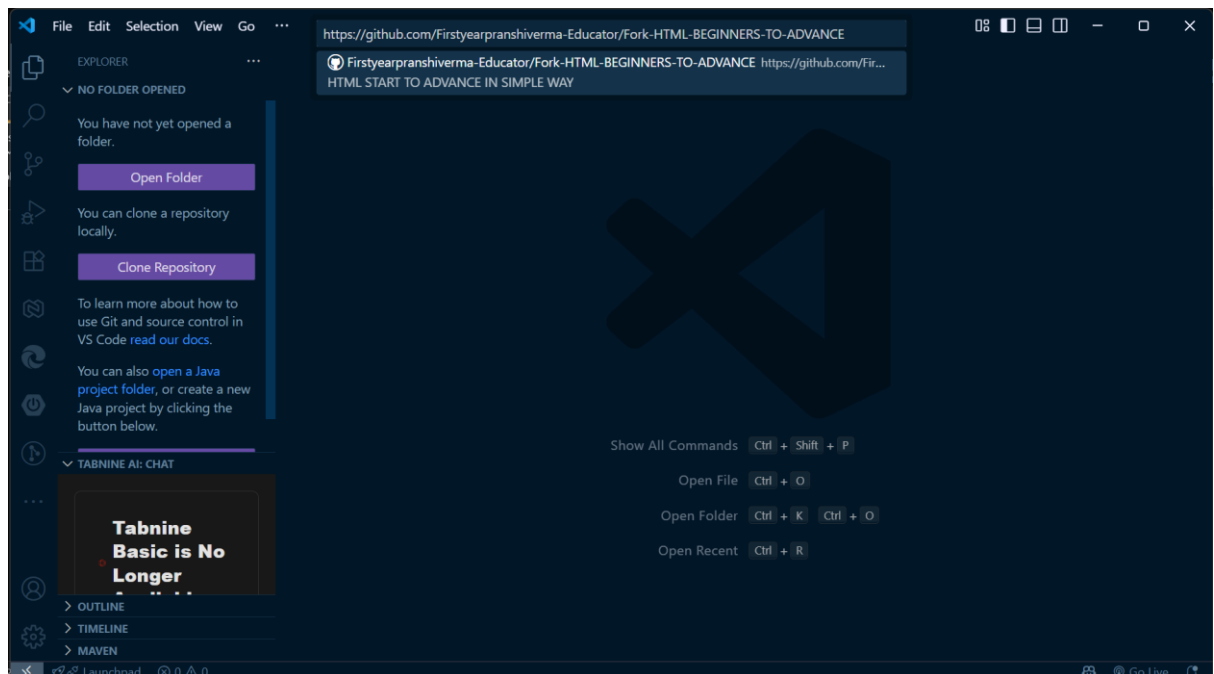
What is Pull Request (PR)?

A **Pull Request** is a request to merge your changes from your fork (or branch) back into the original repository.

- It's the main way developers contribute to open-source projects.

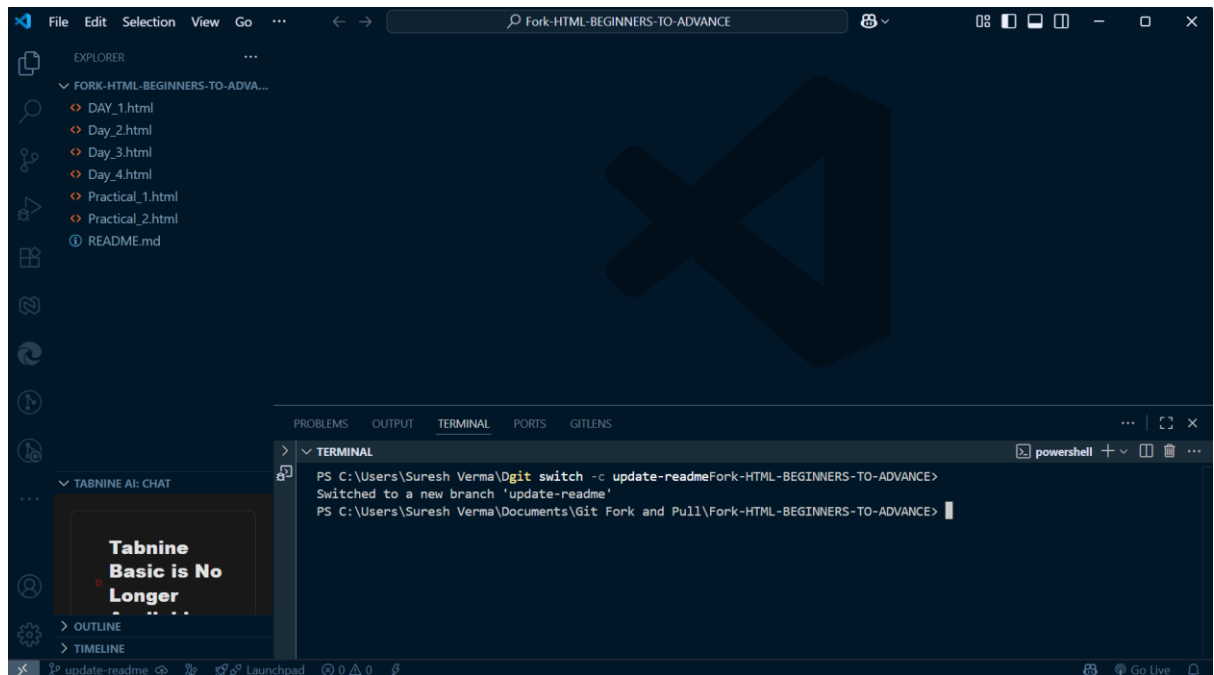
Clone Your Fork Locally

In VS Code:



Create a New Branch for Your Change

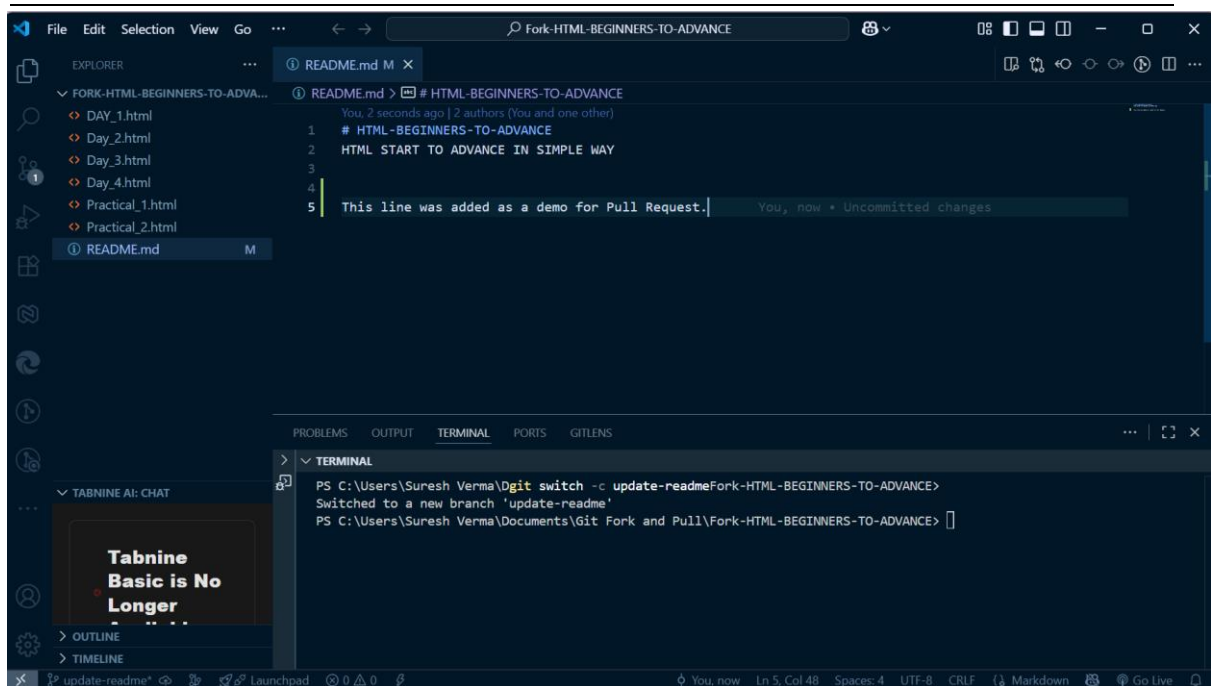
`git switch -c update-readme`



Make a Change

Open README.md and add a new line at the bottom:

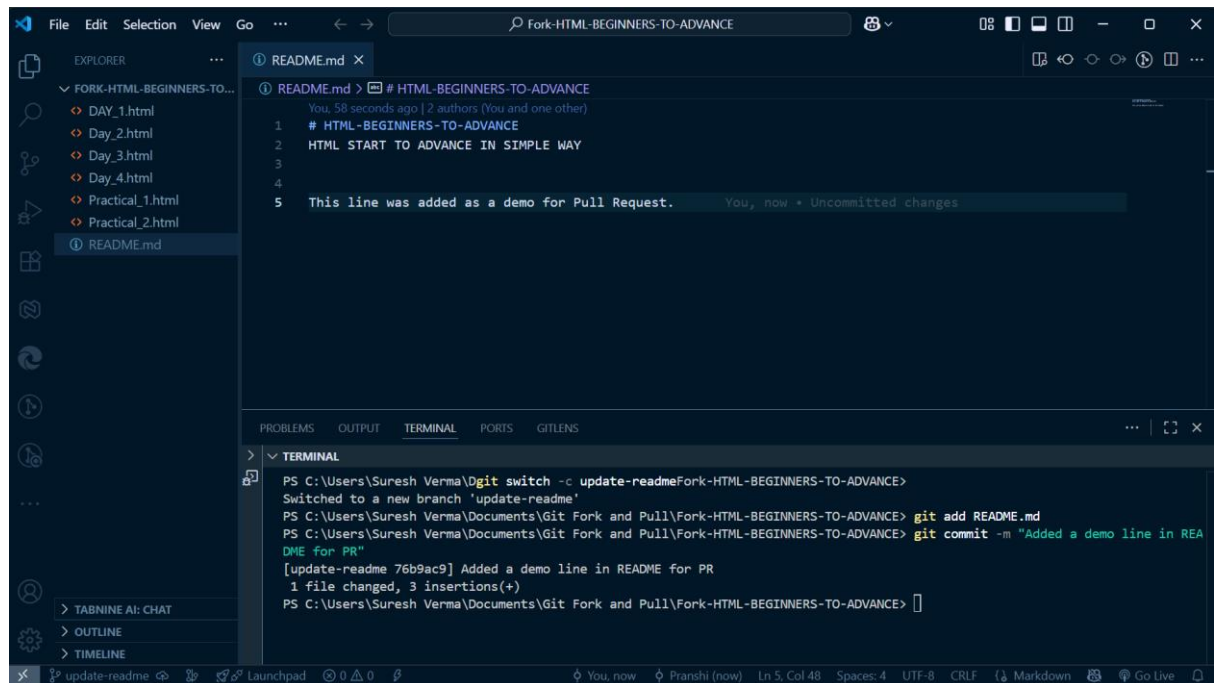
This line was added as a demo for Pull Request.



Commit the Change

`git add README.md`

`git commit -m "Added a demo line in README for PR"`

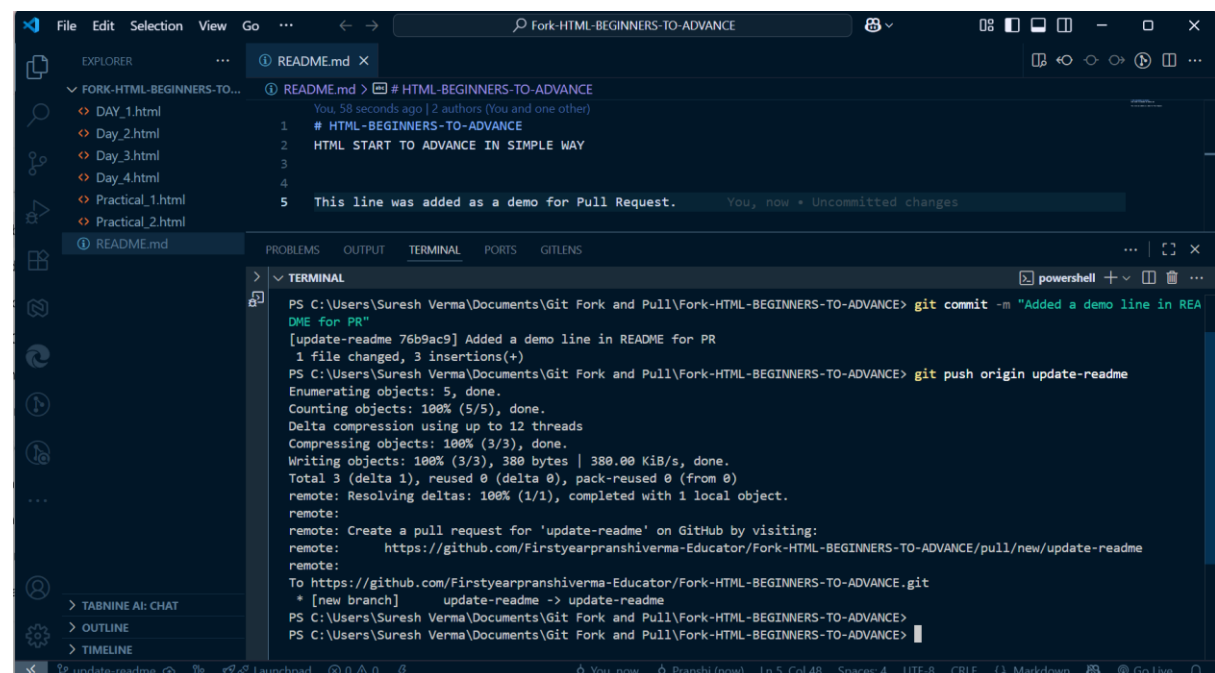


The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'FORK-HTML-BEGINNERS-TO-ADVANCE' with files 'DAY_1.html', 'Day_2.html', 'Day_3.html', 'Day_4.html', 'Practical_1.html', 'Practical_2.html', and 'README.md'. The README.md file is open in the editor, showing its content: '# HTML-BEGINNERS-TO-ADVANCE', 'HTML START TO ADVANCE IN SIMPLE WAY', and a new line 'This line was added as a demo for Pull Request.' which is highlighted with a green background and labeled 'You, now * Uncommitted changes'. The TERMINAL pane at the bottom shows the following commands and output:

```
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE> git switch -c update-readmeFork-HTML-BEGINNERS-TO-ADVANCE
Switched to a new branch 'update-readme'
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE> git add README.md
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE> git commit -m "Added a demo line in README for PR"
[update-readme 76b9ac9] Added a demo line in README for PR
1 file changed, 3 insertions(+)
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE>
```

Push the Branch to Your Fork

`git push origin update-readme`

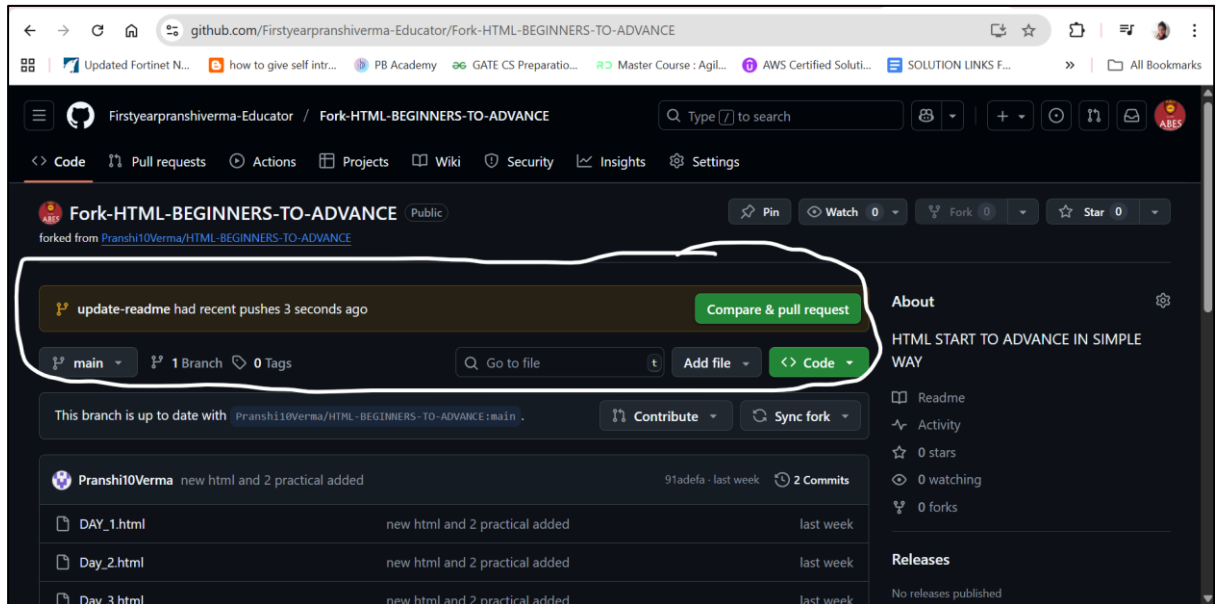


The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows the same project as before. The README.md file is still open in the editor. The TERMINAL pane at the bottom shows the following commands and output:

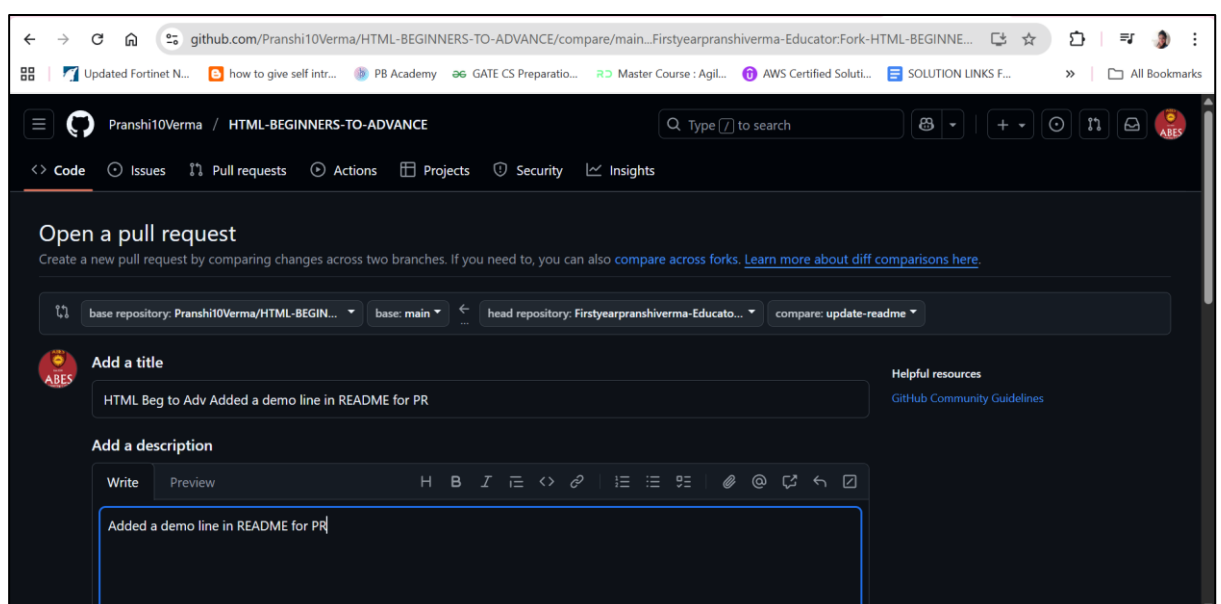
```
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE> git commit -m "Added a demo line in README for PR"
[update-readme 76b9ac9] Added a demo line in README for PR
1 file changed, 3 insertions(+)
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE> git push origin update-readme
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 380 bytes | 380.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'update-readme' on GitHub by visiting:
remote:   https://github.com/Firstyearpranshiverma-Educator/Fork-HTML-BEGINNERS-TO-ADVANCE/pull/new/update-readme
remote:
To https://github.com/Firstyearpranshiverma-Educator/Fork-HTML-BEGINNERS-TO-ADVANCE.git
 * [new branch]      update-readme -> update-readme
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE>
```

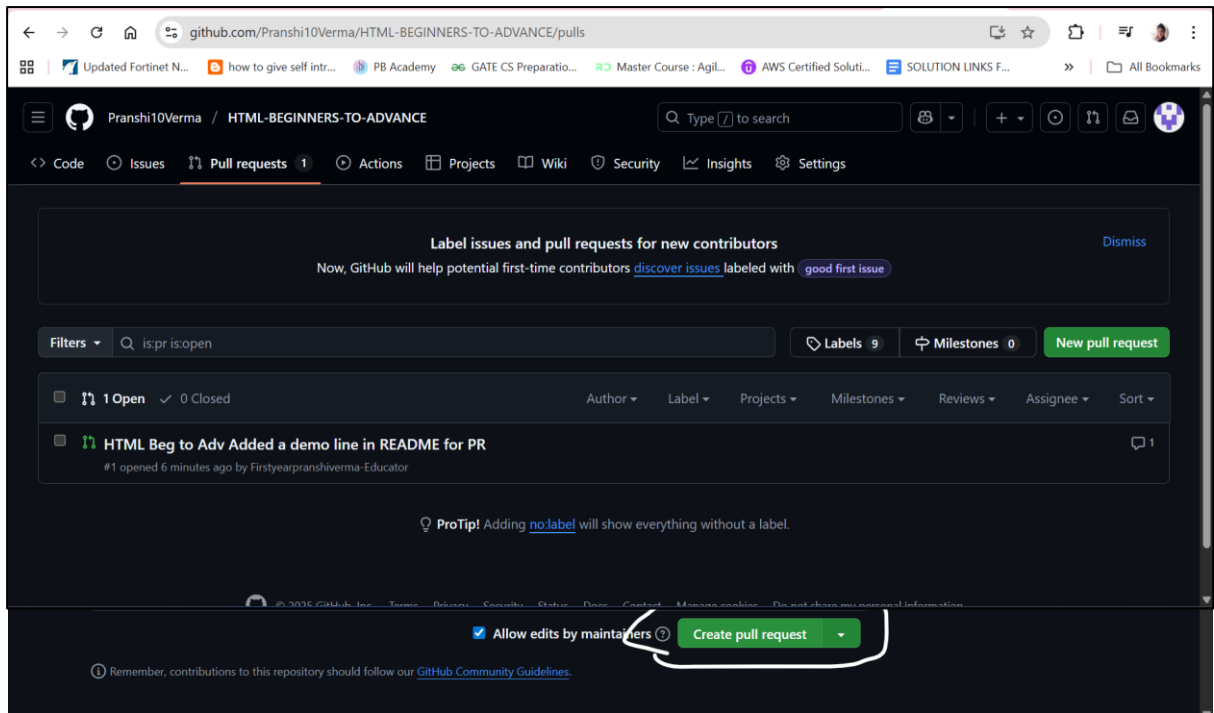
Open a Pull Request

- Go to your fork on GitHub.
- You'll see a message: *"Compare & pull request"*.
- Click it.

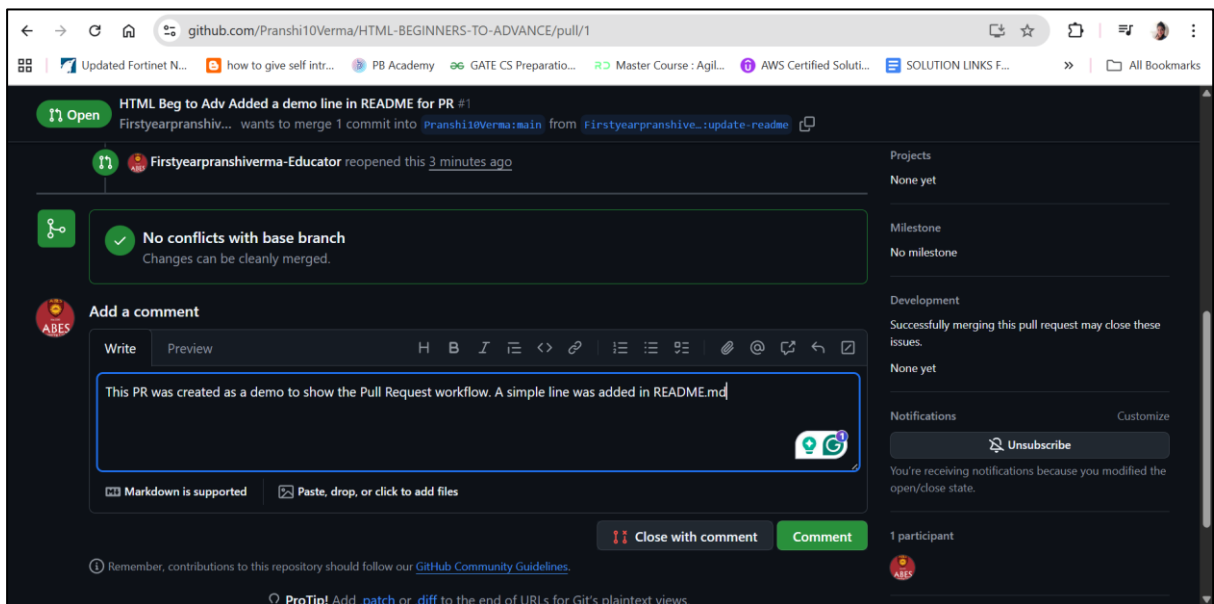


1. GitHub shows a comparison view:
 - **Base repository** → the original repo (octocat/Spoon-Knife).
 - **Head repository** → your fork + branch (your-username/update-readme).
2. Add a title and description.

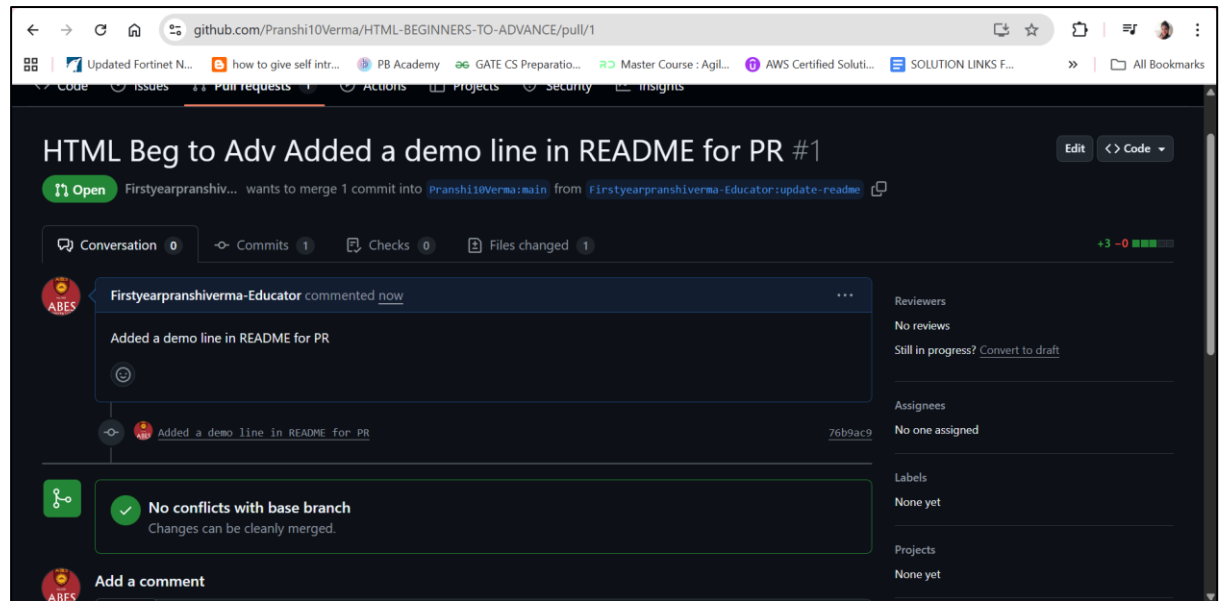




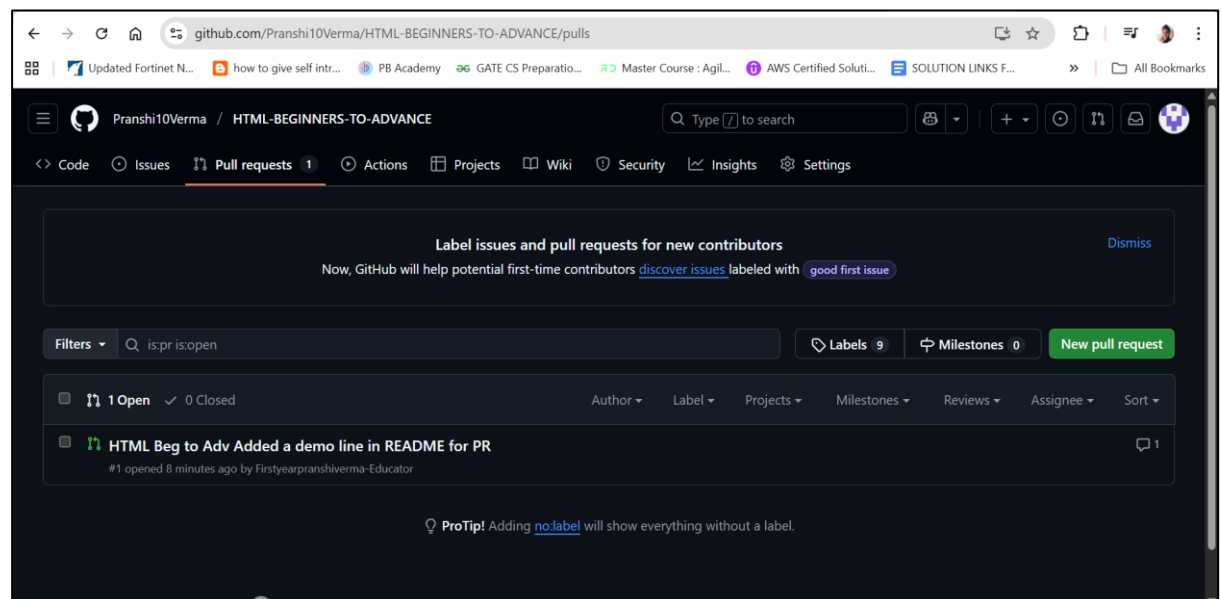
3. Click Create Pull Request.



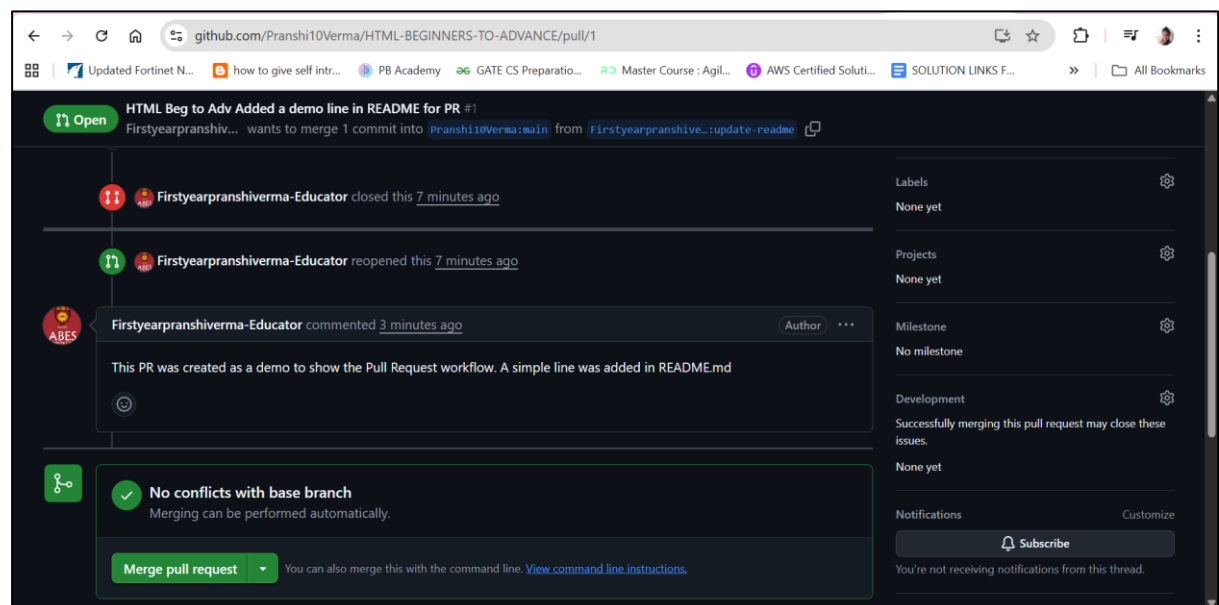
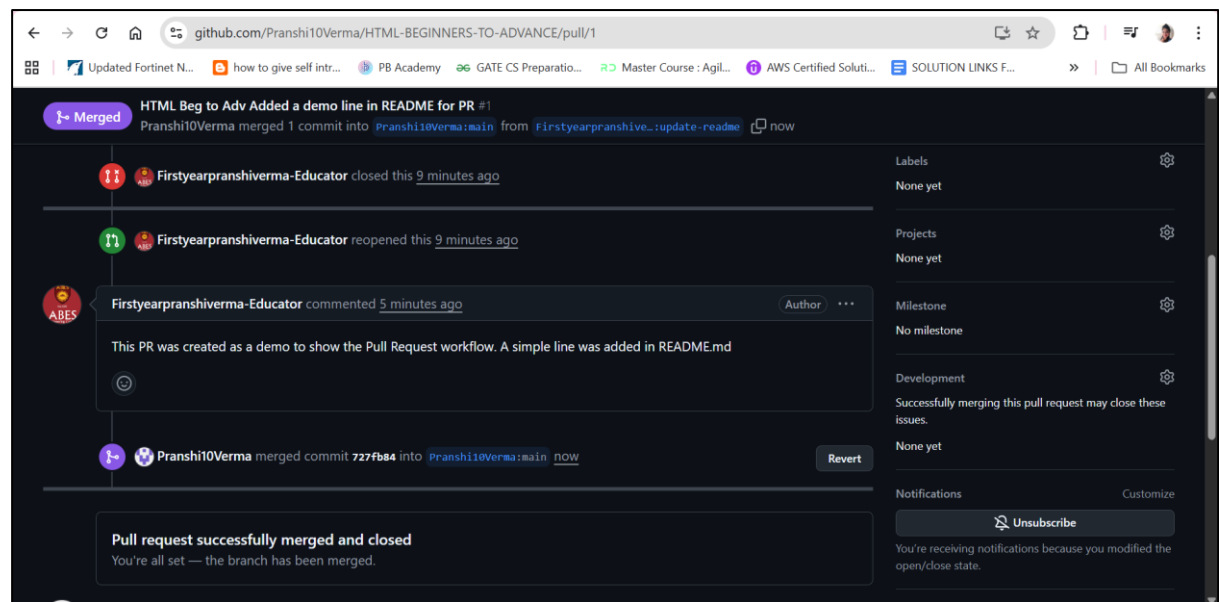
4. Add a comment for high readability. (not mandatory).



After that owner of the url <https://github.com/Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE> received a pull request



- you'll see a **Merge pull request** button.
- Click it → then click **Confirm merge**.



PR merged successfully.

Key Takeaways:

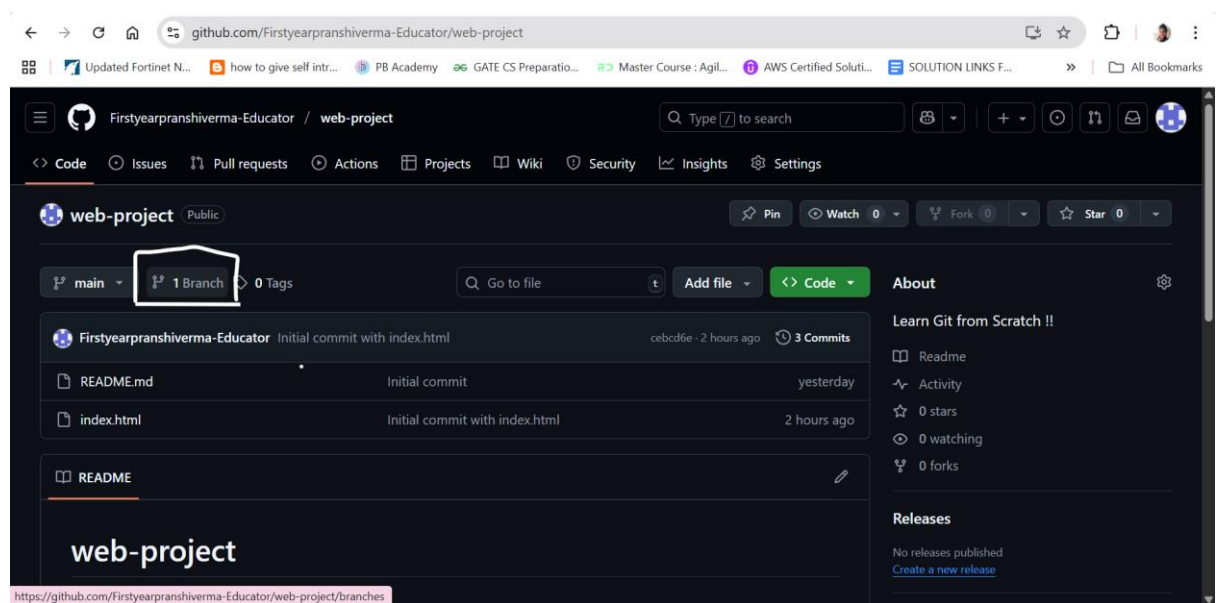
- Fork → Create personal copy of repo.
- Clone → Bring fork to your computer.
- Branch → Make safe changes.
- Commit & Push → Upload to fork.
- PR → Ask original repo owner to accept.
- Merge → Changes officially added.

3.4 Branching: git branch, git checkout, git switch, merging branches, handling merge conflicts

3.4.0 Branch

- A **Branch** is a parallel line of development inside a repository.

By default, every repo has a branch called **main** (or master).



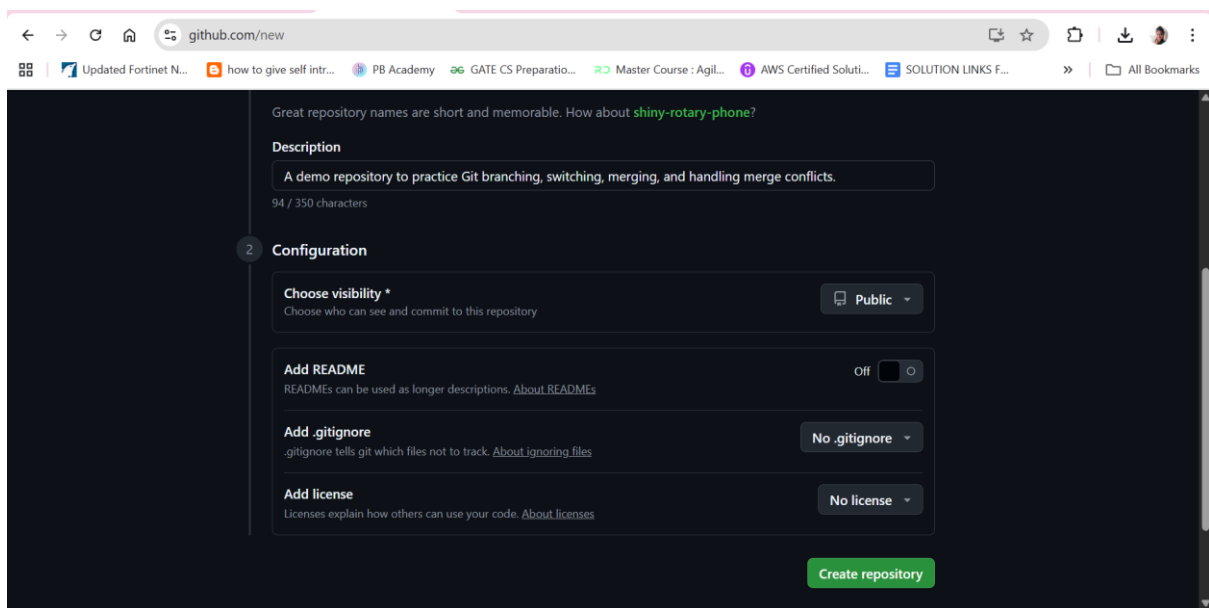
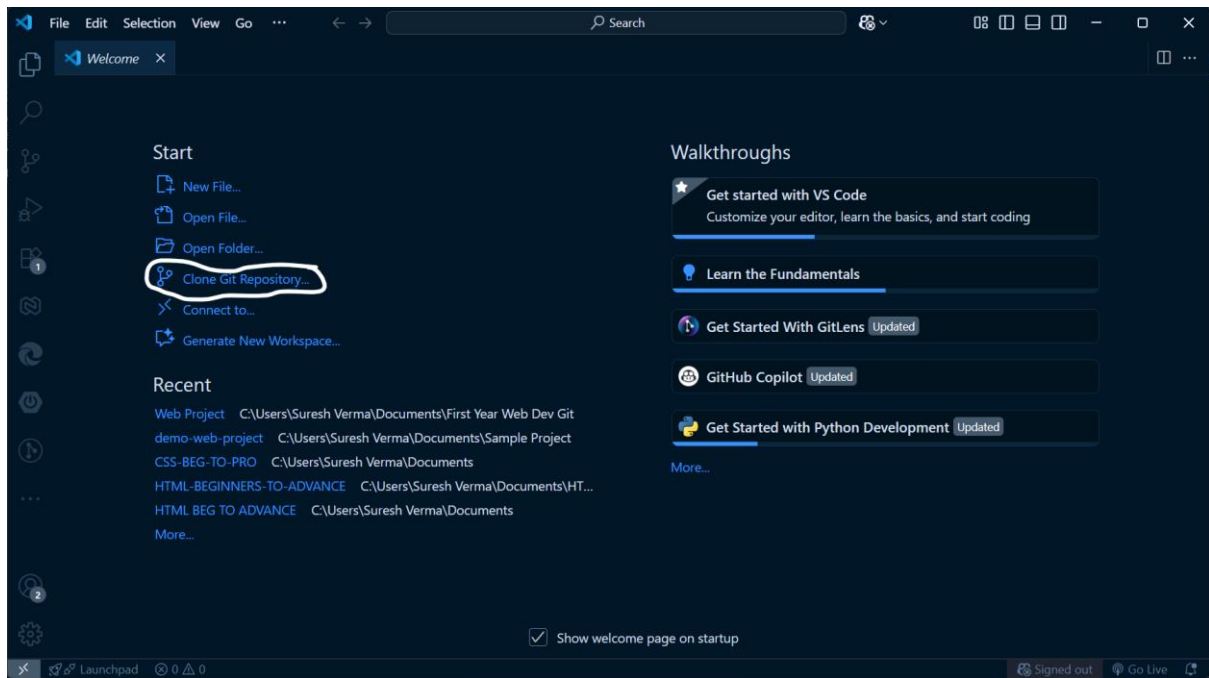
- Developers create new branches to work on features, bug fixes, or experiments **without disturbing the main branch**.

Let's understand how to create branch, switching between branches, making changes, and merging branches.

- You can create new branches to work on features or fixes without disturbing the main project.

Step 1: Create a Repository on GitHub

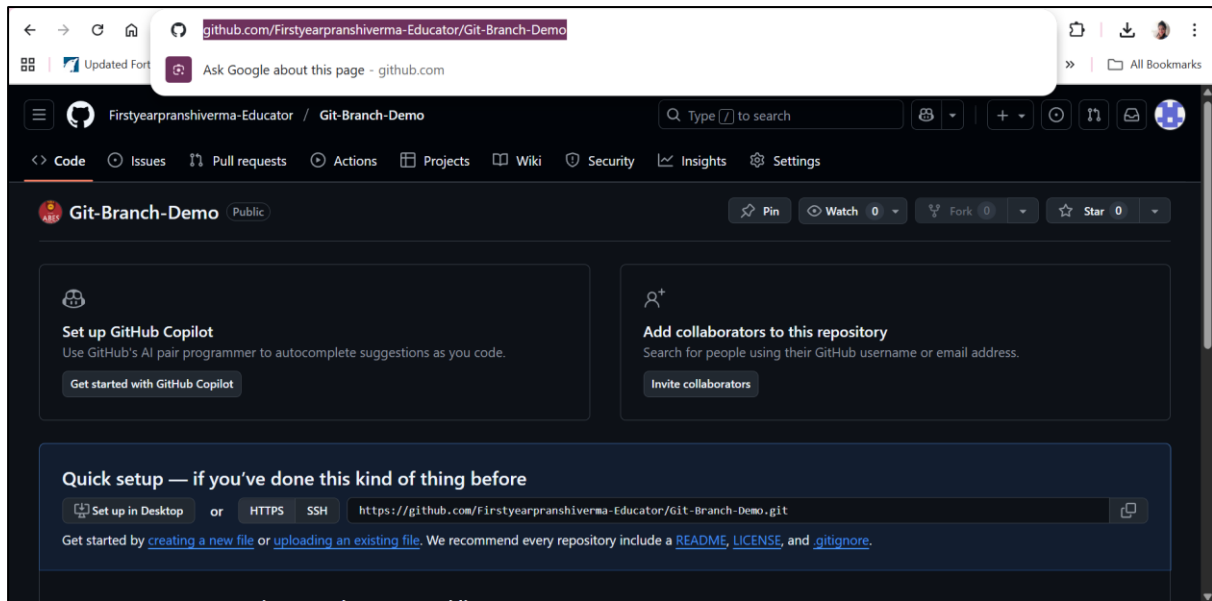
1. Log in to your GitHub account.
2. Click on **New Repository**.
3. Enter the repository name → **Git Branch Demo**.
4. Do **not** check the option *"Initialize this repository with a README"*.
5. Click **Create Repository**.



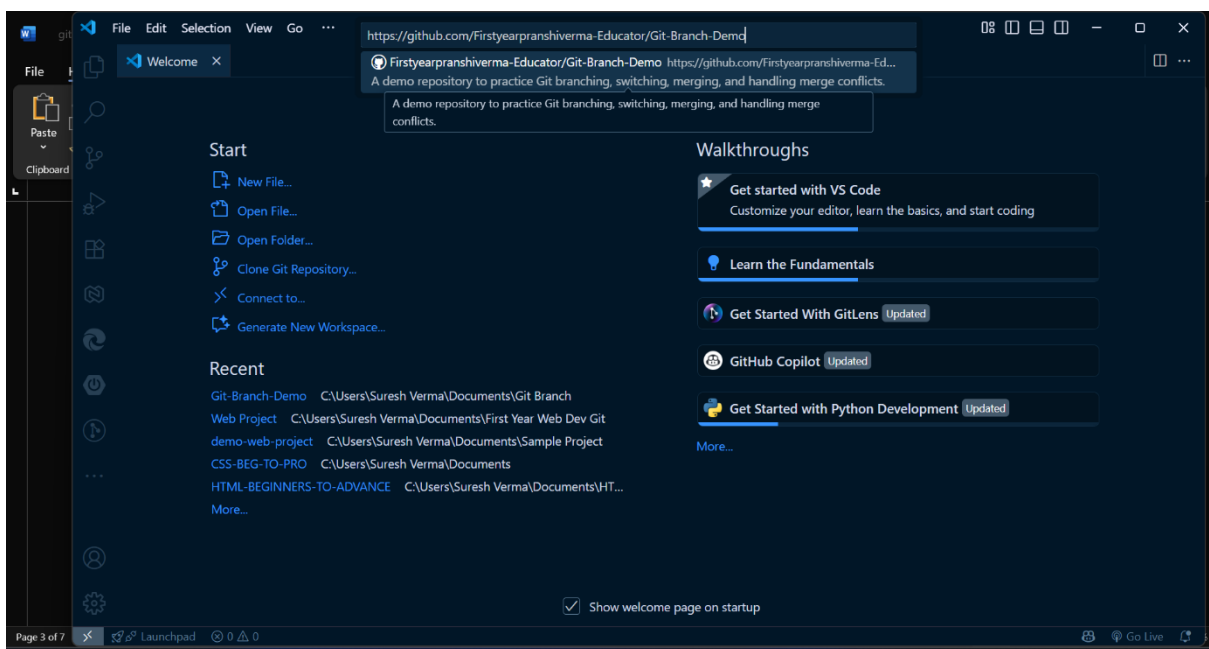
Step 2: Clone the Repository into VS Code

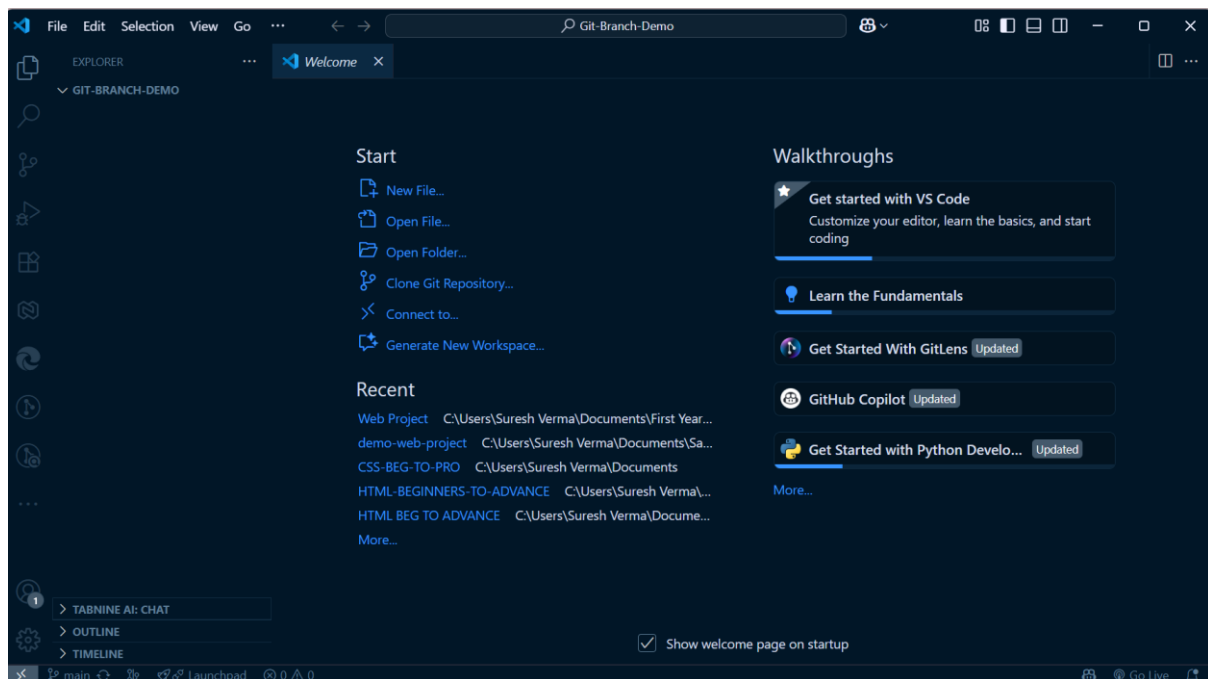
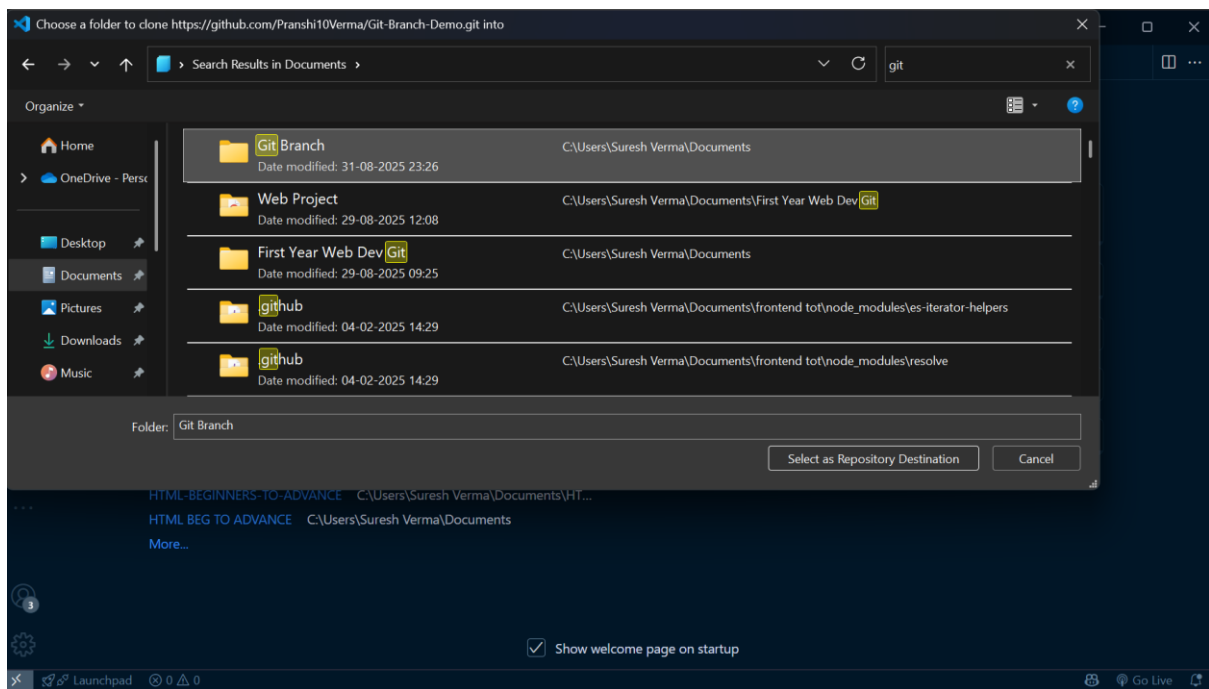
Open the VS Code and click on Clone Git Repository...

Then copy the url from github repo.



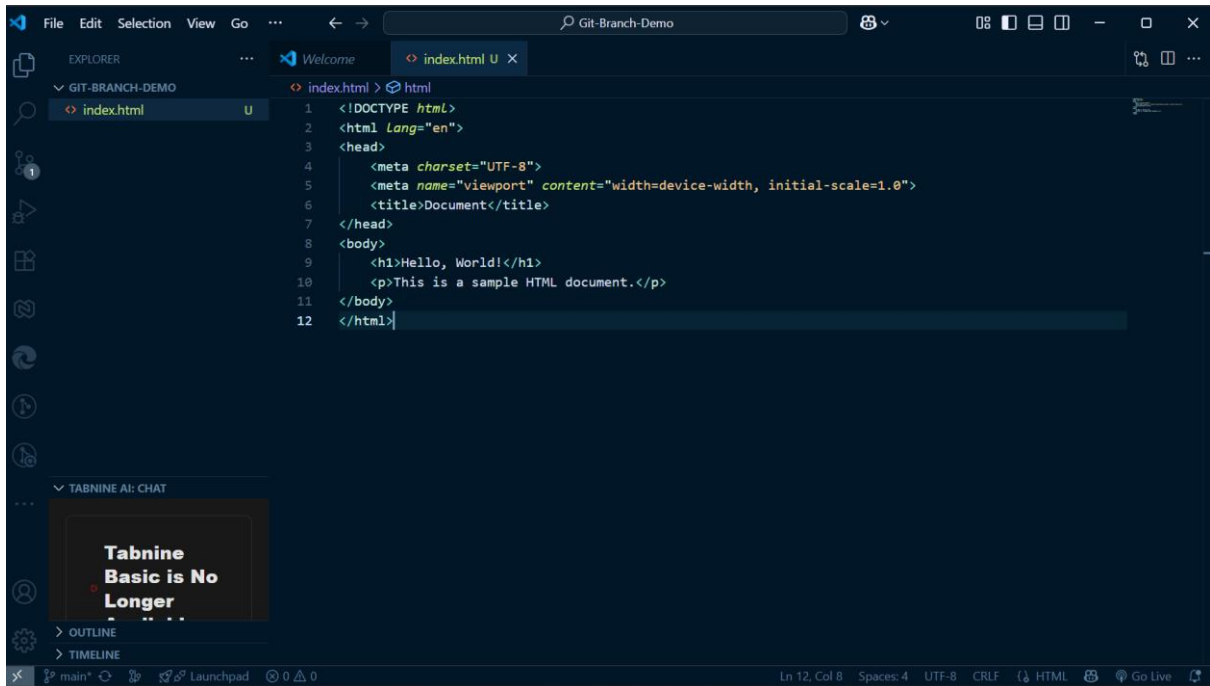
Then select the folder, where you want to clone it





Repository cloned successfully

Step 4: Create a new file named index.html and add the content:



The screenshot shows the Visual Studio Code editor with a file named `index.html` open. The file contains the following HTML code:

```
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11 </body>
12 </html>
```

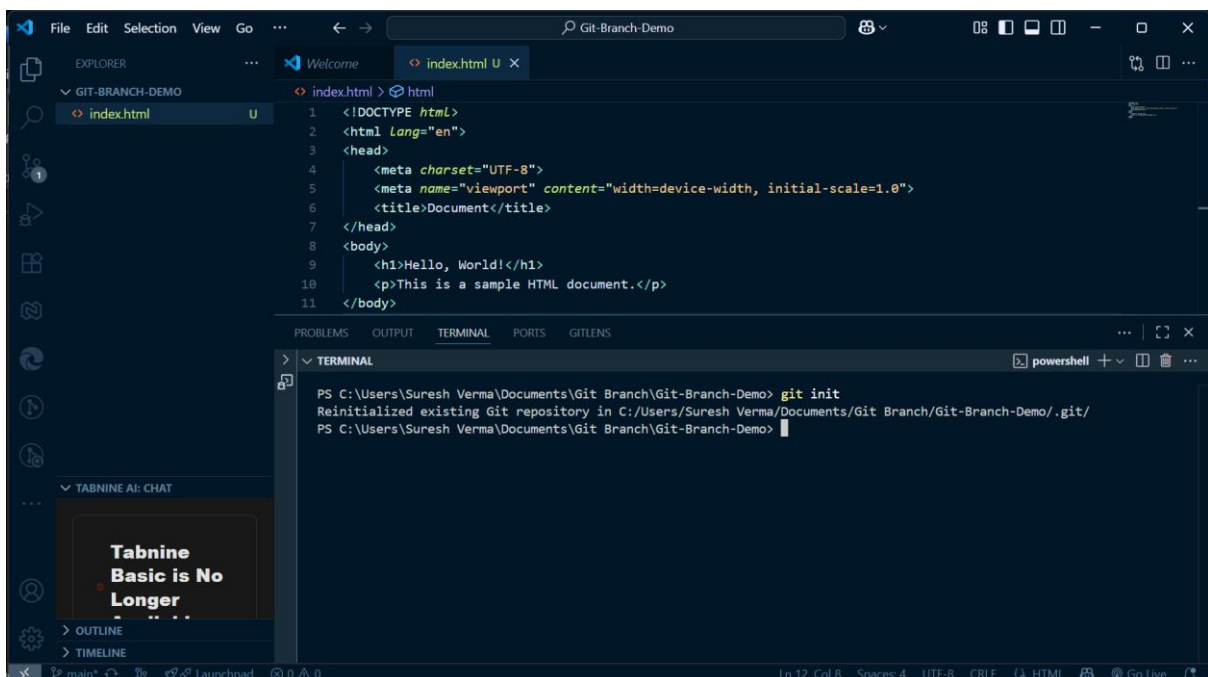
The Explorer sidebar on the left shows the file structure with `index.html` selected. The Tabnine AI Chat sidebar is also visible on the left with the message "Tabnine Basic is No Longer".

Step 5: Initialize Git (if not already initialized)

In the VS Code terminal:

```
git init
```

This command initializes the folder as a Git repository.

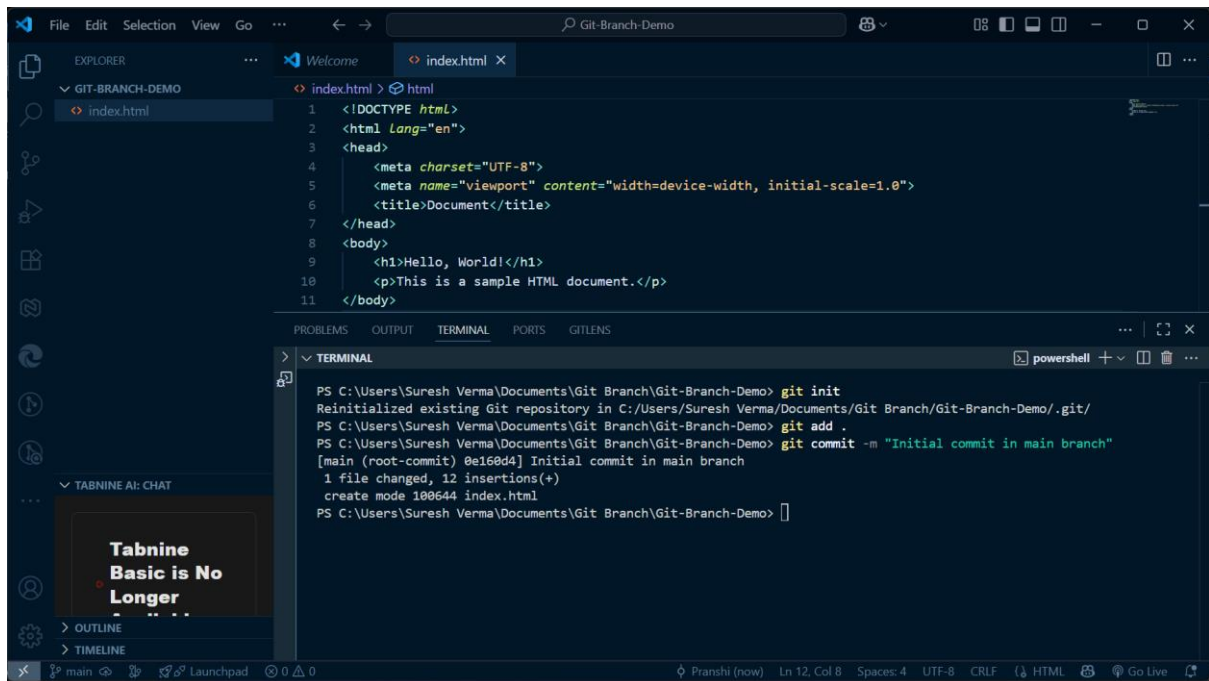


The screenshot shows the Visual Studio Code editor with the `index.html` file open. The terminal at the bottom shows the execution of the `git init` command:

```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git init
Reinitialized existing Git repository in C:/Users/Suresh Verma/Documents/Git Branch/Git-Branch-Demo/.git/
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

The Explorer sidebar on the left shows the file structure with `index.html` selected. The Tabnine AI Chat sidebar is also visible on the left with the message "Tabnine Basic is No Longer".

Step 6: Make the First Commit

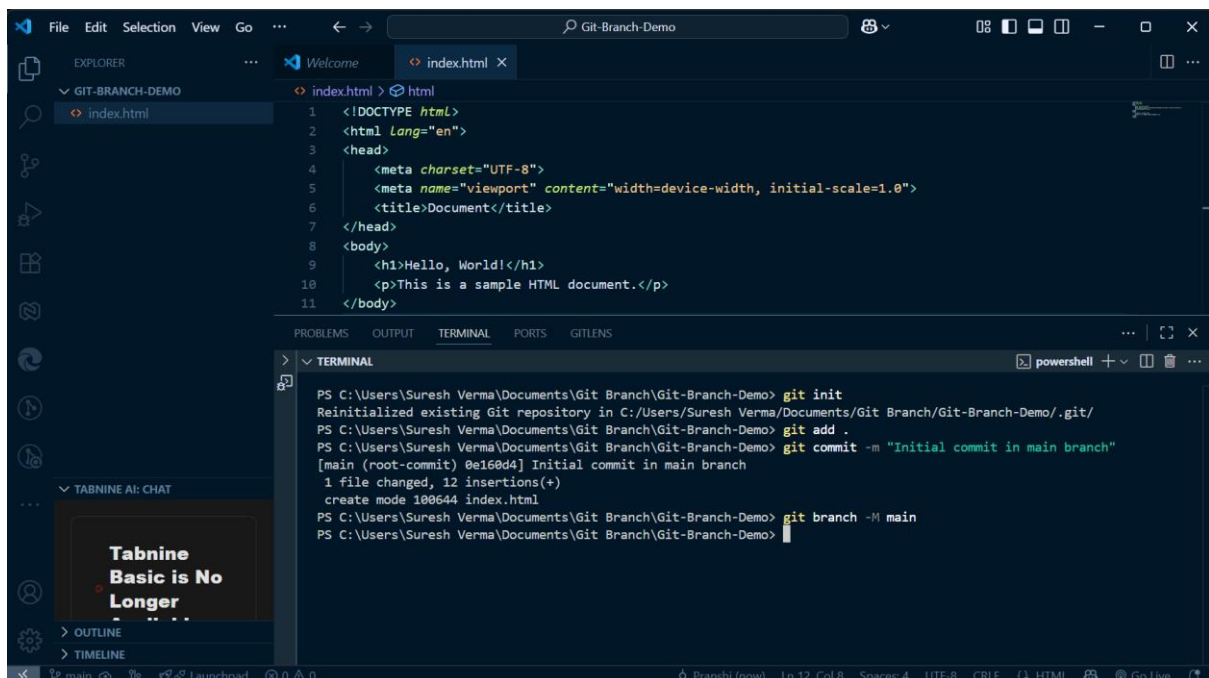


The screenshot shows the Visual Studio Code interface with a file explorer on the left showing a project named 'GIT-BRANCH-DEMO' containing 'index.html'. The editor displays the content of 'index.html', which is a basic HTML document. The terminal at the bottom shows the execution of the following commands:

```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git init
Reinitialized existing Git repository in C:/Users/Suresh Verma/Documents/Git Branch/Git-Branch-Demo/.git/
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add .
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Initial commit in main branch"
[main (root-commit) 0e160d4] Initial commit in main branch
1 file changed, 12 insertions(+)
create mode 100644 index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Step 7: Push the Code to GitHub

git branch -M main



This screenshot is identical to the previous one, showing the same VS Code interface and terminal output for the first commit. The terminal now includes an additional command:

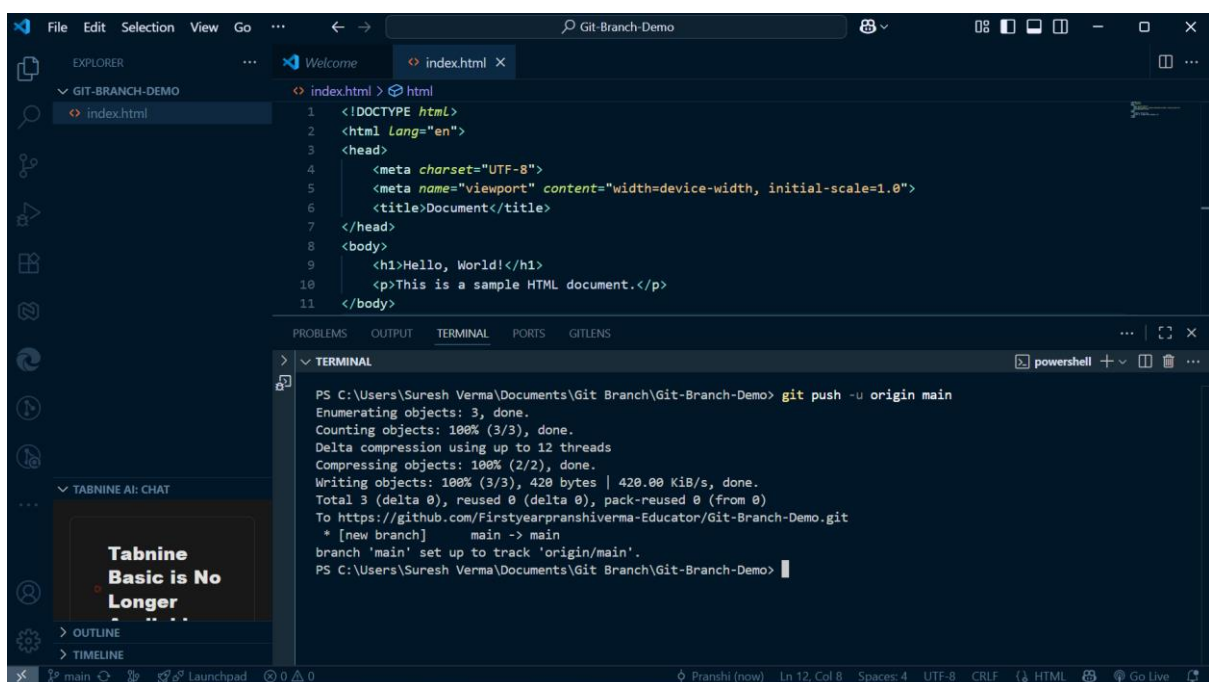
```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch -M main
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Meaning:

- `git branch` → command to manage branches.
- `-M` → option that means **move/rename branch (forcefully if needed)**.
- `main` → the new name for the current branch.

So this command **renames your current branch to main**, even if a branch with that name already exists (it will overwrite).

`git push -u origin main`

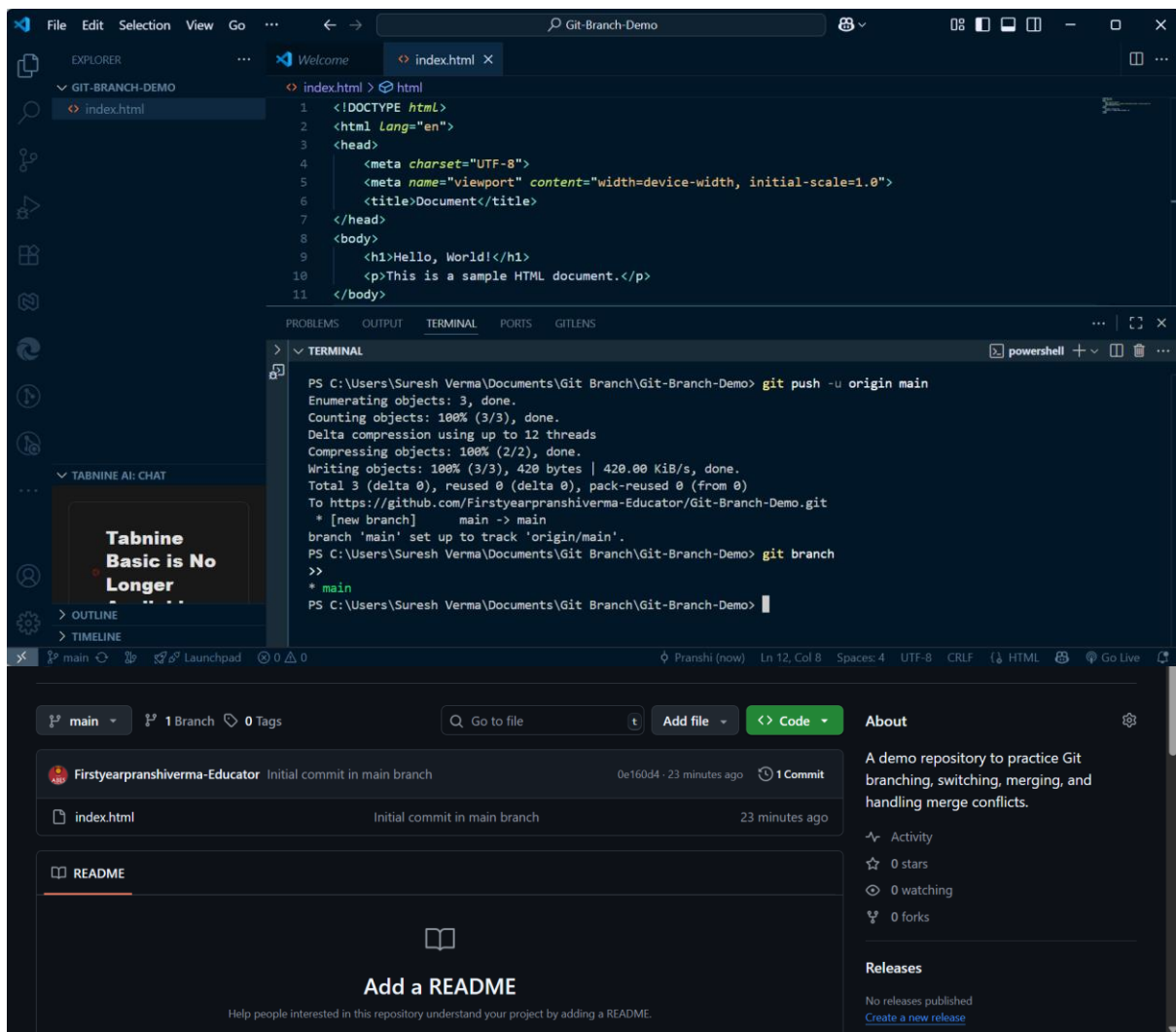


The screenshot shows a Visual Studio Code editor window titled 'Git-Branch-Demo'. The Explorer sidebar on the left shows a file named 'index.html'. The editor window displays the content of 'index.html', which is an HTML document with a title 'Document' and a body containing 'Hello, World!' and 'This is a sample HTML document.' Below the editor, the TERMINAL panel is open, showing the output of the command `git push -u origin main`. The output indicates that the local commit was pushed to the remote repository, creating a new branch 'main' that tracks 'origin/main'.

```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 420 bytes | 420.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Firstyearpranshivarma-Educator/Git-Branch-Demo.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Meaning:

- **git push**
Uploads your local commits (from your computer) to a remote repository (like GitHub).
- **origin**
This is the short name (alias) for the remote repository you linked earlier using
 - `git remote add origin <URL>`
- By convention, the first/primary remote is always called **origin**.
- **main**
The branch you want to push. Here it means: *“Push the local main branch to the remote repository’s main branch.”*



- **-u (or --set-upstream)**
This sets a **tracking relationship** between your local main branch and the remote main branch.
After this first time, you don't need to write git push origin main again.
You can simply run:

git push

and Git will know where to push by default.

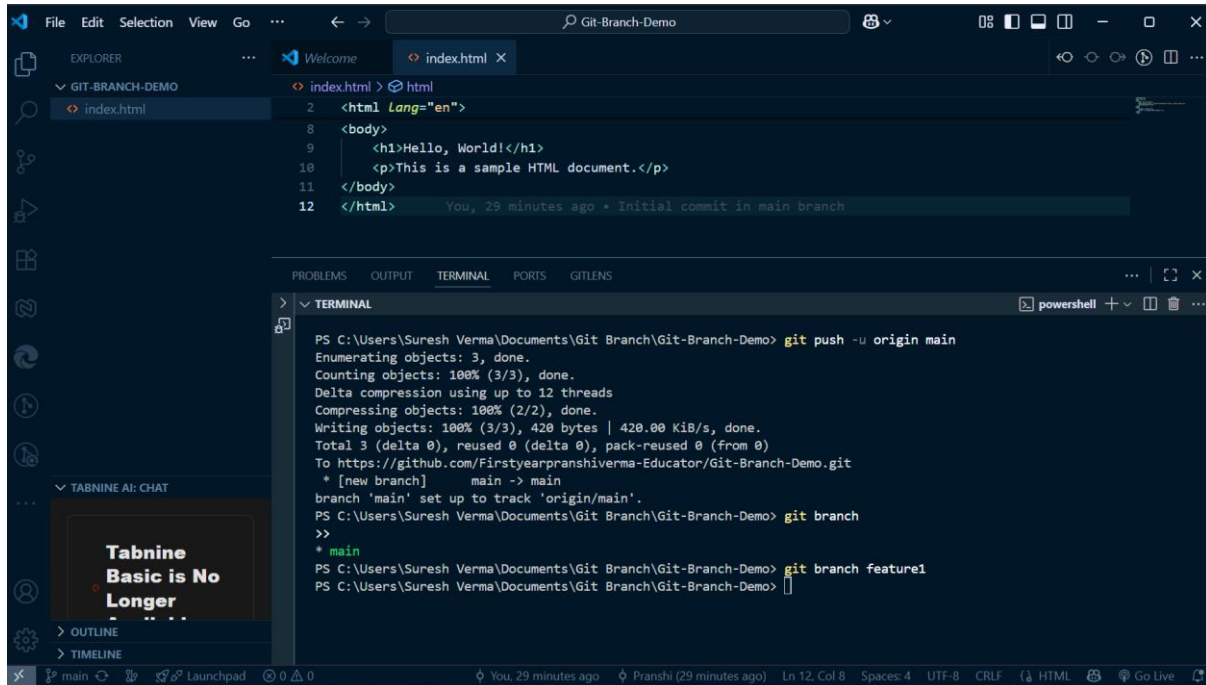
GitHub showing main branch with index.html

Step 7: Check the Current Branch

Output shows all branches. The active branch has a * symbol in front of it (e.g., * main).

Step 8: Create a New Branch

Using the command: `git branch feature1`



The screenshot shows the Visual Studio Code interface with a Git repository named 'Git-Branch-Demo'. The Explorer pane on the left shows the file structure with 'index.html' selected. The editor pane shows the content of 'index.html', which is a simple HTML document. The Terminal pane at the bottom shows the output of the following commands:

```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 420 bytes | 420.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Firstyearpranshiverma-Educator/Git-Branch-Demo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch
>>
 * main
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch feature1
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

This creates a new branch named **feature1**.

Step 9: Switch to the New Branch

There are two methods:

Old method (checkout):

`git checkout feature1`

New method (recommended):

`git switch feature1`

```
index.html > html
2 <html Lang="en">
8 <body>
9 <h1>Hello, World!</h1>
10 <p>This is a sample HTML document.</p>
11 </body>
12 </html>

You, 29 minutes ago • Initial commit in main branch

PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 420 bytes | 420.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Firstyearpranshiverma-Educator/Git-Branch-Demo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch
>>
 * main

PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch feature1
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch feature1
Switched to branch 'feature1'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Now you are working inside the **feature1** branch.

Step 10: Make Changes in index.html

Open your index.html and add one more line under the `<p>` tag. For example:

```
index.html > ...
You, 1 second ago | 1 author (You)
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Document</title>
7 </head>
8 <body>
9 <h1>Hello, World!</h1>
10 <p>This is a sample HTML document.</p>
11 <p>Change made in feature1 branch</p>
12 </body>
13 </html>
14

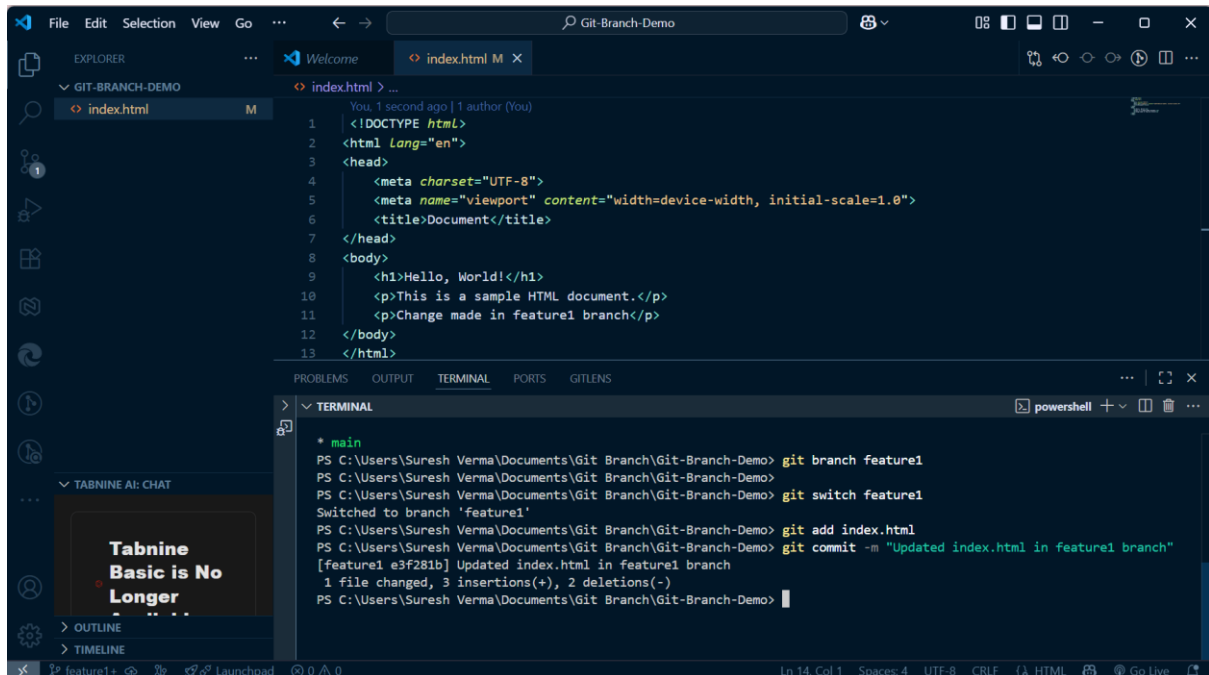
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch feature1
Switched to branch 'feature1'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Step 11: Commit the Changes in feature1 Branch

Run these commands:

```
git add index.html
```

```
git commit -m "Updated index.html in feature1 branch"
```



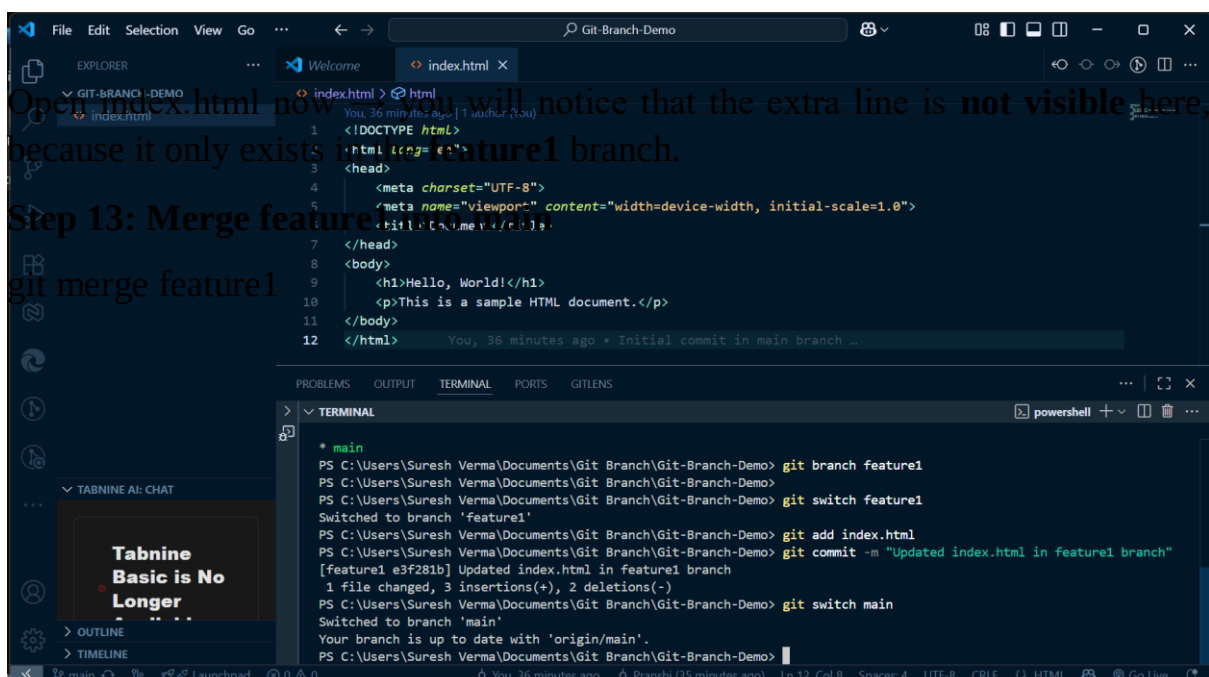
The screenshot shows the Visual Studio Code interface with the Explorer, Source Control, and Terminal panels. The Explorer panel shows the 'index.html' file in the 'feature1' branch. The Source Control panel shows the file is staged for commit. The Terminal panel shows the following commands and output:

```
* main
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch feature1
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch feature1
Switched to branch 'feature1'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Updated index.html in feature1 branch"
[feature1 e3f281b] Updated index.html in feature1 branch
1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Commit message for feature1 branch.

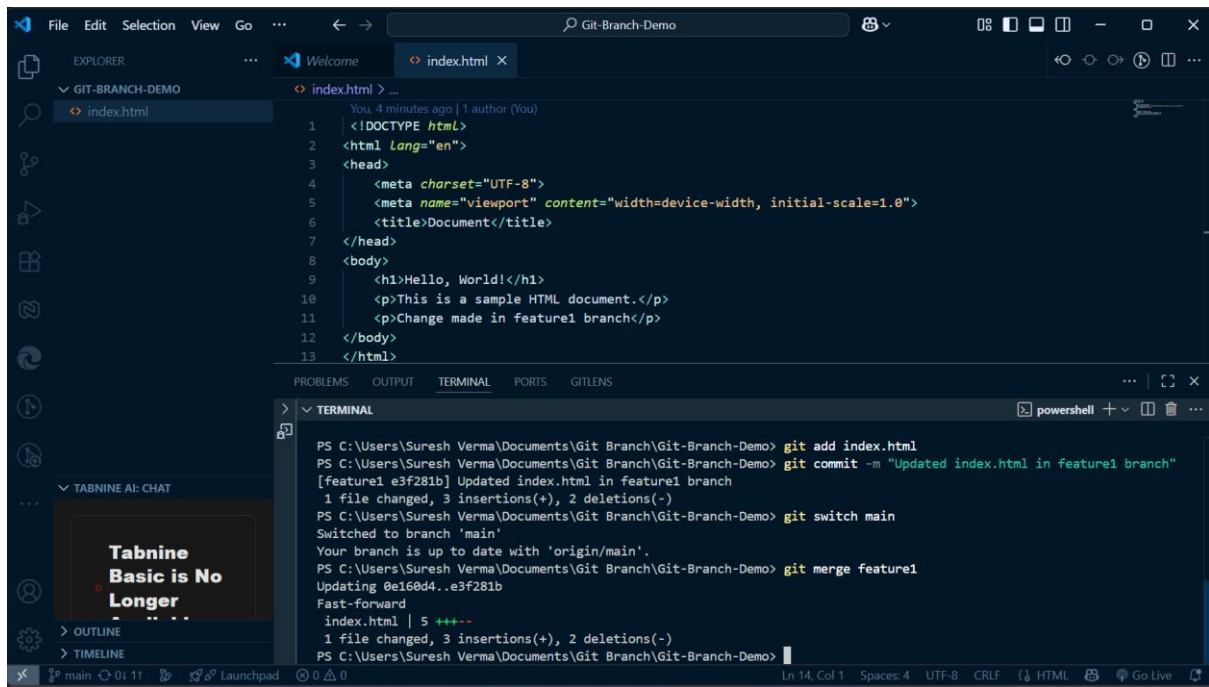
Step 12: Switch Back to Main Branch

```
git switch main
```



The screenshot shows the Visual Studio Code interface with the Explorer, Source Control, and Terminal panels. The Explorer panel shows the 'index.html' file in the 'main' branch. The Source Control panel shows the file is staged for commit. The Terminal panel shows the following commands and output:

```
* main
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch feature1
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch feature1
Switched to branch 'feature1'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Updated index.html in feature1 branch"
[feature1 e3f281b] Updated index.html in feature1 branch
1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```



Now the changes from feature1 are merged into main.
Check index.html again → you will see the new line has appeared.

Step 14 (Optional): Delete the Branch

`git branch -d feature1`

☞ If you don't need the branch anymore, you can delete it safely.

That completes our first **branching + switching + merging demo**.

Step 15 Merge Conflict Demo (with index.html)

From main branch, create a new branch:

`git switch -c feature2`

```
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11  <p>Change made in feature1 branch</p>
12 </body>
13 </html>
```

```
[feature1 e3f281b] Updated index.html in feature1 branch
1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git merge feature1
Updating 0e168d4..e3f281b
Fast-forward
 index.html | 5 +++--
1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch -c feature2
Switched to a new branch 'feature2'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

This creates and switches to a new branch **feature2**.

Step 16: Modify index.html in feature2

Change the second `<p>` line like this:

`<p>This is a sample HTML document. Edited in feature2 branch</p>`

```
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11  <p>This is a sample HTML document. Edited in feature2 branch</p>
12 </body>
13 </html>
```

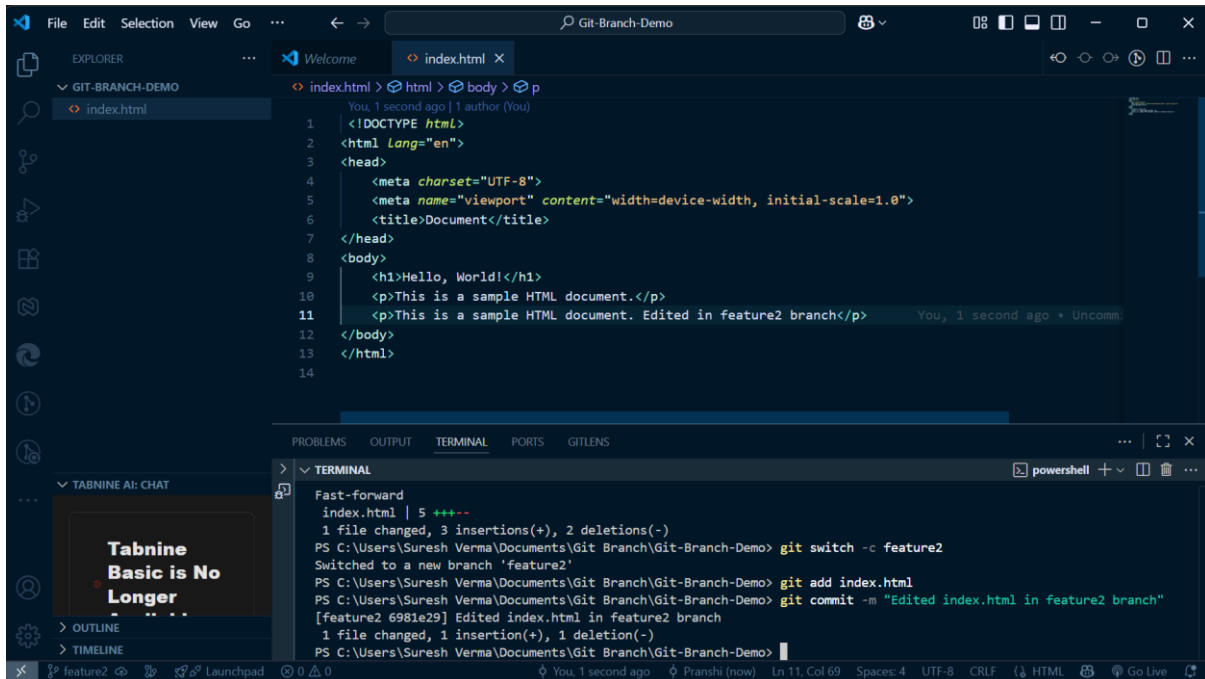
```
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git merge feature1
Updating 0e168d4..e3f281b
Fast-forward
 index.html | 5 +++--
1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch -c feature2
Switched to a new branch 'feature2'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Commit this change:

git add index.html

git commit -m "Edited index.html in feature2 branch"

Commit in feature2 branch

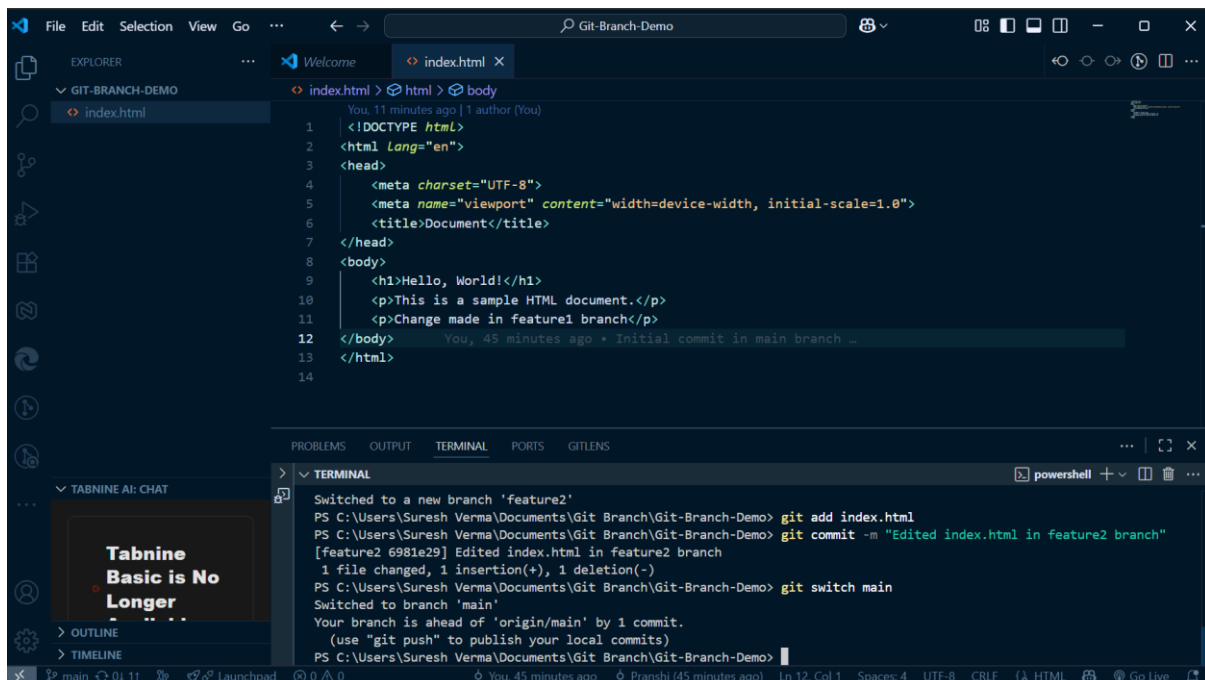


```
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11  <p>This is a sample HTML document. Edited in feature2 branch</p>
12 </body>
13 </html>
14
```

```
Fast-forward
 index.html | 5 +++--
 1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch -c feature2
Switched to a new branch 'feature2'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Edited index.html in feature2 branch"
[feature2 6981e29] Edited index.html in feature2 branch
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Step 17: Switch to main and Make a Different Change

Switch back to main: git switch main

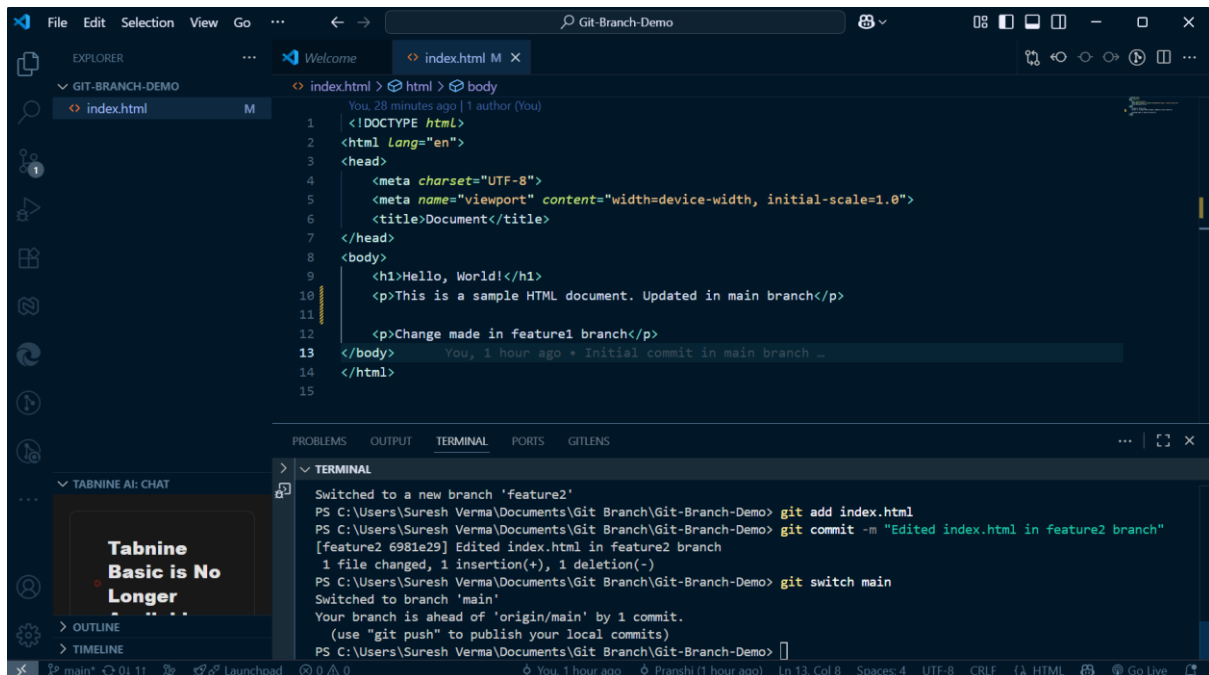


```
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11  <p>Change made in feature1 branch</p>
12 </body>
13 </html>
14
```

```
Switched to a new branch 'feature2'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Edited index.html in feature2 branch"
[feature2 6981e29] Edited index.html in feature2 branch
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```


Now **edit the same line** but write something different:

<p>This is a sample HTML document. Updated in main branch</p>



The screenshot shows the Visual Studio Code editor with a file named `index.html` open. The file content is as follows:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document. Updated in main branch</p>
11
12  <p>Change made in feature1 branch</p>
13 </body>
14 </html>
15
```

The terminal window at the bottom shows the following commands and output:

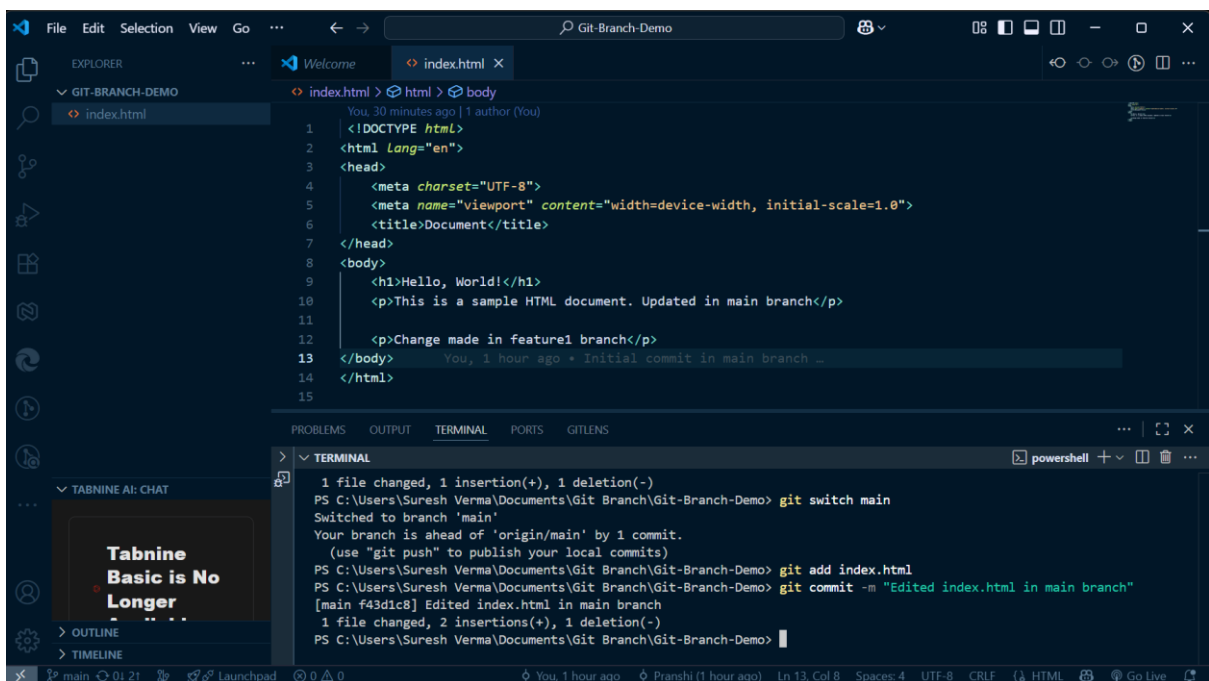
```
Switched to a new branch 'feature2'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Edited index.html in feature2 branch"
[feature2 6981e29] Edited index.html in feature2 branch
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Commit this change:

`git add index.html`

`git commit -m "Edited index.html in main branch"`

Commit in main branch



The screenshot shows the Visual Studio Code editor with the same `index.html` file. The content is identical to the previous screenshot. The terminal window now shows the following commands and output:

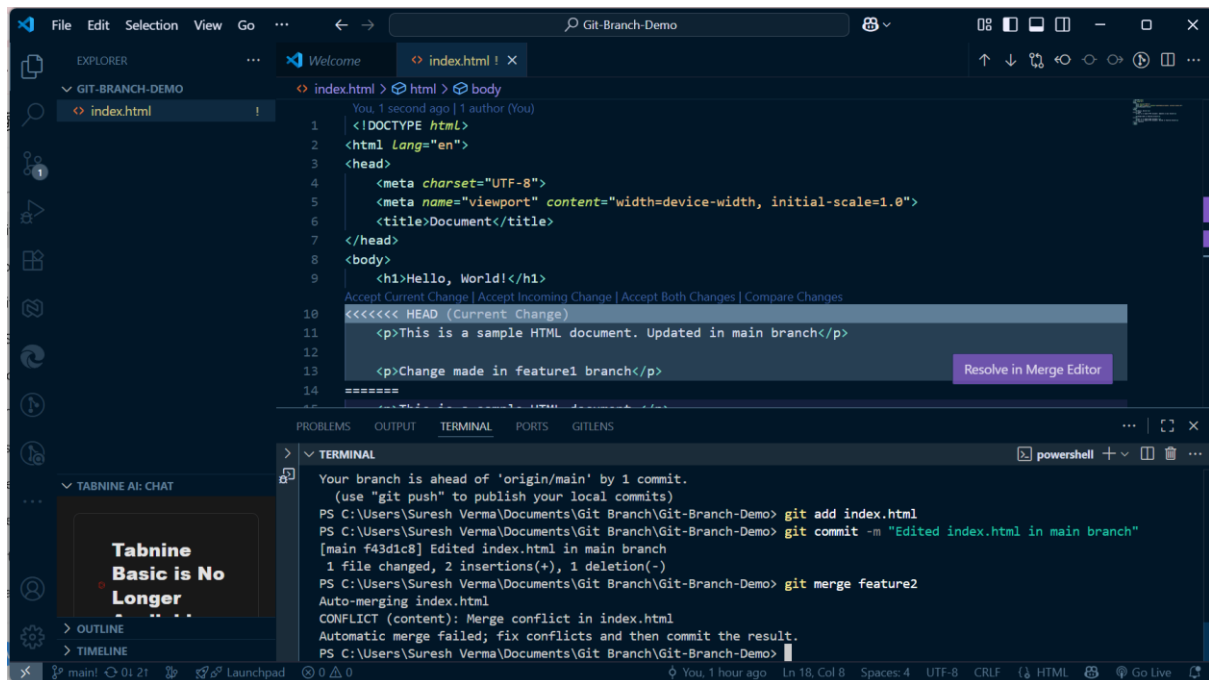
```
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Edited index.html in main branch"
[main f43d1c8] Edited index.html in main branch
1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Step 18: Try Merging feature2 into main

Now merge:

```
git merge feature2
```

Git will detect that **both branches modified the same line differently** → this creates a **merge conflict**.



Merge conflict message in terminal and The index.html file will show conflict markers like this:

<<<<<<< HEAD → your current branch (main) version

===== → separator

>>>>>> feature2 → incoming branch version

Step 20: Resolve the Conflict Manually

Decide how you want the final version to look. For example, keep both lines combined:

```
<body>
```

```
    <h1>Hello, World!</h1>
```

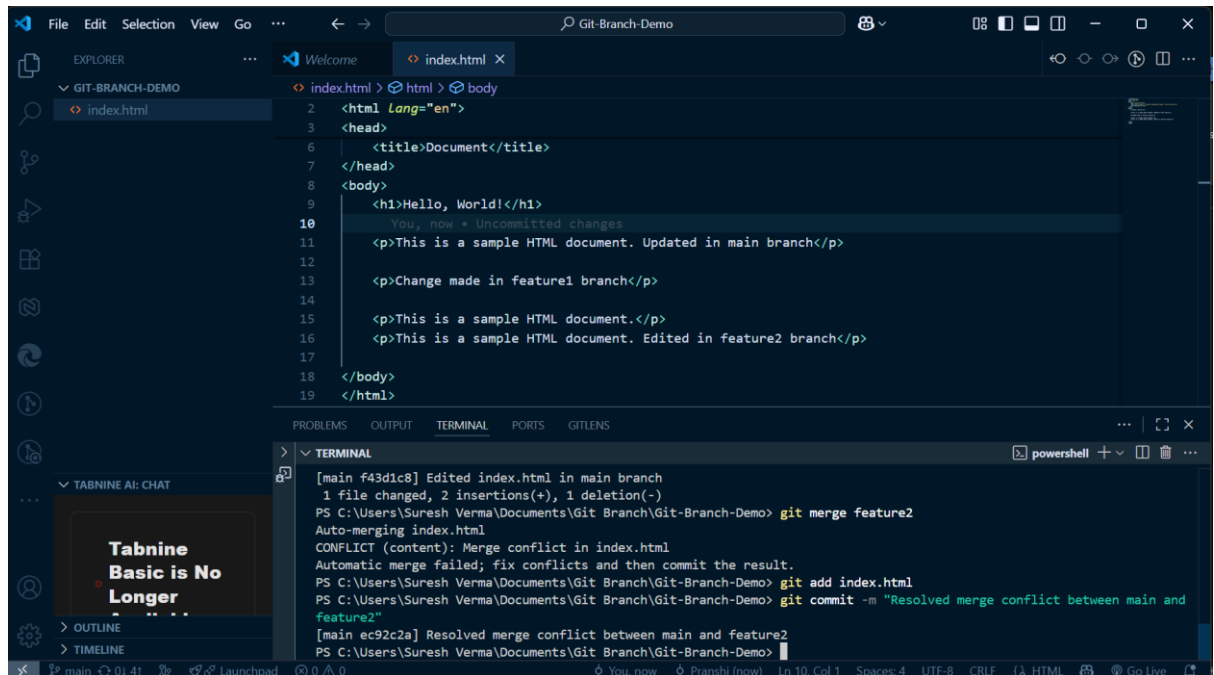
```
    <p>This is a sample HTML document. Updated in main branch & feature2  
branch</p>
```


<p>Change made in feature1 branch</p>
</body>

Step 21: Mark the Conflict as Resolved

git add index.html

git commit -m "Resolved merge conflict between main and feature2"



The screenshot shows the VS Code interface with a file named `index.html` open. The file content is an HTML document. The terminal window at the bottom shows the following commands and output:

```
[main f43d1c8] Edited index.html in main branch
1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git merge feature2
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Resolved merge conflict between main and feature2"
[main ec92c2a] Resolved merge conflict between main and feature2
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

- Merge conflicts occur when **two branches edit the same part of a file differently**.
- Git cannot decide automatically, so it asks *you* to resolve.
- You open the file, remove the conflict markers (`<<<<<<<`, `=====`, `>>>>>>>`), and keep the correct content.
- After resolving, you stage and commit the file.

Summary of Key Commands:

- **Create a new branch:** `git branch <branch-name>`
- **Switch to a branch:** `git checkout <branch-name>`
- **Create and switch in one step:** `git checkout -b <branch-name>`
- **Stage changes:** `git add <file-name>` or `git add .`
- **Commit changes:** `git commit -m "message"`

- **List all branches:** `git branch`
 - **Merge a branch into the current branch:** `git merge <branch-name>`
 - **Delete a branch:** `git branch -d <branch-name>`
 - **Push changes:** `git push origin <branch-name>`
-

3.5 GitHub collaboration workflow, Best practices for commits, branches, and messages

3.5.0 Collaboration Workflow in GitHub

When working in a **team project**, multiple developers collaborate. The most common workflow is:

Step 1: Fork (for open-source projects) / Clone (for team repos)

- If you're contributing to someone else's repository → **Fork** it into your account.
- If you're in a team and already have access → **Clone** the repo.

Forked repo (on GitHub) → then clone it

```
git clone https://github.com/your-username/project.git
```

```
cd project
```

(Already discussed with eg.)

Step 2: Create a New Branch for Your Work

Never work directly on the main branch.

```
git checkout -b feature/login-page
```

Real-life example:

- Project repo = **E-commerce website**
- You → creating **login page** feature → branch name = `feature/login-page`

Step 3: Make Changes Locally

- Edit files, add new features, or fix bugs.
- Stage changes:

```
git add .
```

- Commit with a meaningful message:

```
git commit -m "Added login form with validation"
```

Step 4: Push the Branch to GitHub

```
git push origin feature/login-page
```

Now your branch is visible on GitHub.

Step 5: Create a Pull Request (PR)

- Go to GitHub → Open a **Pull Request** from feature/login-page → main.
- Describe what you did.
- Example PR message:

"Implemented login page with form validation and session handling."

Step 6: Code Review & Discussion

- Teammates review your code.
 - They can request **changes** or **approve**.
-

Step 7: Merge PR

- Once approved → merge into main.
- If conflicts appear → resolve with:

```
git pull origin main
```

```
# Fix conflicts in files
```

```
git add .
```

```
git commit -m "Resolved merge conflicts"
```

```
git push origin feature/login-page
```

Step 8: Delete Branch (after merge)

```
git branch -d feature/login-page # local
```

```
git push origin --delete feature/login-page # remote
```

3.5.1 Best Practices for Commits, Branches, and Messages

A. Branching Best Practices

- Use **descriptive names**:
 - feature/login-page
 - bugfix/cart-issue
 - hotfix/security-patch
 - One branch = One task/feature
-

B. Commit Best Practices

- Small, frequent commits (atomic commits).
 - Avoid large commits like: "updated everything".
 - Good messages:
 - "Added search functionality in product list"
 - "Fixed bug in checkout page discount calculation"
 - Bad messages:
 - "Fixed", "Update code", "Work done"
-

C. Pull Request Best Practices

- Clear **title + description**.
 - Link issue number (if using GitHub issues).
 - Request review from teammates.
-

D. General Workflow Best Practices

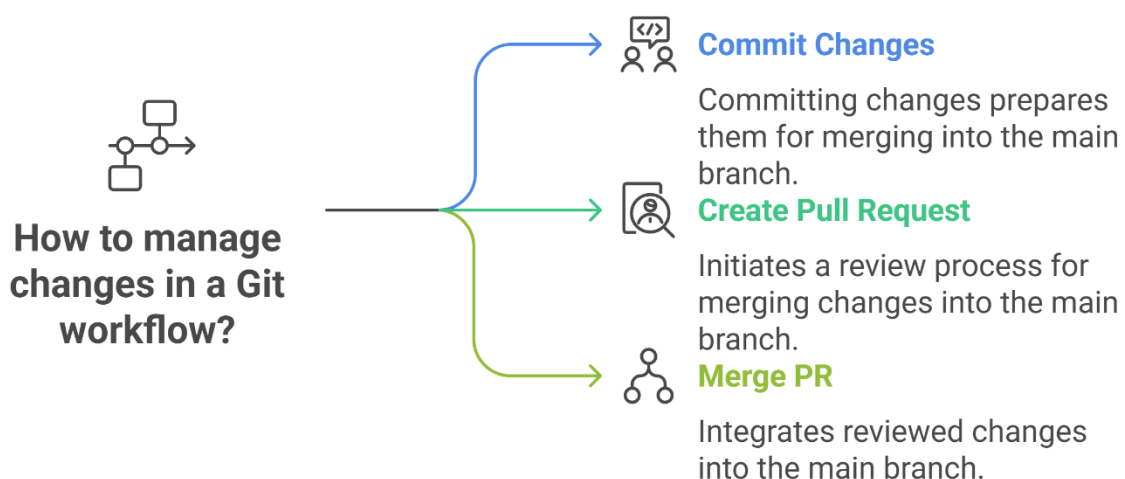
1. **Always pull latest changes before starting work:**
 2. `git pull origin main`
 3. **Work in your own branch.**
 4. **Keep commits clean.**
 5. **Write meaningful PR descriptions.**
 6. **Delete merged branches** to avoid clutter.
-

3.5.2 Real-Life Example Flow

Team Project: *E-commerce Website*

- **Alice** creates feature/login-page.
 - **Bob** creates feature/cart-system.
 - Both work separately.
 - They push branches → make PR → merge into main.
 - The main branch always has **stable, production-ready code**.
-

3.5.3 Diagram of GitHub Collaboration Workflow



Made with Napkin

This workflow ensures **smooth collaboration**, prevents conflicts, and keeps the main branch stable.

3.6 CSS selectors: element, class, ID

"A CSS selector is used to pick an HTML element and give it style."

CSS selector is used style on web page.

There are three fundamental types of CSS selectors.

1. Element Selector
2. Class Selector
3. ID Selector

3.6.0 Element Selector

It selects HTML elements by their name and applies styles to them.

Targets HTML tags directly.

Targets all tags like p, h1, div etc.

Syntax

```
p {  
color: red;  
}
```

3.6.1 Class Selector

Can be used for many elements at once

Selects elements with a specific class

Syntax in CSS: .classname (e.g. .box, .center, .btn)

```
.highlight {  
background: yellow;  
}
```

3.6.2 ID Selector

Each ID must be unique in a page.

affects one specific element

Selects a unique element with a specific id.

Syntax: #idname

```
#title {  
  color: green;  
}
```

3.7 CSS Properties: Color, fonts, text alignment, font-size, Box model: padding, margin, border

Styling Text and Elements

3.7.0 Color

Sets the color of a text

```
P{  
  color: color-name | rgb | rgba | hsl | hsla | hexadecimal;  
}
```

Example (color: red), hexadecimal values (e.g., #FF0000), RGB values (e.g., rgb(255, 0, 0)), or HSL values (e.g., hsl(0, 100%, 50%)).

Sets background color

Sets the background color

```
p {  
  background-color: lightgray;  
}
```

```
p {  
  color: red;  
  background-color: lightgray;  
}
```

Text-align

This property aligns the text in an element at a specific position

text-align: left | right | center | justify | start | end | initial | inherit

```
h1 {  
  text-align: center;  
}
```

3.7.1 Font Size

Sets the size of the text/Font

Setting the text size with pixels (px) gives you full control over the text size.

- em: This unit is relative to the font size of the parent element.
- rem: This unit is relative to the font size of the root HTML element.
- %: This unit is relative to the font size of the parent element
- smaller and larger: These units adjust the font size relative to the parent element.

```
p {  
  font-size: 20px;  
}
```

3.7.2 Box Model

Padding

Space inside the element (between content & border).

It is controlled by the padding property,

You can set padding for each side

padding-top, padding-right, padding-bottom, padding-left

1. padding: 10px; (all sides)
2. padding: 10px 20px; (top-bottom, left-right)
3. padding: 10px 15px 20px 25px; (top, right, bottom, left).

```
div {  
  background-color: blue;  
  padding: 20px;  
}
```


Margin

This is the outermost area, between the element's border and other elements.

They are controlled by the margin property,

You can set margin for each side:

Margin-top, margin-right, margin-bottom, margin-left.

1. `margin: 20px;` (all sides)
2. `margin: 10px 20px;` (top-bottom, left-right)
3. `margin: 10px 15px 20px 25px;` (top, right, bottom, left).

```
div {  
background-color: green;  
margin: 30px;  
}
```

Border that goes around the padding and content

Borders can be applied to each side:

`border-top`, `border-right`, `border-bottom`, `border-left`

Border is always visible and sits between padding and margin.

Three main properties:

- **Width** → thickness (e.g., 10px)
- **Style** → solid, dashed, dotted, double, etc.
 1. **Solid border** → Straight continuous line.
 2. **Dashed border** → Line made of dashes.
 3. **Dotted border** → Line made of dots.
 4. **Double border** → Two parallel lines.
- **Color** → any color (e.g., yellow, blue).
 1. Save files in the same folder:
 2. Save html Code **File Name.html**
 3. `index.html` (File Name :indexl)

```
<!DOCTYPE html>

<html>
<head>

  <title>CSS Border Example </title>

  <link rel="stylesheet" href="New1.css">
</head>
<body>

  <h2>CSS Border </h2>

  <div class="box1">This is a solid border.</div>
  <div class="box2">This is a dashed border.</div>
  .box1 {
    border: 2px solid black;
    padding: 10px;
    margin: 10px;
  }
  .box2 {
    border: 3px dashed red;
    padding: 10px;
    margin: 10px;
  }
  .box3 {
    border: 4px dotted blue;
    padding: 10px;
    margin: 10px;
  }
  .box4 {
    border: 5px double green;
    padding: 10px;
    margin: 10px;
  }
}
```

3.8 Applying styles: inline, internal, external

3.8.0 Inline CSS

Inline CSS has the highest priority compared to Internal and External CSS.

Affects only the specific HTML element where it is applied.

Style is written inside <body> tag

```
<body>

<p style="color: blue; font-size: 16px;">
This is a blue paragraph
</p>
```

3.8.1 Internal CSS

Internal CSS is written inside the <style> tag within the <head> section of an HTML

Internal CSS has less priority than Inline CSS but higher than External CSS.

Styling a single webpage

Style is written inside <style> tag in the <head>.

```
<head>

<title>Internal CSS </title>

<style>

h1 {

    color: red;

    text-align: center;

}

p {

    font-size: 20px;

    color: blue;

}

</style>

</head>
```

3.8.2 External CSS

Style is written in a separate file (style.css).

separate .css file and linked to the HTML document using the <link> element in

<head>

<link rel="stylesheet" href="style.css">

</head>

Best for large projects

Makes code clean, reusable, and easy to maintain.

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>External CSS </title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>This is External CSS</h1>
  <p>This paragraph is styled using External CSS.</p>
</body>
</html>
```

style.css

```
h1 {
  color: purple;
  text-align: center;
}
p {
  font-size: 25px;
  color: red;
}
```

This is External CSS

This paragraph is styled using External CSS.

Assignment 1

Design a web page, which uses padding, margin, and border in External CSS

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Box Model</title>
  <link rel="stylesheet" href="Style11.css">
</head>
<body>
  <h1>CSS Box Model </h1>
  <div class="box">
    <p>ABES Engg College Ghaziabad</p>
  </div>
</body>
</html>
```

style.css

```
.box {
  width: 400px;
  background-color: yellow;
  padding: 20px;
  border: 25px solid blue;
  margin: 30px;
  text-align: center;
}
```

CSS Box Model



4.0 Display: block, inline, inline-block, float and clear

4.0.0 Display: block

- Block-level elements always start on a new line and take up the full available width of their parent container.
- They respect width, height, margin, and padding properties.
- Examples include div, p, h1, ul, li.

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h2>display: block (Headings/Paragraphs)</h2>
    <h1 style="display: block; background: lightblue">
      Amazon - Online Shopping
    </h1>
    <p style="display: block; background: lightgreen">
      This is a full-width paragraph like a blog or product description.
    </p>
  </body>
</html>
```

Output:

display: block (Headings/Paragraphs)

Amazon - Online Shopping

This is a full-width paragraph like a blog or product description.

4.0.1 display: inline

- Inline elements do not start on a new line and only take up as much width as their content requires.
- They do not respect width and height properties. Top and bottom margin and padding are applied but may visually overlap with surrounding content.
- Examples include span, a, strong, em.

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h2>display: inline (Navigation links)</h2>
    <a href="#" style="display:inline; margin-right:10px;">Home</a>
    <a href="#" style="display:inline; margin-right:10px;">Deals</a>
    <a href="#" style="display:inline; margin-right:10px;">Cart</a>
  </body>
</html>
```

Output:

display: inline (Navigation links)

[Home](#) [Deals](#) [Cart](#)

4.0.2 display: inline-block

- Inline-block elements combine characteristics of both inline and block elements.
- They do not start on a new line, allowing other elements to sit alongside them.
- They respect width, height, margin, and padding properties, behaving like block elements in this regard.
- This display value is often used for creating grid-like layouts where elements need to sit side-by-side but also require specific dimensions.

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h2>display: inline-block (Product cards)</h2>
    <div style="display:inline-block; width:150px; height:200px;
background:rgb(182, 182, 47); margin:10px; text-align:center;">
      <h4>Book</h4>
      <p>$10</p>
    </div>
    <div style="display:inline-block;
background:rgb(216, 45, 96); margin:10px; text-align:center;">
      <h4>Headphones</h4>
      <p>$25</p>
    </div>
    <div style="display:inline-block; width:150px; height:200px;
background:lightblue; margin:10px; text-align:center;">
      <h4>Watch</h4>
      <p>$50</p>
    </div>
  </html>
```

Output:

display: inline-block (Product cards)



4.0.3 float

- The float property takes an element out of the normal document flow and positions it to the left or right within its containing element.
- Content (like text) will wrap around the floated element.
- Commonly used for image and text wrapping or creating multi-column layouts, though modern CSS layout methods like Flexbox and Grid are often preferred for complex layouts.
- Values include left, right, none.

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body bgcolor="pink">
    <h2>float (Image in an article)</h2>
    
    <p>
This text flows around the image, just like a newspaper <br>or blog post with an
image aligned to the left.
It continues <br>to wrap neatly around the floated image.
    </p>
  </body>
</html>
```

Output:

float (Image in an article)



This text flows around the image, just like a newspaper or blog post with an image aligned to the left. It continues to wrap neatly around the floated image.

4.0.4 clear

- The clear property is used in conjunction with float to control the positioning of elements that follow a floated element.
- It prevents an element from sitting alongside a floated element on the specified side.
- Values include left (clears floats on the left), right (clears floats on the right), both (clears floats on both sides), and none (default, allows floats on both sides).
- Often used to prevent content from wrapping around a floated element or to ensure a container expands to encompass its floated children (e.g., using a "clearfix" hack).

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <title>Document</title>
  </head>
  <body bgcolor="pink">
    <h2>clear (Footer below floated content)</h2>
    
    <p>This paragraph wraps around the floated image.</p>
    <p style="clear:both; background:lightgray;">This is the footer section,
forced below the floated image using clear:both.</p>
  </body>
</html>
```

Output:

clear (Footer below floated content)



This paragraph wraps around the floated image.

This is the footer section, forced below the floated image using clear:both.

4.1 Introduction to Flexbox: justify-content, align-items, flex-direction

Flexbox

Flexbox (Flexible Box Layout) is a modern CSS3 layout system designed to align, distribute, and organize elements in a container efficiently. It is extremely useful for creating responsive web applications and solving alignment issues that older techniques (like floats and inline-block) could not handle.

4.1.0 Introduction to Flexbox

Flexbox works on a parent-child relationship where the parent (container) uses flex properties, and the children (items) adjust themselves accordingly. Flexbox provides control in two axes:

- **Main Axis** – controlled by flex-direction and justify-content.
- **Cross Axis** – controlled by align-items.

Example:

```
.container {  
  display: flex; /* flexbox activate */  
}
```

4.1.1 flex-direction

The 'flex-direction' property defines the direction of the main axis of the flex container.

Values:

- **row (default)**: Items placed left to right.
- **row-reverse**: Items placed right to left.
- **column**: Items placed top to bottom.
- **column-reverse**: Items placed bottom to top.

Example:

```
.container {  
  display: flex;  
  flex-direction: column;  
}
```

4.1.2 justify-content

The 'justify-content' property aligns flex items along the main axis (horizontal for row, vertical for column).

Values:

- **flex-start:** Items **aligned at the start**.
- **flex-end:** Items aligned at the end.
- **center:** Items aligned at the center.
- **space-between:** Equal space between items; first and last at edges.
- **space-around:** Equal space around items with half space at edges.
- **space-evenly:** Equal space between all items including edges.

Example:

```
.container {  
  display: flex;  
  justify-content: space-between;  
}
```

4.1.3 align-items

The 'align-items' property aligns items along the cross axis (perpendicular to main axis).

Values:

- **stretch (default):** Items stretch to fill container.
- **flex-start:** Items align to start of cross axis.
- **flex-end:** Items align to end of cross axis.
- **center:** Items aligned at center of cross axis.
- **baseline:** Items align based on text baseline.

Example:

```
.container {  
  display: flex;  
  justify-content: space-between;  
}
```

4.1.4 Example

Below is an example showing flex-direction, justify-content, and align-items working together:

Example:

Index.html

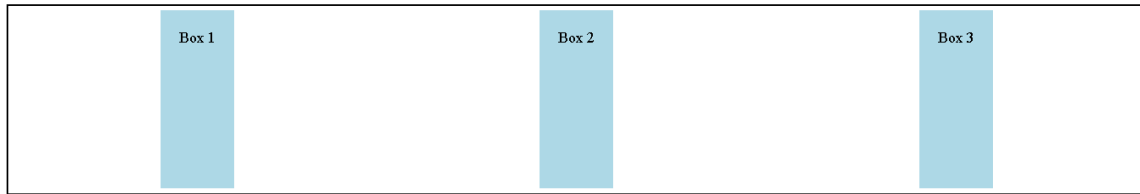
```
<!DOCTYPE html>
<html>
<head>
  <title>Flexbox Example</title>
  <!-- CSS file ko link karna -->
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
  <div class="container">
    <div class="item">Box 1</div>
    <div class="item">Box 2</div>
    <div class="item">Box 3</div>
  </div>
</body>
</html>
```

Stylesheet.css

```
.container {
  display: flex;          /* Flexbox activate */
  flex-direction: row;    /* Items left → right */
  justify-content: space-around; /* Horizontal equal spacing */
  align-items: center;    /* Vertical center alignment */
  height: 200px;         /* Container height */
  border: 2px solid black; /* Border around container */
}

.item {
  background: lightblue; /* Box background color */
  padding: 20px;         /* Inner spacing */
  margin: 5px;           /* Outer spacing between boxes */
}
```

Output



4.1.5 Advantages of Flexbox

- Simplifies centering both horizontally and vertically.
- Provides responsive layouts without using floats or positioning hacks.
- Allows reordering of items easily.
- Adjusts automatically to different screen sizes.

4.1.6 Real-life Use Cases

- Navigation bars using justify-content: space-between.
- Login page content aligned using align-items: center.
- Card layouts with row direction and wrapping.
- Footers distributed evenly using space-evenly.

4.2 Units: px, %, em, rem, vh, vw, Media queries and breakpoints

4.2.0 Pixel (px)

Definition

- px stands for **pixels**.
- It is an **absolute unit** (fixed size).
- 1px = 1 dot on the screen (depending on screen resolution).

Example

```
p {  
  font-size: 16px;  
}
```


Advantages

- ✓ Predictable and precise.
- ✓ Simple for beginners.

Disadvantages

- ✗ Not scalable → does not adapt to user preferences.
- ✗ Can look too small or too big on different screens.

4.2.1 Percentage (%)

Definition

- A **relative unit**.
- Always calculated **relative to the parent element**.

Example

```
div {  
  width: 50%;  
}
```

Advantages

- ✓ Flexible, automatically adjusts to parent size.

Disadvantages

- ✗ Can be confusing when parent has no fixed size.

4.2.2 em

Definition

- Relative to the **font-size of the parent element**.
- 1em = font-size of the parent.

Example

```
div {  
  font-size: 20px;  
}  
  
p {  
  font-size: 2em; /* = 40px */  
}
```

Nested Example (Multiplication Effect)

```
div { font-size: 16px; }  
section { font-size: 1.5em; } /* 24px */  
p { font-size: 2em; } /* 48px */
```

Advantages

✓ Flexible and scalable.

Disadvantages

✗ Can “multiply” in nested elements → hard to control.

4.2.3 rem (Root em)

Definition

- Relative to the **root (<html> element’s font-size**.
- 1rem = font-size of <html>.

Example

```
html { font-size: 16px; }  
p { font-size: 2rem; } /* 32px */
```

Advantages

- ✓ Avoids “multiplication problem” of em.
- ✓ Easier to manage responsive typography.

Disadvantages

✗ Less flexible in some nested designs.

4.2.4 vh (Viewport Height)

Definition

- Relative to the **height of the browser’s viewport**.
- 1vh = 1% of viewport height.

Example

```
div {  
  height: 50vh;  
}
```

Advantages

✓ Adapts to screen size automatically.

Disadvantages

✗ May cause issues with mobile browsers when toolbars appear/disappear.

4.2.5 vw (Viewport Width)

Definition

- Relative to the **width of the browser's viewport**.
- 1vw = 1% of viewport width.

Example

```
div {  
  width: 100vw;  
}
```

Advantages

✓ Very useful for **responsive layouts**.

Disadvantages

✗ sometimes overflows if scrollbars are present.

Unit	Type	Relative To	Example	Use Case
px	Absolute	Screen pixels	16px	Borders, icons, fixed design
%	Relative	Parent element	50%	Fluid layouts, grids
em	Relative	Parent font-size	2em	Flexible text, padding
rem	Relative	Root (<html>) font-size	2rem	Consistent typography
vh	Relative	1% of viewport height	50vh	Fullscreen sections
vw	Relative	1% of viewport width	100vw	Responsive widths

4.3 Media Queries & Breakpoints, Mobile-first design

4.3.0 Introduction

Modern websites must adapt to many devices: mobiles, tablets, laptops, and desktops.

Media queries and **breakpoints** are the core tools in CSS for achieving **responsive design**.

4.3.1 What are Media Queries?

- A **media query** is a CSS feature that applies styles **only if certain conditions are true** (like screen width, height, orientation, resolution, etc.).
- It helps in **customizing design** for different devices.

Syntax

```
@media (condition) {  
    /* CSS styles here */  
}
```

Example

```
@media (max-width: 768px) {  
    body {  
        background-color: lightblue;  
    }  
}
```

4.3.2 Why Use Media Queries?

1. To make websites **responsive**.
2. To improve **usability** on small devices.
3. To **avoid horizontal scrolling**.

To serve different layouts for **mobile, tablet, and desktop** users.

4.3.3 What are Breakpoints?

- **Breakpoints** are specific **screen widths** where the layout of a webpage changes.
- They are used with **media queries**.

Common Breakpoints (industry standard):

- **320px – Small phones**
- **480px – Large phones**
- **768px – Tablets**
- **1024px – Small laptops**
- **1200px+ – Desktops**

Types of Media Queries

a) **Max-width (applies up to a certain width)**

```
@media (max-width: 768px) {  
  p { font-size: 14px; }  
}
```

b) Min-width (applies from a certain width upwards)

```
@media (min-width: 1024px) {  
  p { font-size: 18px; }  
}
```

c) Min & Max Combined (specific range)

```
@media (min-width: 768px) and (max-width: 1023px) {  
  p { font-size: 16px; }  
}
```

d) Other Conditions

Orientation:

```
@media (orientation: landscape) {  
  body { background: lightgreen; }  
}
```

Resolution:

```
@media (min-resolution: 300dpi) {  
  img { width: 80%; }  
}
```

4.3.4 Mobile-First vs Desktop-First Approach

a) Mobile-First (Recommended)

Default styles written for **small screens** first.

Use min-width for larger devices.

Example:

```
/* Default (mobile) */  
p { font-size: 14px; }  
/* Tablet and above */  
@media (min-width: 768px) {  
  p { font-size: 16px; }}
```

```
/* Desktop and above */  
  
@media (min-width: 1024px) {  
  p { font-size: 18px; }  
}
```

b) Desktop-First

- Default styles written for **large screens**.
- Use max-width for smaller devices.

Example:

```
/* Default (desktop) */  
  
p { font-size: 18px; }  
  
  
/* Tablet */  
  
@media (max-width: 1024px) {  
  p { font-size: 16px; }  
}  
  
/* Mobile */  
  
@media (max-width: 768px) {  
  p { font-size: 14px; }  
}
```

4.3.5 Advantages of Media Queries

- ✓ Flexible design for all devices.
- ✓ Better **user experience**.
- ✓ Helps in **SEO ranking** (Google prefers mobile-friendly sites).
- ✓ Saves cost (no need for separate mobile website).

4.3.6 Disadvantages

- ✗ Too many breakpoints = complex CSS.
- ✗ Harder to maintain if not planned properly.

4.3.7 Practical Example – Responsive Box

```
<!DOCTYPE html>
<html>
<head>
<style>
.box {
  width: 100%;
  height: 100px;
  background: green;
}
/* Tablet */
@media (min-width: 768px) {
  .box {
    background: blue;
  }
}
/* Desktop */
@media (min-width: 1024px) {
  .box {
    background: red;
  }
}
</style>
</head>
<body>
  <div class="box"></div>
</body>
</html>
```

4.4 Responsive navigation concepts, Practical

4.4.0 Introduction

Navigation is one of the most important parts of a website.

- On **desktops**, space is available for a full horizontal navigation bar.
- On **mobiles**, screen width is limited → so we must design navigation that is **responsive**.
- A **responsive navigation** adapts its layout depending on the device size.

4.4.1 Types of Navigation Layouts

a) Horizontal Navigation (Desktop)

- Links are shown **side by side** in a row.
- Common for large screens.

Example:

```
nav ul {  
  display: flex;  
  justify-content: space-around;  
}
```

b) Vertical Navigation (Mobile)

- Links are stacked in a **column**.
- Saves space on smaller screens.

Example:

```
nav ul {  
  display: flex;  
  flex-direction: column;  
}
```

c) Hamburger Menu (≡)

- Popular design pattern for mobile.
- A small icon (≡) represents the menu.
- On click/tap → it expands to show navigation links.

4.4.2 Responsive Navigation Techniques

a) Using Flexbox

- Flexbox allows arranging menu items **horizontally or vertically**.
- Example:

```
nav {  
  display: flex;  
  justify-content: space-between;  
}
```

b) Using Media Queries

- Show/hide navigation based on **breakpoints**.
- Example:

```
/* Mobile */  
.menu { display: none; }  
.hamburger { display: block; }  
/* Tablet & Desktop */  
@media (min-width: 768px) {  
  .menu { display: flex; }  
  .hamburger { display: none; }  
}
```

c) Hide/Show Menu

- Use `display: none;` and `display: block;` to toggle menus.
- For mobile → hide menu by default, show on button click.

d) Dropdowns / Sidebars

- On mobile, menus can **slide in from the side**.
- Achieved with **CSS transitions** or **JavaScript toggles**.

4.4.3 Mobile-First Responsive Navigation

a) Default (Mobile)

- Show only logo and hamburger menu (☰).
- Menu hidden by default.

b) Tablet/Desktop

- Show full menu inline.
- Hide hamburger icon.

4.4.4 Practical Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Navigation</title>
  <style>
    body {
      margin: 0;
      font-family: Arial, sans-serif;
    }
    nav {
      display: flex;
      justify-content: space-between;
      align-items: center;
      background: #333;
      padding: 1rem;
    }
    .logo {
      color: #fff;
      font-size: 1.5rem;
    }
    .menu {
      display: none;
      flex-direction: column
```

```
background: #444;
position: absolute;
top: 60px;
right: 20px;
width: 200px;
}
.menu a {
color: #fff;
padding: 1em;
text-decoration: none;
display: block;
}
.hamburger {
font-size: 2rem;
color: #fff;
cursor: pointer;
}
/* Tablet & Desktop */
@media (min-width: 768px) {
.menu {
display: flex;
flex-direction: row;
position: static;
background: none;
width: auto;
}
.hamburger {
display: none;
}
}
```

```
</style>
</head>
<body>
  <nav>
    <div class="logo">MySite</div>
    <div class="hamburger">☰</div>
    <div class="menu">
      <a href="#">Home</a>
      <a href="#">About</a>
      <a href="#">Services</a>
      <a href="#">Contact</a>
    </div>
  </nav>
</body>
</html>
```

4.4.5 Advantages of Responsive Navigation

- ✓ Works across devices (mobile, tablet, desktop).
- ✓ Saves screen space on small devices.
- ✓ Improves user experience.
- ✓ Reduces bounce rate (users stay longer).

4.4.6 Challenges

- ✗ Requires careful design for different screen sizes.
- ✗ Sometimes needs **JavaScript** for toggling hamburger menu.
- ✗ More testing required across devices.

4.5 Project planning and design, Version control in large projects

4.5.0 Project Planning & Design + Version Control (Git)

Why this topic?

- **Engineering is teamwork.** You rarely build software alone.
- Without planning: projects fail, deadlines slip, and bugs pile up.

- Without version control: confusion happens → “Whose code is latest?”
“Who deleted my file?”

Think of **project planning** like preparing a blueprint before building a house.
Think of **version control (Git)** like Google Docs → multiple people editing safely.

4.5.1 Project Planning & Design – Basics

4.51A Steps of Project Planning

1. **Problem Understanding** – What are we building? Who are the users?
Example: “We are building a college management portal.”
2. **Requirements** – Features + constraints.
Example: Login system, student records, attendance, online notices.
3. **Design (Architecture)** – How will it work? (Flowcharts, diagrams).
Example: Client (browser) → Server (backend) → Database.
4. **Scheduling** – Divide work into tasks & assign to team members.
5. **Testing Plan** – How will we check correctness?
6. **Delivery** – Deployment to real users.

4.51B Example PRD (Product Requirement Document) – Mini

Title: College Notice Portal

Problem: Students miss important notices on notice boards.

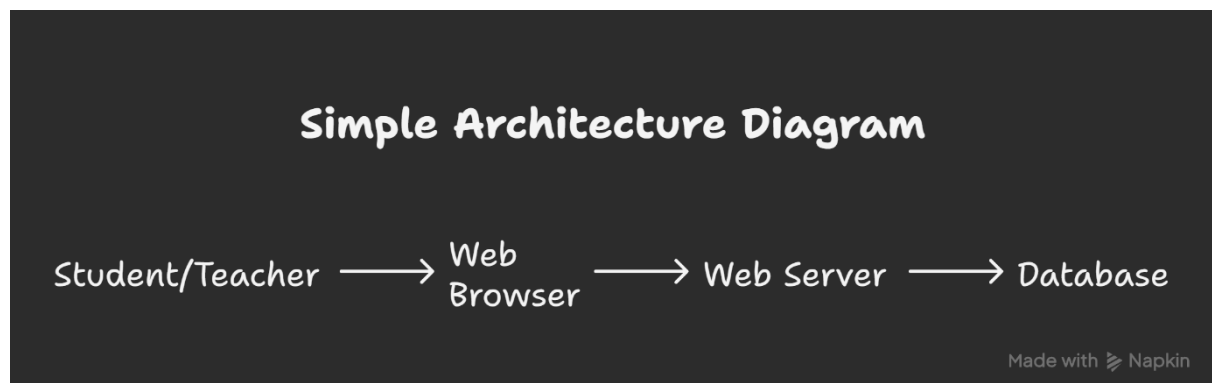
Goal: Digital notice system with instant updates.

Users: Students, Faculty, Admin.

Scope: Create, read, and manage notices.

Constraints: Simple UI, mobile-friendly, limited budget.

4.51C Simple Architecture Diagram (student-level)



4.51D Why Design Matters?

- Saves time later.
- Avoids confusion.
- Makes teamwork easier.

4.5.2 Version Control (Git) – Basics

4.52A What is Git?

- Git = a **tool** that keeps track of changes in code.
- Think of it like a **time machine for your project**.
- Every change = snapshot → you can go back anytime.

Without Git:

- “FinalProject_v1”,
- “FinalProject_v2_reallyfinal”,
- “FinalProject_reallyreallyfinal”

With Git:

- One project folder, history saved, easy teamwork.

4.52B What is GitHub?

- GitHub = online platform to host Git repositories.
- Just like Google Drive but for code.
- Enables collaboration → multiple students can work together.

4.52C Why do we need Version Control in *large projects*?

- Many developers → avoid conflicts.
- Backup in case of mistakes.
- Parallel work → one works on login, another on payment system.
- Track who did what.
- Easy to rollback if something breaks.

4.5.3 Git Basics – Commands for Students

4.53A Setup

```
# Configure Git
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

4.53B Creating a Project

```
# Create new project
mkdir college-portal
cd college-portal
git init # start git
```

4.53C Saving Work

```
echo "Hello Project" > index.html
git add index.html
git commit -m "First commit: added index.html"
```

4.53D Checking History

```
git log
```

Already covered in detail in Git Basics.

4.5.4 Collaboration with GitHub

4.54A Workflow

1. Create repo on GitHub.
2. Clone repo in local system.
3. Create branch → make changes.
4. Push branch → Open Pull Request.
5. Merge after review.

Already covered in detail in Git Advance.

4.54B Example (Student Team Project)

```
# clone repo
git clone https://github.com/username/college-portal.git
```

```
cd college-portal
```

```
# create a new branch
```

```
git checkout -b feature/login-page
```

```
# make changes...
```

```
git add login.html
```

```
git commit -m "Added login page"
```

```
# push branch
```

```
git push origin feature/login-page
```

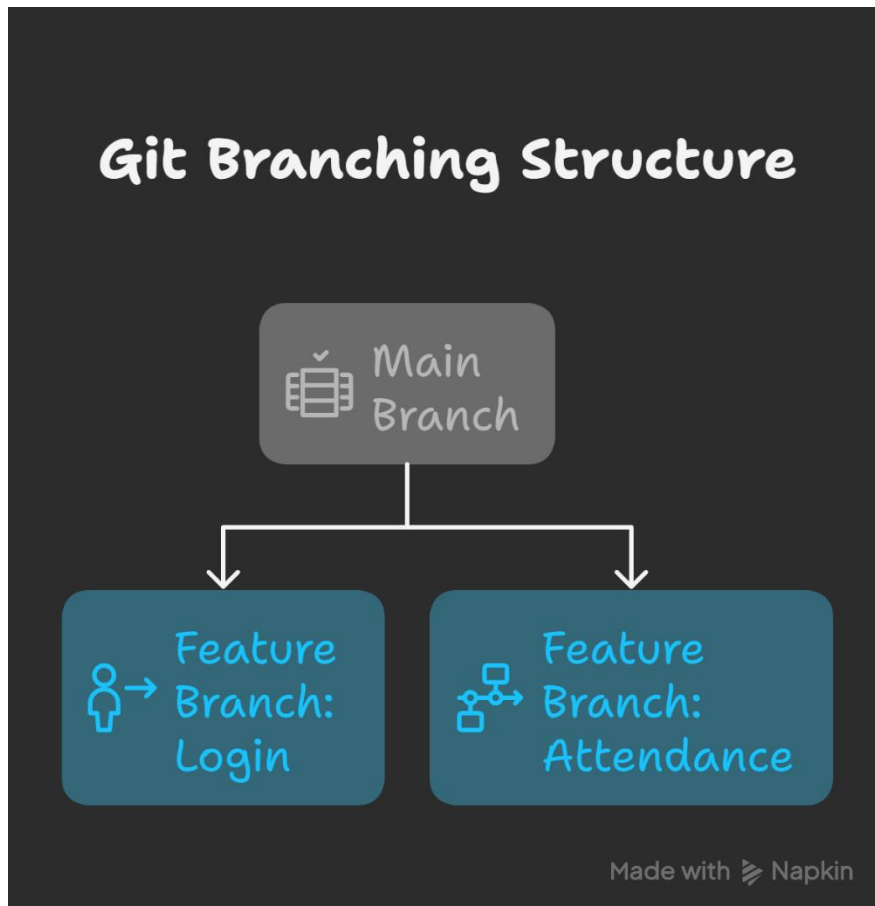
Another teammate reviews and merges on GitHub.

Already covered in detail in Git Advance.

4.5.6 Git Branching – Visual

```
main ----o---o---o-----
      \      \
      o (login) o (attendance)
```

- main → stable version.
- feature/* → new features.
- Later → merged back.



4.5.7 Example Classroom Activity

Task: Create a simple HTML page (index.html), track with Git, commit changes.

Next Step: Pair students → one adds "About Us", another adds "Contact Us". Merge changes.

Students will see conflicts → explain conflict resolution.

4.5.8 Conflict Example

File: index.html

- Student A changes heading to Welcome Students.
- Student B changes heading to Hello World.

When merged → Git asks to resolve conflict.
Student must decide final content.

Already covered in detail in Git Advance.

4.5.9 Git in Large Projects (Advanced but simplified)

- **Branches for features** → multiple teams can code at same time.
- **Pull Requests** → ensure code is reviewed.
- **CI/CD (Continuous Integration/Deployment)** → automatic testing & deployment.

Example: In Flipkart, 100+ developers can push changes daily → Git + GitHub manage safely.

4.6 Deploy using GitHub Pages, Final portfolio polishing

4.6.0 What is GitHub Pages?

Imagine you made a website (HTML + CSS files) in your laptop. Now you want to **show it to the world** — friends, teachers, companies.

- GitHub Pages = free hosting by GitHub.
- You don't need to buy hosting, just upload your files to GitHub → GitHub will give you a live website link like:

<https://yourusername.github.io/>

4.6.1 Folder Structure (very simple)

```
my-portfolio/
├── index.html
├── about.html
├── css/
│   └── style.css
├── images/
│   └── profile.jpg
```

4.6.2 Steps to Deploy (Step by Step)

Step 1 – Create GitHub Repository

1. Go to github.com → Sign in.
2. Click **New Repository** → Name it yourusername.github.io (⚡ this name is special, it makes your main website).
3. Keep it **Public** → Click **Create Repository**.

Step 2 – Upload Code

With Git commands:

- git init
- git add .
- git commit -m "my portfolio first commit"
- git branch -M main

- git remote add origin https://github.com/username/username.github.io.git
- git push -u origin main

Step 3 – Enable GitHub Pages

1. Go to **Settings** → **Pages**.
2. Under **Source**, choose main branch.
3. Save → Wait 1-2 minutes.
4. Visit https://yourusername.github.io → Your site is live!

4.6.4 Final Portfolio Polishing (Checklist)

Content

- Home page = Short intro: *“Hi, I am ____, B.Tech student, learning web development.”*
- About page = Skills, tools you know.
- Projects page = 2–3 mini projects using HTML and CSS.
- Contact = Email + LinkedIn/GitHub links.

Design & Look

- Use **clean colors** (2–3 only).
- Keep **same font style** everywhere.
- Make sure text is **readable on mobile**.
- Add **profile picture or logo** at top.

Technical

Title for each page (<title>My Portfolio</title>).

- Add meta description for SEO:
- <meta name="description" content="Portfolio website of Pranshi, B.Tech student">
- Use **alt=""** text for all images.
- Resize images (don't upload 5 MB photos).